ElevatePrep Senior Java Interview Master Guide - SDE-3

Java 21 baseline | ElevatePrep edition v25 | 9 October 2026

Java, Spring Boot, Hibernate/JPA, microservices, Maven, GCP and Kubernetes.

292 numbered Q&As, core module questions, 60 daily scenarios and 24 interview simulations across 21 parts. Java examples use Java 21 without preview features unless explicitly labeled as a later-version comparison. Framework snippets may require their named dependencies and application types; they are not complete applications.

This edition aligns Parts 01-16, the new Part 21 and all five daily practice sets with the current local ElevatePrep source as of 9 October 2026. It includes the expanded foundation paths, worked explanations, topic-specific follow-ups, annotation and persistence lessons, and reasoning before coding. Parts 17-20 preserve the supplied reviewed guide's additional Q245-Q272 and 24 interview simulations. Those supplemental sections are not currently listed in ElevatePrep's website catalog. The repaired thread pool and complete producer-consumer example are retained as worked alternatives. Part 21 adds Q273-Q292 in four-tier learning ladders, with reviewed explanations and two Java fragments exercised by a Java 21 fixture (preview enabled for Q289). Historical validation claims are not evidence for this export; see Appendix B.

How to use the guide

Explain the mechanism, why it exists, a realistic failure, the trade-off and how to verify it. A definition-only answer is a starting point. Work toward a decision supported by constraints and evidence. Treat all example incidents and metrics as illustrative, and substitute your own experience.

Score	What the answer demonstrates
1	Definition only
2	Correct mechanism, limited reasoning
3	Mechanism, purpose and common pitfall
4	Trade-offs, alternatives and failure handling
5	A defensible decision and production evidence

Contents

1. Part 01: Course and core modules
2. Part 02: Question Bank Vol. 1 - Java, Spring, JPA and microservices
3. Part 03: Question Bank Vol. 2 - Kafka, security, Maven, GCP and Kubernetes
4. Part 04: Question Bank Vol. 3 - Scenarios, design and coding
5. Part 05: Question Bank Vol. 4 - Modern Java and the Spring ecosystem
6. Part 06: Question Bank Vol. 5 - Rapid-fire, puzzles and harder coding
7. Part 07: Question Bank Vol. 6 - Kubernetes, Terraform, APIs and leadership
8. Part 08: Question Bank Vol. 7 - Streams, WebFlux and interview practice
9. Part 09: Coding interview practice track
10. Part 10: Web sourced questions and strong answer rubrics
11. Part 11: Decision walkthroughs for the full practice roadmap
12. Part 12: More Senior-Level Questions (Q154-Q173)
13. Part 13: Spring Boot Internals, Coding-Style and Leadership Questions (Q174-Q193)
14. Part 14: System Design, Data Internals, JVM and Coding Questions (Q194-Q213)
15. Part 15: Full System Design Walkthroughs and DP/Graph Problems (Q214-Q222)
16. Part 16: Hibernate and SQL Questions (Q223-Q244)
17. Part 17: More Hibernate and SQL Questions (Q245-Q272)
18. Part 18: SDE-3 Interview Simulation Lab
19. Part 19: 30-Minute SDE-3 Mock Interview Protocol
20. Part 20: Interviewer Follow-Up Rules
21. Part 21: Production Resilience, Delivery and JVM Operations (Q273-Q292)

Daily practice sets 1-5 · Appendix C: repaired thread pool

Appendix A: complete producer-consumer example · Appendix B: review and validation

Website navigation: Senior Java guide · Daily practice. Alignment uses the local editable website source; a deployed-site comparison was not performed.

Part 01 - Course and core modules

Senior Java Interview Refresher (9+ Years)

Stack: Java · Spring Boot · Hibernate/JPA · Microservices · Maven · GCP

Companion volumes: Vol. 1 (Q1–Q30), Vol. 2 (Q31–Q58), Vol. 3 (Q59–Q74 + designs + programs).

What interviewers want at 9+ yrs: not definitions, but *trade-offs, failure modes, production war stories*. Answer pattern: **Concept → Why it exists → Pitfall → What I did in production.**

2-Week Plan

Days	Focus
1–3	Arrays, hashing, two pointers, sliding windows; Java collections and complexity
4–5	Binary search, intervals, stacks, linked lists; one timed coding round
6–7	Trees, BFS/DFS, heaps; Java equality, ordering, overflow and generics
8–9	Graphs, topological sort, union-find and shortest paths; concurrency exercise
10–11	Backtracking and dynamic programming; one backend design deep dive
12–14	Two full coding mocks, one machine-coding round, failed-problem revision, STAR stories

Use Part 9 for a daily schedule and readiness criteria. Treat cloud and framework chapters as a lookup reference unless the role explicitly emphasizes them.

Topic learning order: start with the foundations

For each topic, begin with its starting question, explain the mechanism, then handle failures and trade-offs. Follow the linked foundation before the advanced question bank; the existing Q/M identifiers remain revision references rather than a required numerical reading order.

For learning: read the direct answer, work through the example, then answer the follow-up before looking at its expected reasoning. If a term is unfamiliar, use the foundation link or the terminology explanations. For code, explain the invariant and trace a small input before reading the implementation.

For the interviewer: begin with a concise definition and purpose, explain the mechanism, give one concrete example, and state the important limit or trade-off. Use the 30-second version as an opening, then expand when asked. Separate documented guarantees from implementation details and state the runtime/database version where it matters. Use your own experience for incident and leadership answers; illustrative examples are not personal evidence.

Self-check: could you explain why it works, predict one failure case and describe how to verify the result? Repeating an API name or memorizing a checklist does not establish those three abilities.

Topic	Starting question	Progression and entry point
Core Java and collections	What is an object, how do references work, and how do we choose a collection?	Java foundations → identity/immutability → List/Set/Map → hashing/ordering → generics → internals and boundary cases.
Multithreading	Why use threads, and how do we create, start and finish one?	Multithreading Questions → lifecycle → shared state → locks/coordination → executors/results → cancellation → production diagnosis.
JVM	Where does code run and where does data live?	JVM foundations → allocation/reachability → collection → latency/retention → profiling and version-specific tuning.
Spring Boot	How does a Java object become a managed bean?	Annotation ladder → registration/injection → configuration/conditions → proxies → MVC/persistence → failures and testing.
Hibernate and SQL	How does Java or Boot connect to the database?	Connection and session foundations → pools/sessions/factories → transactions → annotations → queries → concurrency and data design.
Microservices	What is a service boundary and how do services communicate?	Service foundations → ownership/API → timeouts/failures → consistency/idempotency → migration → scale.
Messaging	How does a producer's event reach a consumer?	Messaging foundations → topics/partitions/offsets → groups → acknowledgements → retries/ordering → delivery guarantees.
Security	Who is calling, and what may they do?	Security foundations → credentials/tokens → verification → authorization → browser threats → operational controls.
Maven	How do sources and dependencies become a tested artifact?	Build foundations → lifecycle/dependencies → plugins/scopes → modules → reproducibility → release.
Cloud and containers	Where does the process run, and who owns its resources?	Cloud foundations and container foundations → runtime → identity/network/data → health/scaling → deployment → diagnosis.

Topic	Starting question	Progression and entry point
Streams	How does a lambda become a stream pipeline?	Stream foundations → source/operations → evaluation → collectors → parallelism and pitfalls.
Caching	What happens on a cache hit, miss or update?	Cache foundations → cache-aside → TTL/invalidation → concurrent fills → overload and consistency.
Testing and observability	What behavior are we proving, and how do we diagnose failure?	Quality foundations → assertions/test boundaries → logs/metrics/traces → performance → flaky-test diagnosis.
Infrastructure as code	How does configuration become real infrastructure?	Terraform foundations → provider/resource → plan/apply → state → locking/drift and recovery.
APIs and reactive programming	How does a request complete, and how does a subscriber receive data?	API foundations → contracts/retries → protocol choice; reactive foundations → subscription/demand → operators → WebFlux failures and testing.
System design	Which requirements and guarantees should we establish first?	Design interview sequence → estimates → API/data → architecture → failures/scale → trade-offs and full walkthroughs.

Module 1: Core Java, Concurrency, JVM

Detailed answers. More in Vols. 1–3 (Q1–Q74).

C1 What should you explain before collection internals

Strong answer: distinguish primitive values from references to objects. Java passes argument values, including reference values, by value; assigning a parameter does not replace the caller's reference, while mutating a shared object can be visible through both references. Explain object identity versus logical equality, and define whether your value is mutable before using it as a collection key.

Follow-up: List models an ordered sequence with duplicates, Set membership, and Map key-to-value association. Choose using lookup/iteration/order requirements, null policy and mutation patterns. Equality and hashing must agree; comparator-based collections use their ordering relation for key/membership decisions. Learn these contracts before bucket trees or iterator internals.

Senior follow-up: challenge null/empty input, duplicate keys, mutable keys, comparator consistency and concurrent mutation. A collection's safety does not imply deep immutability of its elements. Continue with M1.1 and the identity/collection questions in Part 2.

M1.1 How does HashMap work internally?

A good SDE-3 interview answer is:

`HashMap` stores key-value pairs in an internal array of buckets. Java uses the key's `hashCode()` and spreads the hash bits to choose a bucket index. If multiple keys land in the same bucket, that is a **collision**; the bucket starts as a linked structure and can be converted to a red-black tree when the implementation's collision and table-capacity thresholds are met. With a good hash distribution, `get()` and `put()` are expected O(1), while resizing is occasional O(n) work, so insertion is amortized O(1).

First: what does "bin" mean?

Bin is OpenJDK implementation terminology for the entries associated with one position in the internal hash-table array. In an interview, it is perfectly fine to say **bucket**.

Think of it like:

```
HashMap table

index 0  → null
index 1  → entry
index 2  → null
index 3  → entry → entry
index 4  → null
```

So, for this discussion:

```
bucket ≈ bin
```

What happens during `put()`?

Conceptually:

```
key
 ↓
hashCode()
 ↓
hash spreading
 ↓
bucket index
 ↓
find entry / insert entry
```

If another key is already in that bucket, Java compares keys using `equals()` to determine whether it is the same mapping or a collision.

Why are `hashCode()` and `equals()` both needed?

`hashCode()` helps find the **bucket**.

`equals()` helps find the **actual key** inside that bucket.

That is why the `equals()` / `hashCode()` contract is critical.

What happens when there are many collisions?

A bucket can initially contain a linked chain of entries. When the relevant collision threshold is reached, Java may treeify that bucket. However, treeification is not simply "8 entries means tree": the table also needs to be large enough; otherwise resizing may happen first.

The tree is a red-black tree, which is self-balancing. This can reduce collision-search cost when hashes or useful key comparisons guide the search; equal-hash keys without useful ordering can still require a linear scan of the tree bin.

What happens during resizing?

`HashMap` grows its table when the number of mappings crosses the threshold derived from capacity and load factor. The default load factor is `0.75`.

For example:

```
capacity = 16
load factor = 0.75
threshold ≈ 12
```

When the table grows, entries are redistributed. The stored hash can be reused, so Java does not need to call the user's `hashCode()` again simply because the table resized.

Why is lookup normally O(1)?

With a good hash distribution, entries are spread across buckets, so each lookup examines only a small number of entries.

Therefore:

```
get()    → expected O(1)
put()    → expected O(1)
remove() → expected O(1)
```

But do **not** say "HashMap is always O(1)". Collisions and resizing matter.

Important production pitfall: mutable keys

If a key changes after insertion and the changed fields participate in `equals()` or `hashCode()`, the map may no longer find the entry using the mutated key.

So keys should normally be immutable with respect to fields used by `equals()` and `hashCode()`.

SDE-3 takeaway

A strong answer connects four things:

```
hashCode() → bucket selection
equals()   → key identity
collisions → linked structure / possible treeification
resize     → capacity + load factor + amortized cost
```

The Java 19+ `HashMap.newHashMap(expectedMappings)` API expresses an expected mapping count; older constructors interpret the argument as initial capacity. If calculating capacity manually, remember Java integer arithmetic when writing expressions such as `expected / 0.75 + 1`.

30-second version

> "`HashMap` uses an internal array of buckets. It uses the key's hash to select a bucket, and `equals()` to identify the exact key when collisions occur. Collided entries start in a linked structure and can be treeified under the implementation's thresholds. With good hash distribution, lookup and insertion are expected O(1); resizing is occasional O(n) work, so insertion is amortized O(1). Mutable keys are dangerous because changing a key's hash-related state after insertion can make the entry unreachable."

M1.1.1 Red-Black Tree Deep Dive — HashMap Follow-up

If the interviewer asks, **"You said a HashMap bucket can become a red-black tree. Explain the tree."**, do not jump straight into implementation cases. Start with the purpose.

A good SDE-3 interview answer is:

> A red-black tree is a self-balancing binary search tree. Each node has a red or black color, and the tree maintains balancing invariants that keep its height O(log n). With a useful search ordering, search, insertion and deletion are O(log n). HashMap uses tree bins to improve collision handling, but equal-hash keys without useful comparison ordering can still require O(n) lookup.

What problem does it solve?

A normal binary search tree can become skewed:

```
10
  \
   20
     \
      30
        \
         40
```

This is effectively a linked list, so search can become O(n).

A balanced tree keeps the height logarithmic:

```
       20
      /  \
    10    30
            \
             40
```

The exact shape varies, but the important property is:

```
height = O(log n)
search = O(log n)
insert = O(log n)
delete = O(log n)
```

What does “red-black” mean?

Every node has one extra piece of state:

```
RED
BLACK
```

The tree maintains these core properties:

1. Every node is either red or black.
2. The root is black.
3. `null` leaves are treated as black.
4. A red node cannot have a red child.
5. Every path from a node to its descendant `null` leaves has the same number of black nodes (black height).

These rules prevent the tree from becoming arbitrarily skewed.

How does it stay balanced?

Two operations are fundamental:

```
1. Recoloring
2. Rotation
```

There are two rotations:

```
Left rotation
Right rotation
```

A rotation changes the tree structure while preserving the binary-search-tree ordering.

For example:

```
Before:

    10
      \
       20
         \
          30
```

After a left rotation around `10`:

```
      20
     /  \
   10    30
```

The ordering is still:

```
10 < 20 < 30
```

but the height is improved.

Why are new nodes generally inserted as red?

Inserting a black node can immediately increase the black-node count on paths through that node.

Inserting red avoids changing black height immediately, although it can create another violation:

```
red parent
    +
red child
```

The tree then repairs the violation using recoloring and/or rotations.

What happens during insertion?

At a high level:

```
1. Insert as a normal BST node.
2. Color the new node red.
3. Check the red-black invariants.
4. If a red-red violation exists:
      - recolor when the surrounding structure allows it
      - otherwise rotate and recolor
5. Ensure the root is black.
```

You do not need to memorize every insertion case unless the interviewer explicitly asks you to implement a red-black tree.

Why does HashMap need this?

Suppose many different keys collide into the same bucket:

```
bucket 7

A → B → C → D → E → F → G → H
```

A linked structure makes lookup increasingly expensive as the collision chain grows.

Treeification changes the structure conceptually to:

```
          D
        /   \
       B     F
      / \   / \
     A   C E   G
               \
                H
```

The exact tree shape depends on the implementation and keys, but the key idea is:

```
linked collision chain → balanced tree
O(n) chain search → O(log n) tree search when ordering distinguishes keys
Equal hashes without useful key ordering → lookup may still be O(n)
```

Important: the entire HashMap does NOT become a tree

This is a common interview mistake.

`HashMap` still has its array of buckets:

```
HashMap
  |
  +-- table[0] → entry/list
  +-- table[1] → null
  +-- table[2] → entry/list
  +-- table[3] → tree
  +-- table[4] → null
  +-- ...
```

Only a heavily-collided bucket can be treeified.

When does HashMap treeify?

For the OpenJDK implementation relevant to Java 8+ discussions, the commonly discussed constants are:

```
TREEIFY_THRESHOLD     = 8
UNTREEIFY_THRESHOLD    = 6
MIN_TREEIFY_CAPACITY   = 64
```

But do **not** say:

> “At exactly eight entries it always becomes a tree.”

The correct interview explanation is:

> When a bucket becomes sufficiently populated, HashMap can treeify it, but it first checks the table capacity. If the table is still small, resizing is preferred because spreading entries across more buckets may solve the collision problem more cheaply.

The implementation details are version-sensitive, so use the target JDK's OpenJDK source if an interviewer asks for exact thresholds.

Can a treeified bucket become a list again?

Yes.

If the number of entries in that bucket falls sufficiently, the implementation can **untreeify** it.

That avoids keeping tree-management overhead for a bucket that is no longer heavily populated.

Why red-black tree instead of AVL?

A good senior-level answer is:

> Both AVL and red-black trees provide O(log n) operations. AVL is more strictly balanced and can provide slightly shorter search paths, but it generally performs more rebalancing during updates. Red-black trees permit more imbalance while still guaranteeing logarithmic height, which is a useful trade-off for workloads involving both lookup and updates.

Do not claim one is universally faster; workload and implementation matter.

What about `hashCode()` and `equals()`?

Another common follow-up is:

“If the bucket is a tree, does HashMap only use hashCode?”

No.

Conceptually:

```
hashCode()
    ↓
find bucket
    ↓
find candidate entries
    ↓
equals()
    ↓
identify the actual key
```

`hashCode()` narrows the search. It does not uniquely identify a key.

When tree bins need ordering/tie-breaking beyond hash values, the OpenJDK implementation has additional comparison logic. The important interview point is that **HashMap's correctness still depends on the `equals()`/`hashCode()` contract**.

Why can a bad hashCode be dangerous?

If unrelated keys produce the same hash:

```
A → hash 42
B → hash 42
C → hash 42
D → hash 42
```

they are forced into the same bucket.

A well-designed `hashCode()` distributes keys across buckets and reduces collisions.

But:

> Good hashing gives expected performance; it is not a correctness replacement for `equals()`.

What should you say about complexity?

Be precise:

```
Normal HashMap:
get/put/remove → expected O(1)

Treeified bucket:
search/update → O(log n) when hashes or useful comparisons distinguish keys
equal hashes without useful comparison ordering → lookup may still be O(n)

Resize:
O(n) work for the resize operation

Overall put:
expected amortized O(1)
```

Do **not** say:

> "After Java 8 HashMap is O(log n)."

The tree is a defensive mechanism for pathological collision behavior, not the normal complexity of the whole map.

Why can a balanced HashMap bin still need linear lookup?

A red-black tree has logarithmic height. That bounds search only when the query can choose a branch at each comparison. OpenJDK HashMap may search both subtrees for equal-hash keys without a useful comparison ordering. An absent non-comparable key whose hash matches every stored key can therefore require O(n) equality checks even in a treeified bin.

OpenJDK HashMap implementation notes and TreeNode search.

SDE-3 follow-up questions

Q: Why not keep the linked list?

> Because a heavily-collided bucket could degrade lookup toward O(n). A balanced tree can provide logarithmic lookup when hashes or useful key comparisons guide the search. Equal-hash, non-orderable keys can still require O(n) lookup.

Q: Why not treeify immediately?

> Trees have more structural and memory overhead. For small tables, resizing often distributes entries more cheaply.

Q: Why insert a red node?

> It avoids immediately increasing black height; any red-red violation can then be repaired.

Q: What are the two rotation types?

> Left rotation and right rotation.

Q: What are the two main balancing mechanisms?

> Recoloring and rotations.

Q: Does a treeified bucket make every HashMap lookup O(log n)?

> No. Only that bucket uses the tree structure; normal HashMap lookup remains expected O(1).

Q: Red-black tree vs AVL?

> Both are O(log n); AVL is more strictly balanced, while red-black generally trades some balance for fewer update rebalancing operations.

30-second interview answer

> “A red-black tree is a self-balancing BST that maintains color-based invariants so its height stays O(log n). HashMap uses it defensively when a particular bucket has too many collisions. Instead of searching a long linked collision chain, the bucket can be represented as a balanced tree. Lookup can be O(log n) when hashes or useful comparisons distinguish keys, but can remain O(n) for equal-hash keys without useful ordering. Recoloring and rotations keep the tree height logarithmic; they do not guarantee that every lookup follows only one branch. Importantly, only the heavily-collided bucket becomes a tree; HashMap as a whole still provides expected O(1) lookup with a good hash distribution.”

What you should know for SDE-3

Must know: purpose, five properties, rotations, recoloring, O(log n), relationship to HashMap.

Should know: treeification, untreeification, capacity threshold, why resizing may happen first, and red-black vs AVL.

Only if drilled further: individual insertion/deletion cases, exact OpenJDK `TreeNode` implementation details, and tie-breaking logic.

Multithreading Questions: foundations to senior follow-ups

Start here before M1.2–M1.6: why threads → creation → lifecycle → shared state → synchronization → coordination → executors → results → cancellation → collections/context → virtual threads → diagnosis. **Foundation:** explain the API and a small example. **Mid-level:** explain the mechanism and ownership. **Senior/SDE-3:** identify the invariant, capacity limit, failure path and a test that proves the required behavior. Explanations target Java 17/21; virtual-thread APIs require Java 21. These are conceptual walkthroughs and proposed exercises, not new executable examples or benchmarks.

T1 Why do we use multiple threads, and is concurrency the same as parallelism

Strong answer: a thread is an execution path within a process. Threads share process resources and can access the same heap objects, while each executes its own call stack. Concurrency means tasks can make progress over overlapping lifetimes; parallelism means work executes at the same time on multiple execution resources. One CPU core can support concurrency without parallel execution. Threads help overlap waiting and can divide CPU work, but scheduling, coordination and shared-state costs can outweigh the benefit.

Follow-up: an independent request is often a task; the task is not the worker thread. Reusing workers does not make request state safe to share. Creating more workers does not add cores or database connections.

Senior follow-up: first classify CPU work, blocking I/O and downstream capacity. Measure throughput, p99 latency and queueing against the sequential baseline. Do not promise a speedup merely because execution is concurrent.

T2 How do you create and start a thread in Java

Strong answer: describe work with `Runnable`, construct a `Thread` with that task and call `start()`. In Java 21, `Thread.ofPlatform()` and `Thread.ofVirtual()` provide builders. Calling `run()` directly on a platform Thread is an ordinary method call on the current thread; it does not schedule a new execution path. Directly calling run on a virtual Thread does not execute its task. A Thread can be started only once; starting it again throws `IllegalThreadStateException`, even after it has terminated. Executors usually separate task submission from thread creation in application code.

Follow-up: extending Thread couples work to the execution mechanism; implementing Runnable keeps work reusable. `Callable<T>` can return a result and throw checked exceptions, and is normally submitted to an executor rather than passed directly to a Thread constructor.

Senior follow-up: decide who owns completion, uncaught exceptions and cleanup. A daemon thread does not keep the JVM alive; it is not a reliable place for work that must finish. Do not rely on thread names or priorities to provide ordering or fairness.

T3 What are the thread states, and how do sleep, wait and join differ

Strong answer: Java exposes NEW, RUNNABLE, BLOCKED, WAITING, TIMED*WAITING and TERMINATED. RUNNABLE includes execution and readiness for execution; it does not prove a thread is consuming CPU. BLOCKED specifically concerns entering a monitor. Other waits, including explicit locks, can appear as WAITING or TIMED*WAITING. State is a diagnostic snapshot, not a synchronization protocol.

Follow-up: `sleep` pauses the current thread and does not release monitors. `Object.wait` requires ownership of that object's monitor, releases that monitor while waiting and reacquires it before returning; it does not release other held locks. `join` waits for another thread to terminate. A timed join can return before termination, so check completion.

Senior follow-up: sleep and yield create no shared-memory visibility guarantee. A successful termination observation through join provides an ordering/visibility boundary. Avoid sleeps as a way to “ensure the other thread has finished.”

T4 What is a race condition, and how do visibility and atomicity differ

Strong answer: a race condition occurs when correctness depends on an uncontrolled interleaving. A data race specifically involves conflicting accesses without the required happens-before relationship. Visibility concerns observing another thread's writes; atomicity concerns an operation being indivisible relative to other operations; ordering concerns which effects can be observed. `count++` reads, adds and writes, so two threads can overwrite each other's increments even if the field is volatile.

Follow-up: putting each method behind a lock is insufficient if the invariant spans two separately locked calls. “Check stock, then reserve” needs one coherent state transition or an atomic conditional operation.

Senior follow-up: identify the shared state and the exact invariant before choosing a primitive. Prefer immutable data or single ownership when possible. Continue with M1.3 for publication and the Java Memory Model; a test that happened to pass does not prove a data race absent.

T5 How does synchronized work, and when would you use ReentrantLock

Strong answer: synchronized acquires the specified object's monitor and releases it on normal or exceptional exit. An instance synchronized method locks that instance; a static synchronized method locks the class object. Different instances do not share an instance lock. Unlocking and subsequent locking of the same monitor establish visibility as well as mutual exclusion. Reentrant means the owner can reacquire the lock without blocking itself.

Follow-up: `ReentrantLock` provides explicit acquisition, including interruptible/timed forms, and Conditions. Release only a lock you acquired, in a finally block. A timed failed acquisition must not call unlock. Fairness can trade throughput for reduced acquisition variance and is not a universal scheduling guarantee.

Senior follow-up: keep blocking I/O and untrusted callbacks outside critical sections when the invariant permits it. Replacing a lock with a concurrent collection does not automatically preserve a multi-object invariant. M1.2 compares locks, atomics and volatile in more detail.

T6 How do threads coordinate work without polling

Strong answer: for a monitor protocol, check a condition in a while loop while holding the same monitor that protects the state. `wait` can return spuriously; notification means recheck, not that the condition is certainly true. The notifier changes protected state before notifying and does not release the monitor merely by calling notify. Higher-level `BlockingQueue` operations often express producer/consumer coordination more safely.

Follow-up: `notify` selects one waiter without guaranteeing it is the right condition's waiter; `notifyAll` wakes all waiters to compete and recheck. Multiple Conditions can separate wait sets. A bounded queue provides a capacity policy; timed offer/poll lets the operation respect a deadline.

Senior follow-up: distinguish an empty queue from completed production. Design explicit completion, failure and cancellation signals. A poison pill must account for all consumers and must not get permanently stuck behind a full queue during shutdown. Continue with the odd/even exercise in Part 4 after learning the guarded-condition protocol.

T7 When do volatile, atomic classes and LongAdder fit

Strong answer: volatile supports visibility/order for that variable, but not an arbitrary compound state transition. Atomic classes provide operations such as compare-and-set, increment and update on their represented state. CAS checks the expected value and conditionally changes it atomically; retry logic may execute its computation more than once, so keep such functions free from non-repeatable side effects. LongAdder reduces contention for accumulating counters, but `sum()` is not an atomic snapshot under concurrent updates.

Follow-up: use a volatile stop flag only when work will check it and can make progress; it cannot wake every blocked operation. Use an atomic counter for an exact conditional counter protocol and a lock or immutable aggregate for invariants spanning fields. LongAdder is useful for statistics, not an exact admission check.

Senior follow-up: ask whether A→B→A matters, whether retry can starve, and whether the algorithm is actually simpler than a lock. Q161 covers stamped references and ABA; Q163 discusses false sharing. Do not equate lock-free with wait-free or with guaranteed better performance.

T8 What are Executor, ExecutorService and a thread pool

Strong answer: Executor accepts Runnable work and abstracts execution; it need not create a new thread or execute asynchronously. ExecutorService adds submission/results and lifecycle. ThreadPoolExecutor manages workers, a work queue and rejection behavior. For its usual configuration, a new task creates a worker below the core size, then is queued, then may create a worker up to the maximum when queueing fails; otherwise it is rejected. An unbounded queue can prevent normal growth toward the maximum.

Follow-up: fixed thread pools bound workers but ordinarily use an unbounded queue; cached pools can create many threads. Configure capacity deliberately. CallerRunsPolicy can slow the submitter when saturated, but runs work on the submitting thread and discards rejected work after shutdown; it is not universally safe backpressure. AbortPolicy makes rejection visible to the caller.

Senior follow-up: waiting tasks retain memory and consume deadline budget. Separate workloads where justified, bound admission and instrument queue age, active workers, rejections and downstream saturation. M1.5 covers sizing; Q148 explains a maximum-size trap. Never block a worker waiting for child work queued behind it in the same saturated pool.

T9 How do Runnable, Callable, execute, submit and Future fit together

Strong answer: Runnable has no returned result; Callable returns a value and may throw checked exceptions. `execute` accepts a Runnable without returning a Future. `submit` wraps work and returns a Future: get waits for completion and exposes execution failure through ExecutionException. With ordinary ThreadPoolExecutor submission, the wrapper captures task exceptions, so an uncaught-exception handler alone will not report every submitted failure. Observe the Future or another deliberate error channel.

Follow-up: timed get bounds how long the caller waits; a timeout does not automatically stop the task. Future cancellation is a request governed by the implementation and task cooperation. An already completed task cannot be retroactively cancelled.

Senior follow-up: define who owns every submitted result, how abandoned work is cancelled and how failures are aggregated. Submission establishes publication to the task, and successful Future.get establishes result visibility. Avoid fire-and-forget operations that can silently lose failures or outlive their owner.

T10 How does CompletableFuture compose work, and which thread runs a stage

Strong answer: CompletableFuture represents a completion and supports dependent stages. `thenApply` transforms a value; `thenCompose` flattens a function returning another stage; `thenCombine` combines independently obtained results. Non-async callbacks may run on a completing thread or another caller of completion methods. Async variants use their supplied executor or the documented default facility, commonly the common pool. Do not assume every stage has its own thread.

Follow-up: `allOf` completes after all supplied futures complete but does not collect typed results or automatically cancel siblings on failure. `handle` can transform success or failure; `exceptionally` recovers from failure; `whenComplete` observes completion, but a throwing observer can itself affect the dependent result.

Senior follow-up: distinguish a future's timeout/cancellation from cancellation of the actual HTTP/JDBC operation. `CompletableFuture.cancel(true)` does not use the flag to interrupt its computation. Configure underlying I/O deadlines and explicit cancellation support. Use a deliberately owned executor for blocking operations; Q147 explores flattening and M1.6 compares composition with virtual threads.

T11 How do interruption, cancellation and shutdown work

Strong answer: interrupt is a cooperative signal, not a forced stop. Interruptible waits such as sleep/wait/join throw InterruptedException and clear the interrupt status. Either propagate it or, when handling it at a boundary, restore the status and exit or hand control to an owner that will handle it. `Thread.interrupted()` tests and clears the current thread's status; `isInterrupted()` does not clear it. Do not use deprecated stop/suspend/resume APIs for lifecycle control.

Follow-up: shutdown rejects new executor tasks but lets accepted work finish. shutdownNow attempts interruption, returns queued tasks that never started, and does not guarantee termination. Await termination with a bounded budget and report any survivors. Account for returned submitted-task wrappers: their Futures may still need explicit cancellation/completion handling.

Senior follow-up: Java 21 ExecutorService.close waits for termination; try-with-resources is not a bounded shutdown policy for a stuck task. Cancellation of a caller's Future is not proof that a side effect did not happen. Stop admitting work, propagate remaining deadlines, release resources only when ownership ends, and reconcile ambiguous external outcomes.

T12 Which concurrent collections and coordination tools should you know

Strong answer: ConcurrentHashMap provides documented atomic map operations, not thread safety for an arbitrary mutable value inside it. BlockingQueue coordinates handoff with optional bounded capacity. CopyOnWriteArrayList favors frequent traversal and rare writes, with snapshot iterators. CountDownLatch is one-shot completion/starting coordination; CyclicBarrier coordinates a fixed group repeatedly; Phaser supports phased coordination with changing parties; Semaphore bounds permits to a resource.

Follow-up: release a semaphore permit only after successful acquisition, exactly once, when the represented resource is no longer owned. A timeout before acquisition is not a reason to release. A timeout at the caller can occur while the underlying work still uses capacity.

Senior follow-up: choose collection and primitive from the invariant, not their “concurrent” name. Q145 covers mutable map values and Q146 covers slow computeIfAbsent callbacks. Barrier failure, missing countdown and broken ownership can prevent progress; include failure paths in the protocol.

T13 Why does ThreadLocal need cleanup and deliberate propagation

Strong answer: ThreadLocal stores per-thread state. Pooled workers can outlive requests, so values can leak into the next task or retain request objects unless removed in finally at the owning boundary. InheritableThreadLocal copies context at thread creation, not at each submission to an already existing worker. Neither automatically propagates the right request context into every asynchronous stage.

Follow-up: explicitly capture and install required logging/tenant/security context, then restore or clear the prior worker state on exit. Do not carry a mutable EntityManager or assume a Spring transaction moves with a ThreadLocal copy.

Senior follow-up: test two unrelated requests on the same worker, exceptions and nested context installation. Virtual threads still need context ownership, and storing an expensive object per virtual thread can consume large aggregate memory. Q8 in Part 2 develops the cleanup discussion.

T14 When do virtual threads and structured concurrency help

Strong answer: Java 21 virtual threads make many blocking tasks economical; they do not make CPU work faster or increase a database's capacity. Use a virtual thread per task and limit admission/scarce resources separately. Do not build a worker pool to reuse virtual threads. Their daemon status means essential work needs an explicit lifetime owner.

Follow-up: Java 21–23 monitor-related blocking can pin a carrier; JDK 24 removes that monitor pinning. Native/foreign calls still deserve analysis. Check the deployed JDK before mechanically replacing synchronized. Structured concurrency groups child work under a lifetime owner, but the Java 21 API is preview and requires matching build/runtime preview configuration; its shape changes across releases.

Senior follow-up: define child failure, cancellation, deadlines and partial-result policy before choosing an API. Avoid copying a preview example across JDK versions. M1.6 discusses the virtual-thread/composition choice;

Part 11 explains version-sensitive pinning.

T15 How do you distinguish deadlock, livelock, starvation and overload

Strong answer: deadlock is a dependency cycle with no progress path; livelock repeatedly reacts without useful progress; starvation denies a task needed resources; overload admits work faster than useful capacity can finish it. A waiting thread alone does not identify which problem exists. Follow locks, queued child tasks, permits, connections and remote dependencies to the actual cycle or bottleneck.

Follow-up: keep acquisition order consistent and shorten critical sections. Timed acquisition needs a bounded retry policy and a recovery path; adding retries can otherwise create livelock. Thread dumps, JFR, CPU profiles, queue metrics and dependency latency should tell one coherent story.

Senior follow-up: Java 21 ThreadMXBean does not monitor virtual threads; a null deadlock result cannot rule out their deadlock. Use tools that cover the actual threads and trace application resource dependencies. Continue with M1.4 and Q162–Q163 only after this distinction is clear.

T16 How should you test a concurrency contract

Strong answer: assert an invariant after a controlled interleaving. Use latches/barriers or deterministic fakes to hold tasks at the critical boundary, release them deliberately and await completion with a bounded deadline. Do not assert one scheduler-dependent ordering or rely on sleep. A bounded test timeout is a failure guard, not proof that work was cancelled.

Follow-up: cover success, no work, duplicate completion, failure before acquisition, cancellation after acquisition, queue rejection, stale ownership and shutdown while work is in flight. Ensure the test releases its own gates and terminates its executors even after an assertion fails.

Senior follow-up: separate correctness tests from throughput/latency benchmarks and load tests. Repetition can reveal a race but does not prove its absence. Use appropriate stress tools for memory-model claims, representative load for capacity, and JMH for microbenchmarks. Interview answer: state the invariant → name its owner → explain the happens-before edge → define failure/cancellation → show the test and production signal.

Primary references: Java 21 Thread, Object wait/notify, concurrent package contracts, ExecutorService, ThreadPoolExecutor, CompletableFuture, ReentrantLock, and JDK 24 virtual-thread behavior.

Multithreading deep dives after the foundations

The retained M1.2–M1.6 answers below deepen synchronization, memory ordering, deadlocks, pool sizing and async/virtual-thread choices. Then try Q145–Q148 in Part 10, Q161–Q163 in Part 12 and Q183–Q184 in Part 13. Keep the existing question identifiers for revision; use the ladder above as the learning order.

M1.2 `volatile` vs `synchronized` vs atomics vs `ReentrantLock`?

A good SDE-3 interview answer is:

`volatile` provides visibility and ordering between threads, but it does **not** make compound operations such as `count++` atomic. `synchronized` provides both mutual exclusion and the memory-visibility guarantees associated with monitor locking. `ReentrantLock` provides explicit locking with features such as `tryLock`, timed/interruptible acquisition, fairness and multiple `Conditions`. `AtomicInteger`/`AtomicLong` provide atomic single-variable operations using CAS, while `LongAdder` is optimized for high-contention counters where an exact instantaneous snapshot is not required.

Think of the choices like this

```
Need only visibility?
        ↓
     volatile

Need one shared value updated atomically?
        ↓
   AtomicInteger / AtomicLong

Need very high-throughput counter under contention?
        ↓
     LongAdder

Need mutual exclusion around multiple operations/state?
        ↓
 synchronized / ReentrantLock
```

Why isn't `volatile` enough for `count++`?

This:

```
volatile int count;
count++;
```

looks like one operation, but conceptually it is:

```
read count
   ↓
add 1
   ↓
write count
```

Two threads can read the same old value and overwrite each other's update.

So `volatile` is appropriate for things such as:

```
volatile boolean shutdown;
```

where one thread publishes a new value and other threads need to see it.

`synchronized` vs `ReentrantLock`

`synchronized` is usually the simplest choice:

```
synchronized (lock) {
    // critical section
}
```

Use `ReentrantLock` when you genuinely need capabilities such as:

- `tryLock()`
- timed acquisition
- interruptible lock acquisition
- fairness policy
- multiple `Conditions`

With `ReentrantLock`, ownership is manual, so always unlock in `finally`.

AtomicInteger vs LongAdder

`AtomicInteger` is useful when you need an exact atomic value update such as:

```
counter.incrementAndGet();
```

`LongAdder` spreads contention across multiple internal cells. Under heavy concurrent writes, this can provide better throughput than repeatedly updating one atomic variable.

The trade-off is that `sum()` is not an instantaneous transactionally consistent snapshot.

SDE-3 decision rule

Start with the **simplest primitive that gives the required correctness**.

Do not choose a sophisticated concurrency primitive because it sounds faster. First establish:

1. What state is shared?
2. Is the operation single-variable or multi-variable?
3. Do we need atomicity, visibility, ordering, or all three?
4. Is contention actually high?
5. Do we need a consistent snapshot?

Then measure before optimizing.

30-second version

> "`volatile` gives visibility but not atomicity. `synchronized` gives mutual exclusion plus visibility and is my default when I need to protect a critical section. `ReentrantLock` is useful when I need advanced lock features such as `tryLock` or interruptible acquisition. Atomics are good for atomic single-variable operations, while `LongAdder` is useful for highly contended counters. I choose the simplest primitive that satisfies the correctness requirement and optimize only after measuring contention."

M1.3 Explain the Java Memory Model and happens-before.

A good SDE-3 interview answer is:

The **Java Memory Model (JMM)** defines the rules for how threads interact through memory. The key idea is **happens-before**: if action A happens-before action B, the Java memory model guarantees that B can observe the relevant effects of A and constrains reordering between them.

Important happens-before relationships include:

```
monitor unlock  → subsequent lock of the same monitor
volatile write  → subsequent read of that volatile variable
Thread.start()  → actions in the started thread
actions in a thread → successful Thread.join() return
```

There are also special initialization guarantees for properly constructed objects with `final` fields, but those are not a general replacement for safe publication.

Why does this matter?

Without the required synchronization, two threads can have a data race. The compiler, JIT and CPU are allowed to reorder operations as permitted by the JMM, and another thread is not automatically guaranteed to observe ordinary writes in the order you expected.

The common example is double-checked locking:

```
class Singleton {
    private static Singleton instance;

    static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Without `volatile` on `instance`, this pattern is not safely published.

The safer form is:

```
private static volatile Singleton instance;
```

because the volatile write that publishes the reference establishes the required visibility/order relationship for readers.

Happens-before is not "execution order"

This is a common interview trap.

Happens-before does **not** simply mean:

> "This line always executes before that line."

It is a memory-model guarantee about visibility and ordering between actions in different threads.

SDE-3 takeaway

When debugging concurrency, ask:

```
What state is shared?
        ↓
What synchronization establishes visibility?
        ↓
What establishes atomicity?
        ↓
Is there a happens-before relationship?
        ↓
Could the code have a data race?
```

Do not solve a visibility problem with a random lock, and do not assume `volatile` makes a multi-step operation atomic.

30-second version

> "The Java Memory Model defines what visibility and ordering guarantees exist between threads. Happens-before is the key relationship: for example, an unlock happens-before a subsequent lock on the same monitor, and a volatile write happens-before a subsequent volatile read of that variable. Without an appropriate happens-before relationship, ordinary shared mutable state can have visibility and ordering problems. A classic example is double-checked locking, where the instance reference must be volatile for safe publication."

M1.4 How do you detect, prevent and fix deadlocks?

A good SDE-3 interview answer is:

For a resource deadlock, look for mutual exclusion, hold-and-wait, no preemption and circular wait. Prevent cycles with a global lock order, short critical sections and careful resource ownership. Avoid calling foreign code while holding a lock. Timed `tryLock` with a bounded retry policy, or message passing, can help when appropriate.

For platform-thread monitor/ownable-synchronizer deadlocks, inspect `jstack`, `jcmd <pid> Thread.print` and `ThreadMXBean.findDeadlockedThreads()`. Java 21's `ThreadMXBean` does not monitor virtual threads: a null result does not rule out their deadlocks. Use `jcmd <pid> Thread.dump_to_file -format=json <file>` or its text form to inspect virtual-thread stacks, alongside JFR and application state. These dumps do not provide the same lock/ownership information as traditional dumps and are not an automatic virtual-thread deadlock detector. Trace the actual wait/resource dependency cycle.

How to understand it

Worked explanation: Imagine transfer A locks account 1 then waits for account 2, while transfer B holds 2 and waits for 1. Neither can release its first lock because each waits inside the operation. Sorting account IDs before acquiring both locks removes that cycle.

Key terms you should know

- **Deadlock** — Tasks cannot progress because they wait on a cyclic set of dependencies.
- **Livelock** — Tasks react repeatedly without making useful progress.

- **Starvation** — A task repeatedly loses access to needed resources.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a timeout fix correctness? Only if the operation releases owned resources and leaves a valid state; blind retries can recreate the cycle.

30-second version

> Prevent deadlocks with consistent resource order and short critical sections. Traditional deadlock detection covers platform-thread monitors and ownable synchronizers. For Java 21 virtual threads, inspect thread dumps and actual wait dependencies; ThreadMXBean cannot rule out a deadlock.

Java 21 ThreadMXBean, virtual-thread diagnostics.

M1.5 How do you size and configure a thread pool?

A good SDE-3 interview answer is:

For ThreadPoolExecutor, submissions normally create workers up to core size, then enter the queue; if queueing fails, workers may grow to maximum size, then rejection applies. Size CPU work around effective available CPU and waiting-heavy work from measured wait/compute time and downstream limits. Use bounded admission and an explicit rejection contract. CallerRunsPolicy can slow producers but blocks the submitting thread and discards rejected tasks after shutdown. Monitor queue delay, active work and rejection; shut down with a deadline and define treatment of unfinished tasks. Java 21 executor contract.

How to understand it

Worked explanation: If tasks arrive faster than workers finish them, the queue stores delay and memory. With an unbounded queue, a ThreadPoolExecutor normally queues after reaching its core size, so raising maximumPoolSize may do nothing.

Key terms you should know

- **back-pressure** — A mechanism that prevents an upstream producer from overwhelming a downstream consumer.
- **bulkhead** — A resilience pattern that isolates resources such as threads or connections so one overloaded dependency does not exhaust the whole service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What should happen when full? Choose explicit rejection, shedding or bounded caller participation; CallerRunsPolicy can block a request thread and discards tasks after shutdown.

30-second version

> A pool controls workers, queueing and rejection together. An unbounded queue can prevent growth beyond core size. Bound admission, measure queue delay and downstream capacity, and define shutdown and rejection behavior.

M1.6 CompletableFuture vs virtual threads: when do you use which?

A good SDE-3 interview answer is:

Virtual threads are final in Java 21 and fit workloads that spend much of their time waiting. They do not accelerate CPU-bound work or enlarge a database connection pool. Use one virtual thread per task and separately bound admission and scarce resources. On JDK 21–23, blocking while holding `synchronized` monitors can pin carriers; JDK 24 removes this monitor-related pinning, so do not mechanically replace locks on newer JDKs. Native/foreign calls still deserve attention. `CompletableFuture` models composition; configure actual I/O deadlines and cancellation as well as future timeouts. Virtual threads in JDK 24.

How to understand it

Worked explanation: For three independent remote reads, futures can describe the result-combination graph; virtual threads let each read use ordinary blocking code cheaply. Both still wait for the same remote services and share their limits.

Key terms you should know

- **CompletableFuture** — A Java API for representing and composing asynchronous results and their success/failure stages.
- **virtual thread** — A lightweight Java thread managed by the JVM, designed to make high-concurrency blocking I/O easier to scale.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a future timeout stop the remote call? Not necessarily; configure the client's deadline and the cancellation path as well.

30-second version

> Virtual threads are final in Java 21 and fit workloads that spend much of their time waiting. They do not accelerate CPU-bound work or enlarge a database connection pool.

JVM foundations before garbage-collector choices

J1 What do the JDK, JVM, heap and thread stacks do

Strong answer: the JDK provides development tools and a runtime; the JVM executes bytecode through interpretation and compilation while managing memory and other runtime services. Heap objects can be shared by threads; thread stacks hold frames, local values and references for their execution paths. A stack-held reference can keep a heap object reachable. Garbage collection reclaims unreachable heap objects, not objects that an application merely “finished using.”

Follow-up: a growing cache, listener registration or ThreadLocal can retain reachable objects. Closing a socket/database resource is a lifecycle responsibility rather than a promise of prompt garbage collection. Stack exhaustion and heap exhaustion have different causes; increasing one limit does not fix both.

Senior follow-up: establish allocation rate, live retained data and pause/CPU cost before choosing GC flags. Use logs/JFR and retention evidence rather than calling every increase in heap occupancy a leak. Continue with M1.7 and Part 11's JVM walkthroughs.

M1.7 Explain garbage collection and how you tune it.

A good SDE-3 interview answer is:

GC reclaims unreachable objects; the layout and collection process depend on the collector and JDK. G1 uses regions and generational collection; young collections and mixed collections do different work. ZGC and Shenandoah use concurrent techniques to reduce pauses, while Parallel GC targets throughput. Define latency, throughput and memory objectives, then inspect GC logs, allocation rate and live-set retention before tuning. A pause target is a goal rather than a hard deadline, and reducing allocation helps only if it is a measured bottleneck.

How to understand it

Worked explanation: Allocation creates objects; reachability determines which can be reclaimed. A growing heap before GC can be normal, whereas an increasing live set after collections suggests retention. Compare these before changing collector settings.

Key terms you should know

- **G1** — A region-based garbage collector with a pause-time target.
- **GC** — Reclaiming heap storage no longer needed by reachable objects.
- **Tuning goal** — The workload objective, such as throughput or tail latency.
- **Heap** — JVM storage used for object allocation.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Will a larger heap fix a leak? It can delay failure while retaining the same unwanted objects; inspect retaining paths and measured pauses.

30-second version

> Start from reachability and the chosen collector. Measure allocation, retained live data and pauses, then fix the dominant cause. Avoid treating every collector as the same young/old layout or a pause target as a guarantee.

Module 2: Spring & Spring Boot

More in Q11–Q19, Q37–Q42.

Start here: Spring annotation foundations and senior follow-ups. Learn bean registration → injection/lifecycle → configuration → proxies → web/data integration before the auto-configuration and transaction questions below.

M2.1 How does Spring Boot auto-configuration work?

A good SDE-3 interview answer is:

`@SpringBootApplication` combines `@Configuration`, `@ComponentScan` and `@EnableAutoConfiguration`. The latter loads candidate classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from every jar on the classpath, and each is guarded by conditions (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`). Many auto-configurations back off when a matching user bean exists; check the actual conditions and bean types rather than assuming every default is overridable this way. To debug, run with `--debug` or use the Actuator `conditions` endpoint. To build your own starter: an autoconfigure module plus a thin starter POM and `@ConfigurationProperties` for settings.

How to understand it

Worked explanation: Adding a JDBC driver and connection settings can make Boot register database infrastructure when its conditions match. Providing a matching custom bean can make that particular default back off. Component scanning and conditional auto-configuration are separate discovery paths.

Key terms you should know

- **bean** — An object managed by the Spring IoC container.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why is a bean missing? Read the condition report for the unmet class, property or bean condition before adding exclusions.

30-second version

> `@SpringBootApplication` combines `@Configuration`, `@ComponentScan` and `@EnableAutoConfiguration`. The latter loads candidate classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from every jar on the classpath, and each is guarded by conditions (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`).

M2.2 Describe the bean lifecycle and scopes.

A good SDE-3 interview answer is:

Spring creates a bean, populates dependencies, invokes applicable awareness and initialization callbacks, and exposes the processed instance, often a proxy. Annotation processors invoke @PostConstruct during before-initialization processing; InitializingBean and custom init callbacks then participate in the configured lifecycle. Managed singleton destruction can invoke @PreDestroy; prototype cleanup is the caller's responsibility. Singleton means one instance per bean definition per container, not automatic thread safety. Injecting a prototype once into a singleton gives that injection one instance; use a provider for repeated retrieval. Spring lifecycle, scopes.

How to understand it

Worked explanation: The container creates an object, supplies dependencies, initializes it and exposes the resulting bean, potentially through a proxy. A singleton consumer receives a prototype once during injection; the prototype scope does not turn that field into a factory.

Key terms you should know

- **AOP** — Aspect-oriented programming: applying cross-cutting behavior such as transactions, logging or security around method execution.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you obtain a fresh prototype per operation? Ask an ObjectProvider; also define cleanup because Spring does not automatically destroy prototype instances.

30-second version

> Explain construction, dependency population, initialization, post-processing and managed destruction. Singleton scope is container-local and does not imply thread safety. Prototype retrieval and cleanup need deliberate ownership.

M2.3 Why prefer constructor injection?

A good SDE-3 interview answer is:

Dependencies are explicit and `final`, the object is never half-built, it can be unit-tested with `new` and mocks, and an excess of constructor parameters exposes a class doing too much (Single Responsibility). Field injection hides dependencies, needs reflection in tests, and encourages cycles. Use setter injection only for genuinely optional dependencies.

How to understand it

Worked explanation: An OrderService constructed with a PaymentGateway makes the dependency visible at its creation point. A test can supply a fake gateway directly. Missing dependencies become construction failures instead of surprising null fields later.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does final make the dependency thread-safe? No; it prevents reassignment of the reference, not mutation inside the referenced gateway.

30-second version

> Dependencies are explicit and `final`, the object is never half-built, it can be unit-tested with `new` and mocks, and an excess of constructor parameters exposes a class doing too much (Single Responsibility). Field injection hides dependencies, needs reflection in tests, and encourages cycles.

M2.4 How does `@Transactional` work and what are its pitfalls?

A good SDE-3 interview answer is:

A call through a Spring transaction proxy is intercepted by the configured transaction manager. With ordinary JDBC/JPA, work usually participates in a thread-bound transaction. The default rollback rule covers unchecked exceptions and errors; configure checked-exception handling deliberately. Self-invocation bypasses proxy advice. Spring 6 class-based proxies can also intercept eligible protected/package-visible methods; interface proxies require public interface methods. Swallowing an exception does not clear rollback-only state. `REQUIRES_NEW` needs independent resources; `NESTED` depends on savepoint support. An async task does not inherit its caller's transaction, but can start its own by calling a transactional service through its proxy. Keep transaction scope short; `readOnly=true` is not portable write prevention. Spring transaction annotations, transaction implementation.

How to understand it

Worked explanation: Trace an external call through the proxy: begin or join a transaction, invoke the target, then commit or roll back. A target calling its own method skips that proxy boundary, so a different annotation on the inner method may have no effect.

Key terms you should know

- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.
- **Application service** — A component coordinating a business use case; it may be a Spring-managed bean and is not a Kubernetes Service.
- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if an inner call marks rollback-only and the caller catches its exception? The transaction may still fail at commit; catching does not repair it.

30-second version

> A call through a Spring transaction proxy is intercepted by the configured transaction manager. With ordinary JDBC/JPA, work usually participates in a thread-bound transaction.

M2.5 How do you secure a REST API with JWT?

A good SDE-3 interview answer is:

Configure a stateless `SecurityFilterChain` with OAuth2 resource-server JWT support. Set `spring.security.oauth2.resourceserver.jwt.issuer-uri` to the trusted issuer, or configure an equivalent issuer validator on a custom decoder. An explicit `jwk-set-uri` supplies verification keys; that URL alone does not validate the token's `iss` claim. Validate the signature with allowed algorithms, `exp`/`nbf`, issuer and audience. Use Boot's `audiences` property where supported, or an explicit audience validator.

Map scopes/roles using `JwtAuthenticationConverter`; enable method security with `@EnableMethodSecurity` and enforce domain authorization with `@PreAuthorize`. Keep access tokens short-lived, rotate refresh tokens at the authorization server, store passwords with an appropriate adaptive hash where the application owns credentials, and never log tokens. Return 401 for invalid/missing authentication and 403 for denied authorization.

How to understand it

Worked explanation: For an orders request, first verify the token's signature and trusted claims, then map its authorities, then check access to the requested order. A valid token proves neither ownership of every order nor permission for every action.

Key terms you should know

- **JWKS** — A set of keys used here to verify JWT signatures.
- **Issuer (`iss`)** — The token issuer this API explicitly trusts.
- **Audience (`aud`)** — The intended recipient or recipients of a token.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why check audience? A token valid for a different API must not automatically authorize this API.

30-second version

> A JWT resource server needs signature, time, issuer and audience validation. Configure issuer-uri or an explicit issuer validator; a JWKS URL alone only supplies keys. Map authorities, enable method security, and test wrong-issuer and wrong-audience tokens even when their signatures are valid.

Spring Security JWT configuration.

M2.6 What does "production-ready" mean for a Boot service?

A good SDE-3 interview answer is:

Observability (Actuator health/readiness/liveness, Micrometer metrics to Prometheus/Cloud Monitoring, structured logs with trace IDs, OpenTelemetry), resilience (timeouts, circuit breakers, bounded pools, graceful shutdown), configuration (externalised, validated `@ConfigurationProperties`, secrets from a vault), data safety (Flyway migrations, `ddl-auto=validate`), security (HTTPS, headers, dependency scanning), plus tests, CI/CD and runbooks with SLOs and alerts.

How to understand it

Worked explanation: Take one payment request and ask how you detect failure, limit its duration, protect its data and recover its outcome. Each production control should answer one of those operational questions rather than merely add a library.

Key terms you should know

- **observability** — The ability to understand internal system behavior from telemetry such as logs, metrics and traces.
- **trace** — A representation of one request's path through distributed services.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What proves readiness? A tested deployment and recovery path with observable user outcomes, not simply an Actuator endpoint returning UP.

30-second version

> Observability (Actuator health/readiness/liveness, Micrometer metrics to Prometheus/Cloud Monitoring, structured logs with trace IDs, OpenTelemetry), resilience (timeouts, circuit breakers, bounded pools, graceful shutdown), configuration (externalised, validated `@ConfigurationProperties`, secrets from a vault), data safety (Flyway migrations, `ddl-auto=validate`), security (HTTPS, headers, dependency scanning), plus tests, CI/CD and runbooks with SLOs and alerts.

Module 3: Hibernate / JPA

More in Q20–Q25.

Start here: database connections and session management. Learn JDBC/Boot connection setup → pool → Session/EntityManager and factories → transaction manager → entity lifecycle → mappings before N+1 or inheritance strategies.

M3.1 What is the N+1 problem and how do you fix it?

A good SDE-3 interview answer is:

One query loads N parents and then lazily triggers one query per parent for a child association, giving N+1 round-trips that look fine in dev and collapse under production data. Detect it with SQL logging, Hibernate statistics, or tests asserting a query count (datasource-proxy). Fixes: `JOIN FETCH` or `@EntityGraph` for the use case, `@BatchSize` / `hibernate.default_batch_fetch_size` to load children in batches of IN-lists, or DTO projections selecting only needed columns. Caveats: joining multiple to-many associations can multiply rows; fetching multiple bag mappings can additionally trigger `MultipleBagFetchException`, and collection fetch with pagination is applied in memory, so paginate parent IDs first and fetch children second.

How to understand it

Worked explanation: Loading 50 orders in one query and then touching each customer's proxy can cause 50 additional selects. A use-case fetch plan loads the required relationship deliberately, reducing round trips without making every association globally eager.

Key terms you should know

- **N+1** — A query pattern where one query loads N parent rows and then N additional queries load related data, causing excessive database round trips.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why not fetch every collection? Joined rows multiply, and collection pagination may require a separate parent-ID page.

30-second version

> One query loads N parents and then lazily triggers one query per parent for a child association, giving N+1 round-trips that look fine in dev and collapse under production data. Detect it with SQL logging, Hibernate statistics, or tests asserting a query count (datasource-proxy).

M3.2 Explain entity states and the persistence context.

A good SDE-3 interview answer is:

An entity can be new/transient, managed, detached or removed. A persistence context tracks entity identity and changes; modifying a managed entity can produce SQL at flush without another save call. Detachment can follow close, clear or explicit detach. Flush synchronizes changes without committing; AUTO flushing depends on transaction and query semantics and does not mean every query always flushes. Bound context size during bulk work and flush deliberately before clearing pending changes.

How to understand it

Worked explanation: Load an order inside a persistence context, change its status and flush: Hibernate tracks the managed instance and synchronizes the change. Closing the context detaches it; changing that detached object alone no longer schedules an update.

Key terms you should know

- **dirty checking** — Hibernate/JPA's mechanism for detecting changes to managed entities and generating updates during flush.
- **persistence context** — JPA/Hibernate's managed set of entity instances whose changes are tracked and synchronized with the database.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is flush a commit? No; SQL can run and still be rolled back by the surrounding transaction.

30-second version

> The persistence context supplies identity and change tracking. Managed changes reach SQL at flush, while detached changes do not automatically persist. Flush and commit differ, and clearing can discard unflushed work.

M3.3 Lazy vs eager, and `LazyInitializationException`?

A good SDE-3 interview answer is:

Make associations LAZY by default; JPA's defaults for `@ManyToOne`/`@OneToOne` are EAGER, so set `fetch = LAZY` explicitly. The exception occurs when a proxy is touched after the session closed. Open-Session-In-View keeps the persistence context available through web rendering and can hide uncontrolled queries; connection retention

depends on acquisition/release settings. For service APIs, consider disabling `spring.jpa.open-in-view` and defining explicit fetch boundaries. The right fixes: fetch what you need in the service layer (`EntityGraph`/join fetch) or return DTOs.

How to understand it

Worked explanation: A service returns an order whose lines were never loaded. Serialization later touches the lines after the context closed and fails. Build the required DTO while the deliberate fetch boundary is still active.

Key terms you should know

- **persistence context** — JPA/Hibernate's managed set of entity instances whose changes are tracked and synchronized with the database.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.
- **Application service** — A component coordinating a business use case; it may be a Spring-managed bean and is not a Kubernetes Service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does EAGER guarantee one query? No; it specifies fetching semantics, and providers can issue additional selects.

30-second version

> Make associations LAZY by default; JPA's defaults for `@ManyToOne`/`@OneToOne` are EAGER, so set `fetch = LAZY` explicitly. The exception occurs when a proxy is touched after the session closed.

M3.4 Optimistic vs pessimistic locking?

A good SDE-3 interview answer is:

Optimistic uses a `@Version` column: `UPDATE … WHERE id=? AND version=?`; zero rows updated throws `OptimisticLockException`, which you retry or surface as 409. It avoids a pessimistic read lock; the eventual UPDATE still takes database locks until transaction completion. Pessimistic (`PESSIMISTIC_WRITE` → `SELECT … FOR UPDATE`) blocks competing writers, suitable for high contention or scarce resources like inventory; always set lock timeouts and keep transactions short to avoid deadlocks (and use `SKIP LOCKED` for queue-like tables).

How to understand it

Worked explanation: Two transactions read version 4. The first updates with version=4 and advances it; the second update matches no row and detects stale state. A pessimistic strategy instead acquires a database lock before the conflicting work.

Key terms you should know

- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What should be retried? Re-read and re-evaluate the business operation in a new transaction; never blindly replay an external charge.

30-second version

> Optimistic uses a `@Version` column: `UPDATE … WHERE id=? AND version=?`; zero rows updated throws `OptimisticLockException`, which you retry or surface as 409.

M3.5 How do you tune Hibernate performance?

A good SDE-3 interview answer is:

Measure first (slow-query log, `pg_stat_statements`, p6spy). Then: fix N+1; project to DTOs for read paths; add proper indexes (check `EXPLAIN`); enable JDBC batching (`hibernate.jdbc.batch_size`, `order_inserts/updates`); use SEQUENCE generators with pooled optimisers because IDENTITY disables insert batching; use second-level/query caches only for read-mostly data; paginate with keyset; avoid huge `IN` lists; keep transactions short; and use Flyway to manage schema and index changes.

How to understand it

Worked explanation: Separate time spent waiting for a connection, executing SQL, transferring rows and materializing objects. An ORM setting cannot repair a missing index or a transaction holding connections during a slow HTTP call.

Key terms you should know

- **N+1** — A query pattern where one query loads N parent rows and then N additional queries load related data, causing excessive database round trips.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you prove improvement? Compare query count, plans and latency on representative data with the same result contract.

30-second version

> Measure first (slow-query log, `pg_stat_statements`, p6spy). Then: fix N+1; project to DTOs for read paths; add proper indexes (check `EXPLAIN`); enable JDBC batching (`hibernate.jdbc.batch_size`, `order_inserts/updates`); use SEQUENCE generators with pooled optimisers because IDENTITY disables insert batching; use second-level/query caches only for read-mostly data; paginate with keyset; avoid huge `IN` lists; keep transactions short; and use Flyway to manage schema and index changes.

Module 4: Microservices

More in Q26–Q30, Q59–Q74.

F1 What is a microservice and how does one request cross services

Strong answer: a microservice is an independently deployable service around a coherent business capability with explicit API and data ownership. A client/request enters an API, invokes local business logic and may call another service synchronously or publish an event. Crossing a service boundary introduces network delay, partial failure and separate deployment/lifecycle ownership. A modular monolith may satisfy the need with less operational cost.

Follow-up: HTTP/gRPC offers a request/result interaction; messaging separates sending from later processing. Neither makes a multi-service operation automatically atomic. Define contracts, deadlines and who owns retry/idempotency before selecting a transport.

Senior follow-up: start with one concrete user operation and trace what happens when a downstream call times out after accepting the request. Avoid retries that duplicate side effects. Then study boundaries, consistency and migration in M4.1 onward.

M4.1 How do you split a monolith into microservices?

A good SDE-3 interview answer is:

Use the **strangler fig** pattern: put a routing layer (gateway) in front, carve out one bounded context at a time (often the highest-change or most independently scalable, not the shared core), and migrate its data so each service owns its database, with an anti-corruption layer where models differ. Identify boundaries with DDD (event storming), not technical layers, and avoid a *distributed monolith* where services must deploy together. Ensure CI/CD, observability and team ownership exist *before* splitting.

How to understand it

Worked explanation: Extracting inventory means giving it ownership of reservation rules and data, not merely moving its controller into another process. The old application calls that boundary while migration proceeds incrementally.

Key terms you should know

- **DDD** — Domain-Driven Design: modeling software around business domains, boundaries and domain concepts.
- **observability** — The ability to understand internal system behavior from telemetry such as logs, metrics and traces.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: When should extraction stop? If independent deployment and ownership are not achievable, the added network boundary may only create a distributed monolith.

30-second version

> Use the **strangler fig** pattern: put a routing layer (gateway) in front, carve out one bounded context at a time (often the highest-change or most independently scalable, not the shared core), and migrate its data so each service owns its database, with an anti-corruption layer where models differ. Identify boundaries with DDD (event storming), not technical layers, and avoid a *distributed monolith* where services must deploy together.

M4.2 How do you handle transactions across services?

A good SDE-3 interview answer is:

Two-phase commit coordinates atomic commit but can block under failures and couples availability. When that trade-off is unsuitable, consider a **Saga**: a sequence of local transactions, each followed by an event, with compensating actions on failure. *Choreography* (services react to events) is simple for 2–3 steps; *orchestration* (a coordinator holds the state machine) is clearer for complex flows. Use the **transactional outbox** to atomically save state and the event, then publish via a relay or CDC. Design compensations carefully (a refund is not an undo), and make every step idempotent.

How to understand it

Worked explanation: An order flow reserves stock, charges payment and confirms delivery in separate local transactions. If payment fails, release the reservation through a compensating step; persist progress so recovery knows what already happened.

Key terms you should know

- **CDC** — Change Data Capture: publishing database changes by reading the database's change log or transaction log.
- **outbox** — A database table used to store events in the same transaction as business data so publication can happen reliably afterward.
- **Two-phase commit** — A protocol separating preparation from a coordinated commit or abort decision.
- **Saga** — A distributed transaction pattern that coordinates a sequence of local transactions and compensating actions instead of one global database transaction.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is compensation rollback? No; it is another business action that can fail and require retry or manual reconciliation.

30-second version

> Two-phase commit coordinates atomic commit but can block under failures and couples availability. When that trade-off is unsuitable, consider a **Saga**: a sequence of local transactions, each followed by an event, with compensating actions on failure.

M4.3 What delivery guarantees exist and how do you cope?

A good SDE-3 interview answer is:

Brokers provide at-most-once, at-least-once (most common) or exactly-once within narrow scopes. Real systems use at-least-once plus **idempotent consumers** (dedupe on event ID in the same transaction as the business change) so duplicates are harmless. Also handle out-of-order delivery with versions or timestamps, and poison messages through retries and dead-letter queues.

How to understand it

Worked explanation: A consumer commits an order update and crashes before acknowledging the event. Redelivery is expected. Saving the event ID atomically with that update lets the next attempt recognize completed work.

Key terms you should know

- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a separate deduplication cache guarantee this? Not if it can disagree with the business transaction; define an atomic boundary or reconciliation protocol.

30-second version

> Brokers provide at-most-once, at-least-once (most common) or exactly-once within narrow scopes. Real systems use at-least-once plus **idempotent consumers** (dedupe on event ID in the same transaction as the business change) so duplicates are harmless.

M4.4 Which resilience patterns do you apply?

A good SDE-3 interview answer is:

Timeouts on every remote call (shorter than the caller's budget), retries only for idempotent and transient failures with exponential backoff + jitter and a retry budget, **circuit breaker** (closed → open → half-open) to fail fast and give the dependency time to recover, **bulkhead** (separate pools per dependency), rate limiting and load shedding, fallbacks/caches for degraded mode, and health-based routing. Verify with chaos or fault-injection tests. Naive retries amplify outages (retry storms).

How to understand it

Worked explanation: A slow dependency retains caller threads and connections; retries increase its load. Deadlines limit waiting, a bulkhead limits the affected capacity, and a breaker temporarily stops calls likely to fail.

Key terms you should know

- **bulkhead** — A resilience pattern that isolates resources such as threads or connections so one overloaded dependency does not exhaust the whole service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is a fallback always safe? Only when its result preserves the business contract; a stale price or invented payment success is not harmless degradation.

30-second version

> Timeouts on every remote call (shorter than the caller's budget), retries only for idempotent and transient failures with exponential backoff + jitter and a retry budget, **circuit breaker** (closed → open → half-open) to fail fast and give the dependency time to recover, **bulkhead** (separate pools per dependency), rate limiting and load shedding, fallbacks/caches for degraded mode, and health-based routing. Verify with chaos or fault-injection tests.

M4.5 Explain CAP and how it influences design.

A good SDE-3 interview answer is:

CAP says that during a network partition, a system cannot guarantee both linearizable behavior and successful responses to every request at a non-failing node. Preserving the single up-to-date history can require refusing some operations; serving both sides needs a defined consistency/conflict contract. Choose by operation and business invariant, not broad labels such as banking versus catalogue. PACELC additionally asks about latency and consistency when no partition exists.

How to understand it

Worked explanation: During a partition, two replicas cannot always coordinate before answering. Requiring one up-to-date order forces some requests to wait or fail; accepting operations on both sides requires a conflict strategy.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What does consistency mean here? CAP uses linearizability, not simply valid business data or the C in ACID.

30-second version

> Define consistency as linearizability and describe one operation during a partition. Explain which requests may fail or become stale and why the business accepts that trade-off; CAP is not a blanket pick-two product label.

M4.6 How do you observe a distributed system?

A good SDE-3 interview answer is:

Logs (structured JSON, correlation/trace IDs propagated across HTTP and messaging), metrics (RED: rate, errors, duration for services; USE: utilization, saturation, errors for resources; plus business indicators), and traces (OpenTelemetry, sampling, spans for DB and remote calls). Define service-level indicators (SLIs) and their objectives (SLOs), alert on symptoms (error-budget burn rate) rather than causes, and maintain dashboards and runbooks. Tracing is what turns "it's slow somewhere" into a specific span.

How to understand it

Worked explanation: When checkout is slow, a latency metric identifies the widespread symptom, a trace locates the long payment span, and correlated logs explain the failure at that point. Each signal answers a different question.

Key terms you should know

- **span** — One timed operation within a distributed trace, such as an HTTP call or database query.
- **trace** — A representation of one request's path through distributed services.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What is an SLO? A target for a measured user-facing indicator over a stated window, with a defined denominator.

30-second version

> Logs (structured JSON, correlation/trace IDs propagated across HTTP and messaging), metrics (RED for services, USE for resources, plus business KPIs), and traces (OpenTelemetry, sampling, spans for DB and remote calls). Define service-level indicators (SLIs) and their objectives (SLOs), alert on symptoms (error-budget burn rate) rather than causes, and maintain dashboards and runbooks.

Module 5: Maven

B1 How does Maven build a Java application

Strong answer: a POM declares the project, dependencies and build configuration. Maven runs ordered lifecycle phases; plugins provide goals bound to those phases. Running package normally runs preceding default phases such as compilation and tests before producing the artifact. Clean is a separate lifecycle. A dependency supplies classes/libraries; a plugin performs build work; dependencyManagement supplies defaults without itself adding a dependency.

Follow-up: dependency scopes affect classpaths and transitivity. A successful compile does not prove a dependency will be available in the deployed runtime, and skipped tests do not become executed tests because packaging succeeded.

Senior follow-up: inspect the effective POM, resolved graph and final artifact before changing versions. Define the target JDK/release and reproducibility requirements. Continue with mediation, BOMs, test plugins and release behavior below. Maven lifecycle reference.

More in Q43–Q46.

M5.1 Explain the Maven lifecycle, phases, goals and plugins.

A good SDE-3 interview answer is:

Maven has three built-in lifecycles: `clean`, `default` (build) and `site`. The default lifecycle runs ordered **phases**: `validate → compile → test → package → verify → install → deploy`; running a phase runs every earlier phase. A phase does nothing by itself: **plugin goals** are bound to phases (`compiler:compile` to `compile`, `surefire:test` to `test`, `jar:jar` to `package`) and you can attach more through `<executions>`. `install` copies the artifact to the local `~/.m2`, `deploy` publishes it to a remote repository. Invoke a goal directly with `mvn dependency:tree`, or a phase with `mvn verify`.

How to understand it

Worked explanation: Running verify reaches compile, unit tests, packaging and configured integration verification in lifecycle order. A plugin goal does the actual work at a phase; the phase name alone does not install a test runner.

Key terms you should know

- **goal** — A specific action provided by a Maven plugin, such as compiling code or running tests.
- **phase** — A lifecycle stage in Maven, such as compile, test, package or verify.
- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why avoid invoking integration-test alone? Later verification and cleanup phases may be needed to finish the integration-test lifecycle correctly.

30-second version

> Maven has three built-in lifecycles: `clean`, `default` (build) and `site`. The default lifecycle runs ordered **phases**: `validate → compile → test → package → verify → install → deploy`; running a phase runs every earlier phase.

M5.2 `dependencyManagement` vs `dependencies`, and parent vs BOM vs aggregator?

A good SDE-3 interview answer is:

`<dependencies>` declares project dependencies, whose scopes determine their classpaths and transitivity. `<dependencyManagement>` centralizes dependency metadata and controls versions of matching transitive project dependencies as well as defaults for direct declarations. Management alone does not add an artifact, but a child need not explicitly declare every artifact whose transitive version it manages. An explicit version on a direct dependency in that project takes precedence over its managed default. Project dependency management does not manage plugin dependencies.

A **parent POM** supplies inheritance, including properties, plugin configuration and management sections. A **BOM** is imported with `<scope>import</scope><type>pom</type>` inside `dependencyManagement` to align library versions, useful when the project cannot inherit a platform's parent. An **aggregator** lists `<modules>` for a reactor build. One POM may be both parent and aggregator; these roles are distinct.

How to understand it

Worked explanation: A BOM can choose version 2 of a library, but an application still needs a dependency path that brings the library in. A parent supplies inherited configuration; an aggregator decides which modules participate in the reactor.

Key terms you should know

- **BOM** — A POM supplying a compatible set of managed dependency versions.
- **Transitive dependency** — An artifact introduced through another dependency.
- **Aggregator** — A POM listing modules built together by the reactor.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Must parent and aggregator be the same POM? No; inheritance and multi-module build membership solve different problems.

30-second version

> Dependencies add artifacts; dependencyManagement supplies defaults and can control transitive project dependency versions without explicit child declarations for each artifact. It does not add artifacts by itself. Parent means inheritance, BOM means managed library alignment, and aggregator means reactor modules.

Maven dependency mechanism.

M5.3 What are the dependency scopes and how are transitive scopes resolved?

A good SDE-3 interview answer is:

`compile` (default: all classpaths, transitive), `provided` (needed to compile, supplied by the container, not packaged, e.g. servlet API), `runtime` (not needed to compile, e.g. JDBC driver), `test`, `system` (avoid) and `import` (BOM only). A `test`-scoped dependency is not transitive; a `compile` dependency of a `runtime` dependency becomes `runtime`. Use `<optional>true</optional>` so a dependency isn't pulled in by consumers. Common production bug: a library accidentally on the `compile` scope causes version clashes or bloated jars; run `mvn dependency:analyze` to find unused-declared and used-undeclared dependencies.

How to understand it

Worked explanation: A JDBC driver may be needed only at runtime, while JUnit belongs on the test classpath. Scope controls where a dependency is available and how it reaches consumers; it is not just documentation.

Key terms you should know

- **BOM** — Bill of Materials: a dependency-management document that centralizes compatible library versions.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does optional mean unused? No; it tells downstream consumers they must opt in if they need that dependency's functionality.

30-second version

> `compile` (default: all classpaths, transitive), `provided` (needed to compile, supplied by the container, not packaged, e.g. servlet API), `runtime` (not needed to compile, e.g.

M5.4 How do you handle dependency conflicts and vulnerabilities?

A good SDE-3 interview answer is:

Maven picks the *nearest* version in the tree (first declared on ties), which can silently downgrade a library. Diagnose with `mvn dependency:tree -Dverbose`, fix by pinning in `dependencyManagement`, importing the right BOM, or excluding the transitive artifact, and enforce with `maven-enforcer-plugin` (`dependencyConvergence`, `requireUpperBoundDeps`, `banDuplicatePomDependencyVersions`). For vulnerabilities, run OWASP Dependency-Check or Dependabot/Renovate in CI, fail the build on high CVEs, and generate an SBOM (CycloneDX).

How to understand it

Worked explanation: Two libraries request incompatible versions of the same artifact. Inspect the resolved graph first, then choose a compatible aligned version; removing the losing dependency without checking its caller can produce linkage failures.

Key terms you should know

- **BOM** — Bill of Materials: a dependency-management document that centralizes compatible library versions.
- **OWASP** — An organization that publishes widely used application-security guidance, including the OWASP Top 10.
- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a clean vulnerability scan prove compatibility? No; run the relevant integration behavior against the resolved runtime graph.

30-second version

> Maven picks the *nearest* version in the tree (first declared on ties), which can silently downgrade a library. Diagnose with `mvn dependency:tree -Dverbose`, fix by pinning in `dependencyManagement`, importing the right BOM, or excluding the transitive artifact, and enforce with `maven-enforcer-plugin` (`dependencyConvergence`, `requireUpperBoundDeps`, `banDuplicatePomDependencyVersions`).

M5.5 Surefire vs Failsafe, and how do you organise tests in the build?

A good SDE-3 interview answer is:

Surefire runs unit tests (`*Test`) in the `test` phase; failing tests stop the build immediately. Failsafe runs integration tests (`*IT`) across `integration-test` and `verify`, so the build can still run `post-integration-test` (e.g., shut down containers) before failing at `verify`. Add JaCoCo for coverage gates,

Checkstyle/SpotBugs/Sonar for static analysis, and Testcontainers for real dependencies. Keep fast tests on every commit and slower suites on the pipeline.

How to understand it

Worked explanation: Start a database before integration-test, run integration tests, stop it in post-integration-test and report failures in verify. Delaying the failure verdict allows lifecycle cleanup to run.

Key terms you should know

- **phase** — A lifecycle stage in Maven, such as compile, test, package or verify.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Are naming conventions sufficient? Only when the matching plugins and executions are configured; confirm which tests actually ran.

30-second version

> Surefire runs unit tests (`*Test`) in the `test` phase; failing tests stop the build immediately. Failsafe runs integration tests (`*IT`) across `integration-test` and `verify`, so the build can still run `post-integration-test` (e.g., shut down containers) before failing at `verify`.

M5.6 How do you release and publish artifacts?

A good SDE-3 interview answer is:

Publish traceable release artifacts from validated commits into an authorized repository. Releases should be immutable; snapshots are changing development coordinates whose refresh depends on repository update policy, not an unconditional latest-build lookup. Configure publication through distributionManagement and matching server credentials supplied securely to CI. Pin the release version and toolchain, record the commit, and promote the tested artifact or container digest rather than rebuilding it differently for each environment.

How to understand it

Worked explanation: A release should be traceable from an immutable artifact to its source commit and validation. A snapshot is a development coordinate whose resolved contents can change according to repository update policy.

Key terms you should know

- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why promote an existing image? Rebuilding separately for production can produce different bytes from the artifact tested in staging.

30-second version

> A release needs immutable bytes, source/validation provenance and controlled repository credentials. Snapshots can change according to update policy. Promote the artifact that was tested instead of assuming a rebuild is identical.

M5.7 How do you speed up and harden a Maven build?

A good SDE-3 interview answer is:

Use the wrapper (`mvnw`) and pinned plugin versions, parallel builds (`-T 1C`), incremental module builds (`-pl :svc -am`), CI caching of `.m2`, avoid `clean` when not needed, remote build cache (Develocity/Maven build cache extension), and Jib/Buildpacks to build images without a Docker daemon. For reproducibility set `project.build.outputTimestamp`; for security use signed artifacts, trusted mirrors and checksum verification.

How to understand it

Worked explanation: Caching dependencies saves downloads; selecting modules saves build work; parallelism runs independent reactor work concurrently. Each optimization affects a different bottleneck and must preserve required checks.

Key terms you should know

- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can parallel builds always be enabled? Verify plugin thread safety and resource contention, and compare clean reproducibility separately from cache speed.

30-second version

> Use the wrapper (`mvnw`) and pinned plugin versions, parallel builds (`-T 1C`), incremental module builds (`-pl :svc -am`), CI caching of `.m2`, avoid `clean` when not needed, remote build cache (Develocity/Maven build cache extension), and Jib/Buildpacks to build images without a Docker daemon. For reproducibility set `project.build.outputTimestamp`; for security use signed artifacts, trusted mirrors and checksum verification.

Module 6: Google Cloud Platform

G1 What must you understand before choosing a cloud runtime

Strong answer: identify the application process, compute runtime, network boundaries, workload identity, state/storage and deployment lifecycle. A managed runtime changes which operations the provider handles; it does not remove application ownership of timeouts, authorization, data correctness or cost. Choose after establishing traffic shape, statefulness, latency needs and operational capacity.

Follow-up: distinguish scaling application instances from increasing database/downstream capacity. Replicas multiply connections and scheduled work unless you deliberately coordinate them.

Senior follow-up: draw one authenticated request from ingress through application to storage, then trace failure and rollout. Compare runtime choices in M6.1 using that concrete workload rather than a product-name checklist.

More in Q47–Q54.

M6.1 Which compute option do you choose: Cloud Run, GKE, Compute Engine or Cloud Functions?

A good SDE-3 interview answer is:

Choose from workload requirements and operational cost. Cloud Run services fit request-serving applications; Cloud Run also has jobs and worker pools, so a background worker does not automatically require Kubernetes. Check the selected resource's scaling, networking, hardware and regional capabilities. GKE fits workloads needing Kubernetes APIs, scheduling and platform controls. Compute Engine fits VM-level control or specialized legacy requirements. Compare cold starts, steady load, cost, portability and team expertise. Cloud Run resource types.

How to understand it

Worked explanation: Start with whether the workload serves requests, runs to completion or processes continuously. Then compare runtime constraints, scaling behavior, networking and ownership costs for that concrete workload.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why choose GKE over Cloud Run? Name the Kubernetes control you require and the operational cost you accept, rather than using scale alone as the reason.

30-second version

> Choose from workload requirements and operational cost. Cloud Run services fit request-serving applications; Cloud Run also has jobs and worker pools, so a background worker does not automatically require Kubernetes.

M6.2 How do you manage identity and access securely?

A good SDE-3 interview answer is:

IAM binds permissions through roles to principals at the relevant resource scope. Give each workload least-privilege identity and avoid exported service-account keys. Cloud Run can use a runtime service account. Workload Identity Federation for GKE supports direct access by a federated Kubernetes workload principal, or IAM service-account impersonation where needed. Client libraries can obtain short-lived credentials through ADC. Secrets, IAM bindings, audit access and organizational controls still require explicit configuration. GKE workload identity.

How to understand it

Worked explanation: The workload authenticates as an identity; IAM decides what that identity may do to a resource. Short-lived credentials remove a stored key, but overly broad roles still grant overly broad access.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.

- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.
- **IAM** — Identity and Access Management: policies controlling who or what can perform which actions on which resources.
- **service account** — A non-human Google Cloud identity used by workloads or applications.
- **Workload Identity** — A mechanism that lets workloads obtain cloud identities without embedding long-lived service-account keys.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does federation grant permissions automatically? No; the relevant principal or impersonated service account still needs deliberate IAM bindings.

30-second version

> Separate identity from permissions. Use least-privilege workload identities and short-lived credentials; GKE federation can grant direct resource access or use service-account impersonation without distributing private keys.

M6.3 Design a Spring Boot microservice on GCP end to end.

A good SDE-3 interview answer is:

Build with Maven and Jib → image to **Artifact Registry** (vulnerability scan) → deploy to Cloud Run/GKE across multiple zones behind a global HTTPS load balancer with **Cloud Armor** and managed TLS. The service reads config from env/Secret Manager, talks to **Cloud SQL** over private IP using the Cloud SQL connector, publishes domain events through the outbox to **Pub/Sub**, and caches in Memorystore (Redis). Observability: JSON logs correlated with traces, Micrometer metrics to Cloud Monitoring, SLO-based alerting. Infrastructure is Terraform and delivery uses Cloud Build + Cloud Deploy with canary rollouts and automated rollback.

How to understand it

Worked explanation: Follow an order from the load balancer to the service, database commit, outbox relay and subscriber. Explain the identity, timeout and retry boundary at each hop before listing cloud products.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.
- **observability** — The ability to understand internal system behavior from telemetry such as logs, metrics and traces.
- **outbox** — A database table used to store events in the same transaction as business data so publication can happen reliably afterward.
- **Pub/Sub** — Google Cloud's asynchronous messaging service for publishing messages to topics and delivering them to subscriptions.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.
- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Where can partial failure occur? Database commit can succeed before publication; the durable outbox and idempotent consumer handle that gap.

30-second version

> Build with Maven and Jib → image to **Artifact Registry** (vulnerability scan) → deploy to Cloud Run/GKE across multiple zones behind a global HTTPS load balancer with **Cloud Armor** and managed TLS. The service reads config from env/Secret Manager, talks to **Cloud SQL** over private IP using the Cloud SQL connector, publishes domain events through the outbox to **Pub/Sub**, and caches in Memorystore (Redis).

M6.4 How do you use Pub/Sub from Spring correctly?

A good SDE-3 interview answer is:

Use Spring Cloud GCP (`PubSubTemplate`, `@ServiceActivator`/inbound adapter with manual ack). Delivery is at-least-once, so handlers must be idempotent (dedupe by message ID or business key). Tune the ack deadline to processing time, configure retry with backoff and a **dead-letter topic**, use ordering keys only when per-key order matters, set flow control (`maxOutstandingElementCount`) for back-pressure, and monitor oldest unacked message age and DLQ size. Never ack before the work is durably done.

How to understand it

Worked explanation: The handler receives a message, commits its business effect and only then acknowledges it. Losing that acknowledgement can still cause redelivery, so successful work must be recognizable on a second attempt.

Key terms you should know

- **back-pressure** — A mechanism that prevents an upstream producer from overwhelming a downstream consumer.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why bound outstanding messages? It limits memory and downstream demand while slow processing or acknowledgements accumulate.

30-second version

> Use Spring Cloud GCP (`PubSubTemplate`, `@ServiceActivator`/inbound adapter with manual ack). Delivery is at-least-once, so handlers must be idempotent (dedupe by message ID or business key).

M6.5 How do you choose a data store on GCP?

A good SDE-3 interview answer is:

Cloud SQL (PostgreSQL/MySQL) for regional relational OLTP with HA and read replicas; AlloyDB for higher PostgreSQL performance and analytics; **Spanner** for globally consistent, horizontally scalable relational data; Firestore for document data with real-time sync; Bigtable for very high-throughput wide-column/time-series; **BigQuery** for serverless analytics (columnar, partitioned and clustered, never as an OLTP store); Memorystore for caching; Cloud Storage for objects. Match consistency, query patterns, scale and cost; don't pick Spanner "because it's powerful".

How to understand it

Worked explanation: List required queries and invariants first: a financial transaction, a document lookup and an analytical scan need different access patterns. Compare specific products' guarantees and costs against those requirements.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can one product serve everything? Sometimes, but demonstrate the workload fit; another datastore adds synchronization, recovery and operational work.

30-second version

> Cloud SQL (PostgreSQL/MySQL) for regional relational OLTP with HA and read replicas; AlloyDB for higher PostgreSQL performance and analytics; **Spanner** for globally consistent, horizontally scalable relational data; Firestore for document data with real-time sync; Bigtable for very high-throughput wide-column/time-series; **BigQuery** for serverless analytics (columnar, partitioned and clustered, never as an OLTP store); Memorystore for caching; Cloud Storage for objects. Match consistency, query patterns, scale and cost; don't pick Spanner "because it's powerful".

M6.6 How do you design for high availability and disaster recovery?

A good SDE-3 interview answer is:

Define **RTO** (time to recover) and **RPO** (data you can lose) with the business, then pick the pattern: multi-zone within a region (default HA: regional GKE, regional Cloud SQL with failover, regional load balancer), cross-region for DR (read replicas or Spanner multi-region, multi-region storage, traffic failover with a global load balancer). Automate backups with point-in-time recovery, test restores and failover game-days, keep infrastructure as code so a region can be rebuilt, and make deployments backward compatible so rollbacks are safe.

How to understand it

Worked explanation: A zone failure and a region failure are different recovery events. Replicas can improve availability, while backups recover deleted or corrupted data; one does not replace the other.

Key terms you should know

- **RTO** — The target duration for restoring service after a disruption.
- **RPO** — The acceptable recovery point, often expressed as an amount of time of data loss.
- **HA** — Availability maintained through redundancy and failure handling.
- **DR** — Recovery of service and data after a major disruption.
- **PITR** — Restoring a database to a selected point in time within retained recovery history.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you validate RPO and RTO? Restore and fail over with measured data loss and elapsed recovery time, including application dependencies.

30-second version

> Define **RTO** (time to recover) and **RPO** (data you can lose) with the business, then pick the pattern: multi-zone within a region (default HA: regional GKE, regional Cloud SQL with failover, regional load balancer), cross-region for DR (read replicas or Spanner multi-region, multi-region storage, traffic

> failover with a global load balancer). Automate backups with point-in-time recovery, test restores and failover game-days, keep infrastructure as code so a region can be rebuilt, and make deployments backward compatible so rollbacks are safe.

Module 7: System Design Prompts (practice 45 min each)

1. URL shortener 2. Rate limiter 3. Notification service 4. Order/payment platform with saga 5. Ride-hailing/real-time tracking. **Framework:** requirements (functional/non-functional) → capacity estimates → API → data model → high-level design → deep dive (scaling, consistency, failures) → trade-offs.

Module 8: Coding Programs with Solutions

1. LRU Cache (O(1)): classic

Derive the approach: the operations require both key lookup and recency updates. A hash table solves lookup; a doubly linked list allows removal/reinsertion of a known node without searching. Access-ordered `LinkedHashMap` supplies both for the concise solution.

Why this, not the alternatives: an array/list requires O(n) movement or search; a timestamp heap adds O(log n) updates and stale-entry handling. Implement the map plus list yourself when the interviewer disallows library LRU. Exact global LRU under contention may be more expensive than an approximate eviction policy.

What changes: insertion order evicts an old entry even after a recent read. An access-order `get` mutates ordering, so concurrent reads need coordination too. A synchronized wrapper only protects calls through that wrapper; multi-step operations need one shared lock.

Trace and checks: capacity two; put A, put B, read A, put C must evict B. Test replacement, capacity one, invalid capacity and mutable keys. Expected O(1) operations, O(capacity) state.

Strong answer signal: connect each data structure to a required operation and explain the recency invariant.

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LRUCache(int capacity) {
        super(Math.max(1, capacity), 0.75f, true);
        if (capacity < 1) throw new IllegalArgumentException("capacity must be positive");
        this.capacity = capacity;
    }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > capacity; }
}
```

Interviewer follow-up: implement without LinkedHashMap → `HashMap<K, Node>` + doubly linked list with head/tail sentinels; move node to head on access, evict the tail. Make thread-safe with `synchronized` wrapper or a lock (a coarse shared lock can preserve exact LRU; specialized caches trade eviction policy against concurrency).

2. Streams: top 3 earners per department

Derive the approach: the ranking is independent per department, so partition by department, order each group and retain its first three entries. Define whether “three” means three people or three distinct salary ranks.

Why this, not the alternatives: sorting each group is easy to inspect and costs O(sum(m*d log m*d)). For large groups and fixed k, maintain a size-k min-heap per department for O(n log k) processing and O(departments × k) retained candidates. Sorting the whole dataset is unnecessary unless its global order is also required.

What changes: tied salaries need a stable secondary key for reproducibility. A top-three-distinct-salaries requirement can return more than three people. For monetary correctness use minor units or `BigDecimal`, rather than assuming `double` is acceptable. If data lives in SQL, consider a partitioned window ranking query and return only needed rows.

Trace and checks: salaries 90, 100, 100, 80 produce three people but only two distinct ranks. Test fewer than three employees, missing departments and ties. The shown grouping retains O(n) data.

Strong answer signal: explain the business meaning of ranking before optimizing the stream.

```
record Employee(String name, String dept, double salary) {}

Map<String, List<Employee>> top3 = employees.stream()
    .collect(Collectors.groupingBy(Employee::dept,
        Collectors.collectingAndThen(Collectors.toList(),
            l -> l.stream()
                  .sorted(Comparator.comparingDouble(Employee::salary).reversed())
                  .limit(3).toList())));
```

3. Producer–Consumer with BlockingQueue

Derive the approach: producers and consumers run at different speeds and need a safe handoff. A bounded blocking queue combines synchronization, waiting and a memory bound.

Why this, not the alternatives: polling a normal list risks races and wastes CPU; an unbounded queue hides overload as memory growth. A custom wait/notify queue is useful as an interview exercise but adds lifecycle and signaling responsibilities already handled by the library.

What changes: the example assumes one producer, one consumer and a sentinel that cannot be real data. Multiple consumers need an agreed termination protocol, often one terminal signal per consumer after all producers finish. Interrupting a producer before it enqueues the sentinel can strand the consumer; a supervisor must cancel/join both sides on failure. Durable work needs a broker or persisted queue, not only memory.

Trace and checks: fill ten slots; the next put waits until a take frees capacity. Test interruption while full/empty, consumer failure and shutdown. Queue operations are O(1), but waiting time is workload-dependent; storage is O(capacity).

Strong answer signal: distinguish backpressure from a complete shutdown protocol.

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(10);
Runnable producer = () -> { try { for (int i = 0; i < 100; i++) q.put(i); q.put(-1); }
                            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
};
Runnable consumer = () -> { try { int v; while ((v = q.take()) != -1) System.out.println(v); }
                            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
};
new Thread(producer).start(); new Thread(consumer).start();
```

Mention: `put`/`take` block (back-pressure); poison pill for shutdown; always restore the interrupt flag.

4. Token-Bucket Rate Limiter (thread-safe)

Derive the approach: the requirement permits a short burst but limits the average rate. Represent available credit as tokens, refill by elapsed monotonic time and cap credit at capacity.

Why this, not the alternatives: a fixed window can admit two bursts around a boundary; an exact sliding log retains each accepted timestamp. A leaky-bucket queue controls output pacing and introduces waiting, which differs from immediate admit/reject behavior.

What changes: wall-clock adjustments distort refill, so use an injectable monotonic ticker. Validate positive capacity/rate. Without the lock, two requests can spend the same token. Multiple JVMs require coordinated state or an explicitly approximate per-instance allowance. Floating-point refill may need a carefully defined rounding policy at exact boundaries.

Trace and checks: capacity two and one token/second: two immediate successes, one rejection, then one success after one simulated second. Test idle refill capped at capacity and concurrent callers. Local state and work are O(1), excluding lock contention.

Strong answer signal: distinguish burst capacity, refill rate and concurrent request count.

```
class TokenBucket {
    private final long capacity, refillPerSec;
    private double tokens; private long last = System.nanoTime();
    TokenBucket(long capacity, long refillPerSec) { this.capacity = capacity; this.refillPerSec =
refillPerSec; this.tokens = capacity; }
    synchronized boolean tryAcquire() {
        long now = System.nanoTime();
        tokens = Math.min(capacity, tokens + (now - last) / 1e9 * refillPerSec);
        last = now;
        if (tokens >= 1) { tokens--; return true; }
        return false;
    }
}
```

Follow-up: distributed version → Redis + Lua script for atomic refill/consume.

5. Async fan-out with CompletableFuture

Derive the approach: user and order reads do not depend on one another, so start both and combine only after both results are ready. Sequential latency is roughly the sum; parallel latency approaches the slower call plus overhead, subject to capacity.

Why this, not the alternatives: sequential calls are simpler when cheap or dependent. Virtual threads make blocking tasks easier to express; reactive composition fits an already reactive stack. Futures help express the dependency graph, but are not a reason to fan out without limits.

What changes: if the order call requires the user result, compose it after the first call instead. A blanket fallback can hide an authorization or data error. A future timeout does not necessarily stop network work; define deadlines and ownership of cancellation. Waiting on subtasks submitted to the same saturated pool can cause starvation.

Trace and checks: user completes in 100 ms, orders in 300 ms; the successful combination waits for orders. Test each branch failing, both failing, timeout and executor rejection. Two branches mean constant task count, not constant remote cost.

Strong answer signal: identify independent work, result semantics and failure ownership.

```
CompletableFuture<User> u = CompletableFuture.supplyAsync(() -> userSvc.get(id), pool);
CompletableFuture<List<Order>> o = CompletableFuture.supplyAsync(() -> orderSvc.get(id), pool);
Profile p = u.thenCombine(o, Profile::new)
             .orTimeout(2, TimeUnit.SECONDS)
             .exceptionally(ex -> Profile.fallback(id))
             .join();
```

Mention: select an executor deliberately, especially for blocking I/O. `orTimeout` completes a future exceptionally but does not stop the underlying call; `CompletableFuture.cancel(true)` does not interrupt its computation. Configure downstream deadlines and explicit cancellation, and distinguish timeout from business failure before returning a fallback. CompletableFuture API.

6. Thread-safe Singleton with enum and holder alternatives

Derive the approach: one instance is required within one class-loader scope, and concurrent callers must not observe partial construction. Enum initialization or the initialization-on-demand holder uses JVM initialization guarantees.

Why this, not the alternatives: a synchronized accessor is correct but adds a locking path; double-checked locking is more subtle and requires correct volatile publication. In application code, an injected singleton-scoped service often gives better lifecycle control and testability than a global singleton. Enum is an option, not universally “best.”

What changes: a singleton does not make its mutable fields thread-safe, and separate class loaders can each have an instance. A distributed singleton requires coordination beyond this pattern. Lazy construction shifts initialization failure to first use; eager construction discovers it sooner.

Trace and checks: concurrent holder access returns one reference after class initialization. Test access from multiple threads and avoid using static global state as a substitute for dependency injection. Access is O(1); retained state depends on the singleton's contents.

Strong answer signal: state the scope of “one” and separate construction safety from state safety.

```
public enum Config { INSTANCE; /* safe against reflection & serialization */ }

class Holder { private Holder() {}
    private static class H { static final Holder I = new Holder(); }
    static Holder get() { return H.I; } }   // lazy, thread-safe via class loading
```

7. Spring Boot: REST + global error handling + N+1 fix

Derive the approach: the response needs order fields and items, so define that read shape explicitly, map it inside the service transaction and return a DTO with a controlled error contract.

Why this, not the alternatives: serializing an entity can trigger lazy queries, expose fields or recurse through associations. Globally eager relationships overfetch unrelated use cases. An entity graph fits this read; a DTO query can be better when only a few columns are needed.

What changes: fetching a to-many collection with pagination requires care; paginate parent IDs or use an appropriate projection. Two collections can multiply rows. A graph expresses required loading, not a portable guarantee of one SQL statement. Exposing raw exception messages is unsafe unless they are intentionally user-facing.

Trace and checks: existing order with two items yields one DTO; missing order gives 404; an empty order yields an empty items list. Measure queries and rows with realistic cardinality and verify authorization. Cost depends on fetched rows and query plans, not just the Java mapper loop.

Strong answer signal: tie transaction and fetch boundaries to the API contract and verify emitted SQL.

```
@RestController @RequestMapping("/orders") @RequiredArgsConstructor
class OrderController {
    private final OrderService svc;
    @GetMapping("/{id}") OrderDto get(@PathVariable Long id) { return svc.get(id); }
}

@Service @RequiredArgsConstructor
```

```
class OrderService {
    private final OrderRepository repo;
    @Transactional(readOnly = true)
    OrderDto get(Long id) {
        return repo.findWithItemsById(id).map(OrderDto::from)
                   .orElseThrow(() -> new OrderNotFoundException(id));
    }
}

interface OrderRepository extends JpaRepository<Order, Long> {
    @EntityGraph(attributePaths = "items")
    Optional<Order> findWithItemsById(Long id);
}

@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
    }
}
```

8. Quick algorithm warm-ups

Longest unique substring — derive and choose: a repeated character only invalidates windows containing its previous occurrence. Keep the last index and advance the left boundary to `max(left, previous + 1)`. This beats enumerating O(n²) substrings; a set-based shrinking window is equally valid but may perform more removals. On `abba`, moving left backward at the last `a` gives a wrong answer; the maximum is two. The code counts UTF-16 units; code-point semantics change representation. Expected O(n) time, O(distinct characters) state. A strong explanation proves the window remains duplicate-free.

First non-repeating character — derive and choose: counts determine uniqueness; encounter order determines “first.” Count then scan the original string, or use insertion-ordered counts. Sorting loses encounter order, while a plain map's iteration order is not the answer. On `swiss`, return `w`; on `aabb`, report absence. A live stream needs counts plus a queue of candidates. O(n) time and O(alphabet) space; define the same character model as above.

Linked-list cycle — derive and choose: two moving positions inside a cycle gain one relative step each iteration and eventually meet. Floyd's method uses O(1) space; a visited identity set is easier to explain and can expose the first repeated node but costs O(n). Stop when fast or fast.next is null. Duplicate values are not cycles; compare node identity. Test self-cycle, two-node cycle, empty and linear lists. Finding the cycle entry requires the additional reset-and-walk phase. O(n) time.

Merge intervals — derive and choose: sorting by start makes only the last merged interval a possible overlap candidate. Pairwise comparison is O(n²), and a heap is unnecessary for a static union. The full implementation and boundary analysis appear under “Merge overlapping intervals.” Closed `[1,2]` and `[2,3]` merge in that implementation; half-open intervals require an explicit endpoint policy. O(n log n) time plus output space.

Longest substring without repeating characters (sliding window, O(n)):

```
int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> idx = new HashMap<>(); int best = 0, left = 0;
    for (int r = 0; r < s.length(); r++) {
        Integer prev = idx.put(s.charAt(r), r);
        if (prev != null && prev >= left) left = prev + 1;
        best = Math.max(best, r - left + 1);
    }
    return best;
}
```

First non-repeating character: `LinkedHashMap<Character,Integer>` counts, return the first entry with value 1. **Detect cycle in linked list:** Floyd's slow/fast pointers, O(1) space. **Merge intervals:** sort by start, then merge in one pass, O(n log n).

Module 9: Behavioral (Senior Level)

Prepare 5 **STAR** stories with metrics (Situation, Task, Action, **Result**):

1. Production incident you led (root cause, fix, prevention).
2. A big architecture decision and its trade-offs.
3. Performance improvement ("p99 from 2s → 300ms by …").
4. Disagreement with a peer/manager and how it was resolved.
5. Mentoring / raising team standards.

Questions to ask them: deployment frequency, on-call load, tech-debt policy, how architecture decisions are made.

Final Checklist

- Can explain `@Transactional` pitfalls, N+1, HashMap internals, and saga/outbox *without notes*
- Can code LRU, a rate limiter, and a stream grouping in 10 minutes each
- Have 5 STAR stories and 2 system designs rehearsed aloud

Part 02 - Question Bank Vol. 1 - Java, Spring, JPA and microservices

Senior Java Question Bank, Vol. 1 (Detailed Answers)

Companion to the refresher course. Each answer follows what senior interviewers score: **definition → internals → trade-off → pitfall → real-world example.** Practise saying them aloud in 60–90 seconds.

A. Core Java

Learning order: begin with Java foundations; for Q7–Q9, first read the multithreading ladder.

Q1. Explain the `equals()`/`hashCode()` contract and what breaks if you violate it.

A good SDE-3 interview answer is:

If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must hold; equals must be reflexive, symmetric, transitive, consistent and false for null. The reverse is not required (collisions are legal). If you override `equals` without `hashCode`, equal objects land in different buckets, so `HashSet.contains` and `HashMap.get` fail intermittently. Mutating fields used in `hashCode` after inserting into a hash collection "loses" the entry. *Production tip:* use immutable keys or records; for JPA entities use a stable business key.

How to understand it

Worked explanation: Two distinct customer-key objects representing the same customer must compare equal and produce the same hash. The hash narrows the search; equality identifies the mapping. Changing the key after insertion can make lookup search a different bucket.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can unequal keys share a hash? Yes; collisions are legal and must be resolved with equality.

30-second version

> If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must hold; equals must be reflexive, symmetric, transitive, consistent and false for null. The reverse is not required (collisions are legal).

Q2. Why is `String` immutable and what is the String pool?

A good SDE-3 interview answer is:

Immutability gives thread-safety, safe use as map keys (hash cached), security (class names, URLs, credentials cannot be altered after validation) and enables the pool, where literals are interned and shared in the heap (since Java 7). `new String("a")` creates a separate object; `intern()` returns the pooled one. In loops use `StringBuilder` (not thread-safe, fast) or `StringBuffer` (synchronized, rarely needed). Store passwords in `char[]` so they can be wiped.

How to understand it

Worked explanation: Two references can safely share a String because neither can change its characters. Concatenation creates a result rather than editing the old string. The pool reuses selected equal strings; equality of contents still uses equals.

Key terms you should know

- **heap** — JVM memory where Java objects are allocated.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does an immutable password String become erasable? No; a char array offers explicit clearing, although copies and runtime behavior still limit guarantees.

30-second version

> Immutability gives thread-safety, safe use as map keys (hash cached), security (class names, URLs, credentials cannot be altered after validation) and enables the pool, where literals are interned and shared in the heap (since Java 7). `new String("a")` creates a separate object; `intern()` returns the pooled one.

Q3. How do fail-fast and fail-safe iterators differ?

A good SDE-3 interview answer is:

“Fail-safe” is an informal label; distinguish the actual iterator contracts. `ArrayList` and `HashMap` iterators detect many structural changes made through another path and may throw `ConcurrentModificationException`, on a best-effort basis. `CopyOnWriteArrayList` iterators traverse a snapshot and do not throw that exception due to later list changes; writes copy the backing array and can cost O(n). `ConcurrentHashMap` iterators are weakly consistent, not snapshots.

Use `Iterator.remove()` only when that iterator supports it. It is supported by ordinary `ArrayList`/`HashMap` iterators, but a `CopyOnWriteArrayList` iterator throws `UnsupportedOperationException`. For that list, use a supported collection-level operation such as `removeIf`. Choose copy-on-write collections for read-heavy, rarely-modified data such as listener registrations.

How to understand it

Worked explanation: A snapshot iterator keeps seeing its original array after a writer replaces the collection's array. A weakly consistent iterator can observe concurrent changes without promising that snapshot. Fail-fast detection is a debugging aid.

Key terms you should know

- **Fail-fast** — Best-effort detection of incompatible structural modification.
- **Snapshot iterator** — Traversal of a previously captured collection structure.
- **Weak consistency** — Concurrent traversal without a fixed snapshot or fail-fast guarantee.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can ConcurrentModificationException prove thread safety? No; its absence is not a synchronization guarantee, and same-thread structural changes can also cause it.

30-second version

> ArrayList and HashMap iterators are best-effort fail-fast; CopyOnWriteArrayList uses snapshots; ConcurrentHashMap is weakly consistent. Iterator removal is implementation-dependent: it throws for CopyOnWriteArrayList, whose supported collection-level removeIf can be used instead.

Java 21 CopyOnWriteArrayList.

Q4. What is PECS in generics?

A good SDE-3 interview answer is:

PECS means Producer Extends, Consumer Super. With `List<? extends Number>`, values can be read as Number, but adding an arbitrary Integer or Double is unsafe because the actual subtype is unknown; null is the general exception. `List<? super Integer>` can accept Integers, but reads guarantee only Object. Neither wildcard promises immutability. Use a concrete type parameter when an API must both consume and produce the same type. Generic type checks are limited by erasure; for example, `instanceof List<?>` is permitted, while `instanceof List<String>` is not. Java wildcard rules.

How to understand it

Worked explanation: A List of an unknown subtype of Number might actually contain only Doubles, so adding an Integer would be unsafe. A list of an unknown supertype of Integer can accept Integers, but reading guarantees only Object.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does extends make the list immutable? No; supported removal or clearing may still work, and null is the only generally addable value.

30-second version

> Extends gives a readable upper bound; super gives a safe input type. Explain what can be read and written, and remember that wildcard variance does not make a collection immutable.

Q5. Checked vs unchecked exceptions: what is your strategy?

A good SDE-3 interview answer is:

Checked exceptions require a caller to catch or declare them; unchecked exceptions do not. Recoverability is an API design consideration, not a rule determined solely by that classification. Translate lower-level failures at meaningful boundaries while preserving causes, use try-with-resources for owned AutoCloseable resources, and retain suppressed cleanup failures. Catch only when you can recover, add useful context or map to a public contract; avoid swallowing errors or logging the same failure at every layer.

How to understand it

Worked explanation: Distinguish the language rule from your API policy: checked exceptions require catching or declaration; unchecked ones do not. Either category can describe a condition a particular caller chooses to recover from.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Where should translation happen? At a boundary that can add domain meaning while preserving the cause and avoiding duplicate logging.

30-second version

> Explain the compiler rule first, then your recovery and translation policy. Preserve causes and cleanup failures, and handle an exception at the boundary that can take a meaningful action.

Q6. Explain class loading and when you meet `ClassNotFoundException` vs `NoClassDefFoundError`.

A good SDE-3 interview answer is:

Separate loading, linking and initialization. Standard application loading normally delegates to a parent; custom loaders can use other policies. ClassNotFoundException is a checked failure from an explicit class-loading request. NoClassDefFoundError is a linkage failure when a required definition cannot be used, including subsequent use of a class whose initialization previously failed. Inspect the earliest failure and cause chain, resolved dependencies and class-loading logs; a missing jar is one possibility, not the only explanation.

How to understand it

Worked explanation: A plugin loader explicitly asks for a named class and cannot find it: ClassNotFoundException. A class needed during execution cannot be defined or was left erroneous after initialization: a linkage error such as NoClassDefFoundError can follow.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why inspect the cause chain? A later class-initialization failure can hide the earlier exception that first made the class unusable.

30-second version

> Loading, linking and initialization differ. ClassNotFoundException comes from explicit lookup; NoClassDefFoundError can reflect a missing definition or earlier initialization failure. Diagnose the original cause and actual runtime classpath.

Q7. How does `ConcurrentHashMap` achieve thread safety and why does it forbid null?

A good SDE-3 interview answer is:

Java 8+: a table of bins; empty-bin inserts use CAS, non-empty bins lock only the head node (`synchronized`), so contention is per-bucket. Reads are lock-free via volatile reads. Nulls are banned because `get(k) == null` would be ambiguous (absent vs mapped to null) in a concurrent setting where you cannot do a safe `containsKey` follow-up. Use `compute`, `merge`, `putIfAbsent` for atomic compound operations; `check-then-put` is a race.

How to understand it

Worked explanation: Two threads doing get-then-put can both see absence. putIfAbsent or compute combines that decision with the update for one key. The map protects its mappings, not mutable fields inside stored values.

Key terms you should know

- **bin** — A bucket position in HashMap's internal table; OpenJDK source commonly calls the entries at one index a bin.
- **bucket** — One position in a hash table's internal array where entries can be stored.
- **CAS** — Compare-and-set, an atomic CPU/JVM operation that updates a value only if it still equals an expected value.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a slow compute block other work? Yes; keep callbacks short and avoid recursive updates or remote I/O inside them.

30-second version

> Java 8+: a table of bins; empty-bin inserts use CAS, non-empty bins lock only the head node (`synchronized`), so contention is per-bucket. Reads are lock-free via volatile reads.

Q8. What are ThreadLocals' dangers?

A good SDE-3 interview answer is:

They store per-thread state (user context, MDC, date formatters), but with pooled threads values outlive the request unless you `remove()` in a `finally`/filter, causing data leaks between users and classloader memory leaks. They also don't propagate to child/async threads automatically (use a `TaskDecorator`, `InheritableThreadLocal` carefully, or scoped values: preview in Java 21 and final in Java 25). With virtual threads, large per-thread state gets expensive. Scoped values support bounded-lifetime context sharing, not arbitrary mutable thread-local storage. Java 25 ScopedValue.

How to understand it

Worked explanation: A worker handles Alice, then Bob. If Alice's context remains in its ThreadLocal, Bob can inherit the stale value even though requests are unrelated. Restore or remove context at the operation boundary.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does copying context permit sharing mutable state? No; choose which values may cross threads and how they are isolated.

30-second version

> They store per-thread state (user context, MDC, date formatters), but with pooled threads values outlive the request unless you `remove()` in a `finally`/filter, causing data leaks between users and classloader memory leaks. They also don't propagate to child/async threads automatically (use a `TaskDecorator`, `InheritableThreadLocal` carefully, or scoped values: preview in Java 21 and final in Java 25).

Q9. Are parallel streams a good idea?

A good SDE-3 interview answer is:

Parallel streams can help substantial, independent CPU work on efficiently splittable sources when combining results is cheap. Common implementations use fork/join execution, often the common pool; context and implementation matter, so do not treat a fixed executor as a universal Stream API contract. Blocking callbacks can consume shared capacity and hurt unrelated work. Preserve non-interference, associativity and required ordering, measure representative workloads, and prefer explicitly controlled execution for I/O.

How to understand it

Worked explanation: Parallel execution splits work and combines partial results. Splitting, scheduling and merging cost time, so a cheap mapping over a short list can become slower. Shared blocking work also competes for execution capacity.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How would you decide? Benchmark representative CPU work and validate ordering, reduction associativity and side-effect safety first.

30-second version

> Parallel streams add splitting, scheduling and merging costs. Use them only after proving the operation's parallel correctness and measuring a benefit; blocking shared workers and assuming executor isolation are common mistakes.

Q10. Immutability: how do you design an immutable class?

A good SDE-3 interview answer is:

`final` class, private final fields, no setters, defensive copies in constructor and getters for mutable components (`List.copyOf`, `Date` clones), no leaking `this` during construction. Benefits: thread-safety without locks, safe sharing and caching. Java 16+ `record` gives this for shallow immutability; collections inside still need `List.copyOf` in the compact constructor.

How to understand it

Worked explanation: An immutable wrapper around a caller-owned list is unsafe if that list remains mutable through another reference. Copy the structure on entry and avoid exposing mutable internals on exit.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is List.copyOf a deep copy? No; mutable elements are still shared and need their own ownership or immutability policy.

30-second version

> `final` class, private final fields, no setters, defensive copies in constructor and getters for mutable components (`List.copyOf`, `Date` clones), no leaking `this` during construction. Benefits: thread-safety without locks, safe sharing and caching.

B. Spring & Spring Boot

Learning order: start with the Spring annotation ladder, then return here for circular dependencies, AOP, async, caching and API follow-ups.

Q11. How does Spring resolve circular dependencies?

A good SDE-3 interview answer is:

For singleton beans with *setter/field* injection, Spring exposes an early reference via a three-level cache (`singletonObjects`, `earlySingletonObjects`, `singletonFactories`). Constructor injection cannot be resolved (`BeanCurrentlyInCreationException`), and Boot 2.6+ prohibits circular references by default. The right answer: treat a cycle as a design smell and break it by extracting a third service, publishing events, or injecting `ObjectProvider`/`@Lazy` only as a last resort.

How to understand it

Worked explanation: A constructor for A needs B, whose constructor needs A; neither object can finish construction. Early references can help some setter-based cycles but do not make the dependency design healthy.

Key terms you should know

- **Application service** — A component coordinating a business use case; it may be a Spring-managed bean and is not a Kubernetes Service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What is the first refactoring? Identify the shared responsibility or orchestration and move it to a suitable third component.

30-second version

> For singleton beans with *setter/field* injection, Spring exposes an early reference via a three-level cache (`singletonObjects`, `earlySingletonObjects`, `singletonFactories`). Constructor injection cannot be resolved (`BeanCurrentlyInCreationException`), and Boot 2.6+ prohibits circular references by default.

Q12. Explain Spring AOP and its limitations.

A good SDE-3 interview answer is:

Spring AOP applies cross-cutting concerns such as transactions, security, logging and caching through runtime proxies. JDK dynamic proxies expose interfaces; CGLIB creates subclasses. The actual proxy type depends on configuration; Spring Boot commonly defaults to class proxies. Know aspect, advice, pointcut and join point.

Advice applies to eligible calls through the proxy. Self-invocation and private methods bypass ordinary proxy advice. CGLIB cannot subclass a final class or override a final method. However, a JDK interface proxy can advise an interface call whose target implementation method is final: it delegates instead of overriding that method. AspectJ weaving has a different interception model.

`@Around` controls whether and how the target executes. It may call `proceed()` zero times (cached result or rejection), once, or multiple times (a deliberate retry policy). Return-value and exception behavior must preserve the intended API contract; always rethrowing is not a universal rule.

How to understand it

Worked explanation: The caller invokes a proxy, advice executes, and the proxy delegates to the target. A call from one target method to another stays on that target, bypassing the external interception path.

Key terms you should know

- **AOP** — Applying cross-cutting behavior at selected execution points.
- **Proxy** — An intermediary that intercepts calls before delegating.
- **Around advice** — Advice that controls target invocation and the caller-visible result.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why can an interface proxy advise a final implementation method? It delegates through the interface instead of overriding that method.

30-second version

> Spring AOP intercepts eligible calls through proxies. Final-method restrictions apply to subclass proxies; JDK proxies can advise interface calls to final target implementations. Around advice may skip, invoke or repeat the target deliberately, while honoring its result and failure contract.

Proxy mechanisms, around advice.

Q13. How do you manage configuration across environments?

A good SDE-3 interview answer is:

Externalised config precedence: command-line args > env vars > `application-{profile}.yml` > `application.yml`. Use `@ConfigurationProperties` (type-safe, validated with `@Validated`) instead of scattered `@Value`. Secrets come from Secret Manager/Vault, never from Git. In Kubernetes use ConfigMaps/Secrets and, for live reload, Spring Cloud Config or `@RefreshScope`. Same artifact, different config (12-factor).

How to understand it

Worked explanation: The same image can use different connection endpoints through external properties. Grouped typed settings make invalid combinations fail at startup instead of surfacing as scattered parsing errors during requests.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does updating a ConfigMap update every bean? No; environment variables and existing bound objects need an explicit reload or restart strategy.

30-second version

> Externalised config precedence: command-line args > env vars > `application-{profile}.yml` > `application.yml`. Use `@ConfigurationProperties` (type-safe, validated with `@Validated`) instead of scattered `@Value`.

Q14. How does `@Async` work, and what goes wrong?

A good SDE-3 interview answer is:

With async support enabled, an eligible call through the proxy is submitted to the selected executor. Self-invocation bypasses that advice. Configure admission, rejection, deadlines and failure reporting explicitly. In Boot 3.5, the usual auto-configured executor is a ThreadPoolTaskExecutor; with Java 21+ virtual threads enabled it can be a virtual-thread SimpleAsyncTaskExecutor. Thread-bound transactions and context do not automatically transfer. CompletableFuture exposes result/failure composition; void methods need AsyncUncaughtExceptionHandler reporting. Boot task execution.

How to understand it

Worked explanation: A proxy hands work to an executor and the caller continues. The new thread has its own transaction and context boundaries, and a void return gives the caller no future through which to observe failure.

Key terms you should know

- **CompletableFuture** — A Java API for representing and composing asynchronous results and their success/failure stages.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does virtual-thread execution remove overload? No; bound admission and scarce resources even when threads are cheap.

30-second version

> Async advice dispatches proxy calls to a configured executor. Bound work and resources, propagate only intended context, and handle failures. Boot's executor choice changes when virtual threads are enabled.

Q15. Spring caching: how and what are the traps?

A good SDE-3 interview answer is:

`@EnableCaching` + `@Cacheable`/`@CachePut`/`@CacheEvict` through a proxy, backed by Caffeine (local) or Redis (distributed). Traps: self-invocation, caching mutable objects, missing TTL/size limits (memory leak), stale data across instances with local caches, cache stampede (use `sync=true` or request coalescing), and key design (`key = "#id"`). Always ask "what is the invalidation strategy?"

How to understand it

Worked explanation: On a cache hit, advice can return the stored value without running the target method. If the key omits tenant identity, one tenant can receive another's result even though the method itself is correct.

Key terms you should know

- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does sync=true coordinate every instance? Its scope and support depend on the cache provider; it is not a universal distributed lock.

30-second version

> `@EnableCaching` + `@Cacheable`/`@CachePut`/`@CacheEvict` through a proxy, backed by Caffeine (local) or Redis (distributed). Traps: self-invocation, caching mutable objects, missing TTL/size limits (memory leak), stale data across instances with local caches, cache stampede (use `sync=true` or request coalescing), and key design (`key = "#id"`).

Q16. How do you test a Spring Boot application?

A good SDE-3 interview answer is:

Pyramid: plain JUnit 5 + Mockito for domain logic (fast, no Spring); slice tests (`@WebMvcTest` with `MockMvc`, `@DataJpaTest`) for one layer; `@SpringBootTest` sparingly for wiring; **Testcontainers** for real PostgreSQL/Kafka instead of H2 (dialect differences hide bugs); WireMock for HTTP dependencies; contract tests between services. Keep the context cache hot by minimising `@MockBean`/`@DirtiesContext`.

How to understand it

Worked explanation: A plain unit test proves pricing rules, a MVC slice proves request binding and error responses, and a real-database test proves SQL constraints. Select the smallest boundary that can expose the failure.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why can an H2 test pass while PostgreSQL fails? SQL dialects, constraints and transaction behavior can differ.

30-second version

> Pyramid: plain JUnit 5 + Mockito for domain logic (fast, no Spring); slice tests (`@WebMvcTest` with `MockMvc`, `@DataJpaTest`) for one layer; `@SpringBootTest` sparingly for wiring; **Testcontainers** for real PostgreSQL/Kafka instead of H2 (dialect differences hide bugs); WireMock for HTTP dependencies; contract tests between services. Keep the context cache hot by minimising `@MockBean`/`@DirtiesContext`.

Q17. Design a good REST API. What do you check?

A good SDE-3 interview answer is:

Resource-oriented nouns, correct verbs/status codes (201 + `Location`, 204, 400 vs 422, 409 conflict), idempotent `PUT`/`DELETE`, `Idempotency-Key` for `POST` payments, pagination (cursor over offset for large sets), filtering, versioning, consistent error format (`ProblemDetail`), validation (`@Valid`), HATEOAS only if justified, OpenAPI docs, rate limiting, and authentication/authorisation on every endpoint. Never expose entities directly; use DTOs.

How to understand it

Worked explanation: For an order-creation request, explain validation, authorization, creation status, resource location and retry behavior. The API contract includes failure and duplicate requests, not just the JSON shape.

Key terms you should know

- **idempotency** — The property that repeating the same logical request produces the same intended outcome rather than creating duplicate effects.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is repeated DELETE required to return the same status? No; idempotency concerns intended state effects, not identical responses.

30-second version

> Resource-oriented nouns, correct verbs/status codes (201 + `Location`, 204, 400 vs 422, 409 conflict), idempotent `PUT`/`DELETE`, `Idempotency-Key` for `POST` payments, pagination (cursor over offset for large sets), filtering, versioning, consistent error format (`ProblemDetail`), validation (`@Valid`), HATEOAS only if justified, OpenAPI docs, rate limiting, and authentication/authorisation on every endpoint. Never expose entities directly; use DTOs.

Q18. What happens when a request hits a Spring MVC app?

A good SDE-3 interview answer is:

Servlet container (Tomcat) → filter chain (Security, etc.) → `DispatcherServlet` → `HandlerMapping` finds the controller method → `HandlerInterceptor.preHandle` → argument resolvers/`HttpMessageConverter` (Jackson) bind the body → controller runs → return value handler/converter serialises → interceptors `postHandle`/`afterCompletion` → exceptions go to `HandlerExceptionResolver` (`@ExceptionHandler`). Knowing filter vs interceptor distinction is a common probe.

How to understand it

Worked explanation: A filter can reject a request before MVC selects a controller. MVC then resolves arguments, invokes the handler and converts its result; response-body conversion can happen before interceptor postHandle.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Where should exception mapping live? MVC resolvers handle controller-processing exceptions; failures outside MVC need handling at their own boundary.

30-second version

> Servlet container (Tomcat) → filter chain (Security, etc.) → `DispatcherServlet` → `HandlerMapping` finds the controller method → `HandlerInterceptor.preHandle` → argument resolvers/`HttpMessageConverter` (Jackson) bind the body → controller runs → return value handler/converter serialises → interceptors `postHandle`/`afterCompletion` → exceptions go to `HandlerExceptionResolver` (`@ExceptionHandler`). Knowing filter vs interceptor distinction is a common probe.

Q19. How do you handle graceful shutdown and zero-downtime deploys?

A good SDE-3 interview answer is:

`server.shutdown=graceful` with `spring.lifecycle.timeout-per-shutdown-phase`; readiness probe flips to "out of service" so the load balancer stops routing; in-flight requests finish; consumers stop polling and commit offsets; the Kubernetes `terminationGracePeriodSeconds` exceeds the app timeout and a `preStop` sleep covers endpoint propagation lag. Pair with rolling updates (`maxUnavailable: 0`) and backward-compatible DB migrations (expand → migrate → contract).

How to understand it

Worked explanation: Stopping new routing and finishing accepted work take time. Coordinate readiness, endpoint propagation, server draining and the platform termination deadline; keep old and new schema contracts compatible during overlap.

Key terms you should know

- **Shutdown phase** — A stage of Spring lifecycle shutdown with its own configured timeout; distinct from a Maven build phase.
- **readiness probe** — A Kubernetes health check used to decide whether a Pod should receive traffic.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a preStop sleep guarantee safety? No; it consumes the same grace period and must be justified by measured propagation and drain times.

30-second version

> `server.shutdown=graceful` with `spring.lifecycle.timeout-per-shutdown-phase`; readiness probe flips to "out of service" so the load balancer stops routing; in-flight requests finish; consumers stop polling and commit offsets; the Kubernetes `terminationGracePeriodSeconds` exceeds the app timeout and a `preStop` sleep covers endpoint propagation lag. Pair with rolling updates (`maxUnavailable: 0`) and backward-compatible DB migrations (expand → migrate → contract).

C. Hibernate / JPA

Learning order: start with connections, sessions, transactions and mapping annotations, then return here for inheritance, lifecycle, bulk operations and isolation.

Q20. Compare JPA inheritance strategies.

A good SDE-3 interview answer is:

`SINGLE_TABLE`: one table + discriminator, fastest queries, but nullable columns and weak constraints. `JOINED`: normalised table per class, clean schema, joins on every polymorphic query. `TABLE_PER_CLASS`: no joins for concrete reads but UNION-heavy polymorphic queries; rarely recommended. `@MappedSuperclass` shares fields without polymorphism. Default to SINGLE_TABLE for small hierarchies, JOINED when integrity matters.

How to understand it

Worked explanation: A payment hierarchy can use one table with a type column or several joined tables. The choice moves cost between nullability/constraints, writes and polymorphic queries.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does SINGLE_TABLE always win? No; evaluate actual query plans and integrity requirements rather than claiming a universally fastest strategy.

30-second version

> `SINGLE_TABLE`: one table + discriminator, fastest queries, but nullable columns and weak constraints. `JOINED`: normalised table per class, clean schema, joins on every polymorphic query.

Q21. Explain cascade types and `orphanRemoval`.

A good SDE-3 interview answer is:

`CascadeType` propagates operations (PERSIST, MERGE, REMOVE, REFRESH, DETACH) from parent to child; `ALL` combines them. `orphanRemoval=true` deletes a child when it is removed from the parent's collection. Use only on true aggregate ownership (Order → OrderLine); never cascade REMOVE on shared references like `@ManyToOne` to a lookup. Keep both sides of bidirectional associations in sync with helper `addItem/removeItem` methods; the owning side (the one with `@JoinColumn`) writes the FK.

How to understand it

Worked explanation: Removing a privately owned OrderLine can mean deleting that row; removing a shared Product reference must not mean deleting the product. Cascades propagate lifecycle operations, while association ownership controls relationship updates.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why update both sides? It keeps the in-memory graph consistent even though only the owning side writes the relationship.

30-second version

> `CascadeType` propagates operations (PERSIST, MERGE, REMOVE, REFRESH, DETACH) from parent to child; `ALL` combines them. `orphanRemoval=true` deletes a child when it is removed from the parent's collection.

Q22. Why does offset pagination degrade and what's the alternative?

A good SDE-3 interview answer is:

`OFFSET n` forces the DB to scan and discard n rows, so deep pages get slow and unstable under concurrent inserts. **Keyset (seek) pagination** uses `WHERE (created_at, id) < (:lastCreatedAt, :lastId) ORDER BY created_at DESC, id DESC LIMIT :size` with a matching composite index: avoids work proportional to the skipped offset. With a suitable index, seek plus retrieval is approximately O(log n + page size); changing sort keys can still move rows between pages. Spring Data's `Window`/`ScrollPosition` (3.1+) supports it.

How to understand it

Worked explanation: Page 10,000 by offset asks the database to pass many earlier rows. A cursor carries the last stable sort key so the next query can seek after it using an index.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why include a unique tie-breaker? Equal timestamps alone cannot define an unambiguous page boundary.

30-second version

> `OFFSET n` forces the DB to scan and discard n rows, so deep pages get slow and unstable under concurrent inserts. **Keyset (seek) pagination** uses `WHERE (created_at, id) < (:lastCreatedAt, :lastId) ORDER BY created_at DESC, id DESC LIMIT :size` with a matching composite index: avoids work proportional to the skipped offset.

Q23. `save()` **vs** `saveAndFlush()` **vs** `flush()`**, and how does** `merge` **vs** `persist` **differ?**

A good SDE-3 interview answer is:

`persist` makes a *new* transient entity managed (void; ID assigned per generator). `merge` copies the detached state onto a managed instance and returns that instance; the argument stays detached. `save()` calls `persist` if `isNew()` (null ID/version) otherwise `merge`, which may trigger an extra SELECT for assigned IDs. `flush` only syncs the context to the DB; it doesn't commit. Dirty checking means managed entities are updated without calling `save`.

How to understand it

Worked explanation: merge returns the managed destination of copied state; the detached argument does not become managed. Editing only that original argument afterward may never reach the database.

Key terms you should know

- **dirty checking** — Hibernate/JPA's mechanism for detecting changes to managed entities and generating updates during flush.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What does saveAndFlush add? It requests synchronization now, while transaction commit and durability still occur at the enclosing boundary.

30-second version

> `persist` makes a *new* transient entity managed (void; ID assigned per generator). `merge` copies the detached state onto a managed instance and returns that instance; the argument stays detached.

Q24. How do you do bulk updates safely?

A good SDE-3 interview answer is:

Bulk JPQL/native updates bypass normal entity dirty checking and can leave managed entities stale; automatic optimistic-version handling is not guaranteed. Flush pending changes before executing the bulk statement where required, then clear or refresh affected state. With Spring Data, consider both `flushAutomatically=true` and

`clearAutomatically=true`; clearing alone can discard pending in-memory work. Preserve tenant filters, version predicates and row-count checks explicitly. For imports, use chunked transactions and bounded persistence contexts.

How to understand it

Worked explanation: A bulk update changes rows directly while an already loaded entity still contains its old value. Later application logic can act on that stale copy unless the persistence context is refreshed or cleared deliberately.

Key terms you should know

- **dirty checking** — Hibernate/JPA's mechanism for detecting changes to managed entities and generating updates during flush.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why flush before clear? Clearing can otherwise discard pending managed changes that have not reached SQL.

30-second version

> Bulk JPQL/native updates bypass normal entity dirty checking and can leave managed entities stale; automatic optimistic-version handling is not guaranteed. Flush pending changes before executing the bulk statement where required, then clear or refresh affected state.

Q25. What isolation levels exist and which anomalies do they prevent?

A good SDE-3 interview answer is:

Isolation constrains concurrent transaction histories. Read committed prevents dirty reads but may allow different committed results across statements. Repeatable read and serializable behavior must be described for the named database: PostgreSQL repeatable read uses snapshot isolation and can permit write skew; serializable detects unsafe histories and can abort transactions. Constraints, atomic updates and appropriate locking can enforce specific invariants. Retrying a serialization/deadlock failure must restart the whole safe business transaction. PostgreSQL isolation.

How to understand it

Worked explanation: Read committed prevents reading another transaction's uncommitted data but can show different committed results across statements. Stronger isolation changes the allowed histories; database-specific implementation matters.

Key terms you should know

- **Isolation level** — A database contract governing which effects concurrent transactions can observe.
- **Optimistic locking** — Detecting conflicting updates, often with a version predicate; the eventual database write still acquires locks.
- **Serializable** — An isolation guarantee equivalent to some serial ordering of committed transactions.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does serializable mean no failures? No; conflicts may require aborting and retrying an entire transaction.

30-second version

> Name the database and invariant. Read committed, repeatable read and serializable provide different guarantees; serializable can require retries. An isolation annotation requests behavior from the database rather than implementing it itself.

D. Microservices & Distributed Systems

Start with the service boundary and request foundation, then gateways, resilience and distributed consistency in Q26–Q30.

Q26. API Gateway vs service mesh: what is the difference?

A good SDE-3 interview answer is:

A gateway controls edge traffic: routing, authentication, rate limits and public API concerns. A service mesh supplies service-to-service transport controls such as workload identity, encryption, routing and telemetry. Mesh architectures can use sidecars or other data-plane arrangements. Both can be useful, but neither is mandatory; justify their operational cost and keep business rules in their owning application boundary.

How to understand it

Worked explanation: An edge gateway routes a customer's request to a service; a mesh can secure and observe calls between services. Neither should automatically become the owner of order-pricing rules.

Key terms you should know

- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Must every system have both? No; justify each boundary's controls and operational cost. Mesh implementations are not limited to sidecars.

30-second version

> A gateway manages the external API boundary; a mesh manages service-to-service transport. Explain the concrete control required and its cost instead of assuming every system needs both.

Q27. Explain CQRS and Event Sourcing, and when not to use them.

A good SDE-3 interview answer is:

CQRS separates the write model (validates commands, enforces invariants) from read models (denormalised projections optimised for queries), which can share a transactional store; event-driven projections in separate stores are optional and can be eventually consistent. Event Sourcing stores the *sequence of events* as the

source of truth, enabling audit, replay and temporal queries, but adds complexity: schema evolution of events, snapshots, rebuild time, and a steep learning curve. Use for complex domains with audit/read-scale needs, not CRUD apps.

How to understand it

Worked explanation: A command validates and changes an order; a projection serves a convenient order-summary view. CQRS can use one database. Event sourcing additionally makes the sequence of business events the authoritative history.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does CQRS require event sourcing? No; they are independent choices with different complexity and recovery costs.

30-second version

> CQRS separates the write model (validates commands, enforces invariants) from read models (denormalised projections optimised for queries), which can share a transactional store; event-driven projections in separate stores are optional and can be eventually consistent. Event Sourcing stores the *sequence of events* as the source of truth, enabling audit, replay and temporal queries, but adds complexity: schema evolution of events, snapshots, rebuild time, and a steep learning curve.

Q28. How do you choose between Kafka and Pub/Sub (or RabbitMQ)?

A good SDE-3 interview answer is:

Kafka: a distributed log with partition-based ordering, consumer groups, long retention, replay and very high throughput; you manage partitions/rebalancing (or use managed). Pub/Sub: fully managed, global, auto-scaling, per-message ack, no partition management, ordering keys optional, replay via seek/retention; less control. RabbitMQ: smart broker, routing flexibility, low-latency task queues. Explain partition-key choice (ordering per key, hot partitions), DLQs, and idempotent consumers.

How to understand it

Worked explanation: Ask whether consumers need replay, per-key ordering, routing flexibility or managed operations. Then describe how the chosen broker acknowledges work and exposes retry/failure behavior.

Key terms you should know

- **Pub/Sub** — Google Cloud's asynchronous messaging service for publishing messages to topics and delivering them to subscriptions.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does changing brokers remove duplicate effects? No; consumers still need a deliberate processing and deduplication contract.

30-second version

> Kafka: a distributed log with partition-based ordering, consumer groups, long retention, replay and very high throughput; you manage partitions/rebalancing (or use managed). Pub/Sub: fully managed, global, auto-scaling, per-message ack, no partition management, ordering keys optional, replay via seek/retention; less control.

Q29. How do you deploy safely: rolling, blue-green, canary?

A good SDE-3 interview answer is:

Rolling replaces instances gradually (cheap, mixed versions coexist, so API/DB must be backward compatible). Blue-green runs two full environments and switches traffic instantly (fast rollback, double cost). Canary routes a small percentage to the new version while watching SLOs, then ramps up (best risk control; needs good metrics). Feature flags decouple deploy from release. On GKE use Cloud Deploy/Argo Rollouts; on Cloud Run use traffic splitting by revision.

How to understand it

Worked explanation: During a rolling release, old and new instances serve traffic together. A canary adds a measured stop/go decision before wider rollout; blue-green changes traffic between environments.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can traffic rollback undo a schema change? No; data and schema compatibility must be designed separately.

30-second version

> Rolling replaces instances gradually (cheap, mixed versions coexist, so API/DB must be backward compatible). Blue-green runs two full environments and switches traffic instantly (fast rollback, double cost).

Q30. A downstream service is slow and your service is falling over. What do you do?

A good SDE-3 interview answer is:

Immediately: check dashboards (RED metrics), identify saturation, apply a circuit breaker/fallback and shed load. Root causes of cascading failure: missing timeouts, unbounded threads/queues, retry storms, shared connection pools. Fixes: tight timeouts (below the caller's), bulkheads per dependency, bounded pools, retries with backoff + jitter and a retry budget, caching, async decoupling via queue, autoscaling with sensible limits, and a post-mortem with action items. This is the "tell me about an outage" answer; use a real story.

How to understand it

Worked explanation: Trace how one slow dependency fills connections, then worker slots, then the request queue. Limit that chain before adding replicas that may send even more traffic to the same bottleneck.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Which signal proves recovery? End-user error/latency plus reduced saturation, not merely an open circuit breaker.

30-second version

> Immediately: check dashboards (RED metrics), identify saturation, apply a circuit breaker/fallback and shed load. Root causes of cascading failure: missing timeouts, unbounded threads/queues, retry storms, shared connection pools.

Self-Assessment

After each answer, ask: *Did I give a trade-off? A pitfall? An example from my own project?* If not, the answer is junior-level.

Part 03 - Question Bank Vol. 2 - Kafka, security, Maven, GCP and Kubernetes

Senior Java Question Bank, Vol. 2

Kafka · Spring Security · Maven · GCP · Docker/Kubernetes (Q31–Q58) Answer shape: **what it is → how it works → trade-off → pitfall → your experience.**

E. Kafka & Messaging

K1 How do producers, brokers and consumers fit together

Strong answer: a producer sends a record/event to a broker; a consumer reads or receives it and processes it. In Kafka, a topic is split into partitions that hold ordered logs, and an offset identifies a position within one partition. A consumer group tracks its processing position; record ordering within a partition is distinct from business completion order across parallel workers.

Follow-up: committing an offset and committing a database side effect are separate actions unless a deliberate protocol couples them. A retry after failure can repeat processing, so identify the durable deduplication/idempotency boundary. A broker acknowledgement is not proof that the consumer's business operation completed.

Senior follow-up: trace a crash after the side effect but before the position is recorded, then the reverse ordering. Define replay, poison-event handling and per-key ordering before discussing delivery guarantees in Q31–Q36. Kafka introduction.

Q31. Explain Kafka's architecture and how consumer groups scale.

A good SDE-3 interview answer is:

A topic is split into partitions, each an append-only, ordered log replicated across brokers (one leader, N-1 followers). Producers write to a partition chosen by key hash; consumers in a **group** divide partitions so each partition is read by exactly one member of the group, which gives parallelism up to the partition count. Extra consumers sit idle, so partition count is your scaling ceiling and is hard to reduce later. Different groups read independently, which enables pub/sub with replay. Offsets live in `__consumer_offsets`.

How to understand it

Worked explanation: In an ordinary consumer group, each assigned partition has one active consumer; another group can read the same partition independently. More group members than partitions do not add partition-processing parallelism.

Key terms you should know

- **Pub/Sub** — Google Cloud's asynchronous messaging service for publishing messages to topics and delivering them to subscriptions.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can you process records asynchronously within a partition? Yes, but you must preserve required ordering and commit only a safely completed prefix.

30-second version

> A topic is split into partitions, each an append-only, ordered log replicated across brokers (one leader, N-1 followers). Producers write to a partition chosen by key hash; consumers in a **group** divide partitions so each partition is read by exactly one member of the group, which gives parallelism up to the partition count.

Q32. How do you get "exactly-once" with Kafka?

A good SDE-3 interview answer is:

Say honestly: end-to-end exactly-once needs cooperation. Producer idempotence prevents duplicate log entries from supported retries; `acks=all` and the topic/broker `min.insync.replicas` setting define acknowledgement and availability trade-offs. They do not promise zero loss under every failure. Kafka transactions (`transactional.id`) atomically write to multiple partitions and commit consumer offsets (consume-transform-produce within Kafka, with consumers using `isolation.level=read_committed`). Once a side effect leaves Kafka (DB, REST), you need an idempotent consumer: dedupe by event ID in a unique-constrained table, or the transactional outbox. Default design target: at-least-once + idempotency.

How to understand it

Worked explanation: A Kafka transaction can commit produced records and consumed offsets together. Charging a card through HTTP is outside that transaction, so a replay can still repeat the external effect.

Key terms you should know

- **idempotency** — The property that repeating the same logical request produces the same intended outcome rather than creating duplicate effects.
- **outbox** — A database table used to store events in the same transaction as business data so publication can happen reliably afterward.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What closes that gap? Provider idempotency, durable operation state and reconciliation, rather than calling the entire workflow exactly-once.

30-second version

> Say honestly: end-to-end exactly-once needs cooperation. Producer idempotence prevents duplicate log entries from supported retries; `acks=all` and the topic/broker `min.insync.replicas` setting define acknowledgement and availability trade-offs.

Q33. What causes consumer rebalances and how do you reduce their impact?

A good SDE-3 interview answer is:

Membership changes, failures and poll/heartbeat violations can move partition assignments. Disruption depends on the configured group protocol and assignment strategy: classic eager rebalances revoke broadly, while cooperative or newer incremental protocols can reduce movement. Keep processing within the applicable poll

contract, coordinate in-flight completion and offset commits with revocation, and use static membership or batching when suitable. Moving work off the poll thread requires an explicit completion/ordering protocol.

How to understand it

Worked explanation: When partition ownership changes, in-flight work can finish after the old owner lost the assignment. Coordinate revocation, completion and commits to prevent stale ownership from advancing progress.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Are all rebalances stop-the-world? No; cooperative and newer protocols reduce disruption. Explain the deployed client/protocol behavior.

30-second version

> A rebalance changes partition ownership. Explain your protocol's disruption, why polling or membership changed, and how unfinished work and commits are handled. Not every rebalance stops every consumer.

Q34. Auto-commit vs manual commit?

A good SDE-3 interview answer is:

Auto-commit can acknowledge records before they are processed (loss on crash) or reprocess after a crash (duplicates). With Spring Kafka I use `AckMode.RECORD/MANUAL_IMMEDIATE`, commit only after the DB transaction succeeds, and make handlers idempotent. Commit after processing = at-least-once; commit before = at-most-once.

How to understand it

Worked explanation: An offset commit is a progress claim for a partition, not an individual database commit. Advancing past unfinished earlier records can skip their effects after a restart.

Key terms you should know

- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What must asynchronous handlers track? The contiguous completed offset frontier, including failures and revoked assignments.

30-second version

> Auto-commit can acknowledge records before they are processed (loss on crash) or reprocess after a crash (duplicates). With Spring Kafka I use `AckMode.RECORD/MANUAL_IMMEDIATE`, commit only after the DB transaction succeeds, and make handlers idempotent.

Q35. How do you handle poison messages and retries?

A good SDE-3 interview answer is:

Distinguish transient (retry) from permanent (deserialisation, validation) failures. Spring Kafka `DefaultErrorHandler` with `ExponentialBackOff`, non-retryable exception classification, and `DeadLetterPublishingRecoverer` to a DLT carrying headers (original topic, exception, offset). For ordering-insensitive flows use non-blocking retry topics (`@RetryableTopic`) so one bad record doesn't block a partition. Alert on DLT depth and provide a replay tool. Use `ErrorHandlingDeserializer` so bad bytes don't crash-loop the consumer.

How to understand it

Worked explanation: A malformed payload will not become valid after ten retries. Classify permanent errors and preserve the failed record with enough context for diagnosis and controlled replay.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if publishing to the dead-letter topic fails? Do not silently treat the original record as recovered; define and verify the recovery/offset policy.

30-second version

> Distinguish transient (retry) from permanent (deserialisation, validation) failures. Spring Kafka `DefaultErrorHandler` with `ExponentialBackOff`, non-retryable exception classification, and `DeadLetterPublishingRecoverer` to a DLT carrying headers (original topic, exception, offset).

Q36. How do you guarantee ordering and avoid hot partitions?

A good SDE-3 interview answer is:

Kafka orders only **within a partition**, so choose a key whose events must be ordered (e.g., `orderId`, `accountId`). A single hot key remains on one partition after adding partitions. Split/salt keys only if the ordering contract allows it; partition-count changes also require an ordering and key-migration plan. With retries, set `max.in.flight.requests.per.connection ≤ 5` with idempotence to retain order. Schema evolution via Avro/Protobuf + Schema Registry with backward-compatible rules.

How to understand it

Worked explanation: Keying events by order ID groups each order's history into a partition. One extremely hot order stays a bottleneck; salting that key spreads load only by changing the ordering problem.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can adding partitions reorder a key's history? A new mapping may put later events elsewhere, so plan migration and consumption explicitly.

30-second version

> Kafka orders only **within a partition**, so choose a key whose events must be ordered (e.g., `orderId`, `accountId`). A single hot key remains on one partition after adding partitions.

F. Spring Security & OAuth2

A1 How do authentication and authorization fit into a request

Strong answer: authentication establishes the caller's identity; authorization decides what that identity may do to the requested resource. A request passes through configured security processing, establishes or resolves the security context, and is checked against access rules. Possessing a token does not by itself grant access to every object named in a request.

Follow-up: decoding a token is different from validating its authenticity, issuer, audience and validity for this application. Enforce tenant/resource ownership as well as operation-level roles, and make error responses safe.

Senior follow-up: trace an anonymous call, expired/invalid credential, authenticated caller without permission and cross-tenant object access. Then study the filter chain, OAuth2/OIDC and browser threats in Q37–Q42; keep version-specific configuration grounded in the deployed framework. Spring Security servlet architecture.

Q37. Walk through the Spring Security filter chain.

A good SDE-3 interview answer is:

`DelegatingFilterProxy` → `FilterChainProxy` → a `SecurityFilterChain` matched by request. Key filters: `SecurityContextHolderFilter`, `CorsFilter`, `CsrfFilter`, authentication filters (`UsernamePasswordAuthenticationFilter`, `BearerTokenAuthenticationFilter`), `ExceptionTranslationFilter`, and `AuthorizationFilter` last. An `AuthenticationManager` delegates to `AuthenticationProvider`s that produce an `Authentication` stored in the `SecurityContext` (thread-bound, so propagate to async threads deliberately). Configure with one or more `SecurityFilterChain` beans using lambda DSL.

How to understand it

Worked explanation: The servlet delegates to Spring Security's selected chain. Authentication establishes an identity; authorization evaluates the requested action. Enabled filters and their order depend on the chain's configuration.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if multiple chains match? The first matching security chain is selected; test ordering and uncovered routes.

30-second version

> `DelegatingFilterProxy` → `FilterChainProxy` → a `SecurityFilterChain` matched by request. Key filters: `SecurityContextHolderFilter`, `CorsFilter`, `CsrfFilter`, authentication filters (`UsernamePasswordAuthenticationFilter`, `BearerTokenAuthenticationFilter`), `ExceptionTranslationFilter`, and `AuthorizationFilter` last.

Q38. Explain OAuth2/OIDC flows and when to use each.

A good SDE-3 interview answer is:

Authorization Code + PKCE for user login (web, SPA, mobile); the PKCE verifier protects the code from interception. **Client Credentials** for service-to-service. **Refresh token** to renew short-lived access tokens. Implicit and Password grants are deprecated. OAuth2 delegates *authorisation*; OIDC adds an ID token (JWT) for *authentication*. In Spring, services are **resource servers** (validate tokens) and BFF/web apps are **clients**; the IdP (Keycloak, Okta, Google Identity Platform) is the authorisation server.

How to understand it

Worked explanation: A browser exchanges a code using its verifier, receives tokens and calls an API with an access token. The ID token describes authentication for the client; it is not a generic API credential.

Key terms you should know

- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does PKCE replace client authentication? No; confidential clients still use their required authentication, and tokens must target the intended recipient.

30-second version

> **Authorization Code + PKCE** for user login (web, SPA, mobile); the PKCE verifier protects the code from interception. **Client Credentials** for service-to-service.

Q39. JWT vs opaque tokens: how do you revoke a JWT?

A good SDE-3 interview answer is:

A locally validated signed JWT does not by itself tell the API whether that particular token has been revoked. Short lifetimes, refresh-token policy, a denylist or another online policy check can limit exposure; opaque-token introspection centralizes the decision at network and caching cost. Key rotation generally affects more than one token. Verify signature, allowed algorithms, issuer, audience and time claims. A signed JWT payload is readable unless encryption is separately used.

How to understand it

Worked explanation: A signed token can be verified without asking its issuer whether it was revoked. That independence saves a lookup but means revocation requires extra state, a policy check or waiting for expiry.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does key rotation revoke one user? Usually it affects all tokens signed by that key; choose the appropriate revocation granularity.

30-second version

> JWT validation can be local, but immediate selective revocation needs extra state or policy. Compare short expiry, denylisting and introspection, and distinguish signing from encrypting token contents.

Q40. CORS vs CSRF: what are they and how do you configure them?

A good SDE-3 interview answer is:

CORS controls whether browser JavaScript can access cross-origin responses. Configure allowed origins, methods and headers through a `CorsConfigurationSource`; do not use a wildcard origin with credentials. CSRF exploits credentials the browser sends automatically, such as session cookies or cached HTTP Basic credentials. CORS is not a substitute for CSRF protection, and stateless authentication alone does not remove CSRF risk.

Keep CSRF protection for endpoints that accept ambient browser credentials. Disabling it can be appropriate for an API that accepts only bearer tokens explicitly attached by the client in the `Authorization` header, with no cookie, Basic or other ambient-authentication fallback. Check every enabled authentication mechanism and endpoint; a JWT stored in a cookie still needs a CSRF strategy.

How to understand it

Worked explanation: A browser can automatically send a session cookie on a forged request. Blocking JavaScript from reading a cross-origin response does not necessarily prevent that request from changing server state.

Key terms you should know

- **CORS** — Browser-enforced rules for cross-origin response access.
- **CSRF** — Forging authenticated requests using ambient credentials.
- **Ambient credentials** — Credentials automatically attached by the browser.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does storing a JWT in a cookie remove CSRF? No; the browser's automatic credential behavior is the relevant property.

30-second version

> CORS and CSRF solve different problems. Keep CSRF protection when the browser sends credentials automatically, including cookies and Basic authentication. Disable it only after verifying that the relevant API accepts exclusively explicit bearer credentials without an ambient-authentication fallback.

Spring Security CSRF reference.

Q41. How do you store passwords and secure service-to-service calls?

A good SDE-3 interview answer is:

Use adaptive, salted hashes: `DelegatingPasswordEncoder` with BCrypt/Argon2 (work factor tuned to ~250 ms), never SHA/MD5 alone. Service-to-service: mTLS (mesh) or OAuth2 client credentials; on GCP use service-account identity tokens (OIDC) verified by the receiver. Apply least privilege, rotate secrets, scan dependencies (OWASP Dependency-Check/Dependabot), and validate input to prevent injection.

How to understand it

Worked explanation: Password verification deliberately performs expensive hashing using the stored salt and parameters. Service authentication instead proves a workload's identity; authorization still decides which operation it may perform.

Key terms you should know

- **OWASP** — An organization that publishes widely used application-security guidance, including the OWASP Top 10.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Should hash cost be a fixed copied number? No; calibrate against hardware, expected traffic and abuse resistance.

30-second version

> Use adaptive, salted hashes: `DelegatingPasswordEncoder` with BCrypt/Argon2 (work factor tuned to ~250 ms), never SHA/MD5 alone. Service-to-service: mTLS (mesh) or OAuth2 client credentials; on GCP use service-account identity tokens (OIDC) verified by the receiver.

Q42. `@PreAuthorize` **vs URL-based authorisation?**

A good SDE-3 interview answer is:

URL rules (`requestMatchers("/admin/**").hasRole(...)`) give coarse perimeter control; method security (`@EnableMethodSecurity`, `@PreAuthorize("hasAuthority('ORDER_READ') and #userId == authentication.name")`) enforces fine-grained, domain-aware rules close to the business logic and works for non-HTTP entry points (messaging). Use both (defence in depth); method security relies on proxies, so self-invocation bypasses it.

How to understand it

Worked explanation: An /orders route rule can require login, while a service method checks that the caller may read this particular order. The method boundary also covers callers that do not arrive through HTTP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a self-call skip method advice? Yes with ordinary proxies; test the actual invocation path and object ownership checks.

30-second version

> URL rules (`requestMatchers("/admin/**").hasRole(...)`) give coarse perimeter control; method security (`@EnableMethodSecurity`, `@PreAuthorize("hasAuthority('ORDER_READ') and #userId == authentication.name")`) enforces fine-grained, domain-aware rules close to the business logic and works for non-HTTP entry points (messaging). Use both (defence in depth); method security relies on proxies, so self-invocation bypasses it.

G. Maven & Build

Start with the Maven lifecycle and artifact foundation, then dependency mediation, scopes and reproducible builds in Q43–Q46.

Q43. How does Maven resolve conflicting dependency versions?

A good SDE-3 interview answer is:

"Nearest definition wins" (shallowest in the tree), ties broken by first declaration. This can silently select an old, vulnerable version. Control it with `<dependencyManagement>` or importing a **BOM** (`spring-boot-dependencies`, `libraries-bom` for GCP), `<exclusions>`, and the `maven-enforcer-plugin` (`dependencyConvergence`,

`banDuplicatePomDependencyVersions`). Diagnose with `mvn dependency:tree -Dverbose -Dincludes=group:artifact`.

How to understand it

Worked explanation: If the same artifact appears along two paths, Maven mediates versions unless management or an explicit declaration determines the result. The winning dependency can satisfy compilation yet break a transitive caller at runtime.

Key terms you should know

- **BOM** — Bill of Materials: a dependency-management document that centralizes compatible library versions.
- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What do you inspect? The effective POM, resolved dependency tree and actual packaged artifact, followed by relevant compatibility tests.

30-second version

> "Nearest definition wins" (shallowest in the tree), ties broken by first declaration. This can silently select an old, vulnerable version.

Q44. How do you structure a multi-module project?

A good SDE-3 interview answer is:

A parent POM (packaging `pom`) holds `dependencyManagement`, `pluginManagement` and properties; `<modules>` aggregates children (e.g., `api`, `domain`, `service`, `app`). Keep module dependencies acyclic and one-directional; use a "bom" or "platform" module for shared versions. Build selectively with `-pl :service -am`, in parallel with `-T 1C`. Avoid shared "utils" dumping grounds that couple microservices (share contracts, not code).

How to understand it

Worked explanation: A domain module should not depend back on the web application that uses it. Directed module dependencies make ownership and build order visible; common version configuration belongs in deliberate management.

Key terms you should know

- **BOM** — Bill of Materials: a dependency-management document that centralizes compatible library versions.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does listing a module make it inherit the parent? No; reactor aggregation and the child's parent declaration are separate.

30-second version

> A parent POM (packaging `pom`) holds `dependencyManagement`, `pluginManagement` and properties; `<modules>` aggregates children (e.g., `api`, `domain`, `service`, `app`). Keep module dependencies acyclic and one-directional; use a "bom" or "platform" module for shared versions.

Q45. Profiles, plugins and what belongs where?

A good SDE-3 interview answer is:

Profiles activate per environment/OS/property (`-Pci`) to vary plugins or dependencies, but avoid using them for runtime configuration (that belongs to Spring profiles/env vars), because they create non-reproducible builds. Plugin execution is bound to lifecycle phases via `<executions>`; know `compiler`, `surefire`, `failsafe`, `jacoco`, `spring-boot-maven-plugin` (repackage/build-image), `jib`, `versions`, and `enforcer`.

How to understand it

Worked explanation: A Maven profile changes the build model; a Spring profile changes application configuration. Mixing them can create separately built artifacts whose differences are hard to verify during promotion.

Key terms you should know

- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Where should plugin defaults live? pluginManagement centralizes defaults; the build must still activate the relevant plugin execution.

30-second version

> Profiles activate per environment/OS/property (`-Pci`) to vary plugins or dependencies, but avoid using them for runtime configuration (that belongs to Spring profiles/env vars), because they create non-reproducible builds. Plugin execution is bound to lifecycle phases via `<executions>`; know `compiler`, `surefire`, `failsafe`, `jacoco`, `spring-boot-maven-plugin` (repackage/build-image), `jib`, `versions`, and `enforcer`.

Q46. How do you make builds fast, reproducible and secure?

A good SDE-3 interview answer is:

Use the Maven wrapper (`mvnw`), pin plugin and dependency versions, set `project.build.outputTimestamp` for reproducible jars, cache `~/.m2` in CI, build with `-T`, skip nothing in CI (no `-DskipTests`). Pull artifacts through an internal repository manager (Artifact Registry/Nexus) configured in `settings.xml` mirrors with credentials from CI secrets; sign and scan artifacts, generate an SBOM (CycloneDX), and fail on critical CVEs.

How to understand it

Worked explanation: Reproducibility means identical intended inputs yield identical artifact bytes. Pinning versions helps, but toolchains, generated timestamps and environment-sensitive tasks also affect the result.

Key terms you should know

- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you verify it? Compare independent clean-build outputs and keep that check separate from performance benefits of caching.

30-second version

> Use the Maven wrapper (`mvnw`), pin plugin and dependency versions, set `project.build.outputTimestamp` for reproducible jars, cache `~/.m2` in CI, build with `-T`, skip nothing in CI (no `-DskipTests`). Pull artifacts through an internal repository manager (Artifact Registry/Nexus) configured in `settings.xml` mirrors with credentials from CI secrets; sign and scan artifacts, generate an SBOM (CycloneDX), and fail on critical CVEs.

H. Google Cloud Platform

Start with the cloud runtime foundation, then compare services and operational guarantees in Q47–Q52.

Q47. Cloud Run vs GKE vs App Engine vs Compute Engine: how do you decide?

A good SDE-3 interview answer is:

Cloud Run offers several execution models, including services, jobs and worker pools; choose the appropriate model before comparing it with GKE. Prefer GKE when Kubernetes-level control or an existing platform justifies its operational cost. Consider Compute Engine for VM requirements and App Engine when its runtime model fits an existing application. Avoid deciding solely from labels such as “stateless” or “background worker.” Cloud Run overview.

How to understand it

Worked explanation: Compare a stateless HTTP service, a batch import and a runtime needing privileged host control. Each imposes different lifecycle and control requirements, so one product ranking cannot answer all three.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What is the senior trade-off? State the required platform capability and who will operate it when failures occur.

30-second version

> Cloud Run offers several execution models, including services, jobs and worker pools; choose the appropriate model before comparing it with GKE. Prefer GKE when Kubernetes-level control or an existing platform justifies its operational cost.

Q48. Explain Pub/Sub delivery, acknowledgements and ordering.

A good SDE-3 interview answer is:

Publishers send to a topic; subscriptions (pull, push or streaming-pull) get **at-least-once** delivery. If not acked within the ack deadline (extendable), the message is redelivered, so handlers must be idempotent. Configure retry policy with backoff and a **dead-letter topic** (`maxDeliveryAttempts`). Ordering keys give per-key order (single-region effect). Exactly-once delivery is available on pull subscriptions in a region but you still need idempotent side effects. Use Spring Cloud GCP's `PubSubTemplate`/message channel adapters.

How to understand it

Worked explanation: An acknowledgement tells the subscription the message is complete. If a subscriber commits work but its acknowledgement is lost, redelivery can happen; ordering keys do not make business side effects atomic.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does exactly-once delivery remove every duplicate business event? No; separately published duplicates and effects outside the delivery protocol still need handling.

30-second version

> Publishers send to a topic; subscriptions (pull, push or streaming-pull) get **at-least-once** delivery. If not acked within the ack deadline (extendable), the message is redelivered, so handlers must be idempotent.

Q49. How do you run Cloud SQL reliably from Spring Boot?

A good SDE-3 interview answer is:

Enable **HA** (regional, automatic failover), automated backups + PITR, read replicas for read scaling (route read-only transactions explicitly), private IP via VPC, and connect through the **Cloud SQL Java Connector** or Auth Proxy (IAM auth, TLS). Size HikariCP carefully (`pool × instances` ≤ DB `max_connections`), set `maxLifetime` below DB/proxy timeouts, and use PgBouncer/connection pooling for Cloud Run bursts.

How to understand it

Worked explanation: Ten instances with 20 pooled connections can ask for 200 database sessions before administrative or other application demand. Autoscaling must respect the database's total capacity.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **IAM** — Identity and Access Management: policies controlling who or what can perform which actions on which resources.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a connector provide network reachability? No; configure the required private/public network path as well as authentication and encryption.

30-second version

> Enable **HA** (regional, automatic failover), automated backups + PITR, read replicas for read scaling (route read-only transactions explicitly), private IP via VPC, and connect through the **Cloud SQL Java Connector** or Auth Proxy (IAM auth, TLS). Size HikariCP carefully (`pool × instances` ≤ DB `max_connections`), set `maxLifetime` below DB/proxy timeouts, and use PgBouncer/connection pooling for Cloud Run bursts.

Q50. When would you choose Spanner or Bigtable over Cloud SQL?

A good SDE-3 interview answer is:

Choose Spanner when distributed relational transactions and horizontal scale justify its cost and design constraints; choose Bigtable for suitable high-volume keyed/wide-column access patterns. Cloud SQL remains a strong fit for many relational workloads. There is no universal few-TB cutoff. Benchmark the actual query mix, consistency needs, key distribution and operational cost. Availability commitments depend on the selected service configuration and SLA conditions. Spanner configuration guarantees.

How to understand it

Worked explanation: A relational transaction spanning business records, a high-volume keyed time-series lookup and an analytical scan imply different storage requirements. Test the hardest query and invariant before choosing a product.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is there a universal data-size cutoff? No; throughput, indexes, consistency, configuration and cost determine suitability, not one TB threshold.

30-second version

> Start from queries, invariants and measured scale. Spanner, Bigtable and Cloud SQL solve different workload problems; neither a headline SLA nor a fixed data-size threshold is enough to choose.

Q51. Explain IAM, service accounts and Workload Identity.

A good SDE-3 interview answer is:

IAM roles collect permissions granted to principals at resource scopes. Use a dedicated least-privilege workload identity and avoid downloaded service-account keys. GKE federation can authorize the Kubernetes workload principal directly for supported resources or use IAM service-account impersonation. Cloud Run can attach a runtime service account. Test permissions and audit access; identity federation alone does not grant permission. GKE identity model.

How to understand it

Worked explanation: Authentication establishes the workload identity; an IAM policy binds permissions to that identity at a resource boundary. Federation avoids distributing a long-lived private key, but does not replace authorization.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.
- **IAM** — Identity and Access Management: policies controlling who or what can perform which actions on which resources.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.
- **Workload Identity** — A mechanism that lets workloads obtain cloud identities without embedding long-lived service-account keys.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Must GKE always impersonate a Google service account? No; supported resources can grant access directly to the federated workload principal.

30-second version

> Federation supplies an identity and short-lived credentials; IAM grants access. Direct workload-principal bindings and service-account impersonation are distinct supported approaches.

Q52. Describe a secure GCP network and edge design.

A good SDE-3 interview answer is:

Global external Application Load Balancer + **Cloud Armor** (WAF, DDoS, rate limits) + managed certificates → services in a VPC with **private IPs**; Private Google Access / Private Service Connect for managed services; Serverless VPC Access or Direct VPC egress for Cloud Run; Cloud NAT for outbound; firewall rules by tags/service accounts; VPC Service Controls for data-exfiltration perimeters; IAP for internal tools.

How to understand it

Worked explanation: Trace inbound traffic through TLS termination and routing, then trace the service's outbound path to data and external APIs. Ingress controls, egress controls and IAM protect different boundaries.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **VPC Service Controls** — Google Cloud controls that constrain access to supported managed services using service perimeters.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a private IP prove authorization? No; a reachable workload must still authenticate and receive only permitted access.

30-second version

> Global external Application Load Balancer + **Cloud Armor** (WAF, DDoS, rate limits) + managed certificates → services in a VPC with **private IPs**; Private Google Access / Private Service Connect for managed services; Serverless VPC Access or Direct VPC egress for Cloud Run; Cloud NAT for outbound; firewall rules by tags/service accounts; VPC Service Controls for data-exfiltration perimeters; IAP for internal tools.

Q53. How would you design CI/CD on GCP?

A good SDE-3 interview answer is:

Commit → **Cloud Build** trigger: `mvn verify` (unit + Testcontainers integration), static analysis/SCA, build image with Jib/Buildpacks, push to **Artifact Registry** with vulnerability scanning, attestations via **Binary Authorization** → **Cloud Deploy** promotes dev → staging → prod with approvals and canary, automated rollback on failed SLO checks. Infrastructure via Terraform with remote state; GitOps (Config Sync/Argo CD) is an alternative.

How to understand it

Worked explanation: Build and validate an artifact once, then promote its immutable digest across environments. A release controller changes deployment state; rollback and verification require explicit policy and signals.

Key terms you should know

- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does the product name guarantee automated rollback? No; explain the configured checks, failure trigger and actual rollback mechanism.

30-second version

> Commit → **Cloud Build** trigger: `mvn verify` (unit + Testcontainers integration), static analysis/SCA, build image with Jib/Buildpacks, push to **Artifact Registry** with vulnerability scanning, attestations via **Binary Authorization** → **Cloud Deploy** promotes dev → staging → prod with approvals and canary,

> automated rollback on failed SLO checks. Infrastructure via Terraform with remote state; GitOps (Config Sync/Argo CD) is an alternative.

Q54. How do you do observability and cost control on GCP?

A good SDE-3 interview answer is:

Structured JSON logs with `trace` fields (Cloud Logging correlates to Cloud Trace), Micrometer → Managed Service for Prometheus/Cloud Monitoring, OpenTelemetry tracing, **SLO monitoring** with burn-rate alerts, error reporting. Cost: right-size requests/limits, committed-use discounts, Spot VMs for batch, autoscaling, log exclusion/retention policies, BigQuery partitioning/clustering, budgets + alerts, and labels for cost attribution.

How to understand it

Worked explanation: Measure cost per useful outcome alongside total spend. A logging change can reduce storage costs but remove diagnostic evidence; right-sizing can save money but create saturation.

Key terms you should know

- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.
- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.
- **trace** — A representation of one request's path through distributed services.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a budget alert cap expenditure? An alert alone reports a threshold; stopping or limiting spend requires a separate control.

30-second version

> Structured JSON logs with `trace` fields (Cloud Logging correlates to Cloud Trace), Micrometer → Managed Service for Prometheus/Cloud Monitoring, OpenTelemetry tracing, **SLO monitoring** with burn-rate alerts, error reporting. Cost: right-size requests/limits, committed-use discounts, Spot VMs for batch, autoscaling, log exclusion/retention policies, BigQuery partitioning/clustering, budgets + alerts, and labels for cost attribution.

I. Docker & Kubernetes for Java

D1 How do an image, container, Pod and Deployment differ

Strong answer: an image packages an application and its runtime filesystem; a container is an executing instance with configured isolation/resources. A Kubernetes Pod groups containers with a shared network/lifecycle context; a Deployment manages a desired set of replaceable Pods and rollout behavior. Local files and process memory are not automatically durable or shared across replicas.

Follow-up: a Service provides a stable access abstraction to selected workloads; it does not make unhealthy application logic correct. Distinguish starting a process, accepting traffic and remaining healthy. Set resource/deadline/shutdown behavior deliberately.

Senior follow-up: trace a Pod restart and a rolling update while requests and database work are in flight. Establish which state is durable and which work can be repeated before tuning probes, autoscaling or rollout parameters in Q55 onward. Kubernetes Pod model.

Q55. How do you build a production Docker image for Spring Boot?

A good SDE-3 interview answer is:

Multi-stage or Jib/Buildpacks; **layered jars** (`java -Djarmode=tools extract --layers`) so dependencies cache separately from app code; minimal/distroless JRE base, pinned tags/digests, **non-root user**, read-only filesystem, no secrets in layers, `.dockerignore`, vulnerability scanning, and exec-form `ENTRYPOINT` so the JVM receives SIGTERM for graceful shutdown.

How to understand it

Worked explanation: Image layers separate stable dependencies from frequently changing application files, improving reuse. Running without root and removing unnecessary tools reduces privileges and image contents; it does not replace patching.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why use exec-form startup? It allows the intended JVM process to receive lifecycle signals directly.

30-second version

> Multi-stage or Jib/Buildpacks; **layered jars** (`java -Djarmode=tools extract --layers`) so dependencies cache separately from app code; minimal/distroless JRE base, pinned tags/digests, **non-root user**, read-only filesystem, no secrets in layers, `.dockerignore`, vulnerability scanning, and exec-form `ENTRYPOINT` so the JVM receives SIGTERM for graceful shutdown.

Q56. Why do Java containers get `OOMKilled`, and how do you tune the JVM?

A good SDE-3 interview answer is:

Container memory includes heap, metaspace, stacks, direct buffers, code cache, GC structures and other native allocations. Setting Xmx equal to the container limit leaves no headroom when the heap grows; a kill is a risk, not a guaranteed immediate outcome. Measure native and heap demand before choosing MaxRAMPercentage or Xmx. Bound threads and buffers, inspect termination reasons, and distinguish a Java OutOfMemoryError from an external cgroup OOM kill, which may produce no heap dump. CPU throttling can also worsen latency and GC progress.

How to understand it

Worked explanation: A container accounts for heap plus native allocations, stacks and other process memory. A heap below its maximum does not prove the process is below its cgroup limit.

Key terms you should know

- **CAP** — The distributed-systems theorem describing the trade-off among consistency, availability and partition tolerance when a network partition occurs.
- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.
- **heap** — JVM memory where Java objects are allocated.
- **JIT** — Just-in-time compilation: runtime compilation of frequently executed bytecode into optimized native machine code.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Will every container kill produce a Java heap dump? No; an external OOM kill can terminate the process before Java handles any exception.

30-second version

> The heap is only part of process memory. Leave measured headroom, inspect native usage and distinguish JVM allocation failure from a container kill. A fixed heap percentage is a starting hypothesis, not a universal safe setting.

Q57. Core Kubernetes objects and probes you must explain.

A good SDE-3 interview answer is:

Pod (smallest unit) → **Deployment/ReplicaSet** (rolling updates) → **Service** (stable virtual IP, load balancing) → **Ingress/Gateway** (L7 routing); **ConfigMap/Secret**, **HPA** (CPU/custom metrics), **PDB**, **StatefulSet**, **Job/CronJob**. Probes: **startup** (slow JVM boot), **readiness** (take out of rotation; Boot `/actuator/health/readiness`), **liveness** (restart only on unrecoverable state; never include downstream dependencies or you cause restart storms).

How to understand it

Worked explanation: A Deployment maintains desired replicas, a Service selects endpoints, and readiness determines whether an endpoint should receive traffic. A liveness failure asks for restart, which is a different action.

Key terms you should know

- **Deployment** — A Kubernetes controller that manages replicated Pods and supports controlled rollout of new versions.
- **Pod** — Kubernetes' smallest deployable unit, containing one or more containers that share networking and storage context.
- **Service** — A stable Kubernetes networking abstraction that exposes a group of Pods behind a stable endpoint.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why separate startup and liveness? Slow healthy initialization should not repeatedly trigger restarts before the application can become ready.

30-second version

> **Pod** (smallest unit) → **Deployment/ReplicaSet** (rolling updates) → **Service** (stable virtual IP, load balancing) → **Ingress/Gateway** (L7 routing); **ConfigMap/Secret**, **HPA** (CPU/custom metrics), **PDB**, **StatefulSet**, **Job/CronJob**. Probes: **startup** (slow JVM boot), **readiness** (take out of rotation; Boot `/actuator/health/readiness`), **liveness** (restart only on unrecoverable state; never include downstream dependencies or you cause restart storms).

Q58. A pod is in `CrashLoopBackOff`. How do you troubleshoot?

A good SDE-3 interview answer is:

`kubectl describe pod` (events, exit code, OOMKilled, probe failures), `kubectl logs --previous`, check config/secret mounts and env vars, image pull and tag, resource limits, failing liveness/startup probes (JVM too slow to start → raise `startupProbe`), dependency connectivity (DNS, network policy, DB credentials), then `kubectl exec`/ephemeral debug container. Fix root cause and add alerting on restart count.

How to understand it

Worked explanation: CrashLoopBackOff describes delayed restart attempts after repeated exits. Read the previous container's logs and termination reason to distinguish application failure, a killed process and a probe-driven restart.

Key terms you should know

- **OOMKilled** — A container termination status indicating the operating system killed the process because it exceeded its memory limit.
- **Pod** — Kubernetes' smallest deployable unit, containing one or more containers that share networking and storage context.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Will increasing the restart delay fix it? No; repair the cause and verify the workload reaches and remains ready.

30-second version

> `kubectl describe pod` (events, exit code, OOMKilled, probe failures), `kubectl logs --previous`, check config/secret mounts and env vars, image pull and tag, resource limits, failing liveness/startup probes (JVM too slow to start → raise `startupProbe`), dependency connectivity (DNS, network policy, DB credentials), then `kubectl exec`/ephemeral debug container. Fix root cause and add alerting on restart count.

Next: Vol. 3

Scenario-based/behavioral questions, system-design walkthroughs and more coding programs with solutions.

Part 04 - Question Bank Vol. 3 - Scenarios, design and coding

Senior Java Question Bank, Vol. 3

Scenarios · System Design · Coding Programs (Q59–Q74 + 3 designs + 6 programs) For scenarios use: **Detect → Mitigate → Root cause → Fix → Prevent.** Always quote a metric.

J. Production Scenarios

Q59. Production CPU is at 95%. What do you do?

A good SDE-3 interview answer is:

First mitigate (scale out, or roll back the latest deploy if correlated). Then find the hot threads: `top -H -p <pid>`, convert the thread ID to hex, match it in `jstack`/`jcmd Thread.print`; or use async-profiler/JFR for a flame graph. Typical causes: infinite/regex-backtracking loops, excessive GC (check GC logs), serialization hot spots, N+1 queries, or a busy-spinning consumer. Fix, add a load test and CPU alert.

How to understand it

Worked explanation: High CPU is a symptom; a profile identifies where execution time goes. Compare application work with GC, retries and busy loops, then choose mitigation that does not overload downstream services.

Key terms you should know

- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.
- **N+1** — A query pattern where one query loads N parent rows and then N additional queries load related data, causing excessive database round trips.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why capture several samples? A single stack can miss intermittent hot paths and cannot establish their share of CPU time.

30-second version

> First mitigate (scale out, or roll back the latest deploy if correlated). Then find the hot threads: `top -H -p <pid>`, convert the thread ID to hex, match it in `jstack`/`jcmd Thread.print`; or use async-profiler/JFR for a flame graph.

Q60. Heap keeps growing and the pod is OOMKilled. How do you investigate?

A good SDE-3 interview answer is:

Confirm the type: heap (Java `OutOfMemoryError`) vs container kill (native memory/limit). Capture a heap dump (`-XX:+HeapDumpOnOutOfMemoryError`, `jcmd GC.heap_dump`), open it in Eclipse MAT, and read the leak suspects/dominator tree. Common roots: unbounded caches or static maps, ThreadLocals in pools, unclosed streams/connections, listener registration without removal, large result sets. For native growth use NMT (`-XX:NativeMemoryTracking`). Fix with bounded caches (Caffeine max size/TTL) and pagination/streaming.

How to understand it

Worked explanation: Track the live set after collections and follow retaining paths from GC roots. A large temporary allocation and an object retained forever require different fixes.

Key terms you should know

- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.
- **heap** — JVM memory where Java objects are allocated.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if the heap is stable but memory rises? Investigate native/direct memory, threads and container accounting instead of assuming a heap leak.

30-second version

> Confirm the type: heap (Java `OutOfMemoryError`) vs container kill (native memory/limit). Capture a heap dump (`-XX:+HeapDumpOnOutOfMemoryError`, `jcmd GC.heap_dump`), open it in Eclipse MAT, and read the leak suspects/dominator tree.

Q61. An endpoint's p99 jumped from 200 ms to 3 s. Walk me through it.

A good SDE-3 interview answer is:

Start from the symptom: is it all requests or specific ones, and since when (deploy, traffic, data growth)? Use distributed traces to find the slow span. Check DB (slow-query log, `EXPLAIN`, missing index, lock waits), connection-pool wait time (Hikari metrics), GC pauses, downstream latency, thread-pool saturation and cold caches. Fix the dominant span, not guesses; then add a latency SLO alert and a regression performance test.

How to understand it

Worked explanation: A percentile describes the slow tail of a distribution. Break latency into queueing, connection acquisition, dependency calls and local processing, then compare the changed component with earlier behavior.

Key terms you should know

- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.
- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.
- **span** — One timed operation within a distributed trace, such as an HTTP call or database query.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can averages prove the fix? No; validate the relevant percentiles, errors and load distribution under comparable traffic.

30-second version

> Start from the symptom: is it all requests or specific ones, and since when (deploy, traffic, data growth)? Use distributed traces to find the slow span.

Q62. "Connection is not available, request timed out after 30000ms." Diagnose.

A good SDE-3 interview answer is:

Hikari pool is exhausted: either too few connections for the load, or connections are held too long. Look for transactions spanning remote calls, `@Transactional` on large methods, leaked connections (`leakDetectionThreshold`), long queries or lock waits, and Open-Session-In-View. Don't simply raise `maximumPoolSize`: total connections across all pods must stay under the DB limit. Shorten transaction scope, add timeouts, and tune pool sizing (small pools are usually faster).

How to understand it

Worked explanation: Borrowing a connection waits when no usable session is available. Slow SQL, long transactions, leaked ownership or failed connection creation can all produce this symptom.

Key terms you should know

- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why not immediately enlarge the pool? More concurrent SQL can saturate the same database and worsen latency across every instance.

30-second version

> Hikari pool is exhausted: either too few connections for the load, or connections are held too long. Look for transactions spanning remote calls, `@Transactional` on large methods, leaked connections (`leakDetectionThreshold`), long queries or lock waits, and Open-Session-In-View.

Q63. Customers were charged twice. How do you prevent it?

A good SDE-3 interview answer is:

Causes: client retries, gateway timeouts, at-least-once messaging. Defence: an `Idempotency-Key` on the request stored with a unique constraint and the stored response; state machine for payment (CREATED → AUTHORISED → CAPTURED) with optimistic locking; idempotent consumers keyed by event ID; call the PSP with its own idempotency key; reconciliation job comparing PSP records with ours. Then add detection alerts for duplicates.

How to understand it

Worked explanation: The provider may charge successfully while the caller times out before saving the response. A repeated request must refer to the same durable operation and provider idempotency key.

Key terms you should know

- **idempotency** — The property that repeating the same logical request produces the same intended outcome rather than creating duplicate effects.
- **optimistic locking** — Concurrency control that detects conflicting updates, commonly using a version column, instead of holding a database lock throughout the transaction.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if the same key has different payment details? Reject the mismatch rather than returning or executing a result for another request.

30-second version

> Causes: client retries, gateway timeouts, at-least-once messaging. Defence: an `Idempotency-Key` on the request stored with a unique constraint and the stored response; state machine for payment (CREATED → AUTHORISED → CAPTURED) with optimistic locking; idempotent consumers keyed by event ID; call the PSP with its own idempotency key; reconciliation job comparing PSP records with ours.

Q64. How do you migrate the database schema with zero downtime?

A good SDE-3 interview answer is:

Expand → migrate → contract. Deploy a backward-compatible schema first (add nullable column/new table), release code that writes to both (or reads either), backfill in batches, switch reads, and only later drop the old column once no running version depends on it. Use Flyway/Liquibase, avoid long locks (`CREATE INDEX CONCURRENTLY`), and test on production-sized data. Never couple a destructive migration with the same release.

How to understand it

Worked explanation: During rollout, old code still uses the old schema while new code introduces the replacement. Expand support first, backfill with verification, then retire old readers/writers before contraction.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does application rollback restore dropped data? No; preserve a compatible rollback window and test migration failure/restart behavior.

30-second version

> Expand → migrate → contract. Deploy a backward-compatible schema first (add nullable column/new table), release code that writes to both (or reads either), backfill in batches, switch reads, and only later drop the old column once no running version depends on it.

Q65. Kafka consumer lag keeps growing. What now?

A good SDE-3 interview answer is:

Check whether it is a producer spike (temporary) or a slow consumer (persistent). Look at per-partition lag, processing time per record, rebalances, DB/downstream latency, and errors/retries. Options: scale consumers (limited by partitions), batch processing, parallelise per key safely, optimise the slow dependency, increase partitions (with ordering implications), or shed non-critical work. Alert on lag *growth rate* and time-to-drain.

How to understand it

Worked explanation: Lag grows when arrival rate exceeds completion rate. A single slow partition can dominate even while other consumers are idle, so inspect partition-level work rather than only group averages.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you estimate recovery? Compare backlog with sustainable processing capacity above the continuing arrival rate.

30-second version

> Check whether it is a producer spike (temporary) or a slow consumer (persistent). Look at per-partition lag, processing time per record, rebalances, DB/downstream latency, and errors/retries.

Q66. REST or gRPC or messaging between services?

A good SDE-3 interview answer is:

REST (JSON) for public/external APIs and simple CRUD: ubiquitous, cacheable, human-debuggable. gRPC for internal, latency-sensitive, strongly typed contracts with streaming and code generation (HTTP/2, Protobuf), but harder through browsers/gateways. Messaging for asynchronous workflows, fan-out and decoupling in time. Mixed in practice: REST at the edge, gRPC for hot internal paths, events for cross-domain state changes.

How to understand it

Worked explanation: A synchronous call couples the caller's completion to the callee's availability. A message decouples completion in time but adds delivery, freshness and workflow-state questions.

Key terms you should know

- **gRPC** — A high-performance RPC framework commonly using HTTP/2 and Protocol Buffers for service-to-service communication.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is messaging automatically more reliable? Only with durable publication, bounded retries, idempotent processing and an operational recovery path.

30-second version

> REST (JSON) for public/external APIs and simple CRUD: ubiquitous, cacheable, human-debuggable. gRPC for internal, latency-sensitive, strongly typed contracts with streaming and code generation (HTTP/2, Protobuf), but harder through browsers/gateways.

Q67. Describe a major incident you led.

A good SDE-3 interview answer is:

Use STAR with numbers: *Situation* (checkout error rate 18%, revenue impact), *Task* (incident commander), *Action* (declared severity, assigned roles, rolled back, communicated every 15 min, preserved evidence), *Result* (restored in 22 min), then **blameless post-mortem**: root cause (connection leak after a library upgrade), five-whys, action items (alerts, canary, load test) and what you personally changed afterwards.

How to understand it

Worked explanation: Describe your actual responsibility and decisions in the incident, separate from the team's work. Explain what evidence supported mitigation, what restored customer service and how recurrence was reduced.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What if you lack exact numbers? Use honest ranges or qualitative evidence; never adopt the guide's illustrative incident as your own.

30-second version

> Use STAR with numbers: *Situation* (checkout error rate 18%, revenue impact), *Task* (incident commander), *Action* (declared severity, assigned roles, rolled back, communicated every 15 min, preserved evidence), *Result* (restored in 22 min), then **blameless post-mortem**: root cause (connection leak after a library upgrade), five-whys, action items (alerts, canary, load test) and what you personally changed afterwards.

Q68. How do you upgrade from Java 8/Spring Boot 2 to Java 21/Boot 3?

A good SDE-3 interview answer is:

Separate the JDK, framework, dependency and behavioral changes. For a legacy Boot 2 application, Boot 2.7 may be an intermediate migration step, not the final support target. Choose an actively supported destination and verify its dependency matrix. Boot 3 moves Jakarta EE APIs such as persistence and servlet packages from `javax` to `jakarta`; do not globally rename Java SE packages such as `javax.sql` or `javax.crypto`. Revalidate Hibernate queries, serialization, security configuration and performance. Boot 4 uses Spring Framework 7 and has its own migration requirements; “modern Spring” is not synonymous with Boot 3. Boot 4 requirements.

How to understand it

Worked explanation: A JDK upgrade can change runtime behavior while a Boot upgrade changes framework contracts and managed dependencies. Separate these dimensions so a failing test points to a manageable cause.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why not globally rename javax? Java SE packages such as javax.sql remain; only the relevant Jakarta EE APIs migrated.

30-second version

> Separate the JDK, framework, dependency and behavioral changes. For a legacy Boot 2 application, Boot 2.7 may be an intermediate migration step, not the final support target.

Q69. How do you design a multi-tenant SaaS service?

A good SDE-3 interview answer is:

Choose isolation by risk and cost: shared schema with `tenant_id` (cheapest; enforce via Hibernate filters/row-level security), schema-per-tenant (balanced), or database-per-tenant (strong isolation, noisy-neighbour control, higher cost). Resolve the tenant from the JWT claim in a filter, propagate in context (MDC, `TenantContext` cleaned in `finally`), key caches by tenant, rate-limit per tenant, and test for cross-tenant leakage.

How to understand it

Worked explanation: Tenant identity must constrain database access, cache keys and background work. A correctly filtered query is insufficient if a later cache lookup ignores the tenant.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a Hibernate filter cover every access path? No; audit native SQL and other bypasses, and test authorization across tenant boundaries.

30-second version

> Choose isolation by risk and cost: shared schema with `tenant_id` (cheapest; enforce via Hibernate filters/row-level security), schema-per-tenant (balanced), or database-per-tenant (strong isolation, noisy-neighbour control, higher cost). Resolve the tenant from the JWT claim in a filter, propagate in context (MDC, `TenantContext` cleaned in `finally`), key caches by tenant, rate-limit per tenant, and test for cross-tenant leakage.

Q70. Reports need data from five services. How do you build them?

A good SDE-3 interview answer is:

Avoid cross-service joins at query time. Publish domain events to build a **read model** (a reporting store, BigQuery or Elasticsearch) via a streaming pipeline (Dataflow, Kafka Connect/CDC). Accept eventual consistency, expose freshness, and version event schemas. API composition is acceptable for small, low-volume cases but couples availability.

How to understand it

Worked explanation: A report assembled synchronously from five services inherits five latency and availability dependencies. A projection makes reads simpler by accepting a defined delay between source changes and reporting visibility.

Key terms you should know

- **CDC** — Change Data Capture: publishing database changes by reading the database's change log or transaction log.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you recover a bad projection? Retain a rebuildable source and reconcile missing, duplicate and out-of-order updates.

30-second version

> Avoid cross-service joins at query time. Publish domain events to build a **read model** (a reporting store, BigQuery or Elasticsearch) via a streaming pipeline (Dataflow, Kafka Connect/CDC).

Q71. You disagree with your architect's design. What do you do?

A good SDE-3 interview answer is:

Show collaborative maturity: understand their constraints, write down the options with trade-offs and data (a quick prototype or benchmark), propose a decision record (ADR) and a reversible experiment, then **disagree and commit** once a decision is made, and revisit with evidence if it fails. Never go around people or argue opinions without data.

How to understand it

Worked explanation: Turn disagreement into a decision about explicit constraints. Compare two feasible designs against the same criteria and document what evidence would justify revisiting the choice.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What makes the answer credible? A real example of listening, changing or defending your view, and supporting the resulting decision.

30-second version

> Show collaborative maturity: understand their constraints, write down the options with trade-offs and data (a quick prototype or benchmark), propose a decision record (ADR) and a reversible experiment, then **disagree and commit** once a decision is made, and revisit with evidence if it fails. Never go around people or argue opinions without data.

Q72. How do you improve code quality across a team?

A good SDE-3 interview answer is:

Automate first: formatter, Checkstyle/SpotBugs/Sonar quality gates, coverage on new code, dependency scanning, in CI. Then human practices: small PRs, review checklist (design, tests, security, observability), pairing, ADRs, a testing pyramid, runbooks, and tech-debt budget (e.g., 15–20% per sprint). Mentor through review comments that explain *why*.

How to understand it

Worked explanation: Choose one recurring defect class, add a useful automated check, and coach reviewers on the underlying reasoning. Measure fewer escapes or faster reviews rather than the number of rules added.

Key terms you should know

- **observability** — The ability to understand internal system behavior from telemetry such as logs, metrics and traces.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a quality gate replace review? No; it detects encoded conditions and still needs judgment about behavior and design.

30-second version

> Automate first: formatter, Checkstyle/SpotBugs/Sonar quality gates, coverage on new code, dependency scanning, in CI. Then human practices: small PRs, review checklist (design, tests, security, observability), pairing, ADRs, a testing pyramid, runbooks, and tech-debt budget (e.g., 15–20% per sprint).

Q73. Deadline is tight and scope is large. How do you handle it?

A good SDE-3 interview answer is:

Clarify the business goal, slice into a thin end-to-end MVP, rank by value/risk, surface risks and options early (scope, date, resources: pick two), use feature flags to ship incrementally, avoid skipping tests on critical paths, and record deliberate shortcuts as tracked debt. Communicate proactively rather than surprise stakeholders.

How to understand it

Worked explanation: Identify the smallest complete user outcome and make excluded scope visible. Reducing scope can preserve validation of that outcome; secretly omitting critical checks changes delivery risk.

Key terms you should know

- **Business goal** — The outcome the team is trying to achieve, used to prioritize scope and evaluate delivery trade-offs.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Do more people always recover the date? No; onboarding and coordination can increase the critical path, so explain realistic options.

30-second version

> Clarify the business goal, slice into a thin end-to-end MVP, rank by value/risk, surface risks and options early (scope, date, resources: pick two), use feature flags to ship incrementally, avoid skipping tests on critical paths, and record deliberate shortcuts as tracked debt. Communicate proactively rather than surprise stakeholders.

Q74. How do you decide between monolith and microservices for a new product?

A good SDE-3 interview answer is:

Start with a **modular monolith** unless there are clear reasons: independent scaling, separate team ownership, different tech/availability needs, regulatory isolation. Microservices add network failures, distributed data, deployment and observability overhead; a well-structured monolith with enforced module boundaries (ArchUnit, Spring Modulith) can be split later when pressure is proven.

How to understand it

Worked explanation: A modular monolith can keep inventory and billing boundaries explicit within one deployment. Splitting later is easier if ownership and APIs are already clear.

Key terms you should know

- **Modular monolith** — A single deployment with explicit internal module boundaries.
- **Microservice** — An independently deployable service with a defined responsibility and data ownership.
- **Distributed monolith** — Separate services whose coupling forces coordinated changes or deployment.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What evidence justifies a service split? A concrete need for independent change, scaling or isolation that outweighs distributed-operation costs.

30-second version

> Start with a **modular monolith** unless there are clear reasons: independent scaling, separate team ownership, different tech/availability needs, regulatory isolation. Microservices add network failures, distributed data, deployment and observability overhead; a well-structured monolith with enforced module boundaries (ArchUnit, Spring Modulith) can be split later when pressure is proven.

K. System Design Walkthroughs (condensed)

Use the requirements-first design sequence before the examples: requirements → estimates → API/data → architecture → failure/scaling analysis → trade-offs. These prompts assume that interview process.

Design 1: URL Shortener. Requirements: 100M new URLs/month, 10:1 read:write, low-latency redirect. API: `POST /urls`, `GET /{code}` → 301/302. Code: base62 of a unique ID (a distributed ID generator or pre-allocated ranges per node; avoid hash-collision retries). Storage: key-value (Bigtable/DynamoDB-style) `code → longUrl, ttl`; cache hot codes in Redis (read-heavy, ~95% hit rate); CDN for popular links. Scale: stateless app tier behind a load balancer, shard by code, async click analytics via Pub/Sub. Trade-offs: 301 (cacheable, loses analytics) vs 302; custom aliases need a uniqueness check; abuse protection with rate limiting and malware checks.

Design 2: Notification Service. API accepts event + user + channel. Flow: validate → persist (outbox) → Pub/Sub/Kafka topic per channel (email, SMS, push) → channel workers with provider adapters, retry with backoff, DLQ, per-provider circuit breaker and fallback provider. Preferences/DND/ templates in a store; deduplicate with an idempotency key; rate-limit per user; track delivery status via webhooks. Priorities: separate topics for OTP (high priority) vs marketing. Observability: delivery latency SLO, failure rate by provider.

Design 3: Order & Payment (Saga). Orchestrated saga: `Order(PENDING)` → reserve inventory → authorise payment → confirm order; on failure run compensations (release stock, void payment). Each service uses a local transaction + outbox; events carry the order ID as the Kafka key to keep order; consumers are idempotent. Timeouts for stuck sagas, a state machine persisted by the orchestrator, and a reconciliation job. Expose order status via a read model; clients poll or receive a push.

L. Coding Programs with Solutions

1. Print odd/even numbers alternately using two threads

Derive the approach: the shared counter and whose turn it is form one invariant. Test the predicate and update the counter under the same monitor; a thread that cannot proceed waits and releases that monitor.

Why this, not the alternatives: sleeping is timing-dependent and does not establish the required ordering. Busy-waiting wastes CPU. Two semaphores can encode turns more directly, while a single-thread loop is the practical choice when parallel work serves no purpose.

What changes: replacing the predicate loop with a one-time test permits spurious wakeups to violate the rule. `notify` can wake an unsuitable waiter in a generalized version; `notifyAll` plus predicate checks is easier to reason about. When one worker is interrupted, cancel and wake its partner; otherwise the survivor can wait forever.

Trace and checks: even starts with n=1, waits; odd prints 1 and notifies; even prints 2. Test either starting order, cancellation and final termination. O(max) work, O(1) shared state; lock scheduling affects elapsed time.

Strong answer signal: identify the shared predicate and give a cancellation timeline.

```
class OddEven {
    private int n = 1; private boolean cancelled;
    private final int max = 10; private final Object lock = new Object();
    void print(boolean odd) {
        synchronized (lock) {
            while (!cancelled && n <= max) {
                if ((n % 2 == 1) == odd) { System.out.println(Thread.currentThread().getName() + ":
" + n++); lock.notifyAll(); }
                else { try { lock.wait(); } catch (InterruptedException e) {
                    cancelled = true; lock.notifyAll();
                    Thread.currentThread().interrupt(); return;
                } }
            }
            lock.notifyAll();
        }
    }
    public static void main(String[] a) {
        OddEven o = new OddEven();
        new Thread(() -> o.print(true), "odd").start();
        new Thread(() -> o.print(false), "even").start();
    }
}
```

Say: `wait` always in a `while` loop (spurious wakeups); the lock must be held to call `wait/notify`.

2. Retry utility with exponential backoff + jitter

Derive the approach: a transient failure may recover, but immediate synchronized retries increase pressure. Retry only safe operations, with a bounded attempt count, capped delay, jitter and an overall deadline.

Why this, not the alternatives: fixed delays synchronize clients; immediate retries amplify overload; indefinite retries consume resources and hide permanent errors. Fail fast for invalid input or authorization failure. A library can centralize policy, but cannot decide business idempotency for you.

What changes: the short snippet is a sketch, not a complete retry policy: `isTransient` is application-specific, arguments need validation, exponential arithmetic needs saturation, and there is no overall deadline. `InterruptedException` should propagate rather than become retryable. A timed-out payment may already have completed, so reuse a stable idempotency key and reconcile uncertain outcomes.

Trace and checks: fail transiently twice then succeed; assert exactly three invocations. Also test permanent failure, attempt exhaustion, cancellation and overflow bounds using an injected sleeper/ticker. Work is bounded by attempts plus task cost; waiting contributes to the deadline.

Strong answer signal: explain when retrying changes correctness, not merely its delay formula.

```
static <T> T retry(Callable<T> task, int maxAttempts, long baseMs) throws Exception {
    for (int attempt = 1; ; attempt++) {
        try { return task.call(); }
        catch (Exception e) {
            if (attempt >= maxAttempts || !isTransient(e)) throw e;
            long delay = (long) (baseMs * Math.pow(2, attempt - 1));
            Thread.sleep(ThreadLocalRandom.current().nextLong(delay / 2, delay + 1)); // jitter
        }
    }
}
```

Say: retry only idempotent operations and transient errors; cap total time; in Spring use Resilience4j `@Retry`.

3. Top K frequent elements (min-heap, O(n log k))

Derive the approach: first count values, then retain only the k largest frequencies. A min-heap exposes the weakest retained candidate for eviction.

Why this, not the alternatives: sorting d distinct values costs O(d log d) and is simple when k is large. A heap costs O(d log k), after expected O(n) counting, and suits small k. Frequency buckets can use O(n + d) space for expected linear processing. A max-heap holding everything uses more state than needed.

What changes: heap iteration is not sorted; explicitly sort the final k if ranked output is required. Ties need a declared rule; for deterministic results include a secondary key in both selection and output. Exact online counts require updating priorities, not just mutating heap entries in place.

Trace and checks: counts A=4, B=2, C=3, k=2 retain A and C. Test k=0, k larger than d, all ties and negative values. Total space O(d + k); counting dominates storage.

Strong answer signal: distinguish input size n, distinct count d and requested count k.

```
List<Integer> topK(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int x : nums) freq.merge(x, 1, Integer::sum);
    PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>
(Map.Entry.comparingByValue());
    for (var e : freq.entrySet()) { pq.offer(e); if (pq.size() > k) pq.poll(); }
    return pq.stream().map(Map.Entry::getKey).toList();
}
```

Contract: validate `0 <= k <= distinctCount` if exactly k results are required. Heap iteration is not ranked output; define ordering and tie-breaking. *Alternative:* frequency buckets trade more space for linear expected processing.

4. Group anagrams

Derive the approach: two words belong together when their character multisets match. Build a canonical representation, then group by that representation.

Why this, not the alternatives: sorting each word works for a broad declared character alphabet and yields an immutable string key. A fixed count vector avoids sorting for a known alphabet. Comparing every pair of words repeats work and is roughly quadratic in word count.

What changes: concatenating counts without separators can create ambiguous keys; using `int[]` directly as a HashMap key compares array identity unless wrapped correctly. Case folding and Unicode normalization are business decisions, not automatic anagram rules. Sorted UTF-16 units are not a complete Unicode grapheme model.

Trace and checks: `eat`, `tea`, `bat` form two groups. Test duplicates, empty strings, mixed case and agreed character boundaries. For n words of maximum length k, sorting keys costs O(n k log k), plus retained keys and output.

Strong answer signal: prove that equal keys mean exactly the intended equivalence relation.

```
Collection<List<String>> groupAnagrams(String[] words) {
    return Arrays.stream(words).collect(Collectors.groupingBy(w -> {
        char[] c = w.toCharArray(); Arrays.sort(c); return new String(c); })).values();
}
```

Complexity: O(n · k log k) for n words of length at most k, plus output storage. A 26-count key gives O(n · k) only under an explicit lowercase-English alphabet contract; encode the counts without ambiguous concatenation.

5. Custom `@LogExecutionTime` with Spring AOP

Derive the approach: timing is repeated across methods, so intercept the method boundary and record elapsed monotonic time in `finally` to cover failure as well as success.

Why this, not the alternatives: hand-written timers are transparent but repetitive. A servlet filter measures an HTTP boundary; it does not measure arbitrary service methods. For aggregated latency and percentiles, use a metrics instrument rather than one log line per call.

What changes: proxy self-invocation bypasses this advice. A method returning a future/publisher can finish before its work does; measuring return time is not completion latency. Do not swallow the exception or use a return in `finally`. Avoid payload logging and expensive high-volume logs.

Trace and checks: a method that throws must still record a duration and propagate the same failure. Test proxied external calls, self-invocation and asynchronous completion separately. Timing adds constant bookkeeping, while logging and the target method have their own costs.

Strong answer signal: define exactly which interval the measurement represents.

```
@Target(METHOD) @Retention(RUNTIME) public @interface LogExecutionTime {}

@Aspect @Component @Slf4j
class TimingAspect {
    @Around("@annotation(LogExecutionTime)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature().toShortString(), (System.nanoTime()
- start) / 1_000_000); }
    }
}
```

Say: needs `spring-boot-starter-aop`; works only through the proxy (no self-invocation); prefer Micrometer `@Timed` for real metrics.

6. Create and fix a deadlock

Derive the approach: opposite lock acquisition orders permit a cycle. Impose a total order on accounts and acquire both locks in that order, independent of transfer direction.

Why this, not the alternatives: one global lock is simpler but serializes unrelated transfers. `tryLock` with timeout avoids endless waiting but introduces retry/livelock policy. Per-account ordering preserves concurrency between independent accounts.

What changes: IDs must be unique and stable and must identify canonical lock objects. Equal IDs on different objects or different lock instances for the same account break the assumed order. A process-local lock does not coordinate other JVMs. Also, two Java mutations are not automatically transactional: if credit fails after debit, locking alone does not undo the debit.

Trace and checks: A→B and B→A both lock the lower-ID object first. Test same account, insufficient funds, invalid amount, overflow and failure between mutations; validate before mutating or use a transactional persistence boundary. O(1) local work, excluding waiting.

Strong answer signal: separate deadlock prevention from atomic business effects.

```
// Deadlock: thread A locks a then b; thread B locks b then a.
// Fix: impose a global lock order.
void transfer(Account from, Account to, long amt) {
    Account first = from.id < to.id ? from : to, second = first == from ? to : from;
    synchronized (first) { synchronized (second) { from.debit(amt); to.credit(amt); } }
}
```

Say: alternatives are `tryLock(timeout)` with back-off, or a single-writer design; detect with `jstack` ("Found one Java-level deadlock").

7. Idempotent event consumer with PostgreSQL and Spring JDBC

Derive the approach: redelivery must not apply a business change twice. A unique event marker and local business update must commit together; let the database arbitrate concurrent duplicate claims.

Why this, not the alternatives: `exists` then `insert` races; an in-memory set disappears on restart and is not shared across instances. Committing the marker separately can permanently suppress an event whose business update failed. An outbox solves reliable publication and complements, rather than replaces, consumer deduplication.

What changes: changing consumer-name scope can make old events appear new. Expiring dedupe records before the allowed replay horizon re-enables old effects. The same event ID with different payloads should be rejected or investigated. External payments require the provider's own idempotency contract.

Trace and checks: two deliveries race; one inserts and updates, the other observes the conflict. If the winner rolls back, a later delivery must still apply the event. Test commit-before-ack crash, business rollback and concurrent duplicates using PostgreSQL. Unique-index and business-query cost dominate.

Strong answer signal: explain the crash windows and identify the atomic boundary.

```
CREATE TABLE processed_event (
    consumer_name text NOT NULL,
    event_id text NOT NULL,
    PRIMARY KEY (consumer_name, event_id)
);
```

```
@Service
@RequiredArgsConstructor
class PaymentEventHandler {
    private final JdbcTemplate jdbc;
    private final PaymentService payments;

    @Transactional
    public void handle(PaymentEvent event) {
        int inserted = jdbc.update("""
            INSERT INTO processed_event (consumer_name, event_id)
            VALUES (?, ?)
            ON CONFLICT (consumer_name, event_id) DO NOTHING
            """, "payment-handler-v1", event.eventId());
        if (inserted == 0) return;
        payments.applyLocal(event);
    }
}
```

`applyLocal` must participate in the same physical transaction and datasource; it must not perform an uncoordinated external payment or use `REQUIRES_NEW`. A business failure must roll back both writes. Acknowledge the broker only after commit. PostgreSQL's conflict clause skips the specified duplicate without aborting the transaction; unrelated errors still fail. PostgreSQL INSERT.

Do not catch a JPA constraint violation and assume the surrounding transaction remains usable: it may be rollback-only and fail on commit. Spring propagation. Define event-ID scope and payload consistency; keep dedupe records for the supported replay horizon. Test concurrent duplicates, failure after dedupe insertion,

replay after commit, and unrelated constraint failures against the real database.

Final Readiness Checklist

- ~100 Q&As covered across the course and Vols. 1–3; explain 20 of them aloud without notes
- Code LRU, rate limiter, odd/even, top-K, and the idempotent consumer in under 10 minutes each
- Rehearse 5 STAR stories and 3 system designs against a timer

---

Part 05 - Question Bank Vol. 4 - Modern Java and the Spring ecosystem

Senior Java Question Bank, Vol. 4

Modern Java · Design · Spring Ecosystem · Databases · Security · Quality (Q75–Q98 + 4 programs)

M. Modern Java & Design

Stream questions before pitfalls

ST1 How do lambdas, functional interfaces and streams fit together

Foundation: A lambda supplies behavior for a functional interface, such as a predicate or mapping function. A stream describes a computation over a source; it is not a collection storing the results. Explain source → intermediate operations such as filter/map → terminal operation such as collect/reduce.

Mid-level follow-up: Intermediate operations are lazy. A terminal operation drives evaluation, which can short-circuit or omit work through valid optimizations. Do not rely on side effects in a mapping function to perform required work.

ST2 How do you choose map, flatMap, reduction and collection

Foundation: map transforms each element; flatMap flattens resulting streams. Reduction combines values; collection accumulates a result such as a list or grouped map.

Senior follow-up: State the empty-input result, duplicate-key policy, encounter-order requirement and resource ownership. Parallel execution requires suitable non-interfering behavior and associative reduction; measure before choosing it. Continue with Q75–Q77 for language features, pitfalls and absence handling.

Baseline/reference: Java 21 Stream API. These questions explain contracts; they add no runnable program.

Q75. Which modern Java features do you use, and why?

A good SDE-3 interview answer is:

Records (Java 16) are shallowly immutable data carriers with generated accessors, equality and display methods; mutable components still require deliberate copying/ownership. Sealed types (17) restrict variants, and pattern matching for switch (21) can make handling those variants explicit. Use these for suitable DTOs and domain results, alongside features such as text blocks and virtual threads when their contracts fit. Explain a concrete benefit and runtime baseline rather than assuming every feature is always preferable. Record contract.

How to understand it

Worked explanation: A record removes boilerplate for a data shape; a sealed hierarchy restricts its permitted variants. These solve different problems. A record containing a mutable list still exposes mutable contents unless ownership is handled.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Which feature would you defend in an interview? Explain one real simplification and its compatibility or mutability limit, rather than reciting release names.

30-second version

> Records reduce data-carrier boilerplate but are shallowly immutable. Sealed types and pattern matching express bounded alternatives. Justify a feature through clearer behavior, its limits and the supported JDK.

Q76. What are the common Stream pitfalls?

A good SDE-3 interview answer is:

Streams are lazy: nothing runs without a terminal operation, and a stream can be consumed only once. Pitfalls: side effects in `map/peek` (breaks with parallelism), `Collectors.toMap` throwing `IllegalStateException` on duplicate keys (supply a merge function), boxing overhead (use `IntStream`), unreadable long pipelines (extract methods), checked exceptions inside lambdas, and using streams for simple loops where performance or debuggability matters. Know `flatMap` (flatten nested), `groupingBy` with downstream collectors, `partitioningBy`, `reduce` and short-circuit ops (`findFirst`, `anyMatch`).

How to understand it

Worked explanation: A stream pipeline describes transformations; a terminal operation produces the result. Short-circuiting and optimization mean not every intermediate callback must run, so required side effects do not belong in peek.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What happens with duplicate map keys? Decide whether to reject, merge or group them; an arbitrary merge can silently lose business data.

30-second version

> Streams are lazy: nothing runs without a terminal operation, and a stream can be consumed only once. Pitfalls: side effects in `map/peek` (breaks with parallelism), `Collectors.toMap` throwing `IllegalStateException` on duplicate keys (supply a merge function), boxing overhead (use `IntStream`), unreadable long pipelines (extract methods), checked exceptions inside lambdas, and using streams for simple loops where performance or debuggability matters.

Q77. How should `Optional` be used?

A good SDE-3 interview answer is:

As a **return type** that signals "may be absent", chained with `map`, `flatMap`, `filter`, `orElseGet` (lazy, unlike `orElse` which always evaluates), and `orElseThrow`. Don't use it for fields, method parameters, collections (return empty collections instead) or serialization, and never call `get()` without checking. `Optional.of(null)` throws; use `ofNullable`. Return `Optional` from repository finders and handle absence explicitly at the service boundary.

How to understand it

Worked explanation: A missing customer is an expected result distinct from a failed database call. Optional can model absence, while an exception can still carry the operational failure.

Key terms you should know

- **Application service** — A component coordinating a business use case; it may be a Spring-managed bean and is not a Kubernetes Service.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why prefer orElseGet for expensive fallback? Its supplier is invoked only when empty; an orElse argument is evaluated before the method call.

30-second version

> As a **return type** that signals "may be absent", chained with `map`, `flatMap`, `filter`, `orElseGet` (lazy, unlike `orElse` which always evaluates), and `orElseThrow`. Don't use it for fields, method parameters, collections (return empty collections instead) or serialization, and never call `get()` without checking.

Q78. Explain SOLID with real examples.

A good SDE-3 interview answer is:

S: a class has one reason to change (separate `OrderService` from `EmailSender`). **O**: extend behaviour without editing existing code (new `PaymentStrategy` implementation, not another `if` branch). **L**: subtypes must be usable wherever the base type is (a `Square extends Rectangle` that breaks width/height contracts violates it). **I**: small, role-specific interfaces rather than a fat one. **D**: depend on abstractions; Spring's DI injects implementations. Interviewers want a story: "We had a 600-line switch on payment type; replaced it with strategies registered in a map keyed by type."

How to understand it

Worked explanation: When adding a payment method, isolate the varying behavior behind a useful contract. The design succeeds if callers can use every implementation without new special cases that violate its promised behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does SOLID require an interface for every class? No; add boundaries where they clarify responsibility or variation, not mechanically.

30-second version

> **S**: a class has one reason to change (separate `OrderService` from `EmailSender`). **O**: extend behaviour without editing existing code (new `PaymentStrategy` implementation, not another `if` branch).

Q79. Which design patterns does Spring itself use?

A good SDE-3 interview answer is:

Explain patterns through actual collaboration: BeanFactory creates and resolves beans; AOP proxies intercept calls; DispatcherServlet centralizes request dispatch; strategies such as PasswordEncoder vary behavior; application events decouple publishers from listeners. Spring template classes commonly combine a fixed

resource-management workflow with callbacks, so calling every JdbcTemplate use classic inheritance-based Template Method is imprecise. Singleton bean scope is per bean definition per container, not a JVM-wide GoF singleton.

How to understand it

Worked explanation: A transaction proxy wraps a service call; a strategy such as PasswordEncoder supplies replaceable behavior. Show how the call moves through the objects and why that indirection helps.

Key terms you should know

- **AOP** — Aspect-oriented programming: applying cross-cutting behavior such as transactions, logging or security around method execution.
- **bean** — An object managed by the Spring IoC container.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Are all classes named Template literal Template Method implementations? No; Spring templates also rely heavily on callbacks. Explain their actual structure.

30-second version

> Choose a pattern and trace the objects involved. A proxy intercepts, a strategy varies behavior, and a template manages a workflow with callbacks. Explain scope and structure instead of matching names to a pattern list.

Q80. Describe the JVM architecture and JIT compilation.

A good SDE-3 interview answer is:

Class loaders load bytecode into the runtime data areas: heap (objects), metaspace (class metadata, native memory), per-thread stacks (frames, locals), PC registers and the code cache. The interpreter starts executing immediately; hot methods are compiled by **C1** (fast, lightly optimised) then **C2** (aggressive: inlining, escape analysis, loop unrolling) via tiered compilation, with deoptimisation if assumptions break. Consequences: warm-up matters for benchmarks (use JMH), and cold-start-sensitive workloads (serverless) benefit from CDS/AppCDS, CRaC or GraalVM native image.

How to understand it

Worked explanation: A frequently executed method may be compiled using runtime profile information. If later behavior invalidates an optimization assumption, execution can deoptimize and return to a less specialized path.

Key terms you should know

- **heap** — JVM memory where Java objects are allocated.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why does warm-up matter? Early measurements include loading and compilation costs that differ from steady-state service behavior.

30-second version

> Class loaders load bytecode into the runtime data areas: heap (objects), metaspace (class metadata, native memory), per-thread stacks (frames, locals), PC registers and the code cache. The interpreter starts executing immediately; hot methods are compiled by **C1** (fast, lightly optimised) then **C2** (aggressive: inlining, escape analysis, loop unrolling) via tiered compilation, with deoptimisation if assumptions break.

N. Spring Ecosystem

Complete the Spring annotation ladder and JPA session/transaction foundations before repository, batch and integration follow-ups.

Q81. Spring Data JPA: derived queries, `@Query`, Specifications, projections.

A good SDE-3 interview answer is:

Derived queries (`findByStatusAndCreatedAtAfter`) are fine for simple cases but become unreadable; use `@Query` (JPQL/native) for clarity, **Specifications** or Querydsl for dynamic filters, and **interface/DTO projections** to select only needed columns. Prefer `Pageable` with a stable sort, avoid `findAll()` on big tables, return `Slice` when you don't need the count query, and don't expose entities through the API.

How to understand it

Worked explanation: A repository proxy interprets a derived method name or declared query and binds parameters. The selected result shape determines how much data is fetched and exposed.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why choose Slice instead of Page? If only next-page availability is needed, avoiding a total count can remove expensive work.

30-second version

> Derived queries (`findByStatusAndCreatedAtAfter`) are fine for simple cases but become unreadable; use `@Query` (JPQL/native) for clarity, **Specifications** or Querydsl for dynamic filters, and **interface/DTO projections** to select only needed columns. Prefer `Pageable` with a stable sort, avoid `findAll()` on big tables, return `Slice` when you don't need the count query, and don't expose entities through the API.

Q82. When do you use Spring Batch?

A good SDE-3 interview answer is:

Use Spring Batch for restartable, high-volume processing such as ETL, reconciliation and nightly billing. A `Job` contains `Step`s. In a typical chunk step, an `ItemReader` reads items, an `ItemProcessor` transforms them, and an `ItemWriter` writes a chunk within a transaction. The job repository records executions and restart state; configure skip/retry policies and readers/writers that support the intended restart contract.

A `JobInstance` is identified by the job name and identifying `JobParameters`. Reuse those parameters when restarting the same logical run. A new timestamp or random identifying parameter creates a new instance and can start the work again; it does not provide idempotency. Protect business writes with stable item/business keys, uniqueness or deduplication, and use provider idempotency for external side effects. Scale using partitioning or remote chunking when needed. For simpler periodic work, scheduling with an appropriate distributed lock may be sufficient; that lock also does not make side effects idempotent.

How to understand it

Worked explanation: A failed chunk can restart using recorded execution state, but restarting with new identifying parameters creates a different logical job instance. That can repeat business work rather than resume it.

Key terms you should know

- **JobInstance** — A logical run identified by job name and identifying parameters.
- **JobExecution** — One execution attempt for that logical run.
- **ExecutionContext** — Persisted state used by restart-aware components.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What protects a charge on restart? Stable business/provider idempotency and durable state, not the mere existence of a job repository.

30-second version

> Spring Batch supplies chunk processing and persisted restart metadata. Keep identifying JobParameters stable for retries of the same logical run; changing them creates a new JobInstance. Business idempotency still requires stable keys and deduplication or equivalent safeguards.

Spring Batch domain model.

Q83. Which Spring Cloud components are still relevant?

A good SDE-3 interview answer is:

Gateway (routing/filters), OpenFeign or declarative HTTP interfaces, Config Server (centralised config; on Kubernetes ConfigMaps/Secrets often replace it), Resilience4j (circuit breaker, retry, rate limiter, bulkhead), and Micrometer Tracing. Netflix Eureka/Ribbon are largely superseded because Kubernetes provides service discovery and load balancing via Services/DNS. Show you choose platform features over libraries when available.

How to understand it

Worked explanation: Ask which capability the deployment platform already supplies, then fill actual gaps with application libraries. Discovery, request balancing and resilience remain separate responsibilities even when packaged together.

Key terms you should know

- **bulkhead** — A resilience pattern that isolates resources such as threads or connections so one overloaded dependency does not exhaust the whole service.

- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is a Kubernetes Service a retry policy? No; the caller still needs deadlines and deliberate handling of dependency failures.

30-second version

> Gateway (routing/filters), OpenFeign or declarative HTTP interfaces, Config Server (centralised config; on Kubernetes ConfigMaps/Secrets often replace it), Resilience4j (circuit breaker, retry, rate limiter, bulkhead), and Micrometer Tracing. Netflix Eureka/Ribbon are largely superseded because Kubernetes provides service discovery and load balancing via Services/DNS.

Q84. `RestTemplate` **vs** `WebClient` **vs** `RestClient`**, and how do you set timeouts?**

A good SDE-3 interview answer is:

`RestClient` is the synchronous fluent client; `WebClient` supports reactive composition and streaming. `RestTemplate` was in maintenance mode in Spring 6 and is deprecated in Spring 7 in favor of `RestClient`. Match the answer to the interview's version. Configure connection acquisition, connect, response/read and overall deadlines on the actual underlying client; retries and circuit breakers depend on the failure contract. Spring REST clients.

How to understand it

Worked explanation: A synchronous client waits in the calling thread; a reactive client returns a composition representing eventual signals. Either can hang or exhaust connections if underlying timeout and capacity controls are missing.

Key terms you should know

- **reactive** — A programming model centered on asynchronous streams and non-blocking processing with explicit demand/back-pressure.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Which timeout is being discussed? Distinguish pool acquisition, connection establishment, response reading and total operation budget.

30-second version

> `RestClient` is the synchronous fluent client; `WebClient` supports reactive composition and streaming. `RestTemplate` was in maintenance mode in Spring 6 and is deprecated in Spring 7 in favor of `RestClient`.

Q85. How do you handle validation and errors consistently?

A good SDE-3 interview answer is:

Bean Validation (`@Valid`, `@NotNull`, `@Size`, custom constraint validators) at the API boundary; domain invariants inside entities/services. A single `@RestControllerAdvice` maps exceptions to RFC 9457 problem details (`ProblemDetail`; RFC 9457 supersedes RFC 7807) with stable error codes (not stack traces), HTTP status by

cause (400 validation, 404, 409 conflict, 422, 503), and a correlation ID. Log once at the boundary with the right level (client errors at `warn`/`info`, server errors at `error`).

How to understand it

Worked explanation: Reject malformed input at the boundary and enforce business invariants where state changes occur. Translate the resulting error into a stable public contract while preserving detailed diagnostics privately.

Key terms you should know

- **bean** — An object managed by the Spring IoC container.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does bean validation authorize an update? No; a syntactically valid request can still target another user's resource.

30-second version

> Bean Validation (`@Valid`, `@NotNull`, `@Size`, custom constraint validators) at the API boundary; domain invariants inside entities/services. A single `@RestControllerAdvice` maps exceptions to RFC 9457 problem details (`ProblemDetail`; RFC 9457 supersedes RFC 7807) with stable error codes (not stack traces), HTTP status by cause (400 validation, 404, 409 conflict, 422, 503), and a correlation ID.

Q86. What is `@TransactionalEventListener` and why is it useful?

A good SDE-3 interview answer is:

`@TransactionalEventListener(AFTER_COMMIT)` runs after successful commit, but is not durable delivery: a process can fail before an email or message is sent. Original transactional resources may still be accessible, yet further writes do not become a new committed transaction automatically. Put required follow-up writes behind a separate transactional boundary and use an outbox when the event must survive process failure. Listener failure cannot undo the original commit. TransactionalEventListener API.

How to understand it

Worked explanation: A transaction can commit and the process can die before an AFTER_COMMIT listener sends its email. Running after commit gives ordering but no durable delivery guarantee.

Key terms you should know

- **outbox** — A database table used to store events in the same transaction as business data so publication can happen reliably afterward.
- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: When is an outbox needed? When the follow-up must be recoverable after process failure, with duplicate-safe handling by the receiver.

30-second version

> `@TransactionalEventListener(AFTER_COMMIT)` runs after successful commit, but is not durable delivery: a process can fail before an email or message is sent. Original transactional resources may still be accessible, yet further writes do not become a new committed transaction automatically.

Q87. What changed with Boot 3 observability and native images?

A good SDE-3 interview answer is:

Boot 3's Micrometer Observation integration can feed metrics and traces when the required instrumentation, bridges and exporters are configured. GraalVM native images can improve startup and memory for suitable applications, but need AOT processing, compatible libraries and reflection/proxy hints; builds and steady-state behavior differ from the JVM. Compare actual cold start, resource use, throughput and diagnostic needs. Neither millisecond startup nor lower memory is a universal promise for every application.

How to understand it

Worked explanation: Instrumentation must create observations and connect them to exporters before useful metrics or traces appear. Native compilation changes startup and runtime trade-offs; it is not merely another packaging extension.

Key terms you should know

- **Cloud Run** — GCP's managed container execution platform where the platform handles servers and scales container instances.
- **JIT** — Just-in-time compilation: runtime compilation of frequently executed bytecode into optimized native machine code.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How would you compare JVM and native? Use the same workload, resource limits, startup and steady-state measurements, including failure diagnostics.

30-second version

> Observation requires configured instrumentation and export. Native compilation can trade build complexity and runtime behavior for startup or memory gains; choose from measurements of the real application.

O. Databases & Caching

Begin database questions with the connection, session and transaction sequence, then the SQL foundations. Q88–Q90 cover indexing, query diagnosis and storage choices after those prerequisites.

Q88. How do database indexes work and how do you design them?

A good SDE-3 interview answer is:

A B-tree index keeps keys ordered. Seeking a key or range start is typically O(log n); retrieving k matching entries adds work, commonly modeled as O(log n + k). An ordered full scan still visits O(n) entries. Real query cost also includes pages read, table lookups, filtering, visibility checks and any additional sorting.

A composite B-tree often benefits from equality constraints on leading columns followed by a range/order requirement. Some engines can exploit later columns through skip scans or other access paths. A covering index contains the needed columns and may avoid table reads, but engine-specific visibility rules still apply. Indexes cost storage and write work, so design them around actual queries and execution plans. Expressions such as `lower(email)` may need expression indexes; leading-wildcard searches may need a specialized index. Low selectivity can make a sequential scan cheaper; do not assume an index is either always used or always ignored.

How to understand it

Worked explanation: An index narrows candidate rows by an ordered key or another access structure. Fetching many matches can still touch many pages, so an index is not a promise of cheap execution.

Key terms you should know

- **Selectivity** — How much of the data a predicate matches.
- **Covering index** — An index containing the columns required by a query.
- **MVCC visibility** — Whether a row version is visible to a transaction snapshot.
- **Database heap** — Table-row storage; distinct from the JVM object heap.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why inspect actual plans? Selectivity, statistics and required columns determine whether the index reduces total work.

30-second version

> A B-tree can find a range start in logarithmic time, but reading k entries adds work: typically O(log n + k), plus database-specific costs. Composite and covering indexes must match the query. Verify the plan and MVCC visibility costs instead of claiming every indexed query is O(log n).

PostgreSQL index types, index-only scans.

Q89. A query is slow. How do you tune it?

A good SDE-3 interview answer is:

For PostgreSQL, `EXPLAIN (ANALYZE, BUFFERS)` executes the statement, so use a safe environment or a carefully controlled read-only query: look for sequential scans on large tables, row-estimate mismatches (stale stats; `ANALYZE`), expensive sorts/hash joins and nested loops over big sets. Fixes: add or reshape indexes, rewrite (avoid `SELECT *`, replace correlated subqueries with joins, use keyset pagination), reduce data returned, partition large tables, cache results, or precompute with materialised views. Verify with production-sized data and watch lock contention and long transactions.

How to understand it

Worked explanation: Compare estimated and actual row counts, then locate expensive scans, joins, sorts and waits. Optimize the dominant work while preserving results; replacing syntax alone does not prove improvement.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why is EXPLAIN ANALYZE sensitive? It executes the statement, including writes and other effects; choose a safe verification context.

30-second version

> For PostgreSQL, `EXPLAIN (ANALYZE, BUFFERS)` executes the statement, so use a safe environment or a carefully controlled read-only query: look for sequential scans on large tables, row-estimate mismatches (stale stats; `ANALYZE`), expensive sorts/hash joins and nested loops over big sets. Fixes: add or reshape indexes, rewrite (avoid `SELECT *`, replace correlated subqueries with joins, use keyset pagination), reduce data returned, partition large tables, cache results, or precompute with materialised views.

Q90. SQL vs NoSQL: how do you choose?

A good SDE-3 interview answer is:

Choose a database from required queries, transactions, consistency, scale, durability and operational cost. Relational systems support rich joins and constraints; document, key-value, wide-column, graph and search systems emphasize other models. Some non-relational products support transactions, and some SQL products scale horizontally, so SQL versus NoSQL is not itself a guarantee. Use additional stores only when their measured benefit justifies data synchronization and recovery work.

How to understand it

Worked explanation: Compare the exact invariant, query shape and operational requirements against a specific database. SQL/NoSQL labels do not by themselves determine transactions, consistency or horizontal scalability.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What is the cost of a second store? Data propagation, reconciliation, backup and failure handling must be owned and tested.

30-second version

> Start with the invariant and access pattern, then compare specific products. SQL/NoSQL labels do not determine all transaction or scaling guarantees, and every extra store adds operational obligations.

Cache questions before Redis patterns

CA1 What is a cache and how does cache-aside serve a read

Foundation: A cache stores reusable results to reduce repeated work. In cache-aside, the application checks a key, returns a hit, or reads the primary store on a miss and fills the cache. Define keys, capacity and expiry; distinguish this from the authoritative store.

Mid-level follow-up: Explain TTL expiry, eviction, negative caching and invalidation after a committed write. An absent cache entry and a cached “not found” result need distinct representations.

CA2 Why can caching return stale data or overload the database

Senior follow-up: A reader can load an old value, race with a write and its invalidation, then refill stale data. TTL alone does not ensure strong consistency. Specify acceptable staleness, versioning/invalidation strategy, bounded loaders and cache-outage behavior. Test duplicate fills, expiry, failed loads and write/read races before choosing the Redis patterns in Q91.

Reference: Redis cache-aside. This is a design discussion, not a live Redis integration.

Q91. Which Redis caching patterns do you know, and what are the failure modes?

A good SDE-3 interview answer is:

Cache-aside (read: check cache, miss → DB → populate; write: update DB then evict), *write-through*, *write-behind*. Always set TTLs (with jitter) and eviction policy. Failure modes: **stampede** (many requests rebuild an expired hot key: use locking/single-flight or early refresh), **penetration** (queries for non-existent keys: cache negatives/Bloom filter), **avalanche** (mass expiry: jittered TTLs), stale data and cache/DB inconsistency. For a distributed lock use `SET key value NX PX ttl` with a unique token and a Lua release script, and note locks don't give strict safety without fencing tokens.

How to understand it

Worked explanation: A hot cache entry expires and many callers simultaneously load it from the database. Coalescing reduces duplicate loads; expiry jitter spreads unrelated expirations. Neither alone solves every stale-write race.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why is a lease token insufficient for strict ownership? An expired holder can continue working; the protected resource may need to reject stale fencing values.

30-second version

> *Cache-aside* (read: check cache, miss → DB → populate; write: update DB then evict), *write-through*, *write-behind*. Always set TTLs (with jitter) and eviction policy.

P. Security, Performance & Quality

Start security with the authentication and authorization foundation. For quality questions, establish the following before discussing benchmarks or flaky tests.

Quality questions before performance and flakiness

QE1 What makes an automated test useful and what should it exercise

Foundation: Arrange a known input/state, perform an operation and assert its observable result or failure. A unit test isolates a small behavior; integration tests exercise component boundaries; end-to-end tests exercise a user journey. Choose the boundary that can expose the defect rather than treating coverage as proof.

Mid-level follow-up: Include empty/invalid inputs, failures and repeated operations. Control clocks, randomness and shared state. A mock proves behavior against a simulated collaborator; verify database, framework and network contracts at the relevant integration boundary.

QE2 How do logs, metrics and traces help diagnose a failure

Foundation: Logs describe events, metrics summarize measurements and traces connect spans along an operation. Correlate them to distinguish one failed request from a system-wide symptom.

Senior follow-up: Define the user-facing success/latency target, inspect saturation and downstream time, and protect sensitive data. Test correctness separately from load; use Q93–Q95 for logging, measurement and flaky-test diagnosis, and Q170 for stronger suite evaluation.

References: JUnit 5.11 assertions and lifecycle, OpenTelemetry observability primer. No new framework example or infrastructure claim is introduced.

Q92. Which OWASP vulnerabilities matter most in a Java API, and how do you prevent them?

A good SDE-3 interview answer is:

Injection (use prepared statements/JPA parameters, never concatenate SQL), broken access control (check ownership on every object access, not just roles: IDOR), cryptographic failures (TLS everywhere, strong hashing, no hard-coded secrets), insecure deserialization (avoid Java native deserialization; configure Jackson safely), SSRF (allow-list outbound URLs), vulnerable components (SCA scanning), security misconfiguration (disable actuator exposure, verbose errors), XSS (output encoding, CSP) and insufficient logging/monitoring. Add rate limiting and input validation.

How to understand it

Worked explanation: Trace untrusted input from request to database, outbound HTTP and rendered output. Each sink needs the matching control: parameterized values, destination policy or output encoding, plus object-level authorization.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can an allowlisted hostname alone stop SSRF? No; redirects, DNS changes and resolved private addresses must be included in the outbound policy.

30-second version

> Injection (use prepared statements/JPA parameters, never concatenate SQL), broken access control (check ownership on every object access, not just roles: IDOR), cryptographic failures (TLS everywhere, strong hashing, no hard-coded secrets), insecure deserialization (avoid Java native deserialization; configure Jackson safely), SSRF (allow-list outbound URLs), vulnerable components (SCA scanning), security misconfiguration (disable actuator exposure, verbose errors), XSS (output encoding, CSP) and insufficient logging/monitoring. Add rate limiting and input validation.

Q93. What are your logging best practices?

A good SDE-3 interview answer is:

Structured JSON logs through SLF4J, with correlation IDs in the MDC (set in a filter, cleared in `finally`, propagated to async tasks), appropriate levels, and no sensitive data (PII, tokens, card numbers). Use parameterised messages (`log.info("order {}", id)`), log once per failure at the boundary, avoid logging in tight loops, and sample high-volume debug logs. Logs explain *what happened*; metrics tell *how much*; traces show *where*.

How to understand it

Worked explanation: Log enough context to connect a failed operation to its trace without exposing credentials or private payloads. A correlation field must be cleaned up when a pooled thread finishes the request.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Will parameterized logging make unsafe data safe? No; parameterization avoids eager formatting, but sensitive data and log-injection boundaries still need policy.

30-second version

> Structured JSON logs through SLF4J, with correlation IDs in the MDC (set in a filter, cleared in `finally`, propagated to async tasks), appropriate levels, and no sensitive data (PII, tokens, card numbers). Use parameterised messages (`log.info("order {}", id)`), log once per failure at the boundary, avoid logging in tight loops, and sample high-volume debug logs.

Q94. How do you performance-test and benchmark?

A good SDE-3 interview answer is:

Define SLOs, then load-test realistic scenarios (k6, Gatling, JMeter) against a production-like environment with representative data, measuring throughput, p50/p95/p99 latency and errors, plus saturation (CPU, GC, pool waits). Find bottlenecks with profiling (JFR, async-profiler) before changing code. Use **JMH** for micro-benchmarks (warm-up, forks, blackhole to avoid dead-code elimination); hand-rolled `System.nanoTime` loops are misleading. Run soak tests to catch leaks and put a regression test in CI.

How to understand it

Worked explanation: Define the traffic model and success criteria before generating load. A service can report low latency by rejecting most requests, so latency without throughput and error rates is incomplete.

Key terms you should know

- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a JMH result predict endpoint p99? No; microbenchmarks isolate code while endpoint latency includes queues, I/O and contention.

30-second version

> Define SLOs, then load-test realistic scenarios (k6, Gatling, JMeter) against a production-like environment with representative data, measuring throughput, p50/p95/p99 latency and errors, plus saturation (CPU, GC, pool waits). Find bottlenecks with profiling (JFR, async-profiler) before changing code.

Q95. How do you deal with flaky tests?

A good SDE-3 interview answer is:

Treat them as defects: find the cause (time/timezone, ordering, shared state, async waits with `sleep`, random data, external dependencies). Fixes: inject a `Clock`, use Awaitility instead of sleeps, isolate data per test (transactions or Testcontainers per class), make tests independent, mock only external boundaries, and quarantine rather than ignore with a ticket and deadline. A flaky suite erodes trust in CI.

How to understand it

Worked explanation: A test waiting a fixed 100 ms assumes a scheduling deadline it does not control. Wait for the observable condition with a bound, and isolate the data and clock driving it.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Do retries fix flakiness? They can hide a defect; diagnose the nondeterministic contract and make failures reproducible.

30-second version

> Treat them as defects: find the cause (time/timezone, ordering, shared state, async waits with `sleep`, random data, external dependencies). Fixes: inject a `Clock`, use Awaitility instead of sleeps, isolate data per test (transactions or Testcontainers per class), make tests independent, mock only external boundaries, and quarantine rather than ignore with a ticket and deadline.

Q96. What are the 12-factor principles that matter most for cloud services?

A good SDE-3 interview answer is:

One codebase, many deploys; explicit dependencies; **config in the environment**; backing services as attached resources; strict build/release/run separation; **stateless processes** (state in DB/Redis); port binding; scale out via processes; **disposability** (fast start, graceful shutdown); dev/prod parity; logs as event streams to stdout; admin tasks as one-off jobs. Tie each to a concrete practice (immutable images, readiness probes, no local file state).

How to understand it

Worked explanation: Build immutable application bits, combine them with explicit configuration for a release, and run disposable instances whose durable state lives elsewhere. Restarting an instance should not erase required business data.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does stateless mean no memory or cache? No; it means required durable/session state cannot depend on one disposable process.

30-second version

> One codebase, many deploys; explicit dependencies; **config in the environment**; backing services as attached resources; strict build/release/run separation; **stateless processes** (state in DB/Redis); port binding; scale out via processes; **disposability** (fast start, graceful shutdown); dev/prod parity; logs as event streams to stdout; admin tasks as one-off jobs. Tie each to a concrete practice (immutable images, readiness probes, no local file state).

Q97. How do you design backward-compatible APIs and events?

A good SDE-3 interview answer is:

Compatibility includes syntax and semantics. Additive optional fields often help, but strict parsers, new enum values, changed defaults or tighter validation can still break consumers. Preserve field meanings, reserve removed Protobuf numbers, use schema-compatibility checks and consumer tests, and plan deprecation windows. Tolerant deserialization is one tool, not permission to accept unsafe or misspelled input indiscriminately. Test old and new components in the rollout combinations that must work.

How to understand it

Worked explanation: A new optional field can still break a strict parser, and a new enum value can break an exhaustive consumer. Compatibility includes consumer behavior and semantic assumptions, not just schema syntax.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you prove independent rollout? Test old/new producer and consumer combinations and keep a deliberate deprecation window.

30-second version

> Additive changes are not automatically safe. Validate old/new consumer behavior, preserve field meanings and use explicit migration windows for breaking changes.

Q98. How do you approach technical debt and refactoring safely?

A good SDE-3 interview answer is:

Make it visible (tracked backlog with impact), pay it continuously (boy-scout rule, a fixed share of each sprint), and refactor behind tests: write characterisation tests first, use small steps, branch-by-abstraction or feature flags for large changes, and the strangler pattern for legacy replacement. Prioritise debt that slows delivery or causes incidents, and explain value in business terms (lead time, defect rate).

How to understand it

Worked explanation: Before changing internals, capture externally required behavior. Make a small change, verify it and observe whether the targeted delivery or reliability problem improves.

Key terms you should know

- **Technical debt** — Design or implementation choices that increase future delivery or operational cost.
- **Characterisation test** — A test recording existing behavior before a change.
- **Branch by abstraction** — Gradually replacing an implementation behind a stable boundary.
- **Strangler pattern** — Incrementally replacing legacy functionality.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What distinguishes useful refactoring from churn? A named risk or cost is reduced with evidence, rather than only changing code shape.

30-second version

> Make it visible (tracked backlog with impact), pay it continuously (boy-scout rule, a fixed share of each sprint), and refactor behind tests: write characterisation tests first, use small steps, branch-by-abstraction or feature flags for large changes, and the strangler pattern for legacy replacement. Prioritise debt that slows delivery or causes incidents, and explain value in business terms (lead time, defect rate).

Q. More Coding Programs

1. Sealed types with exhaustive pattern-matching `switch` (Java 21)

Derive the approach: the operation has a finite set of meaningful outcomes. Model each outcome as a type so each consumer must handle the known alternatives.

Why this, not the alternatives: a boolean plus nullable fields permits inconsistent combinations. Exceptions fit exceptional failures but can obscure ordinary expected outcomes. An interface with polymorphic behavior may be better when behavior belongs to each subtype; switching is useful when operations vary across a closed model.

What changes: adding a permitted subtype makes newly compiled exhaustive consumers require handling it. Independently deployed clients still need compatibility planning. A `default` can hide a missing case; `null` is not automatically a modeled variant. Records remain shallowly immutable.

Trace and checks: Success maps to a value message, Failure to an error message. Test each variant and the declared null policy. Branch selection is bounded by the small variant set, but rendering a value may cost more.

Strong answer signal: explain which invalid states the type model prevents and where evolution is constrained.

```
sealed interface Result<T> permits Success, Failure {}
record Success<T>(T value) implements Result<T> {}
record Failure<T>(String error) implements Result<T> {}

static <T> String describe(Result<T> r) {
    return switch (r) {
        case Success<T> s -> "OK: " + s.value();
        case Failure<T> f -> "ERR: " + f.error();   // no default needed: compiler checks
exhaustiveness
    };
}
```

2. Dynamic filtering with Spring Data `Specification`

Derive the approach: optional filters compose into a conjunction; independently add predicates for supplied values and let the database filter before pagination.

Why this, not the alternatives: derived-method names become unwieldy across many combinations. Fetching all rows and filtering in Java wastes memory and can paginate incorrectly. A fixed JPQL query may be simpler when there are only one or two stable combinations.

What changes: absent filters currently match all rows, so enforce authorization/tenant predicates separately and cap page size. Empty text is different from absent input. Sorting only by creation time is unstable on ties; append a unique ID. Joins may duplicate roots and complicate count queries.

Trace and checks: no optional filters, status only, date only and both must produce intended SQL predicates. Check inclusive date boundaries, null policy and equal timestamps across pages. Complexity follows the execution plan and result cardinality, not the number of Java predicates.

Strong answer signal: connect composability to query correctness, security and stable pagination.

```
interface OrderRepository extends JpaRepository<Order, Long>, JpaSpecificationExecutor<Order> {}

static Specification<Order> filter(String status, Instant from) {
    return (root, q, cb) -> {
        List<Predicate> p = new ArrayList<>();
        if (status != null) p.add(cb.equal(root.get("status"), status));
        if (from != null)   p.add(cb.greaterThanOrEqualTo(root.get("createdAt"), from));
        return cb.and(p.toArray(Predicate[]::new));
    };
}
// repo.findAll(filter("PAID", from), PageRequest.of(0, 20, Sort.by("createdAt").descending()));
```

3. Bounded concurrency with virtual threads

Derive the approach: blocking requests can be represented as lightweight threads, but the downstream permits only a fixed number of concurrent calls. A semaphore represents that separate resource budget.

Why this, not the alternatives: a bounded platform-thread pool also works but limits execution contexts and downstream access together. Unlimited submission without a downstream limit can overwhelm connection pools. A semaphore protects active calls; admission limits also control waiting tasks and retained inputs.

What changes: acquiring permits after task creation permits an unbounded number of waiting virtual threads. Acquire/admit before submission with failure cleanup, or process bounded batches. Release only after successful acquisition and in `finally`. Per-JVM limits multiply across replicas. Waiting in executor close can exceed a request deadline if tasks do not terminate.

Trace and checks: with two permits and five tasks, at most two fetches run concurrently. Test failures and interruption without leaking permits; verify admitted count as well as active count. The shown code retains O(number of IDs) futures.

Strong answer signal: distinguish cheap threads from bounded resource and memory use.

```
Semaphore limit = new Semaphore(50);                     // protect the downstream service
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<String>> results = ids.stream().map(id -> exec.submit(() -> {
        limit.acquire();
        try { return client.fetch(id); } finally { limit.release(); }
    })).toList();
    for (var f : results) System.out.println(f.get());
}   // close() waits for all tasks
```

Say: this limits active downstream calls but still submits one task and retains one future per input. Bound admitted work or process batches for huge inputs; add deadlines and cancellation. `close()` waits for completion, so a hanging task can delay scope exit.

4. Merge overlapping intervals (O(n log n))

Derive the approach: after sorting by start, a new interval either extends the last union interval or starts a disjoint one. Earlier merged intervals cannot overlap it without also overlapping the last one.

Why this, not the alternatives: pairwise comparisons cost $O(n^2)$; an interval tree is useful for dynamic queries but unnecessary for one static merge. If the input is already sorted, skip sorting and scan in O(n).

What changes: the code sorts the input and reuses/mutates its inner arrays. Copy when the caller requires preservation. Closed intervals that touch merge under the current condition; half-open semantics may need a different rule. Reject start > end rather than silently interpreting it.

Trace and checks: `[1,4], [2,3], [6,8]` becomes `[1,4], [6,8]`; the contained interval must not shrink the right endpoint. Test empty input, touching endpoints, duplicates and nested intervals. O(n log n) time, O(n) output, plus sorting workspace.

Strong answer signal: state endpoint and mutation contracts before the sweep.

```
int[][] merge(int[][] in) {
    Arrays.sort(in, Comparator.comparingInt(a -> a[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] cur : in) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0]) out.add(cur);
        else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], cur[1]);
    }
    return out.toArray(new int[0][]);
}
```

Part 06 - Question Bank Vol. 5 - Rapid-fire, puzzles and harder coding

Senior Java Question Bank, Vol. 5

Rapid-Fire Revision · Output-Prediction Puzzles · More System Designs · Harder Coding Use this the night before: say each answer aloud in one breath, then expand if the interviewer probes.

R. Rapid-Fire (one-breath answers)

Use these as recall prompts after the detailed answers. For an interview follow-up, expand the mechanism and example from the topic learning paths; a one-line comparison is not a full explanation.

Java

1. **HashMap vs Hashtable vs ConcurrentHashMap?** Unsynchronized with one null key; fully synchronized legacy; per-bin CAS/locks, no nulls.
2. **ArrayList vs LinkedList?** Array-backed O(1) index and cache-friendly vs O(n) index; ArrayList wins almost always.
3. `final`, `finally`, `finalize`**?** Assignment/extension restrictions, not deep immutability; cleanup on normal or abrupt control flow, but not guaranteed after process termination; finalization is deprecated for removal. Prefer explicit resource ownership and try-with-resources.
4. **Abstract class vs interface?** State and constructors vs multiple inheritance of type; interfaces have `default`/`static` methods since 8.
5. `==` **vs** `equals`**?** Reference identity vs logical equality.
6. **Stack vs heap?** Per-thread frames and primitives/references vs shared objects managed by GC.
7. **Strong, soft, weak, phantom references?** Strong reachability keeps an object alive; softer reference types permit different GC processing and notification. Weak references do not guarantee a cleanup deadline; phantom references support post-reachability cleanup coordination.
8. `wait()` **vs** `sleep()`**?** `wait` requires ownership and releases that object's monitor while waiting, then reacquires it; `sleep` releases no monitors.
9. **Callable vs Runnable?** Returns a value and may throw checked exceptions.
10. **Serialization risks?** Version mismatch, security (gadget chains); prefer JSON/Protobuf and set `serialVersionUID` if you must.
11. **Shallow vs deep copy?** Copies references vs copies the object graph; prefer copy constructors over `clone()`.
12. **Why is** `HashMap` **capacity a power of two?** Fast index masking `(n-1) & hash` and cheap resize splitting.
13. **Fork/Join?** Work-stealing pool for recursive divide-and-conquer tasks.
14. `CountDownLatch` **vs** `CyclicBarrier` **vs** `Semaphore`**?** One-shot countdown; reusable rendezvous; permit-based throttling.

Spring / Hibernate

1. `@Component` **vs** `@Bean`**?** Class-level discovery candidate vs method-level factory registration, useful for third-party or application classes.
2. `@Controller` **vs** `@RestController`**?** View resolution vs `@Controller` + `@ResponseBody`.
3. `@Qualifier` **vs** `@Primary`**?** Narrow eligible candidates by qualifier metadata versus prefer one candidate for single-valued injection.
4. `@Value` **vs** `@ConfigurationProperties`**?** Single property vs typed, validated, grouped binding.
5. `@RequestParam` **vs** `@PathVariable` **vs** `@RequestBody`**?** Request parameters, URI-template variables, and a body decoded by a message converter; the body need not be JSON.
6. **Filter vs Interceptor?** Servlet-level, before `DispatcherServlet` vs Spring MVC handler-level with access to the handler.
7. **Entity lookup vs reference?** `find`/legacy `get` returns an entity or null and may use the persistence context/cache; `getReference` can defer loading and absence detection. Legacy `load` behavior and availability depend on Hibernate version.
8. `@OneToMany` **owning side?** For the usual bidirectional pair, `@ManyToOne` owns the foreign key and `mappedBy` identifies the inverse side. Unidirectional and join-table mappings need their own ownership explanation.
9. `Page` **vs** `Slice` **vs** `List`**?** Page reports total information and may need a count query; Slice reports whether more results exist. A List can still be bounded by a Pageable/limit. Pageable is a request parameter, not a result type.
10. **JPQL vs native vs Criteria?** Portable object queries vs DB-specific SQL vs type-safe dynamic queries.

Architecture / Cloud

1. **Orchestration vs choreography?** Central coordinator vs event-driven reactions.
2. **Idempotency?** Repeating an operation has the same effect as doing it once.
3. **Circuit breaker states?** Closed → open → half-open.
4. **Sticky sessions: good?** Affinity can serve specific needs, but complicates balancing and failover; avoid making durable session correctness depend on one disposable instance.
5. **Blue-green vs canary?** Full parallel switch vs gradual percentage rollout.
6. **SLI vs SLO vs SLA?** The measurement, the internal target, the contractual promise.
7. **Eventual consistency?** Replicas/read models converge after writes if no new updates occur.
8. **Pub/Sub vs queue?** Fan-out to many subscribers vs competing consumers on one queue.
9. **Cloud Run vs GKE one-liner?** Managed service/job/worker execution models versus Kubernetes platform control; compare the selected resource's capabilities and operating cost.
10. **Liveness vs readiness?** Restart me if broken vs stop sending me traffic until ready.
11. **Docker image vs container?** Immutable template vs running instance.
12. **Maven** `install` **vs** `deploy`**?** Local repo vs remote repository.
13. `mvn clean install` **vs** `verify`**?** `verify` reaches configured verification without local installation. `clean install` also cleans first and installs the artifact; integration tests require their plugin configuration.
14. **IaC benefit?** Reproducible, reviewable, versioned infrastructure (Terraform).
15. **PUT vs PATCH vs POST?** Create/replace the target representation idempotently; partial modification with operation-dependent idempotency; resource-specific processing that is not inherently idempotent.
16. **401 vs 403?** Not authenticated vs authenticated but not allowed.

S. "What Will This Print?" Puzzles

1. For `Integer a=127,b=127,c=128,d=128`, `a == b` is `true`; `c == d` is commonly `false` but is not guaranteed because implementations may cache more values. Use `equals` for value equality. **2.** `"a"+"b" == "ab"` → `true` (compile-time constant folding); `new String("ab") == "ab"` → `false`. **3.** `try { return 1; } finally { return 2; }` → returns **2**; a `return` in `finally` overrides and swallows exceptions. Never do it. **4.** `Arrays.asList(1,2).add(3)` and `List.of(1).add(2)` → `UnsupportedOperationException` (fixed-size / structurally unmodifiable). **5.** Mutating equality/hash-relevant state after insertion can make a `HashSet` lookup fail; the exact output is not guaranteed for every mutation. The key must remain stable while stored. **6.** Removing through the list inside an enhanced loop may trigger `ConcurrentModificationException`; fail-fast behavior is best effort and some cases end iteration without throwing. Use `Iterator.remove()` where the iterator supports it, or a supported collection-level `removeIf`. `CopyOnWriteArrayList` supports the latter but not iterator removal. **7.** `0.1 + 0.2 == 0.3` → `false`; use `BigDecimal` (constructed from `String`) for money. **8.** Loading and initialization are different. Static initialization occurs on active use, with superclass initialization first where required. Constructing an instance runs superclass construction before subclass instance initializers and the subclass constructor body.

T. More System Designs (condensed)

Use the requirements-first design sequence before the examples: requirements → estimates → API/data → architecture → failure/scaling analysis → trade-offs. These prompts assume that interview process.

Design 4: Distributed Rate Limiter. Requirements: per-user/API-key limits, <5 ms overhead, works across many instances. Algorithms: *fixed window* (simple, burst at boundaries), *sliding window log/counter* (accurate, more memory), *token bucket* (allows bursts, most common), *leaky bucket* (smooths output). Implementation: enforce at the API gateway or a sidecar; keep state in Redis with an atomic Lua script (`INCR` + `EXPIRE`, or token refill math) keyed by `user:route`. Return `429` with `Retry-After` and `X-RateLimit-*` headers. Trade-offs: Redis is a single point (use cluster, fail open vs fail closed per endpoint), clock skew (use Redis time), local pre-limiters to reduce round trips. Also support tiered limits and rules in config.

Design 5: Chat / Messaging System. Clients hold WebSocket connections to stateless gateway nodes; a presence/session registry (Redis) maps `userId → gateway`. Sending: gateway → message service assigns a sequence ID per conversation, persists (wide-column store, partitioned by `conversationId`, clustered by time) and publishes to Kafka/Pub/Sub; delivery workers look up the recipient's gateway and push, or store for offline push notification. Guarantees: at-least-once with client acks and dedupe by message ID; ordering per conversation via the partition key. Scale: shard by conversation, fan-out on write for small groups and on read for large channels, media in object storage + CDN, end-to-end encryption optional. Discuss read receipts, typing indicators (ephemeral, not persisted), and retention.

Design 6: Payments Ledger. Use **double-entry bookkeeping**: every transfer writes balanced debit/credit entries in one ACID transaction; entries are immutable and balances are derived (or a materialised snapshot). Requests carry an idempotency key with a unique constraint. Concurrency via optimistic version on accounts or serialised writes per account (partition by account). Integrate with PSPs asynchronously: state machine (INITIATED → PENDING → SETTLED/FAILED), webhooks verified by signature, a **reconciliation job** matching PSP files against the ledger and alerting on mismatches. Use outbox for events, audit logs, encryption of PII, PCI scope reduction through tokenisation. Mention rounding rules (minor units as `long`) and multi-currency.

U. Harder Coding Programs

1. Bounded blocking queue from scratch

Derive the approach: represent capacity with a circular array and protect head, tail and count together. Producers wait for not-full; consumers wait for not-empty.

Why this, not the alternatives: two conditions target the relevant waiter class. One monitor plus `notifyAll` can also work but wakes more threads. A linked queue allocates nodes; a circular array gives a fixed storage bound. Use the standard library outside the learning exercise unless special behavior is required.

What changes: zero capacity is not a rendezvous queue in this implementation; reject it. Replacing `while` with `if` breaks predicate rechecking after wakeups. Leaving removed references in the array retains objects unnecessarily. Fair locks can reduce starvation risk but may reduce throughput; fairness is not FIFO completion of arbitrary caller work.

Trace and checks: capacity two; put A/B, take A, put C wraps tail, then take B/C. Test multiple producers/consumers, interruption on both conditions, null policy and full/empty waits. O(1) operations excluding waiting, O(capacity) storage.

Strong answer signal: state `0 <= count <= capacity` and preserve it through every transition.

```
class BoundedQueue<T> {
    private final Object[] items; private int head, tail, count;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition(), notEmpty = lock.newCondition();
    BoundedQueue(int capacity) {
        if (capacity < 1) throw new IllegalArgumentException("capacity must be positive");
        items = new Object[capacity];
    }

    void put(T t) throws InterruptedException {
        Objects.requireNonNull(t);
        lock.lockInterruptibly();
        try {
            while (count == items.length) notFull.await();     // loop guards spurious wakeups
            items[tail] = t; tail = (tail + 1) % items.length; count++;
            notEmpty.signal();
        } finally { lock.unlock(); }
    }
    @SuppressWarnings("unchecked")
    T take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == 0) notEmpty.await();
            T t = (T) items[head]; items[head] = null; head = (head + 1) % items.length; count--;
            notFull.signal(); return t;
        } finally { lock.unlock(); }
    }
}
```

Say: two conditions avoid waking the wrong waiters; this is what `ArrayBlockingQueue` does.

2. Sliding-window rate limiter (per key)

Derive the approach: an exact rolling-window contract requires remembering accepted timestamps still inside the window. A deque makes chronological expiry cheap; one lock protects purge/check/add as a unit.

Why this, not the alternatives: fixed windows permit boundary bursts; token buckets intentionally allow configured bursts; approximate counters use less state but relax accuracy. `ConcurrentHashMap` protects lookup, not the mutable deque stored inside it.

What changes: read time inside the lock so timestamps remain ordered. `>= window` expiry defines the boundary explicitly. Naively removing an idle key while another caller still holds its deque can create two independent counters. Multiple instances require coordinated state or a documented approximation.

Trace and checks: limit two, timestamps 0 and 1; at exactly window-length after 0, its slot expires under this policy. Test simultaneous callers and an injectable clock. A call may purge O(limit) entries, but each accepted timestamp is removed once; state is O(active retained keys × limit), unbounded in the current key-retention sketch.

Strong answer signal: explain both per-key atomicity and key-lifecycle races.

```
class SlidingWindowLimiter {
    private final int limit; private final long windowNanos;
    private final ConcurrentHashMap<String, Deque<Long>> hits = new ConcurrentHashMap<>();
    SlidingWindowLimiter(int limit, Duration window) {
        if (limit < 1 || window.isZero() || window.isNegative())
            throw new IllegalArgumentException("positive limit and window required");
        this.limit = limit;
        this.windowNanos = window.toNanos();
    }

    boolean allow(String key) {
        Deque<Long> q = hits.computeIfAbsent(key, k -> new ArrayDeque<>());
        synchronized (q) {
            long now = System.nanoTime(); // sample under the lock to preserve timestamp order
            while (!q.isEmpty() && now - q.peekFirst() >= windowNanos) q.pollFirst();
            if (q.size() < limit) { q.addLast(now); return true; }
            return false;
        }
    }
}
```

Say: this demonstration retains every key, so total memory is unbounded as key count grows. Idle-key eviction must coordinate with active users of the same deque; naïve removal can create two counters for one key. Use a carefully bounded implementation for production.

3. Build order with cycle detection (Kahn's topological sort)

Derive the approach: a module becomes buildable when all prerequisites have completed. Track remaining prerequisite counts and reverse edges to dependents; removing a ready node reduces those counts.

Why this, not the alternatives: repeatedly scanning for buildable modules can become quadratic. DFS with visiting/visited colors is equally valid and can expose a cycle path, but needs careful stack handling. Kahn's method naturally exposes a set of ready jobs.

What changes: map iteration does not promise deterministic ordering among multiple ready nodes. Use a priority queue if lexicographic order is part of the contract, accepting O(V log V + E). Count duplicate edges consistently or normalize them. Dependencies mentioned only as values are still vertices.

Trace and checks: A depends on B and C: emit B/C in either allowed order, then A. A↔B leaves both with nonzero indegree. Test empty graph, disconnected graph, self-cycle and missing map keys. O(V + E) time and space.

Strong answer signal: derive why emitting fewer than V vertices proves a remaining cycle.

Relevant to Maven reactor ordering or task dependencies.

```
List<String> buildOrder(Map<String, List<String>> deps) {   // module -> modules it depends on
    Map<String, Integer> indeg = new HashMap<>();
    Map<String, List<String>> dependents = new HashMap<>();
    deps.forEach((m, ds) -> { indeg.putIfAbsent(m, 0);
        for (String d : ds) { indeg.putIfAbsent(d, 0); indeg.merge(m, 1, Integer::sum);
                              dependents.computeIfAbsent(d, k -> new ArrayList<>()).add(m); } });
    Deque<String> ready = new ArrayDeque<>();
    indeg.forEach((m, n) -> { if (n == 0) ready.add(m); });
    List<String> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        String m = ready.poll(); order.add(m);
        for (String next : dependents.getOrDefault(m, List.of()))
            if (indeg.merge(next, -1, Integer::sum) == 0) ready.add(next);
    }
    if (order.size() != indeg.size()) throw new IllegalStateException("Cyclic dependency");
    return order;
}
```

Complexity: O(V + E). If `order.size() < V` there is a cycle, which is exactly how Maven reports "The projects in the reactor contain a cyclic reference".

4. Search in a rotated sorted array (O(log n))

Derive the approach: with distinct values, at least one half around the midpoint is sorted. Decide whether the target lies in that half's value range, then discard the other half.

Why this, not the alternatives: a linear scan is correct but O(n). Finding the rotation pivot then binary searching is also O(log n), with extra boundary logic. Sorting would destroy original indices and cost O(n log n).

What changes: duplicates can hide which half is sorted; shrinking ambiguous equal endpoints can degrade to O(n). If the input is not a rotation of a sorted sequence, the elimination proof no longer holds. Return original index, not insertion position.

Trace and checks: `[4,5,6,1,2,3]`, target 2: identify the sorted left half, exclude it, then find 2 on the right. Test no rotation, one element, absent target and pivot at an endpoint. O(log n) time, O(1) extra space for distinct values.

Strong answer signal: justify the discarded half and name the distinctness assumption.

```
int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                                  // left half sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {                                                // right half sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
```

Say: this search assumes distinct values. With duplicates, choosing a sorted half may be ambiguous and worst-case search can become O(n). For nonnegative array indices, the unsigned-shift midpoint is valid; `lo + (hi - lo) / 2` is easier to explain.

5. Find the K-th largest element (quickselect idea vs heap)

Derive the approach: retain the k greatest values seen so far in a min-heap. Its smallest retained value is the kth greatest at each prefix.

Why this, not the alternatives: full sorting is simpler for one small static input or if full order is needed. Quickselect gives expected O(n) selection with mutation and pivot considerations. A size-k heap gives O(n log k) work and fits streaming input without retaining all n values.

What changes: duplicates count toward rank unless “distinct kth” is explicitly requested. A max-heap of all elements needs O(n) state. For k near n, selecting the smaller complementary rank may save space when n is known. Invalid k must fail predictably.

Trace and checks: stream 7, 2, 9, 5 with k=2 ends with 7 and 9; return 7. Test ties, k=1, k=n and negative numbers. O(k) retained state.

Strong answer signal: choose based on streaming, mutation and output requirements rather than declaring one universal fastest method.

```
int kthLargest(int[] nums, int k) {
    if (k < 1 || k > nums.length) throw new IllegalArgumentException("invalid k");
    PriorityQueue<Integer> minHeap = new PriorityQueue<>(k);
    for (int n : nums) { minHeap.offer(n); if (minHeap.size() > k) minHeap.poll(); }
    return minHeap.peek();                                        // O(n log k), O(k) space
}
```

Say: quickselect averages O(n) but O(n²) worst case; the heap works for streams too.

How to Run the Final 48 Hours

- **Day -2:** read Section R aloud, redo 3 coding programs from memory on paper.
- **Day -1:** one 45-minute system design against a timer, rehearse 5 STAR stories, review your resume project by project (be ready to defend every line).
- **Interview day:** clarify requirements first, think aloud, state trade-offs, test your code with edge cases (null, empty, duplicates, large input), and ask thoughtful questions at the end.

Part 07 - Question Bank Vol. 6 - Kubernetes, Terraform, APIs and leadership

Senior Java Question Bank, Vol. 6

Kubernetes in Depth · Terraform · gRPC/GraphQL · Architecture & Leadership · Mock Interview Script (Q99–Q122)

V. Kubernetes in Depth

First explain images, containers, Pods and Deployments; then use Q99–Q105 to discuss reconciliation, traffic, resources and rollout diagnosis.

Q99. What happens when you run `kubectl apply –f deployment.yaml`?

A good SDE-3 interview answer is:

`kubectl` sends the manifest to the **API server**, which authenticates, authorises (RBAC), runs admission controllers and persists desired state in **etcd**. The **Deployment controller** (in controller-manager) creates a ReplicaSet, which creates Pod objects. The **scheduler** assigns each unscheduled Pod to a node (resources, affinity, taints). The node's **kubelet** sees the assignment, asks the container runtime to pull the image and start containers, wires volumes/secrets, runs probes and reports status. **kube-proxy**/CNI program networking so the Service routes to ready Pods. Everything works by *reconciling desired vs actual state*, so the answer shows you understand controllers, not just commands.

How to understand it

Worked explanation: The API stores desired state; controllers repeatedly compare it with observed state and take corrective action. The successful apply response therefore precedes actual Pod readiness.

Key terms you should know

- **Deployment** — A Kubernetes controller that manages replicated Pods and supports controlled rollout of new versions.
- **Pod** — Kubernetes' smallest deployable unit, containing one or more containers that share networking and storage context.
- **proxy** — An object that wraps another object and intercepts method calls, commonly used by Spring for transactions, security and AOP.
- **Service** — A stable Kubernetes networking abstraction that exposes a group of Pods behind a stable endpoint.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does apply success prove rollout success? No; inspect deployment conditions, scheduling, image startup and readiness separately.

30-second version

> `kubectl` sends the manifest to the **API server**, which authenticates, authorises (RBAC), runs admission controllers and persists desired state in **etcd**. The **Deployment controller** (in controller-manager) creates a ReplicaSet, which creates Pod objects.

Q100. How do requests, limits and QoS classes affect a Java service?

A good SDE-3 interview answer is:

Requests inform scheduling and resource allocation; limits constrain runtime usage. CPU limits normally throttle, while memory-limit breaches can trigger OOM termination. Under container-level QoS rules, Guaranteed requires matching nonzero CPU and memory requests/limits for every container; equal memory settings alone are insufficient. Measure Java heap and native headroom plus startup/steady-state CPU before selecting values. Omitting CPU limits is a platform-policy trade-off, not a universal recommendation. Kubernetes QoS.

How to understand it

Worked explanation: Scheduling uses requests to decide placement; limits constrain runtime use. CPU throttling slows work, whereas memory pressure can kill a container, so their failure symptoms differ.

Key terms you should know

- **GC** — Garbage collection: automatic identification and reclamation of heap memory that is no longer reachable.
- **JIT** — Just-in-time compilation: runtime compilation of frequently executed bytecode into optimized native machine code.
- **OOMKilled** — A container termination status indicating the operating system killed the process because it exceeded its memory limit.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does equal memory request/limit guarantee Guaranteed QoS? No; the required CPU and memory settings must match for every relevant container under the QoS rules.

30-second version

> Requests, limits and QoS have different roles. Distinguish throttling from memory kills, account for every container, and choose Java resource settings from measured demand rather than a copied percentage.

Q101. HPA vs VPA vs Cluster Autoscaler: how do you scale?

A good SDE-3 interview answer is:

HPA adds/removes Pod replicas from CPU, memory or custom metrics (requests per second, Kafka lag via KEDA). **VPA** recommends or applies per-Pod resource requests according to its update mode. Some modes evict/recreate Pods; newer supported clusters can apply some changes in place and fall back to recreation. Avoid having VPA and HPA independently control the same metric without an explicit policy. **Cluster Autoscaler**

(or GKE Autopilot) adds nodes when Pods are unschedulable. Java specifics: scale-up lag from JVM warm-up, so set sensible stabilisation windows, min replicas, readiness only after warm-up, and HPA stabilization/minimum replicas to manage replica scale-down. A PDB constrains voluntary evictions through the Eviction API, such as node drains; it does not block a Deployment or HPA from reducing desired replicas.

How to understand it

Worked explanation: HPA changes replica count, VPA changes per-Pod resource requests, and a node autoscaler changes available cluster capacity. A new replica remains Pending if no suitable node can host it.

Key terms you should know

- **GKE** — Google Kubernetes Engine, Google's managed Kubernetes service.
- **Pod** — Kubernetes' smallest deployable unit, containing one or more containers that share networking and storage context.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Will a PDB block HPA scale-down? No; it constrains supported voluntary evictions, not every change to desired replicas.

30-second version

> **HPA** adds/removes Pod replicas from CPU, memory or custom metrics (requests per second, Kafka lag via KEDA). **VPA** adjusts requests per Pod (restarts them; good for right-sizing, don't combine with HPA on the same metric).

Q102. How does Kubernetes networking work (Service types, Ingress, NetworkPolicy)?

A good SDE-3 interview answer is:

Every Pod gets its own IP (flat network via the CNI plugin). A **Service** gives a stable virtual IP/DNS name (`svc.namespace.svc.cluster.local`) load-balancing across ready Pods: `ClusterIP` (internal), `NodePort`, `LoadBalancer` (cloud LB), `Headless` (direct Pod IPs for StatefulSets). **Ingress/Gateway API** provides L7 routing, TLS and host/path rules. **NetworkPolicy** is default-allow until you add rules; adopt default-deny plus explicit allows. Mention DNS caching and `ndots` as a classic source of latency.

How to understand it

Worked explanation: A Service gives clients a stable discovery/routing boundary while Pod addresses change. Network policy controls permitted flows when supported by the network implementation; DNS and routing still need to work.

Key terms you should know

- **plugin** — A Maven component that provides build functionality such as compilation, testing or packaging.
- **Pod** — Kubernetes' smallest deployable unit, containing one or more containers that share networking and storage context.
- **Service** — A stable Kubernetes networking abstraction that exposes a group of Pods behind a stable endpoint.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does one policy isolate every Pod? No; isolation depends on selected Pods, direction and applicable policies. Test required ingress and egress explicitly.

30-second version

> Every Pod gets its own IP (flat network via the CNI plugin). A **Service** gives a stable virtual IP/DNS name (`svc.namespace.svc.cluster.local`) load-balancing across ready Pods: `ClusterIP` (internal), `NodePort`, `LoadBalancer` (cloud LB), `Headless` (direct Pod IPs for StatefulSets).

Q103. Deployment vs StatefulSet, and how does storage work?

A good SDE-3 interview answer is:

Deployments suit stateless, interchangeable Pods. **StatefulSets** give stable identities (`app-0`), ordered rollout, stable network names via a headless Service and a per-replica **PersistentVolumeClaim**, which databases, Kafka and Zookeeper need. Storage: a PVC requests capacity from a StorageClass that dynamically provisions a PV (a persistent disk). Prefer managed databases (Cloud SQL) over running them in-cluster unless there is a strong reason.

How to understand it

Worked explanation: A StatefulSet preserves per-replica identity and storage association across replacement. That helps a database recognize its member, but database replication and consistency still belong to the database.

Key terms you should know

- **Service** — A stable Kubernetes networking abstraction that exposes a group of Pods behind a stable endpoint.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a persistent volume replace backup? No; persistent storage can retain corruption or deletion and needs an independent recovery strategy.

30-second version

> Deployments suit stateless, interchangeable Pods. **StatefulSets** give stable identities (`app-0`), ordered rollout, stable network names via a headless Service and a per-replica **PersistentVolumeClaim**, which databases, Kafka and Zookeeper need.

Q104. How do you do safe rollouts and keep availability during node maintenance?

A good SDE-3 interview answer is:

Rolling update with `maxUnavailable: 0, maxSurge: 25%`, accurate readiness probes, `preStop` sleep plus graceful shutdown, **PodDisruptionBudget** (e.g., `minAvailable: 2`) so drains don't evict too many Pods, anti-affinity/topology spread constraints to distribute replicas across zones, and canary or progressive delivery (Argo Rollouts, Cloud Deploy) with automated rollback on SLO breach. Rollback with `kubectl rollout undo`.

How to understand it

Worked explanation: During a drain, keep enough ready capacity elsewhere, allow in-flight work to finish and prevent replacement replicas from receiving traffic too early. Availability requires both placement and lifecycle behavior.

Key terms you should know

- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can rollout undo reverse external effects? No; schema and business side effects require their own compatibility and recovery plan.

30-second version

> Rolling update with `maxUnavailable: 0, maxSurge: 25%`, accurate readiness probes, `preStop` sleep plus graceful shutdown, **PodDisruptionBudget** (e.g., `minAvailable: 2`) so drains don't evict too many Pods, anti-affinity/topology spread constraints to distribute replicas across zones, and canary or progressive delivery (Argo Rollouts, Cloud Deploy) with automated rollback on SLO breach. Rollback with `kubectl rollout undo`.

Q105. Helm vs Kustomize, and how do you manage config and secrets?

A good SDE-3 interview answer is:

Helm templates charts with values and versioned releases (good for packaging and third-party apps; templating can get unreadable). Kustomize overlays plain YAML per environment with patches (no templating, built into `kubectl`). Config via ConfigMaps (mounted files or env), secrets via the cloud secret manager with the CSI driver or External Secrets Operator rather than committing base64 Secrets to Git. Use GitOps (Argo CD/Config Sync) so Git is the source of truth and drift is reverted.

How to understand it

Worked explanation: Helm renders parameterized manifests; Kustomize applies overlays to manifests. In either case, inspect the final resources and how secret values reach the runtime.

Key terms you should know

- **Helm** — A tool for packaging, templating and managing Kubernetes application releases.
- **Kustomize** — A tool for composing Kubernetes manifests using overlays and patches.
- **GitOps drift** — A difference between declared state in Git and the running environment.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Are base64 Kubernetes Secrets encrypted by that encoding? No; access control, storage encryption and secret lifecycle remain necessary.

30-second version

> Helm templates charts with values and versioned releases (good for packaging and third-party apps; templating can get unreadable). Kustomize overlays plain YAML per environment with patches (no templating, built into `kubectl`).

W. Terraform / Infrastructure as Code

Terraform questions before state internals

TF1 How does Terraform turn configuration into infrastructure

Foundation: Configuration describes desired resources; providers communicate with platform APIs. A resource represents a managed object, a data source reads information, and a module groups configuration.

Mid-level follow-up: Explain init → validate → plan → review → apply. A plan proposes changes using configuration, state and refreshed observations; apply performs the selected changes. Check replacement/destruction before approving a plan.

TF2 Why does infrastructure as code still need state and recovery

Senior follow-up: State associates configuration addresses with remote objects. A failed apply can leave some changes completed; inspect reality and state before planning recovery. Concurrent applies, manual drift, provider changes and secrets require backend-specific locking, access controls and reviewed plans. Continue with Q106–Q108 for state, modules and CI workflows.

Reference: Terraform core workflow. These are workflow questions; no cloud provisioning is performed.

Q106. How does Terraform state work and why is the backend important?

A good SDE-3 interview answer is:

Terraform maps your configuration to real resources through a **state file**. Keep it in a **remote backend** (GCS bucket with versioning) with **state locking** so concurrent applies don't corrupt it, restrict access (state may contain secrets), and never edit by hand (use `terraform state mv/rm`, `import`). Separate state per environment/component to reduce blast radius.

How to understand it

Worked explanation: State connects a configuration address to an existing remote object, allowing Terraform to update rather than recreate it. Losing or sharing the wrong state can change which resources an apply believes it owns.

Key terms you should know

- **bucket** — One position in a hash table's internal array where entries can be stored.
- **Terraform state** — Terraform's record of resources it manages and the mapping between configuration and real infrastructure.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does every remote backend lock state? No; verify that backend's locking and recovery semantics instead of assuming remote storage implies coordination.

30-second version

> Terraform maps your configuration to real resources through a **state file**. Keep it in a **remote backend** (GCS bucket with versioning) with **state locking** so concurrent applies don't corrupt it, restrict access (state may contain secrets), and never edit by hand (use `terraform state mv/rm`, `import`).

Q107. Explain plan/apply, drift, modules and workspaces.

A good SDE-3 interview answer is:

A plan compares desired configuration with state and refreshed remote observations; apply performs planned changes. Drift is an out-of-band difference: decide whether to restore the declaration or deliberately adopt the change. Import associates an existing unmanaged object with a configuration address; it is not a generic fix for drift in already managed resources. Modules package configuration; separate state boundaries constrain impact. Lifecycle protections such as prevent*destroy and ignore*changes have configuration-dependent limits and need review.

How to understand it

Worked explanation: Drift means real infrastructure diverged from the declared or recorded model. Decide whether to restore configuration or intentionally adopt the change; import associates an unmanaged object rather than automatically fixing all drift.

Key terms you should know

- **drift** — A difference between infrastructure declared in Terraform and infrastructure that actually exists.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can prevent_destroy guarantee permanent protection? No; it has configuration-dependent limits and must be complemented by permissions, review and recovery controls.

30-second version

> Explain configuration, state and reality separately. Resolve drift deliberately; import is for ownership association. Review lifecycle exceptions and state boundaries before applying changes.

Q108. How do you run IaC in CI/CD safely?

A good SDE-3 interview answer is:

PR → `fmt`/`validate`/`tflint`/policy checks (OPA/Sentinel, Checkov) → `plan` posted to the PR → human approval → `apply` from a pipeline using a least-privilege service account with Workload Identity Federation (no long-lived keys). Pin provider and module versions, keep secrets in Secret Manager (not variables files), tag/label resources for cost, and protect the main branch.

How to understand it

Worked explanation: A reviewed plan describes intended mutations at a point in time. Protect its artifact and apply the approved changes under controlled credentials; changed inputs or intervening state require renewed planning.

Key terms you should know

- **Plan** — Terraform output describing proposed infrastructure changes.
- **State locking** — Coordination that prevents concurrent writers to the same supported state backend.
- **Workload identity federation** — Exchanging trusted workload identity for cloud access without a long-lived service-account key.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can marking a variable sensitive keep it out of state? It mainly redacts display; protect state and plan data according to actual provider behavior.

30-second version

> PR → `fmt`/`validate`/`tflint`/policy checks (OPA/Sentinel, Checkov) → `plan` posted to the PR → human approval → `apply` from a pipeline using a least-privilege service account with Workload Identity Federation (no long-lived keys). Pin provider and module versions, keep secrets in Secret Manager (not variables files), tag/label resources for cost, and protect the main branch.

X. gRPC, GraphQL & API Styles

API questions before protocol choices

API1 How does a client request become a server response

Foundation: Identify the endpoint, request method, headers/body, authentication and response status/body. Define the contract before selecting REST, RPC or GraphQL. Distinguish transport success from business success and include validation/error behavior.

Mid-level follow-up: Trace the request through routing, authorization, application logic and its data/dependency calls. Set timeouts and cancellation boundaries; explain versioning, pagination and compatibility.

API2 What happens if the response is lost after the server commits

Senior follow-up: A timeout can leave an ambiguous outcome. Retrying a write needs a safe method contract or an application-level idempotency strategy. HTTP idempotency concerns the intended server effect, not identical responses. Establish retry and ordering guarantees before Q109–Q112 compare gRPC, GraphQL and streaming styles.

Reference: HTTP semantics, RFC 9110. This is a request-contract walkthrough, not a deployed API.

Q109. When and how do you use gRPC?

A good SDE-3 interview answer is:

gRPC uses HTTP/2 and Protobuf (compact binary, schema-first, generated clients) and supports four call types: unary, server streaming, client streaming and bidirectional streaming. Strengths: low latency, strict contracts, multiplexing, built-in deadlines/cancellation. Practices: always set **deadlines**, propagate them downstream, use status codes, evolve schemas by only adding fields with new numbers (never reuse or renumber), use client-side load balancing or a mesh because long-lived HTTP/2 connections defeat L4 load balancers, and add interceptors for auth, tracing and metrics. Weaknesses: poor browser support (needs gRPC-Web), harder debugging.

How to understand it

Worked explanation: A client stub serializes a typed request, performs an RPC and receives a result or status. A deadline bounds the caller's wait; the server must also observe cancellation and stop unnecessary work.

Key terms you should know

- **gRPC** — A high-performance RPC framework commonly using HTTP/2 and Protocol Buffers for service-to-service communication.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why can L4 balancing be uneven? Many RPCs share a long-lived connection, so balancing connections does not necessarily balance individual requests.

30-second version

> gRPC uses HTTP/2 and Protobuf (compact binary, schema-first, generated clients) and supports four call types: unary, server streaming, client streaming and bidirectional streaming. Strengths: low latency, strict contracts, multiplexing, built-in deadlines/cancellation.

Q110. GraphQL vs REST: trade-offs and pitfalls?

A good SDE-3 interview answer is:

GraphQL lets clients select fields through a typed schema, useful for different front-end data needs. It can reduce client round trips while still causing excessive backend work. Batch resolver loads with DataLoader; enforce authorization at the relevant object/field boundary, query complexity/depth limits, timeouts and appropriate pagination.

HTTP POST supports query and mutation operations; servers may also support GET for queries. GET queries can support HTTP/CDN caching, and persisted query identifiers can keep URLs manageable. Cache keys must account for the operation, variables and relevant authorization context. Mutations must not execute through GET. HTTP status depends on the error stage and negotiated response format; execution errors may appear in an HTTP 200 response, but not every failure is a 200.

REST can be simpler for resource-oriented, cache-friendly public APIs. In Spring for GraphQL, know `@QueryMapping` and `@BatchMapping` and the server's enabled transports.

How to understand it

Worked explanation: A small GraphQL query can trigger many resolver/database calls. Batch related loads and authorize each relevant object or field, while measuring total backend work rather than only HTTP request count.

Key terms you should know

- **Resolver** — Code that obtains a requested field value.
- **DataLoader** — A facility for batching and caching related loads within its configured scope.
- **Persisted query** — A stored query identified by an agreed identifier.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can all query results share one cache key? No; variables and authorization context can change both data and visibility.

30-second version

> GraphQL offers flexible field selection but needs resolver batching, authorization and query-cost controls. It is not POST-only: supported GET queries can use HTTP caching, while mutations use POST. Cache keys and error handling must follow the actual operation and transport contract.

GraphQL over HTTP.

Q111. WebSocket vs SSE vs long polling vs webhooks?

A good SDE-3 interview answer is:

Long polling: simple fallback, high overhead. **SSE**: one-way server → client over HTTP, auto-reconnect, great for notifications/feeds. **WebSocket**: full-duplex, for chat/collaboration, needs sticky-less design with a pub/sub backplane and heartbeat handling. **Webhooks**: server-to-server callbacks; require signature verification, retries, idempotency and replay protection.

How to understand it

Worked explanation: SSE delivers server events over an HTTP response; WebSocket allows both ends to send messages. A webhook instead calls another server, whose acknowledgement and retries form a delivery protocol.

Key terms you should know

- **idempotency** — The property that repeating the same logical request produces the same intended outcome rather than creating duplicate effects.
- **Pub/Sub** — Google Cloud's asynchronous messaging service for publishing messages to topics and delivering them to subscriptions.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does reconnect restore lost messages? Only if the application defines replay, cursors or resynchronization; transport reconnection alone is insufficient.

30-second version

> Long polling: simple fallback, high overhead. **SSE**: one-way server → client over HTTP, auto-reconnect, great for notifications/feeds.

Q112. What is contract-first API design?

A good SDE-3 interview answer is:

Agree the API schema and behavioral contract with consumers first, then generate suitable stubs/clients and validate implementations in CI. Include errors, compatibility, authorization and retry semantics, not only fields. Linting, schema checks and consumer contracts reduce drift but cannot make it impossible; custom code and semantic changes still need tests and review.

How to understand it

Worked explanation: Review the request, response and failure contract with consumers before generating code. Generation reduces mechanical mismatch, while compatibility tests check whether implementations still honor the agreed behavior.

Key terms you should know

- **API contract** — An agreed description of requests, responses and observable behavior.
- **Contract drift** — A mismatch between an API specification and its implementation or consumer expectations.
- **Consumer-driven contract test** — A test checking a provider against interactions required by a consumer.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Do generated clients prevent all drift? No; validation and tests must detect custom implementation changes and semantic differences.

30-second version

> Contract-first aligns producers and consumers before implementation. Generated code helps, while validation and behavioral compatibility tests catch the drift generation alone cannot prevent.

Y. Architecture & Leadership

First explain service boundaries, then follow the design interview sequence. For distributed-event questions, review saga, outbox and idempotency before choosing an architecture.

Q113. What are the pitfalls of event-driven architecture?

A good SDE-3 interview answer is:

Hard-to-follow flows (no single place shows the process: need tracing and an event catalogue), eventual consistency surprises, duplicate/out-of-order events, schema evolution, poison messages, accidental coupling through event contents, and debugging complexity. Mitigate with clear event ownership, versioned schemas in a registry, idempotent consumers, DLQs with replay tooling, correlation IDs, and choosing events for facts ("OrderPlaced") rather than commands.

How to understand it

Worked explanation: An event describes a fact another service may react to later. That delay creates visible intermediate states, and duplicate delivery means the same fact may be handled again.

SDE-3 interview checkpoints

Follow-up with expected reasoning: When is a command appropriate? When requesting an action from a responsible handler; name the intent explicitly instead of disguising it as a past-tense fact.

30-second version

> Hard-to-follow flows (no single place shows the process: need tracing and an event catalogue), eventual consistency surprises, duplicate/out-of-order events, schema evolution, poison messages, accidental coupling through event contents, and debugging complexity. Mitigate with clear event ownership, versioned schemas in a registry, idempotent consumers, DLQs with replay tooling, correlation IDs, and choosing events for facts ("OrderPlaced") rather than commands.

Q114. Outbox vs CDC vs dual writes: what is the difference?

A good SDE-3 interview answer is:

Dual write (save to DB then publish) can lose or duplicate events if one fails: avoid. **Transactional outbox**: write the event to an `outbox` table in the same transaction as the business data; a poller/relay publishes and marks it sent (at-least-once). **CDC** (Debezium) tails the DB log and publishes outbox rows (or table changes) without polling load. Consumers must be idempotent either way; clean the outbox periodically.

How to understand it

Worked explanation: Committing business data and an outbox row together closes the database/publication gap. The relay can still publish twice if it crashes before recording success, so consumers remain duplicate-safe.

Key terms you should know

- **CDC** — Change Data Capture: publishing database changes by reading the database's change log or transaction log.
- **dual write** — Writing related state to two independent systems in one application operation, which can become inconsistent if one write succeeds and the other fails.
- **outbox** — A database table used to store events in the same transaction as business data so publication can happen reliably afterward.
- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Is CDC an alternative to the outbox invariant? It can relay outbox rows; raw table changes still need deliberate business-event semantics.

30-second version

> **Dual write** (save to DB then publish) can lose or duplicate events if one fails: avoid. **Transactional outbox**: write the event to an `outbox` table in the same transaction as the business data; a poller/relay publishes and marks it sent (at-least-once).

Q115. Explain DDD essentials you actually apply.

A good SDE-3 interview answer is:

DDD starts with a shared business language and bounded contexts in which each model has a clear meaning. Aggregates group state whose invariants a root controls; entities have identity, while value objects are defined by value. Bounded contexts can inform service boundaries but do not require one service each. Keeping transactions within an aggregate is a useful design guideline, not a universal prohibition on broader atomic work. Apply domain events and anti-corruption layers where they clarify real complexity.

How to understand it

Worked explanation: An Order aggregate protects rules such as valid status transitions; its root controls changes to owned parts. A bounded context defines the meaning of that model, rather than dictating one deployment per concept.

Key terms you should know

- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.
- **transaction** — A unit of database work treated as one logical operation with defined commit/rollback behavior.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Must each aggregate be a microservice? No; model boundaries and deployment boundaries are distinct decisions.

30-second version

> Explain one business invariant and its model boundary. Aggregates control consistent state changes; bounded contexts define meaning. Deployment choices and transaction scope still require judgment.

Q116. What is hexagonal (ports & adapters) architecture?

A good SDE-3 interview answer is:

The domain core depends on nothing; it exposes **ports** (interfaces) and the outside world connects via **adapters** (REST controller, JPA repository, Kafka publisher). Dependencies point inward, so you can test business logic without frameworks, swap infrastructure, and enforce rules with ArchUnit. In Spring: `domain` and `application` packages free of Spring annotations where practical, with adapters in `infrastructure`.

How to understand it

Worked explanation: An application use case asks a payment port to charge; an adapter translates that request into a provider API. The use case can be tested with a controlled implementation of the port.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Who owns the interface? The boundary should express application needs rather than expose every detail of the external SDK.

30-second version

> The domain core depends on nothing; it exposes **ports** (interfaces) and the outside world connects via **adapters** (REST controller, JPA repository, Kafka publisher). Dependencies point inward, so you can test business logic without frameworks, swap infrastructure, and enforce rules with ArchUnit.

Q117. How do you scale a read-heavy system?

A good SDE-3 interview answer is:

In order of cost: optimise queries and indexes → add caching layers (CDN, application cache, Redis) → read replicas (account for replication lag; route read-your-writes to the primary) → CQRS read models → partition/shard data (choose a key that avoids hotspots; cross-shard joins and transactions become hard) → precompute. State the trade-off at each step and measure before moving up.

How to understand it

Worked explanation: Measure which reads dominate and whether they tolerate staleness. An index reduces database work; a cache reuses results; a replica shifts reads but may lag behind writes.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you preserve read-your-writes? Route the relevant reads appropriately or use a verified consistency mechanism instead of assuming replicas are current.

30-second version

> In order of cost: optimise queries and indexes → add caching layers (CDN, application cache, Redis) → read replicas (account for replication lag; route read-your-writes to the primary) → CQRS read models → partition/shard data (choose a key that avoids hotspots; cross-shard joins and transactions become hard) → precompute. State the trade-off at each step and measure before moving up.

Q118. How do you do back-of-the-envelope capacity estimates?

A good SDE-3 interview answer is:

Example: 10M DAU × 20 requests/day = 200M/day ≈ 2.3k req/s average, ×3–5 for peak ≈ 10k req/s. Storage: 1 KB/record × 100M/day ≈ 100 GB/day ≈ 36 TB/year before replication. Bandwidth: 10k req/s × 10 KB ≈ 100 MB/s. Use round numbers, state assumptions, and derive node counts from per-node throughput. Interviewers want structured reasoning, not exact figures.

How to understand it

Worked explanation: Convert daily volume to average rate, state a peak multiplier, then estimate bytes and concurrent work. Each estimate should expose an assumption that can be revised.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you estimate concurrency? Arrival rate multiplied by average time in the system gives average in-flight work under the model's stable conditions.

30-second version

> Example: 10M DAU × 20 requests/day = 200M/day ≈ 2.3k req/s average, ×3–5 for peak ≈ 10k req/s.
> Storage: 1 KB/record × 100M/day ≈ 100 GB/day ≈ 36 TB/year before replication.

Q119. Explain SLOs, error budgets and incident severity.

A good SDE-3 interview answer is:

Define the denominator first. A request-based 99.9% SLO permits 0.1% bad eligible requests; at ten million requests that is 10,000 bad requests. A time-based 99.9% availability SLO permits 43.2 minutes in a 30-day window. These are not interchangeable when traffic varies. Define success, latency thresholds, exclusions and measurement windows, then use burn-rate alerts and a documented response policy. Incident severity follows customer impact, not a single infrastructure metric. Google SRE workbook.

How to understand it

Worked explanation: An SLO defines which outcomes count as good and what fraction must meet that rule over a window. The remaining allowance is the error budget, which can be consumed quickly during an incident.

Key terms you should know

- **SLO** — Service Level Objective: a measurable reliability target, such as 99.9% successful requests.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Are request-based and time-based budgets interchangeable? No; traffic variation changes their meaning and the associated response policy.

30-second version

> Define the denominator first. A request-based 99.9% SLO permits 0.1% bad eligible requests; at ten million requests that is 10,000 bad requests.

Q120. How do you lead and mentor as a senior engineer?

A good SDE-3 interview answer is:

Set direction through design docs/ADRs, review for learning not gatekeeping, pair on hard problems, delegate with clear context and ownership, grow others by giving stretch tasks and feedback, and raise the bar through shared standards and automation. Measure impact through team outcomes (lead time, change failure rate, onboarding time) rather than personal output. Have a concrete mentoring example with a result.

How to understand it

Worked explanation: Describe a person or team capability that improved because of your coaching or design work. Explain what you delegated, what support you supplied and how ownership increased.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What is the senior evidence? A concrete team outcome and what you learned, not a list of mentoring practices.

30-second version

> Set direction through design docs/ADRs, review for learning not gatekeeping, pair on hard problems, delegate with clear context and ownership, grow others by giving stretch tasks and feedback, and raise the bar through shared standards and automation. Measure impact through team outcomes (lead time, change failure rate, onboarding time) rather than personal output.

Q121. Tell me about a time you made a wrong technical decision.

A good SDE-3 interview answer is:

Pick a real, moderately significant mistake: what you decided and why it seemed right, the signal that showed it was wrong (metric/incident), how you owned it, how you corrected course (rollback, migration plan), and the **process change** afterward (ADR with revisit date, load test, spike). Seniors are valued for learning speed and honesty, not perfection.

How to understand it

Worked explanation: Reconstruct the decision using what was known then, identify the later evidence that invalidated an assumption, and explain your correction. Owning the mistake includes changing the decision process.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Should you choose a harmless fake weakness? No; use a real example you can discuss honestly without disclosing confidential details.

30-second version

> Pick a real, moderately significant mistake: what you decided and why it seemed right, the signal that showed it was wrong (metric/incident), how you owned it, how you corrected course (rollback, migration plan), and the **process change** afterward (ADR with revisit date, load test, spike). Seniors are valued for learning speed and honesty, not perfection.

Q122. Why do you want to leave your current company / why this role?

A good SDE-3 interview answer is:

Be positive and forward-looking: what you have achieved there and what you want next (larger scale, domain, ownership, cloud-native architecture), tied to something specific about the new company you researched. Never criticise your employer.

How to understand it

Worked explanation: Connect the role's actual responsibilities to the work you want to do and experience you can contribute. Explain motivation specifically enough that it would not fit every employer.

Key terms you should know

- **Role fit** — How the position matches the responsibilities and growth you seek.
- **Evidence** — Specific examples supporting your reasons for the move.

SDE-3 interview checkpoints

Follow-up with expected reasoning: How do you substantiate the fit? Refer to verified role/team information and your own relevant experience, avoiding invented company claims.

30-second version

> Be positive and forward-looking: what you have achieved there and what you want next (larger scale, domain, ownership, cloud-native architecture), tied to something specific about the new company you researched. Never criticise your employer.

Z. Config "Programs" Interviewers Ask You to Write

Kubernetes Deployment with probes and graceful shutdown

Derive the approach: startup, routing readiness and unrecoverable liveness are different states. Give each its own probe, reserve resources and allow draining before termination.

Why this, not the alternatives: one liveness check for slow startup can cause repeated restarts; restarting on a downstream outage can amplify that outage. An immediate kill loses in-flight work. A fixed preStop delay is only a simple allowance for routing propagation, not proof that traffic has drained.

What changes: the preStop delay consumes part of the termination grace period; keep enough remaining budget for application shutdown. The sample exec hook requires `sleep` in the image, which a distroless image may lack. The sample omits CPU limits, so memory request=limit alone does not imply Guaranteed QoS. Set limits from measured behavior and platform policy.

Trace and checks: simulate a slow startup, readiness failure and SIGTERM during an active request. Verify actual probe paths/security, image commands and total shutdown time. This is an operational exercise; judge capacity and failure behavior rather than Big O.

Strong answer signal: connect every setting to an observable lifecycle transition.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: orders }
spec:
  replicas: 3
  strategy: { rollingUpdate: { maxUnavailable: 0, maxSurge: 25% } }
  selector: { matchLabels: { app: orders } }
  template:
    metadata: { labels: { app: orders } }
    spec:
      terminationGracePeriodSeconds: 40
      containers:
      - name: orders
        image: europe-docker.pkg.dev/proj/repo/orders:1.4.2
        ports: [{ containerPort: 8080 }]
        env: [{ name: JAVA_TOOL_OPTIONS, value: "-XX:MaxRAMPercentage=70" }]
        resources: { requests: { cpu: 500m, memory: 1Gi }, limits: { memory: 1Gi } }
        startupProbe:   { httpGet: { path: /actuator/health/liveness,  port: 8080 },
failureThreshold: 30, periodSeconds: 2 }
        readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8080 }, periodSeconds:
5 }
        livenessProbe:  { httpGet: { path: /actuator/health/liveness,  port: 8080 }, periodSeconds:
10 }
        lifecycle: { preStop: { exec: { command: ["sleep", "10"] } } }
```

Terraform: Cloud Run service with a dedicated service account

Derive the approach: declare a reproducible service resource and bind it to a workload identity with explicitly granted permissions. The resource definition makes desired deployment state reviewable.

Why this, not the alternatives: console changes are quick but harder to reproduce and audit. A shared broad-privilege identity increases blast radius. Creating a dedicated service account alone grants no application permissions; add only required IAM bindings separately.

What changes: maximum instances influence cost and downstream capacity, but do not replace application concurrency controls. Secrets should be referenced from a secret store. Remote Terraform state needs access control and locking. The snippet assumes project/provider setup, deployment permissions and service enablement already exist.

Trace and checks: inspect a plan, apply in a nonproduction project, then check runtime identity and access to one allowed and one denied resource. Never interpret successful parsing as proof of working IAM. Consider warm-instance cost and rollout behavior rather than algorithmic complexity.

Strong answer signal: separate resource creation, deployment identity and runtime permissions.

```
resource "google_service_account" "orders" { account_id = "orders-runtime" }

resource "google_cloud_run_v2_service" "orders" {
  name     = "orders"
  location = "europe-west1"
  template {
    service_account = google_service_account.orders.email
    scaling {
      min_instance_count = 1
      max_instance_count = 20
    }
    containers {
      image = "europe-docker.pkg.dev/proj/repo/orders:1.4.2"
      resources { limits = { cpu = "1", memory = "1Gi" } }
    }
  }
}
```

Protobuf contract (additive evolution rules)

Derive the approach: messages need a stable wire identity independent of source field ordering. Assign durable field numbers and reserve removed numbers before later schema changes.

Why this, not the alternatives: JSON is easy to inspect and may be sufficient; Protobuf gives compact typed contracts and code generation. It introduces tooling and schema-evolution obligations. Generated types do not eliminate domain validation.

What changes: reusing a field number can make an old payload mean something different. Adding a field is not automatically behaviorally safe if old clients require new semantics. Money needs currency and units; an int64 total alone is incomplete. A scalar default zero may not distinguish absence unless presence is modeled deliberately.

Trace and checks: encode with an old schema and decode with the new, then reverse direction where supported. Test omitted fields, unknown values and currency mismatch. Serialization cost scales with payload size; compatibility is the main concern.

Strong answer signal: distinguish wire readability from business compatibility.

```
syntax = "proto3";
service OrderService { rpc GetOrder (GetOrderRequest) returns (Order); }
message GetOrderRequest { string order_id = 1; }
message Order { string id = 1; string status = 2; int64 total_minor_units = 3; reserved 4; }
```

Say: never reuse field numbers (`reserved`), only add new fields, money in minor units.

AA. Mock Interview Script (with follow-ups)

Round 1: Technical deep dive (45 min).

1. "Walk me through the architecture of your last project." → *Follow-ups:* Why microservices? How did you split data? What broke in production? What would you change?
2. "How do you make a Spring service handle 5× traffic?" → profile first, DB/index, caching, pool sizing, horizontal scaling, async decoupling, load test.
3. "A `@Transactional` method isn't rolling back." → proxy/self-invocation, checked exception, swallowed exception, wrong propagation, non-transactional datasource.
4. Live coding: LRU cache or rate limiter → *Follow-ups:* thread safety, complexity, tests, distributed version.

Round 2: System design (45 min). "Design an order & payment platform on GCP." Clarify requirements and scale → API and data model → services and events (saga + outbox) → data stores → failure handling and idempotency → security → observability → deployment on GKE/Cloud Run → cost and trade-offs. *Probe:* "What if Pub/Sub delivers twice? If the payment provider times out? If a region fails?"

Round 3: Managerial/behavioural (30 min). Production incident, conflict with a colleague, mentoring a junior, delivering under a deadline, a mistake you owned, biggest impact. Use STAR with metrics and a "what I learned" close.

Closing questions to ask: "What does success look like in 6 months?" "How are architecture decisions made?" "What's your on-call and incident culture like?" "How do you handle tech debt?"

Reminder

Depth beats breadth at 9+ years: for each technology on your resume, be ready to explain *how it works internally, what went wrong, and what you'd do differently.*

Part 08 - Question Bank Vol. 7 - Streams, WebFlux and interview practice

Senior Java Question Bank, Vol. 7

Kafka Streams · WebFlux/Reactive · Security Hardening · LeetCode-Style Coding · Google-Style Design · Amazon Leadership Principles (Q123–Q133)

AB. Kafka Streams

Begin with producers, brokers and consumers. Then Q123–Q125 introduce stream/table views before state management and windows.

Q123. KStream vs KTable vs GlobalKTable?

A good SDE-3 interview answer is:

A **KStream** is an unbounded sequence of independent events (every record matters: clicks, payments). A **KTable** is a changelog view where each key holds its *latest* value (updates replace, a null tombstone deletes), such as the current customer profile. A **GlobalKTable** is fully replicated to every instance, so you can join a stream against it without co-partitioning (suitable only for small reference data). The stream-table duality is the key idea: a table is a stream of updates, and a stream can be aggregated into a table.

How to understand it

Worked explanation: A stream records each customer-address change; a table keeps the latest address per customer. Joining orders to that table answers a different question from preserving every address-change event.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why not always use GlobalKTable? Each instance bears the full replicated reference-state cost, so size and update rate matter.

30-second version

> A **KStream** is an unbounded sequence of independent events (every record matters: clicks, payments). A **KTable** is a changelog view where each key holds its *latest* value (updates replace, a null tombstone deletes), such as the current customer profile.

Q124. How does Kafka Streams manage state, scaling and fault tolerance?

A good SDE-3 interview answer is:

Kafka Streams assigns partition-based tasks to instances sharing an application.id. Stateful operations can use local stores with changelogs for restoration and optional standby replicas. Parallelism depends on the topology and its partitioning. Keyed joins commonly need compatible partitioning; GlobalKTable joins differ. Changing a

key can cause a later key-dependent DSL operation to repartition, while Processor API paths may need explicit handling. Exactly-once processing covers supported Kafka transactional work, not arbitrary external effects. Measure lag and state restore time.

How to understand it

Worked explanation: A stateful task updates local state while its changelog supports restoration elsewhere. Partitioning determines which task owns a key and whether related records meet for a join.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Must every join repartition? No; existing co-partitioning and join type matter, and GlobalKTable joins have different requirements.

30-second version

> Explain key ownership, local state and changelog restoration. Scaling and repartitioning depend on the topology; join requirements and transactional guarantees must be stated for the operation being used.

Q125. Windowing and late data?

A good SDE-3 interview answer is:

Window types: *tumbling* (fixed, non-overlapping), *hopping* (fixed, overlapping), *sliding*, and *session* (gaps of inactivity). Processing uses **event time**; records that arrive after the window end are handled by the **grace period**, after which they are dropped for window-close semantics. Use `Suppressed.untilWindowCloses(...)` with a strict buffer for final window results; `untilTimeLimit` has different behavior. Closure follows stream-time progress, so an idle input may not emit just because wall-clock time passes. Explain the trade-off: a longer grace allows more late events but delays final output and increases state/buffer needs. Test idle streams as well as late events.

How to understand it

Worked explanation: A window groups events by their timestamps; stream-time progress determines when it is too late to update the window. Wall-clock waiting alone may not close it during idle input.

Key terms you should know

- **Event time** — Time assigned to an event, as distinct from when the processor handles it.
- **Grace period** — The allowed lateness before a window closes under Kafka Streams stream-time rules.
- **Window** — A grouping of records over a defined time interval or session.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Why increase grace? It accepts more late data at the cost of later final results and more retained state.

30-second version

> Window types: *tumbling* (fixed, non-overlapping), *hopping* (fixed, overlapping), *sliding*, and *session* (gaps of inactivity). Processing uses **event time**; records that arrive after the window end are handled by the **grace period**, after which they are dropped, and `suppress()` emits only the final result.

AC. Reactive Programming & Spring WebFlux

First establish the thread lifecycle and executor foundation and request contract. Q126 already supplies the reactive foundation: publisher/subscriber → subscription/demand → Mono/Flux → operators. Continue with Q127–Q129 for blocking mistakes, runtime choices and testing.

Q126. Explain Mono, Flux and back-pressure.

A good SDE-3 interview answer is:

`Mono<T>` represents zero or one value; `Flux<T>` represents zero to many. Both also signal completion or failure. Cold, deferred publishers start their work on subscription, but this is not a universal property of all publishers or of the expressions used to construct them. Hot sources may produce independently of subscribers. `Mono.just(blockingCall())` evaluates the call immediately; `Mono.fromCallable(() -> blockingCall())` defers it. Scheduling blocking work safely is a separate decision.

Reactive Streams defines `Publisher`, `Subscriber`, `Subscription` and `Processor`. A subscriber signals demand with `request(n)`; a compliant publisher must honor demand for `onNext` signals. This does not remove the need for bounded buffering and an overflow policy at push-source boundaries. Operators such as `map`, `flatMap`, `zip`, `concatMap`, `retryWhen` and `timeout` compose work. `flatMap` can merge concurrent inner publishers and interleave output; `concatMap` processes them sequentially while preserving order.

How to understand it

Worked explanation: The subscriber requests capacity, the publisher emits permitted values, and completion or error terminates the sequence. A Mono may complete empty; a Flux may emit many values. Source and scheduling semantics determine when work starts.

Key terms you should know

- **Cold publisher** — A source that starts its source work for each subscription.
- **Hot publisher** — A source that can share production independently of individual subscribers.
- **Back-pressure** — Demand signaling that regulates delivery to a subscriber.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does back-pressure eliminate memory limits? No; buffers and push-source overflow policies still need explicit bounds.

30-second version

> Mono models zero or one value and Flux zero to many. Cold deferred work starts on subscription, but hot sources and eager assembly-time calls are exceptions. Back-pressure controls delivery demand; buffering, overflow and blocking-call scheduling still need deliberate policies.

Reactor hot versus cold.

Q127. What are the biggest WebFlux mistakes?

A good SDE-3 interview answer is:

Blocking the event loop stalls unrelated requests. Wrap a synchronous dependency as `Mono.fromCallable(() -> blockingCall()).subscribeOn(Schedulers.boundedElastic())`; `publishOn` changes downstream execution and is not a general fix for blocking source work. Return the publisher from a WebFlux handler and let the

framework subscribe. Preserve context, set bounded concurrency and deadlines, and test error/cancellation paths. Eager work inside `Mono.just(blockingCall())` already ran before subscription. Reactor blocking-call guidance.

How to understand it

Worked explanation: One blocking database call on an event loop can delay many unrelated connections. Deferring that call and assigning it to an appropriate bounded scheduler separates it from the event loop.

Key terms you should know

- **Mono** — A Reactor type representing zero or one asynchronous value.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does publishOn fix work already evaluated in Mono.just? No; that expression ran before the reactive pipeline could schedule it.

30-second version

> Blocking the event loop stalls unrelated requests. Wrap a synchronous dependency as `Mono.fromCallable(() -> blockingCall()).subscribeOn(Schedulers.boundedElastic());` `publishOn` changes downstream execution and is not a general fix for blocking source work.

Q128. WebFlux or Spring MVC with virtual threads?

A good SDE-3 interview answer is:

Choose from the entire workload and library stack. MVC with virtual threads can suit blocking JDBC/JPA workflows while retaining straightforward control flow. WebFlux can suit non-blocking composition, streaming and demand control. A mixed stack is possible with deliberate offloading, bounded concurrency and context handling, but keeps the blocking dependency's limits and adds integration cost. Neither model removes database capacity limits or automatically improves throughput.

How to understand it

Worked explanation: Choose based on the whole dependency path. A blocking JDBC workload fits ordinary blocking code well; an end-to-end reactive path can compose streaming and demand control.

Key terms you should know

- **back-pressure** — A mechanism that prevents an upstream producer from overwhelming a downstream consumer.
- **reactive** — A programming model centered on asynchronous streams and non-blocking processing with explicit demand/back-pressure.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a mixed stack work? Yes with deliberate offloading and limits, but it retains blocking resource costs and added integration complexity.

30-second version

> Compare the complete dependency path and required streaming/concurrency behavior. Virtual threads simplify many blocking workloads; reactive composition can help suitable non-blocking flows. Mixed stacks need explicit scheduling and limits.

Q129. How do you test reactive code?

A good SDE-3 interview answer is:

`StepVerifier` asserts emitted items, errors and completion, with `withVirtualTime` for delays/timeouts; `WebTestClient` tests endpoints; Testcontainers provide real brokers/DBs. Verify cancellation and error paths, not only the happy path.

How to understand it

Worked explanation: Describe the expected signals in order: values, terminal success or error, and cancellation behavior. Virtual time lets time-based operators be exercised without real delays when publishers are created appropriately.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Can a passing value assertion miss a resource leak? Yes; verify termination, cancellation and cleanup as well as emitted data.

30-second version

> `StepVerifier` asserts emitted items, errors and completion, with `withVirtualTime` for delays/timeouts; `WebTestClient` tests endpoints; Testcontainers provide real brokers/DBs. Verify cancellation and error paths, not only the happy path.

AD. Security Hardening

Q130. How do you manage secrets properly?

A good SDE-3 interview answer is:

Never in Git, images, env files committed to repos or logs. Use a secret manager (GCP Secret Manager, Vault) with IAM-scoped access, versioning and audit logs; load at startup or mount as a volume (prefer files over env vars, which leak in dumps and `ps`); **rotate** automatically and design the app to reload or restart on rotation; use short-lived credentials (Workload Identity, IAM database authentication) wherever possible; scan repos for leaked secrets (gitleaks) and revoke immediately if exposed.

How to understand it

Worked explanation: A secret has an owner, permitted readers, expiry/rotation policy and response to exposure. Updating the stored value is only half of rotation; running clients must begin using it safely.

Key terms you should know

- **IAM** — Identity and Access Management: policies controlling who or what can perform which actions on which resources.
- **Workload Identity** — A mechanism that lets workloads obtain cloud identities without embedding long-lived service-account keys.

SDE-3 interview checkpoints

Follow-up with expected reasoning: What happens during overlap? Plan how old and new credentials coexist, refresh, fail and are revoked without exposing values in logs.

30-second version

> Never in Git, images, env files committed to repos or logs. Use a secret manager (GCP Secret Manager, Vault) with IAM-scoped access, versioning and audit logs; load at startup or mount as a volume (prefer files over env vars, which leak in dumps and `ps`); **rotate** automatically and design the app to reload or restart on rotation; use short-lived credentials (Workload Identity, IAM database authentication) wherever possible; scan repos for leaked secrets (gitleaks) and revoke immediately if exposed.

Q131. What is zero-trust and how do you implement it between services?

A good SDE-3 interview answer is:

"Never trust, always verify": no implicit trust from network location. Implementation: **mTLS** between services (mesh-issued short-lived certificates), service identities (SPIFFE or cloud service accounts), per-call authorisation (propagate the user's JWT plus service-level scopes, enforce in each service, not only at the gateway), default-deny network policies, least-privilege IAM, and audit logging. The gateway is one control, not the whole perimeter.

How to understand it

Worked explanation: Authenticating a service's certificate proves its workload identity, not its right to read every user's data. Each hop must enforce the intended authorization context and trusted token audience.

Key terms you should know

- **IAM** — Identity and Access Management: policies controlling who or what can perform which actions on which resources.
- **Service** — An application or platform capability exposed through a defined interface; a Kubernetes Service is a specific networking resource.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Should the same user token be forwarded everywhere? Only when intended for that recipient and permitted by the delegation model; otherwise use appropriate exchange or service credentials.

30-second version

> "Never trust, always verify": no implicit trust from network location. Implementation: **mTLS** between services (mesh-issued short-lived certificates), service identities (SPIFFE or cloud service accounts), per-call authorisation (propagate the user's JWT plus service-level scopes, enforce in each service, not only at the gateway), default-deny network policies, least-privilege IAM, and audit logging.

Q132. How do you secure the software supply chain?

A good SDE-3 interview answer is:

Pin and verify dependencies, scan with SCA (Dependabot/OWASP/Snyk), generate an **SBOM**, sign artifacts/images (cosign) and enforce signature and vulnerability policy at deploy time (Binary Authorization), use minimal/distroless base images, build in isolated CI with ephemeral credentials, require code review and branch protection, and prevent dependency-confusion by using a private repository with explicit namespaces. Know the Log4Shell lesson: have an inventory so you can answer "where do we use library X?" within minutes.

How to understand it

Worked explanation: Trace a deployed image back to source, dependencies and a trusted build. Signatures verify who signed particular bytes; policy and provenance decide whether those bytes should run.

Key terms you should know

- **OWASP** — An organization that publishes widely used application-security guidance, including the OWASP Top 10.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does signing make a vulnerable image safe? No; inventory, review, vulnerability response and deployment enforcement remain distinct controls.

30-second version

> Pin and verify dependencies, scan with SCA (Dependabot/OWASP/Snyk), generate an **SBOM**, sign artifacts/images (cosign) and enforce signature and vulnerability policy at deploy time (Binary Authorization), use minimal/distroless base images, build in isolated CI with ephemeral credentials, require code review and branch protection, and prevent dependency-confusion by using a private repository with explicit namespaces. Know the Log4Shell lesson: have an inventory so you can answer "where do we use library X?" within minutes.

Q133. What is your hardening checklist for a Spring Boot service?

A good SDE-3 interview answer is:

Expose only needed Actuator endpoints (and secure them), disable stack traces in responses, set security headers (HSTS, CSP, `X-Content-Type-Options`), enforce HTTPS and TLS 1.2+, validate and size-limit input (request body size, upload types), rate-limit and add timeouts, parameterised queries only, strict CORS, disable

default accounts and verbose error pages, run as non-root in a read-only container, keep dependencies patched, and log security events (auth failures, access denials) without sensitive data.

How to understand it

Worked explanation: For each attack surface, name a control and an observable failure test: unauthorized object access, oversized payloads, unsafe outbound destinations or exposed management endpoints.

Key terms you should know

- **Actuator** — Spring Boot endpoints for operational information and application management.
- **Least privilege** — Granting only the access needed for a responsibility.
- **Hardening** — Reducing unnecessary exposure and strengthening deployed defaults.
- **Input limits** — Bounds on request sizes and processing cost.

SDE-3 interview checkpoints

Follow-up with expected reasoning: Does a checklist prove security? No; test the real configuration and request paths, including alternate entry points and failure responses.

30-second version

> Expose only needed Actuator endpoints (and secure them), disable stack traces in responses, set security headers (HSTS, CSP, `X-Content-Type-Options`), enforce HTTPS and TLS 1.2+, validate and size-limit input (request body size, upload types), rate-limit and add timeouts, parameterised queries only, strict CORS, disable default accounts and verbose error pages, run as non-root in a read-only container, keep dependencies patched, and log security events (auth failures, access denials) without sensitive data.

AE. LeetCode-Style Programs

1. Valid parentheses (stack)

Derive the approach: the most recently opened bracket must close first, which is exactly LIFO behavior. Store expected closing characters so a close is checked against the top in one step.

Why this, not the alternatives: a single balance counter works for one bracket type but cannot detect crossed types such as `([)]`. Repeated string replacement can be quadratic. A stack makes nesting order explicit.

What changes: the code assumes only bracket characters; other characters are rejected. If parsing source text, decide how to treat strings, escapes and comments. Empty input is valid. General wildcard brackets require a different state representation.

Trace and checks: `([])` pushes `)` then `]`, and consumes them in reverse order. Test `([)]`, leading close, trailing open and empty input. O(n) time, O(n) stack; `toCharArray` also allocates O(n).

Strong answer signal: explain why counts alone cannot represent nesting.

```
boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> st.push(')');
            case '[' -> st.push(']');
            case '{' -> st.push('}');
            default -> { if (st.isEmpty() || st.pop() != c) return false; }
        }
    }
    return st.isEmpty();
}
```

2. Number of islands (DFS, O(rows × cols))

Derive the approach: each island is a connected component. At an unvisited land cell, count one component and mark every reachable land cell so none can start another component.

Why this, not the alternatives: DFS and BFS have the same O(rows × columns) time; iterative traversal avoids recursion-depth failure. Union-find is useful for incremental additions but is more machinery for a single static count. Checking only neighbors' counts cannot reliably merge arbitrarily connected shapes.

What changes: four-neighbor versus eight-neighbor adjacency changes the answer. The snippet mutates land to water and assumes a rectangular grid. Preserve input with a visited structure when required. Deep snake-shaped islands can overflow the recursive stack.

Trace and checks: two diagonally touching land cells count as two under four-neighbor rules. Test all water, all land, one row and a deep component. Worst-case auxiliary traversal space is O(rows × columns), even with in-place marking.

Strong answer signal: distinguish traversal marking, mutation and stack space.

```
int numIslands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { sink(g, r, c); count++; }
    return count;
}
void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
    sink(g, r + 1, c); sink(g, r - 1, c); sink(g, r, c + 1); sink(g, r, c - 1);
}
```

Say: deep recursion can overflow the stack on huge grids; use BFS/iterative DFS or union-find. Mention that you mutate the input and how to avoid it with a `visited` array.

3. Coin change: fewest coins (DP, O(amount × coins))

Derive the approach: after choosing a final coin c, the remaining problem is the best solution for amount−c. Let dp[a] be the minimum coin count for exact total a, with dp[0]=0 and an unreachable sentinel.

Why this, not the alternatives: greedy fails without special denomination guarantees: for {1,3,4}, amount 6, 4+1+1 loses to 3+3. Naive recursion repeats amounts; memoization or bottom-up DP reuses them. BFS over reachable amounts is valid but adds frontier bookkeeping.

What changes: counting combinations is a different recurrence with order-sensitive loop choices. Limited coin supplies require additional state or a bounded-knapsack method. Zero/negative denominations break progress; huge amounts make O(amount) memory impractical even with few coins.

Trace and checks: for {2}, odd totals remain unreachable; zero amount returns zero. Test {1,3,4}/6, duplicates and invalid input under the chosen contract. O(amount × denominations) time, O(amount) space.

Strong answer signal: define the state and prove the transition before writing loops.

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);                    // "infinity"
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int coin : coins)
            if (coin <= a) dp[a] = Math.min(dp[a], dp[a - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Contract: require nonnegative, allocation-bounded `amount` and strictly positive denominations; reject invalid input. Greedy fails for coin sets like {1, 3, 4}, which is why DP is needed. This is O(amount × number of coins) time and O(amount) space.

4. Binary tree level-order traversal (BFS)

Derive the approach: output groups nodes by distance from the root, so process a queue breadth-first. Snapshot the queue size before a level so newly enqueued children stay in the next level.

Why this, not the alternatives: DFS can group by depth too, but BFS directly matches the requested output. Repeatedly traversing from the root for each depth repeats work. A stack alone changes the visitation order.

What changes: a graph may contain cycles/shared vertices and needs visited tracking. A very wide tree can make the queue large; a deep tree makes recursive DFS risky. Zigzag output can reverse each level or change insertion positions without changing the level boundary.

Trace and checks: root 1 with children 2/3 emits `[1]`, then `[2,3]`. Test null root, one-sided tree, broad tree and duplicate values. O(n) time, O(maximum width) queue, O(n) output.

Strong answer signal: explain why the level-size snapshot is taken before processing children.

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Queue<TreeNode> q = new ArrayDeque<>(List.of(root));
    while (!q.isEmpty()) {
        int n = q.size(); List<Integer> level = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            TreeNode t = q.poll(); level.add(t.val);
            if (t.left != null) q.add(t.left);
            if (t.right != null) q.add(t.right);
        }
        out.add(level);
    }
    return out;
}
```

AF. Google-Style System Design (condensed)

Use the requirements-first design sequence before the examples: requirements → estimates → API/data → architecture → failure/scaling analysis → trade-offs. These prompts assume that interview process.

Design 7: Distributed Job Scheduler. *Clarify:* one-off and recurring (cron) jobs, millions per day, at-least-once execution with idempotent jobs, retries, priorities, delays up to months. *API:* `POST /jobs {payload, runAt|cron, retryPolicy}`, `GET /jobs/{id}`, `DELETE`. *Storage:* jobs table partitioned by `runAt` bucket (Spanner/Cloud SQL/Bigtable) with `status`, `nextRunAt`, `attempt`, `leaseUntil`. *Dispatch:* a scheduler tier scans each time bucket for due jobs and enqueues them to Pub/Sub; workers pull, take a **lease** (visibility timeout), execute and mark complete; expired leases are re-dispatched. *Reliability:* heartbeats extend leases, exponential backoff, DLQ after N attempts, deduplication by a stable logical execution ID (for recurring jobs: job ID plus scheduled occurrence), reused across retries; keep attempt count as separate metadata. Fence stale lease holders where external effects require it. *Scaling:* partition the scheduler by hash(jobId) with leader election (or lease-based ownership) per partition; avoid hot buckets at "00:00" by jittering. *Trade-offs:* a scheduler alone cannot guarantee exactly-once arbitrary external effects; combine retryable delivery with idempotency or a transactional effect boundary; clock skew (use server time), long-running jobs need checkpointing.

Google-style interviews reward: clear requirements, quantified scale, trade-offs stated explicitly, and deep dives chosen by the interviewer (be ready to go deep on any component).

AG. Amazon Leadership Principles Pack

Prepare **one story per principle** (many stories cover several). Use STAR with the **Result quantified** and speak in **"I"** not "we".

Principle	Story prompt
Customer Obsession	A decision you made from the customer's pain, not the roadmap
Ownership	Something outside your scope you fixed and owned to completion
Invent and Simplify	You replaced a complex process/system with a simpler one
Are Right, A Lot	A call made with incomplete data; how you validated it
Learn and Be Curious	A new technology you learned fast and applied
Hire and Develop the Best	Mentoring someone to promotion; a hiring bar you raised
Insist on the Highest Standards	You refused to ship, or raised quality, at a cost
Think Big	A bold proposal that changed direction or scale
Bias for Action	A fast reversible decision that unblocked the team
Frugality	Doing more with less (cost cut X% by …)
Earn Trust	Admitting a mistake or giving hard feedback
Dive Deep	A root cause found by digging into data/logs/code
Have Backbone; Disagree and Commit	You challenged a senior decision, then committed
Deliver Results	Delivered under adversity with measurable impact
Strive to be Earth's Best Employer / Success and Scale Bring Broad Responsibility	Inclusion, team wellbeing, responsible impact of your system

Story template (90 seconds): *Situation (15 s) → Task (10 s) → Action (45 s: what *I** did, why, trade-offs)* → *Result (20 s: numbers + what I learned)*. Expect 3–4 deep follow-ups ("What would you do differently?", "What was the pushback?"), so know your stories' details.

Part 09 - Coding interview practice track

The aim is to solve unfamiliar problems, explain why the solution is correct, and adapt it when constraints change. Use the forty problems below as a practice pool over two to four weeks. For a fourteen-day sprint, choose twenty to twenty-four based on your weakest patterns and reserve time for mocks and repeat attempts.

What to prioritize

Priority	Work	Exit criterion
First	Arrays, hashing, binary search, sliding windows, trees, heaps, graphs	Derive a solution without naming a memorized template first
First	Java implementation, invariants, complexity, boundary tests	Produce correct code and explain every state variable
Next	Dynamic programming, backtracking, union-find, monotonic stacks	State the recurrence or maintained invariant before coding
Next	Machine coding and concurrency	Define contracts, bound state, handle cancellation and test interactions
Maintain	System design, databases and project depth	Explain one realistic failure timeline and a recovery plan
Role-dependent	Detailed GCP, Kubernetes and Terraform trivia	Study when it appears in the job description or your resume

This is a suggested allocation, not a claim about any company's interview process. Read the recruiter-provided format first.

Forty problems organized by pattern

For each problem, first clarify input contracts, give a correct baseline, improve it, state an invariant, code, and test. The names below identify practice prompts; the solutions need not come from a particular platform.

Pattern	Problems	Recognition cue	Follow-up to rehearse
Hashing and prefix sums	1. Two Sum; 2. Subarray Sum Equals K; 3. Longest Consecutive Sequence	Fast membership or a relationship between prefixes	Negative numbers, duplicate values, integer overflow
Two pointers	4. Three Sum; 5. Container With Most Water; 6. Trapping Rain Water	Sorted choices or boundaries that move monotonically	Prove why a discarded choice cannot improve the result
Sliding window	7. Longest Substring Without Repeats; 8. Minimum Window Substring; 9. Permutation in String	Maintain a condition on a contiguous range	Explain why negative numbers break some sum-window arguments
Binary search	10. Lower Bound; 11. Search Rotated Array; 12. Minimum Feasible Processing Speed	Ordered search space or a monotone feasibility predicate	Duplicates, no feasible answer, overflow in midpoint/ceil division

Pattern	Problems	Recognition cue	Follow-up to rehearse
Intervals	13. Merge Intervals; 14. Meeting Rooms II; 15. Insert Interval	Sorting exposes overlap decisions	Closed versus half-open intervals; whether touching endpoints overlap
Linked lists	16. Reverse List; 17. Detect Cycle; 18. Merge K Sorted Lists	Pointer ownership or ordered heads	Empty list, aliasing, cycles, whether input nodes may be reused
Stack	19. Valid Parentheses; 20. Daily Temperatures; 21. Largest Rectangle in Histogram	Unresolved elements await a later boundary	Equal values and why total pushes/pops remain linear
Trees	22. Level Order; 23. Lowest Common Ancestor; 24. Serialize and Deserialize a Tree	Recursive subproblem or breadth levels	Missing target nodes, duplicate values, skewed-tree stack depth
Heap	25. Top K Frequent; 26. Median of a Stream; 27. Kth Largest	Retain an ordered frontier or a small best set	Tie-breaking, invalid k, exact versus approximate streaming answers
Unweighted graphs	28. Number of Islands; 29. Clone Graph; 30. Course Schedule	Reachability, distance by edges, or dependency order	Disconnected nodes, repeated edges, cycle detection
Weighted graphs	31. Network Delay; 32. Minimum-Cost Grid Path	Nonnegative weighted shortest path	Why BFS is insufficient; what changes with negative weights
Union-find	33. Redundant Connection; 34. Accounts Merge	Repeated undirected connectivity queries	Duplicate edges, self edges, deletions and rollback
Backtracking	35. Combination Sum; 36. Subsets With Duplicates; 37. Word Search	Enumerate choices under constraints	Avoid duplicate outputs; explain pruning and output-size cost
Dynamic programming	38. Coin Change; 39. Edit Distance; 40. Longest Increasing Subsequence	Repeated subproblems with sufficient state	Base cases, transitions, iteration order, reconstructing a solution

Do not count reading an answer as solving a problem. Mark each attempt as independent, hinted, or read. Retry hinted/read problems after two days, then a week, with no solution visible.

A daily practice session

Allow about 150 minutes: 10 minutes recalling earlier invariants, two 40-minute problem attempts, 25 minutes fixing and testing the weaker attempt, 20 minutes of Java/backend depth, and 15 minutes recording mistakes. On mock days, replace new problems with a timed interview and review.

Day	Coding focus	Short companion revision
1	Arrays and hashing	`equals`, `hashCode`, mutable keys
2	Prefix sums and two pointers	`long`, comparator overflow, collection costs

Day	Coding focus	Short companion revision
3	Sliding windows	UTF-16 versus code points and input contracts
4	Binary search	Loop invariants and boundary conventions
5	Intervals and linked lists	Mutation, aliasing and defensive copies
6	Trees and traversal	Recursion stack, queue space, serialization
7	One 45-minute coding mock; redo two failures	Explain trade-offs aloud
8	Heaps and monotonic stacks	Amortized complexity and heap ordering
9	Graph BFS/DFS and topological ordering	Disconnected state and cycle detection
10	Dijkstra and union-find	Preconditions and counterexamples
11	Backtracking and one-dimensional DP	Search-tree size and pruning
12	Two-dimensional DP; retry weak patterns	State compression and reconstruction
13	One 90-minute machine-coding exercise	Concurrency, lifecycle and tests
14	One 45-minute coding mock and 45-minute design mock	Project stories and final error-log review

The 45-minute coding round

Minutes	Action	What the interviewer can assess
0–5	Clarify input size, duplicates, ordering, mutation and output	Whether you solve the actual problem
5–10	Explain a baseline and the bottleneck	Whether optimization has a reason
10–15	Choose a representation and state an invariant	Whether the algorithm is derived
15–32	Implement, narrating decisions rather than each keystroke	Correctness and readable Java
32–40	Walk through normal and adversarial tests	Ability to find your own bugs
40–45	Explain time/space cost and handle one changed constraint	Depth and adaptability

Practice rubric: score each of contract, reasoning, implementation, verification, and communication from 0 to 2. Zero means missing/incorrect, one means partial or prompted, and two means independent and clear. Aim for at least 8/10 twice on unseen problems, with no zero in correctness or verification. This is a self-assessment tool, not a hiring prediction.

Java details that matter during live coding

- Use `ArrayDeque` for ordinary stacks/queues and `PriorityQueue` for an ordered frontier. Heap iteration is not sorted iteration.
- Prefer `Integer.compare`, `Long.compare` or comparator factories; `a - b` can overflow. A `TreeSet` also uses comparator equality to decide whether an element is already present.
- Cast before arithmetic: `(long) a * b`, not `(long) (a * b)`. Use `long` for counts of subarrays, accumulated weights and large totals.
- `char` is a UTF-16 code unit. Decide whether text problems mean ASCII, code units, Unicode code points, or user-perceived characters; these are different contracts.
- State whether a function changes its input. Sorting the caller's array or returning references to mutable interval arrays can be observable behavior.
- Track recursion depth as auxiliary space. DFS can be O(V + E) time and still overflow the Java call stack on a deep input.
- State expected, amortized and worst-case complexity separately. “HashMap is O(1)” needs assumptions; “two nested loops means O(n²)” is false when both pointers only advance across the input once.
- Do not stream away an algorithm's invariant. A short loop is often easier to implement, inspect and debug under a timer.
- Describe the output contract for ties, empty results and impossible cases. Reject invalid inputs explicitly when that is the agreed contract.

Eight worked solutions

The companion InterviewAlgorithms.java contains all eight implementations and executable checks. The blocks below are members of that class and use `java.util` imports; the file is the complete runnable artifact.

From the directory containing the file, run `java InterviewAlgorithms.java`. It was executed on OpenJDK 21.0.9 and reported **PASS: 3116 checks across eight algorithms**. Tests include explicit edge cases and deterministic comparisons against simpler implementations for prefix sums, binary search, monotonic stacks and running medians. Separate checks passed for the revised LRU cache, bounded queue, concurrent limiter and odd/even cancellation examples. This does not imply every earlier framework excerpt was compiled or integration-tested.

Solution 1 Lower bound

Derive and compare: a linear scan establishes the meaning of the first acceptable index. Sortedness makes `a[i] >= target` monotone, so halve the remaining candidate range. A library binary search may return any equal position, requiring extra handling for the first duplicate; the half-open formulation encodes that contract directly.

What if: changing `< target` to `<= target` searches for the first strictly larger value. For `[1,2,2,4]` and target 2, lower bound is 1 and upper bound is 3. Returning `-1` for an absent value loses the insertion-position contract. If ordering is removed, binary search is invalid.

Strong answer signal: state what is known on both sides of the open search interval and show why every iteration makes progress.

Contract: sorted ascending array; return the first index whose value is at least the target, or array length. Do not add an O(n) sortedness scan when claiming O(log n) search.

Reasoning: all indices before `lo` are too small; all indices at or after `hi` are known candidates or past the array. A half-open interval handles an empty array and an insertion after the last element without a special branch.

```
// 1. Binary search. Precondition: a is sorted ascending. Returns a.length if absent above all
values.
public static int lowerBound(int[] a, int target) {
    Objects.requireNonNull(a);
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Cost: O(log n) time, O(1) extra space. **Tests:** empty array, target before/after the range, repeated target, and a single element. **Follow-up:** change the comparison to find the first value strictly greater than the target; use the two boundaries to count occurrences.

Solution 2 Subarray sum equals a target

Derive and compare: enumerate all start/end pairs as a baseline, then observe that every segment sum is a difference of two prefixes. Frequencies matter because multiple earlier prefixes can give distinct valid starts. Sorting prefixes loses position information; a simple sliding window fails when values can be negative.

What if: store only a set and `[0,0]` with target zero is undercounted. Insert the current prefix before querying and you count an empty segment. With target 1 in `[1,−1,1]`, all three valid segments are counted. For longest length, retain earliest prefix indices; for shortest length, reason about which index should be retained.

Strong answer signal: derive the complement equation and explain query-before-update.

Contract: integer elements may be negative; count nonempty contiguous subarrays. The answer and prefix sums use `long`.

Reasoning: if the current prefix is p, an earlier prefix p − target completes a valid subarray. Seed the empty prefix once and query before inserting the current prefix; otherwise target zero incorrectly counts an empty subarray. A simple shrinking window is not correct for arbitrary negative values.

```
// 2. Prefix sums. Counts nonempty subarrays; negative numbers and zero are supported.
public static long countSubarraysWithSum(int[] a, long target) {
    Objects.requireNonNull(a);
    Map<Long, Long> counts = new HashMap<>();
    counts.put(0L, 1L);
    long prefix = 0, result = 0;
    for (int value : a) {
        prefix += value;
        result += counts.getOrDefault(prefix - target, 0L);
        counts.merge(prefix, 1L, Long::sum);
    }
    return result;
}
```

Cost: expected O(n) time and O(n) space under ordinary hash distribution. **Tests:** all zeros, cancellation between positive/negative values, no match, and totals exceeding `int`. **Follow-up:** for longest matching subarray, store earliest indices instead of frequencies.

Solution 3 Next warmer day

Derive and compare: the quadratic baseline searches right from every day. The repeated work is asking whether a later value resolves earlier candidates. Keep only unresolved indices; a new warmer value discharges a suffix of that stack. A heap could resolve candidates but adds O(log n) operations without improving the static task.

What if: popping on equality incorrectly treats an equal temperature as warmer. Storing only values loses the index distance. On `[30,30,31]`, answers are `[2,1,0]`. For a circular input, scan a second pass with controlled insertion so each original index is resolved once.

Strong answer signal: justify linear total work by charging one push and at most one pop to each index.

Contract: for each temperature, return distance to the next strictly warmer day, or zero.

Reasoning: the stack stores unresolved indices with non-increasing temperatures. A warmer day resolves all smaller values on top. Each index enters and leaves at most once, so the nested loop is linear in total.

```
// 3. Monotonic stack. Equal temperatures do not count as warmer.
public static int[] daysUntilWarmer(int[] temperatures) {
    Objects.requireNonNull(temperatures);
    int[] result = new int[temperatures.length];
    Deque<Integer> unresolved = new ArrayDeque<>();
    for (int i = 0; i < temperatures.length; i++) {
        while (!unresolved.isEmpty()
                && temperatures[i] > temperatures[unresolved.peek()]) {
            int previous = unresolved.pop();
            result[previous] = i - previous;
        }
        unresolved.push(i);
    }
    return result;
}
```

Cost: O(n) time, O(n) auxiliary stack and O(n) output. **Tests:** equal temperatures, decreasing input, one element, empty input. **Follow-up:** adapt to a circular array while avoiding repeated output resolution.

Solution 4 Dijkstra shortest paths

Derive and compare: BFS minimizes edge count, not weight. Nonnegative weights allow repeatedly choosing the smallest tentative distance. A heap makes that choice efficient; a dense graph may justify an O(V²) array selection implementation. Bellman–Ford handles negative weights but costs more.

What if: marking a node final when enqueued freezes an early expensive route. With edges A→B=10, A→C=1, C→B=1, B must improve to 2. Negative edges invalidate the greedy proof. Removing the stale-entry guard repeats work; stopping when the destination is first enqueued can return 10 instead of 2.

Strong answer signal: explain why a popped minimum is safe under the weight contract and distinguish tentative from final distance.

Contract: directed adjacency list, nodes numbered 0 through V − 1, nonnegative integer weights. `Long.MAX_VALUE` denotes unreachable. Validate every edge, including edges in unreachable components.

Reasoning: the cheapest current tentative distance cannot be improved through a more expensive frontier when all edge weights are nonnegative. Because Java's priority queue has no decrease-key operation, insert updated candidates and ignore stale entries when popped.

```
// 4. Weighted graph. Directed adjacency list, nodes 0..V-1, nonnegative int weights.
public record Edge(int to, int weight) {}
private record State(int node, long distance) {}

public static long[] shortestPaths(List<List<Edge>> graph, int source) {
    Objects.requireNonNull(graph);
    Objects.checkIndex(source, graph.size());
    for (List<Edge> edges : graph) {
        Objects.requireNonNull(edges);
        for (Edge edge : edges) {
            Objects.requireNonNull(edge);
            Objects.checkIndex(edge.to(), graph.size());
            if (edge.weight() < 0) throw new IllegalArgumentException("Negative edge weight");
        }
    }
    long[] distance = new long[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    distance[source] = 0;
    PriorityQueue<State> queue = new PriorityQueue<>(Comparator.comparingLong(State::distance));
    queue.add(new State(source, 0));
    while (!queue.isEmpty()) {
        State current = queue.remove();
        if (current.distance() != distance[current.node()]) continue;
        for (Edge edge : graph.get(current.node())) {
            long candidate = current.distance() + edge.weight();
            if (candidate < distance[edge.to()]) {
                distance[edge.to()] = candidate;
                queue.add(new State(edge.to(), candidate));
            }
        }
    }
```

```
    return distance;
}
```

Cost: for this lazy-entry implementation, O(V + E log(E + 1)) time and O(V + E) extra space, excluding the input adjacency list. For simple graphs, log E is O(log V). Distances use `long` and weights use nonnegative `int`; a shortest simple path fits this representation. **Tests:** zero-weight cycle, unreachable vertex, competing routes, stale entry, invalid edge, and large total weight. **Follow-up:** BFS for unit weights, 0–1 BFS for weights zero/one, or a negative-edge algorithm when the contract changes.

Solution 5 Disjoint sets

Derive and compare: repeated connectivity questions need component membership, not the path itself. Store representatives and merge sets; path compression and size-based linking keep representative queries cheap. BFS is simpler for one traversal and is needed when returning a path. Maintaining a full reachability matrix uses much more memory.

What if: an undirected edge joins already-connected nodes, it closes a cycle under the chosen graph rules. Directed cycle detection needs another algorithm. Duplicate edges and self edges require an explicit contract. Deleting an edge may split a component, which this structure cannot discover.

Trace and strong answer: union 0–1 then 1–2; find(0)=find(2), while 3 remains separate. Explain why roots, rather than arbitrary parent entries, must be compared.

Contract: fixed node set, incremental undirected connectivity, single-threaded mutation.

Reasoning: represent each component as a rooted tree. Union attaches the smaller component to the larger; path compression shortens future searches. Decrease the component count only when two different roots merge.

```
// 5. Disjoint sets. Mutable and single-threaded; union returns false for an existing connection.
public static final class DisjointSet {
    private final int[] parent, size;
    private int components;

    public DisjointSet(int n) {
        if (n < 0) throw new IllegalArgumentException("Negative size");
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    public int find(int node) {
        Objects.checkIndex(node, parent.length);
        while (node != parent[node]) {
            parent[node] = parent[parent[node]];
            node = parent[node];
        }
        return node;
    }

    public boolean union(int a, int b) {
```

```
        int rootA = find(a), rootB = find(b);
        if (rootA == rootB) return false;
        if (size[rootA] < size[rootB]) { int tmp = rootA; rootA = rootB; rootB = tmp; }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        components--;
        return true;
    }

    public int components() { return components; }
}
```

Cost: O(n) initialization and space; amortized O(α(n)) for find/union over a sequence of operations. **Tests:** duplicate union, self union, transitive connectivity, zero nodes, invalid node. **Follow-up:** ordinary union-find does not support arbitrary deletions; discuss rebuilding, offline reversal or rollback variants instead of promising an O(1) deletion.

Solution 6 Edit distance

Derive and compare: brute recursion explores the final insert/delete/replace choices repeatedly. Prefix lengths identify repeated subproblems. Bottom-up DP makes dependencies explicit; memoization is equally valid but uses recursion. Two rows suffice only because the answer is a distance, not a full edit script.

What if: reusing a row without saving the diagonal corrupts the replacement transition. Different operation costs change the recurrence; allowing transposition is a different distance definition. For `cat→cut`, replacing one character costs one; empty→`abc` costs three. Unicode code points need a different input representation from `charAt`.

Strong answer signal: define dp[i][j] precisely and explain why current-row left and previous-row entries have different roles.

Contract: minimum unit-cost insertions, deletions and replacements over UTF-16 code units. For code-point semantics, convert input to code-point sequences first.

Reasoning: the distance between two prefixes depends on matching their last character or paying for one of three operations. Only the previous and current DP rows are needed when returning the distance alone.

```
// 6. Dynamic programming. Unit-cost insert/delete/replace over UTF-16 code units.
public static int editDistance(String a, String b) {
    Objects.requireNonNull(a);
    Objects.requireNonNull(b);
    if (a.length() < b.length()) { String tmp = a; a = b; b = tmp; }
    int[] previous = new int[b.length() + 1], current = new int[b.length() + 1];
    for (int j = 0; j <= b.length(); j++) previous[j] = j;
    for (int i = 1; i <= a.length(); i++) {
        current[0] = i;
        for (int j = 1; j <= b.length(); j++) {
            current[j] = a.charAt(i - 1) == b.charAt(j - 1)
                ? previous[j - 1]
                : 1 + Math.min(previous[j - 1], Math.min(previous[j], current[j - 1]));
        }
        int[] tmp = previous; previous = current; current = tmp;
    }
    return previous[b.length()];
}
```

Cost: O(mn) time and O(min(m, n)) extra space. **Tests:** empty strings, equal strings, repeated letters, and a mixed insert/delete/replace example. **Follow-up:** reconstructing the actual edit sequence needs additional state or a divide-and-conquer reconstruction approach.

Solution 7 Combination sum

Derive and compare: choose a candidate, reduce the remaining target, and recurse. Keep a nondecreasing candidate-index order so each multiset appears once. A DP decision algorithm is better if the question asks only whether a target is reachable; enumeration must pay for the output.

What if: restarting at index zero generates permutations such as `[2,3]` and `[3,2]`. Advancing to i+1 forbids reuse. A zero candidate can recurse forever. For {2,3} and target 6, return `[2,2,2]` and `[3,3]`. Duplicate candidates require normalization or branch-level skipping.

Strong answer signal: explain both termination and uniqueness, and count output construction in complexity.

Contract: positive candidate values, unlimited reuse, nonnegative target, unique combinations. A zero target has one solution: the empty combination. Input is not mutated.

Reasoning: normalize duplicate candidates, sort, and only choose indices at or after the current index. This permits reuse while avoiding permutations of the same combination. Positivity makes the remaining target decrease; zero or negative candidates would break that termination argument.

```
// 7. Backtracking. Unique combinations, unlimited reuse, positive candidates, nonnegative target.
public static List<List<Integer>> combinationSum(int[] candidates, int target) {
    Objects.requireNonNull(candidates);
    if (target < 0) throw new IllegalArgumentException("Negative target");
    for (int value : candidates) {
        if (value <= 0) throw new IllegalArgumentException("Candidates must be positive");
    }
    int[] unique = Arrays.stream(candidates).distinct().sorted().toArray();
    List<List<Integer>> result = new ArrayList<>();
    combinations(unique, 0, target, new ArrayList<>(), result);
    return result;
}

private static void combinations(int[] values, int start, int remaining,
                                 List<Integer> path, List<List<Integer>> result) {
    if (remaining == 0) { result.add(List.copyOf(path)); return; }
    for (int i = start; i < values.length && values[i] <= remaining; i++) {
        path.add(values[i]);
        combinations(values, i, remaining - values[i], path, result);
        path.remove(path.size() - 1);
    }
}
```

Cost: output-sensitive and potentially exponential. Let d = floor(target / minimum candidate) and k be the number of distinct candidates. A loose search bound is O((k + 1)^(d + 1)), plus normalization and O(total emitted elements) to copy results; do not label it O(n). Auxiliary space is O(k + d), excluding output. Large d can overflow the call stack. **Tests:** duplicate candidates, no result, empty candidates, target zero, invalid candidate, preserved input. **Follow-up:** allow each candidate once, then explain how duplicate skipping changes.

Solution 8 Running median

Derive and compare: sorting every prefix repeats work; a sorted array has O(n) insertion. Two heaps retain exactly the two middle boundaries while allowing O(log n) insertion. A balanced ordered structure can work too, especially if deletions or other rank queries are required.

What if: balancing sizes without preserving the value partition produces the wrong median. Integer addition can overflow before division unless widened first. For 5,1,9, the medians are 5,3,5. Sliding windows need arbitrary expiry support, while memory-bounded unending streams may require approximate quantiles.

Strong answer signal: state both the size invariant and the cross-heap ordering invariant; neither alone is sufficient.

Contract: exact median after each insertion; no deletions; single-threaded. Querying an empty stream throws.

Reasoning: a max-heap holds the lower half and a min-heap the upper half. Every lower value is at most every upper value; the lower heap has either the same size or one extra element. Cast before adding the two middle values to avoid integer overflow.

```
// 8. Two heaps. Single-threaded; grows with the stream length.
public static final class RunningMedian {
    private final PriorityQueue<Integer> lower = new PriorityQueue<>(Comparator.reverseOrder());
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();

    public void add(int value) {
        if (lower.isEmpty() || value <= lower.peek()) lower.add(value);
        else upper.add(value);
        if (lower.size() > upper.size() + 1) upper.add(lower.remove());
        else if (upper.size() > lower.size()) lower.add(upper.remove());
    }

    public double median() {
        if (lower.isEmpty()) throw new NoSuchElementException("Empty stream");
        if (lower.size() != upper.size()) return lower.peek();
        return ((long) lower.peek() + upper.peek()) / 2.0;
    }
}
```

Cost: O(log n) insertion, O(1) median lookup, O(n) space. **Tests:** odd/even sizes, duplicate values, mixed signs, integer extremes, sorted and reverse inputs. **Follow-up:** a sliding median needs deletion support; an unbounded stream with a memory cap may require approximate quantiles rather than exact retained history.

Machine coding beyond algorithms

For these exercises, agree on requirements before designing classes. Start with one working vertical path, then add the smallest abstractions justified by a second behavior. Do not introduce a database, framework or hierarchy merely to demonstrate familiarity.

Exercise	Minimal scope	Senior-level follow-up	Useful tests
Bounded LRU cache	Generic get/put, replacement, eviction, positive capacity	Access-order reads mutate state; define thread safety and null policy	Update existing entry, access changes eviction, capacity one, concurrent invariants

Exercise	Minimal scope	Senior-level follow-up	Useful tests
Rate limiter	One algorithm, injected monotonic ticker, per-client limits	Admission versus waiting, idle-key eviction races, distributed failure policy	Exact expiry boundary, simultaneous requests, invalid config, large key population
In-memory task scheduler	One-off jobs, cancellation and bounded execution	Stable execution ID, retry policy, shutdown and task exceptions	Cancel before dispatch, duplicate dispatch, failure, no late work after shutdown
Inventory reservation service	Reserve/release, expiry, no negative available stock	Idempotency, concurrent updates and clock policy	Duplicate request, competing reservations, expired reservation, duplicate release

Suggested 90-minute structure: 10 minutes on contracts, 15 on data model and operations, 40 implementing the core flow, 15 on tests, 10 on concurrency/failure trade-offs. Keep real HTTP/database integration out of the exercise unless requested.

For concurrency tests, coordinate threads with latches/barriers and bounded completion waits. Avoid tests whose correctness depends on `Thread.sleep`. Verify invariants such as “accepted reservations never exceed stock,” not just the absence of thrown exceptions.

Six deeper backend follow-ups

A timeout fires. Has the work stopped? Not necessarily. Separate stopping the caller's wait, completing a future, interrupting a worker, canceling I/O and undoing a committed effect. Preserve interruption or propagate it to an owner; do not retry it as a transient business failure. Configure actual I/O deadlines and account for work that cannot be canceled. `CompletableFuture.cancel(true)` does not interrupt its computation. CompletableFuture cancellation.

Can two transactions preserve each row's version and still break a rule? Yes. Suppose two doctors are on call and each transaction removes a different doctor after observing the other. They update different rows, so per-row versions alone do not preserve “at least one doctor remains.” Enforce a common guard/locking protocol or use serializable transactions with retry handling. PostgreSQL repeatable read can permit serialization anomalies. PostgreSQL isolation.

The payment provider timed out. Should we mark the payment failed? A timeout may mean the outcome is unknown. Persist an operation identity, reuse the provider's idempotency key, reconcile status, and expose a pending/unknown state until evidence resolves it. A local transaction cannot roll back a completed external charge. This is a design scenario; the exact policy depends on the provider contract.

How much concurrency does a service need? Under stable conditions, Little's Law relates mean in-flight work to mean arrival rate multiplied by mean time in the system. At 800 requests/second and 250 ms mean response time, mean in-flight work is about 200. This is not a thread-pool setting: account separately for queueing, burstiness, time spent holding each resource and safety margin. Do not substitute p99 latency into a mean-value relationship.

Why can an additive API change still break clients? New enum values can break exhaustive client logic; changed null/default behavior can alter business outcomes; wider numbers can overflow clients; additional JSON fields can fail strict readers. Specify producer/consumer compatibility, test representative older consumers and keep rollback viable during mixed-version rollout.

How do you represent money and time? Agree on currency and rounding rules before choosing minor-unit integers or `BigDecimal`. Scale-sensitive `BigDecimal.equals` differs from numeric `compareTo`; avoid binary-floating constructors for decimal input. BigDecimal API. For time, distinguish an instant from a local calendar time and region, inject `Clock` in business logic, and use elapsed monotonic time for durations rather than wall-clock subtraction. Test rounding boundaries and daylight-saving gaps/overlaps when relevant.

Three mock rounds

1. **Coding, 45 minutes:** count subarrays with a target sum. Introduce negatives after the baseline. Ask for longest matching subarray, overflow handling, and a brute-force reference test.
2. **Coding, 45 minutes:** compute shortest paths in a directed weighted graph. Introduce unreachable vertices, zero-weight edges, stale heap entries and then a negative edge. Ask which assumptions stop holding.
3. **Machine coding, 90 minutes:** implement a bounded per-client limiter with an injected ticker. Add simultaneous requests, a maximum client count, and idle eviction. Ask what changes across multiple JVMs.

For each mock, record the first incorrect assumption, time to a compiling solution, the test that exposed the bug, and the explanation that remained unclear. An example log entry is: “Treated negative-value sums as a shrinking window; replace with prefix frequencies; retry after two days.”

Version reference and important corrections

The stable practice target is Java 21. The Java 25 entries below describe that release specifically; preview status must be checked again for a different target release. This table is not a declaration that Java 25 is the newest JDK.

Topic	Version-aware answer	Source
Virtual threads	Final in Java 21; monitor-related pinning changes in JDK 24	JEP 444, JDK 24 guide
Scoped values	Final API in Java 25; not final in Java 21	Java 25 API
Structured concurrency	`StructuredTaskScope` is preview in Java 25; API shapes changed between previews	Java 25 API

Topic	Version-aware answer	Source
Transaction method visibility	Spring 6 class proxies can support protected/package-visible methods; interface proxies require public methods	Spring annotations
HTTP clients	`RestTemplate` is deprecated in Spring 7; use the correct framework version when answering	Spring client reference
Boot 4	Uses Spring Framework 7 and requires at least Java 17; verify the exact release matrix	Boot 4 requirements

The revised earlier chapters also fix the failed-transaction dedupe example, async transaction wording, optimistic-locking claims, keyset complexity, reactive blocking guidance, request-based versus time-based error budgets, and several coding boundary cases. Primary-source links are placed alongside the affected material. Cloud product capabilities and framework defaults remain version-dependent; verify the exact target before using an example in a real application.

Part 10 - Web sourced questions and strong answer rubrics

Sources checked on **5 October 2026**. Q134–Q141 adapt public coding-practice prompts and link to their original pages. Q142–Q153 are interview-style scenarios derived from official Java documentation; those pages establish the technical behavior, not that a particular employer asks the question. All answer outlines and implementations here are original explanations.

“Strong answer signals” below are coaching judgments based on the problem contract and published interview guidance. They are not leaked questions, guaranteed questions, or a company's private scoring key. A good interview answer explains a decision that fits the constraints; it need not match this guide word for word.

What interviewers publicly say they assess

Source	Published emphasis	How to demonstrate it
Amazon SDE II preparation	Executable, robust, scalable code and edge-case testing	Implement the agreed solution, test it, then explain its bottleneck
Microsoft technical interviews	Clarifying ambiguity, planning, coding, testing and complexity	State assumptions and an invariant before coding; check boundaries afterward
Atlassian engineering interviews	Data structures, code design, thought process and trade-offs	Compare credible options and adapt when the interviewer changes a constraint

A reusable spoken structure is: **“Given these constraints, the baseline is … Its bottleneck is … The property I can exploit is … I choose … because … The invariant is … If this assumption changes, I would … I will test …”** Use it to organize an explanation you understand, not as a memorized script.

Public coding prompts with original Java solutions

The complete runnable companion is WebInterviewPractice.java. Run `java WebInterviewPractice.java` from its directory. It passed **12,417 checks on Java 21**, including fixed-seed comparisons with brute-force sliding-window and parentheses solutions and a reference minimum stack. Each code block below is extracted from that file; imports and the enclosing class are in the companion.

Q134 Copy a list whose nodes also have arbitrary references

Public prompt: construct independent nodes while preserving both link relationships. Copy List with Random Pointer.

Derive it: copying values is insufficient because edges must lead to copied nodes. First create one copy per original identity; then reconnect both edge types through that mapping.

Choose versus alternatives: the two-pass identity map is easy to verify and preserves the input, using O(n) extra mapping space. Interleaving copies between original nodes saves auxiliary mapping space but temporarily changes the original list and complicates restoration/failure handling. A shallow copy leaves links into the original structure.

What if: duplicate node values rule out a value-keyed map. Cycles in `random` are harmless because copies exist before wiring. A cyclic `next` chain violates this implementation's precondition; general graph cloning needs visited traversal.

Trace/tests: two nodes with equal values pointing randomly to each other must become two distinct new objects with reciprocal new links. Test null, self-random, null-random and input preservation. O(n) time and mapping space, plus O(n) required output.

Strong answer signals: identity versus equality; no copied edge reaches an original object; honest mutation trade-off.

```
// Q134. Acyclic next chain; random references must stay inside that chain or be null.
public static final class RandomNode {
    public final int value;
    public RandomNode next, random;
    public RandomNode(int value) { this.value = value; }
}
public static RandomNode copyRandomList(RandomNode head) {
    Map<RandomNode, RandomNode> copies = new IdentityHashMap<>();
    for (RandomNode node = head; node != null; node = node.next)
        copies.put(node, new RandomNode(node.value));
    for (RandomNode node = head; node != null; node = node.next) {
        RandomNode copy = copies.get(node);
        copy.next = copies.get(node.next);
        copy.random = copies.get(node.random);
    }
    return copies.get(head);
}
```

Q135 Decide whether a string can be segmented using a dictionary

Public prompt: determine whether dictionary words can cover the input in order, with reuse allowed. Word Break.

Derive it: a reachable prefix followed by a matching word makes a longer prefix reachable. A boolean DP table avoids re-solving the same prefix.

Choose versus alternatives: greedy longest-prefix selection fails: `cars` with `car`, `ca`, `rs` requires `ca` + `rs`. Plain recursion revisits suffixes; memoized recursion is valid. This implementation tests dictionary words with `startsWith`, avoiding repeated substring allocation; a trie can reduce repeated prefix comparisons for large shared-prefix dictionaries.

What if: requesting every segmentation requires output construction and can be exponential. Empty words must be handled explicitly; this implementation rejects them and treats empty input as segmentable.

Trace/tests: mark 0 reachable, then 2 via `ca`, then 4 via `rs`. Test impossible suffix, word reuse and overlapping choices. For n input length, D distinct words and maximum word length L, O(nDL) matching time; preprocessing also reads the dictionary. O(n + D) auxiliary references/state.

Strong answer signals: define reachability precisely and give a counterexample to greedy selection.

```
// Q135. UTF-16 string matching; empty input is segmentable. Empty dictionary words are rejected.
public static boolean wordBreak(String text, Collection<String> dictionary) {
    Objects.requireNonNull(text);
    List<String> words = List.copyOf(new LinkedHashSet<>(dictionary));
    for (String word : words)
        if (word.isEmpty()) throw new IllegalArgumentException("Empty dictionary word");
    boolean[] reachable = new boolean[text.length() + 1];
    reachable[0] = true;
    for (int start = 0; start < text.length(); start++) {
        if (!reachable[start]) continue;
        for (String word : words)
            if (text.startsWith(word, start)) reachable[start + word.length()] = true;
    }
    return reachable[text.length()];
}
```

Q136 Implement exact-word and prefix lookup

Public prompt: support insertion, exact search and prefix search. Implement Trie.

Derive it: words sharing prefixes can share the same traversal path. A terminal marker distinguishes a complete word from an intermediate prefix.

Choose versus alternatives: a hash set handles exact search well but does not directly index prefixes. Scanning every stored word is expensive for repeated prefix queries. A trie spends memory on nodes; sparse maps or compressed edges can reduce waste for larger alphabets.

What if: `app` is a valid prefix after inserting `apple`, but not a stored word until inserted. The array representation assumes lowercase ASCII. This version explicitly supports an empty word/prefix extension; deletion needs terminal/path-liveness handling. Thread safety is not implied.

Trace/tests: insert `apple`, check exact `app` false and prefix `app` true, then insert `app`. Reject invalid letters before partial mutation. O(L) per operation and O(total stored prefix nodes) space with fixed alphabet.

Strong answer signals: separate prefix existence from word termination and discuss the memory/alphabet trade-off.

```
// Q136. Lowercase ASCII only. Empty word/prefix accepted as an explicit extension.
public static final class Trie {
    private static final class Node {
        final Node[] children = new Node[26];
        boolean terminal;
    }
    private final Node root = new Node();
```

```
    private static void validate(String text) {
        Objects.requireNonNull(text);
        for (int i = 0; i < text.length(); i++)
            if (text.charAt(i) < 'a' || text.charAt(i) > 'z')
                throw new IllegalArgumentException("Expected lowercase ASCII");
    }
    public void insert(String word) {
        validate(word); // Validate before mutating any nodes.
        Node node = root;
        for (int i = 0; i < word.length(); i++) {
            int index = word.charAt(i) - 'a';
            if (node.children[index] == null) node.children[index] = new Node();
            node = node.children[index];
        }
        node.terminal = true;
    }
    private Node find(String text) {
        validate(text);
        Node node = root;
        for (int i = 0; i < text.length(); i++) {
            node = node.children[text.charAt(i) - 'a'];
            if (node == null) return null;
        }
        return node;
    }
    public boolean search(String word) { Node node = find(word); return node != null &&
node.terminal; }
    public boolean startsWith(String prefix) { return find(prefix) != null; }
}
```

Q137 Return the maximum for every fixed-size sliding window

Public prompt: report the maximum as a fixed-width range advances. Sliding Window Maximum.

Derive it: an older smaller value can never win while a newer larger value remains in the window: the newer one also expires later. Retain only useful candidate indices in decreasing-value order.

Choose versus alternatives: rescanning each window costs O(nk). A heap requires expiry management and logarithmic work; a deque exploits the fixed left-to-right expiry order for O(n) total time. Arbitrary range queries need a different structure, such as a segment tree.

What if: storing values alone loses expiry positions. Keeping older equal values is valid with correct expiry; removing them is simpler because the newer equal value lasts longer. If k is zero or exceeds n, reject it.

Trace/tests: for `[4,1,3,2]`, k=2, return `[4,3,3]`. Test all equal, decreasing data, k=1 and k=n. O(n) time, O(k) deque and O(n−k+1) output.

Strong answer signals: prove dominance and charge each index one insertion/removal.

```
// Q137. Requires 1 <= k <= values.length; preserves the input.
public static int[] slidingWindowMaximum(int[] values, int k) {
    Objects.requireNonNull(values);
    if (k < 1 || k > values.length) throw new IllegalArgumentException("Invalid window size");
    int[] result = new int[values.length - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i < values.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() <= i - k) deque.removeFirst();
        while (!deque.isEmpty() && values[deque.peekLast()] <= values[i]) deque.removeLast();
        deque.addLast(i);
        if (i >= k - 1) result[i - k + 1] = values[deque.peekFirst()];
    }
    return result;
}
```

Q138 Select the k points nearest the origin

Public prompt: retain the requested number of nearest coordinates. K Closest Points to Origin.

Derive it: square root preserves nonnegative ordering, so squared distance suffices. Maintain a size-k max-heap whose root is the worst retained candidate.

Choose versus alternatives: full sorting is simple but costs O(n log n). Quickselect gives expected linear selection but often mutates input and needs pivot handling. The heap fits streaming selection and small k; this implementation additionally sorts the retained points for deterministic output.

What if: distance ties need a declared rule; this code uses coordinates. `int` multiplication can overflow before assignment, so widen first. Even two squared arbitrary int extremes can exceed signed long; the code bounds coordinates to $\pm 10^9$. Different origins change the arithmetic bounds.

Trace/tests: distances 25, 2, 2 with k=2 keep the latter two. Test k=0, ties, duplicates and coordinate limits. O(n log(k+2) + k log(k+1)) time, including the linear validation pass when k=0; O(k+1) auxiliary storage plus O(k) output. Input is preserved.

Strong answer signals: relate heap direction to eviction and state arithmetic assumptions.

```
// Q138. Coordinates bounded to +/-1e9 so the squared-distance sum fits long.
public record Point(int x, int y) {
    public Point {
        if (Math.abs((long) x) > 1_000_000_000L || Math.abs((long) y) > 1_000_000_000L)
            throw new IllegalArgumentException("Coordinate outside supported range");
    }
    long squaredDistance() { return (long) x * x + (long) y * y; }
}
public static List<Point> kClosest(List<Point> points, int k) {
    Objects.requireNonNull(points);
    if (k < 0 || k > points.size()) throw new IllegalArgumentException("Invalid k");
    Comparator<Point> order = Comparator.comparingLong(Point::squaredDistance)
        .thenComparingInt(Point::x).thenComparingInt(Point::y);
    PriorityQueue<Point> heap = new PriorityQueue<>(order.reversed());
    for (Point point : points) {
        heap.add(Objects.requireNonNull(point));
        if (heap.size() > k) heap.remove();
    }
    List<Point> result = new ArrayList<>(heap);
    result.sort(order); // Deterministic output, including ties.
    return result;
}
```

Q139 Build a stack with constant-time minimum lookup

Public prompt: support ordinary stack operations plus retrieval of its current minimum. Min Stack.

Derive it: after a pop, the minimum must revert to the minimum of the previous prefix. Store the prefix minimum alongside each pushed value.

Choose versus alternatives: scanning on every minimum query costs O(n); a heap supports a different removal order. A second minimum stack is valid but needs careful handling of repeated minima. A linked entry per push avoids backing-array resize and gives O(1) structural work per operation.

What if: storing only one global minimum cannot restore the previous minimum after popping. Arithmetic encoding tricks require overflow care. Concurrent use needs synchronization around the whole state transition; this implementation is single-threaded.

Trace/tests: push 2, −1, −1; after one pop the minimum remains −1, and after the next it becomes 2. Test integer extremes and empty-stack behavior. O(n) stored entries; output operations are constant-time structurally.

Strong answer signals: explain why each entry stores historical information, including duplicate minima.

```
// Q139. Single-threaded linked stack; all four operations have O(1) structural work.
public static final class MinStack {
    private record Entry(int value, int minimum, Entry previous) {}
    private Entry head;
    public void push(int value) {
        head = new Entry(value, head == null ? value : Math.min(value, head.minimum()), head);
    }
    private Entry topEntry() {
        if (head == null) throw new NoSuchElementException("Empty stack");
        return head;
    }
    public int pop() { Entry entry = topEntry(); head = entry.previous(); return entry.value(); }
    public int top() { return topEntry().value(); }
    public int minimum() { return topEntry().minimum(); }
}
```

Q140 Decode nested repetition expressions

Public prompt: expand nested count-and-bracket expressions. Decode String.

Derive it: entering brackets suspends an outer prefix and repetition count. A frame stack preserves that context until the matching close arrives.

Choose versus alternatives: recursive descent is equally natural but consumes call-stack depth. Repeated regex replacement obscures nesting and repeats scans. An explicit stack exposes parser state and enables clear validation.

What if: multiple digits form one count; unmatched brackets or missing counts are errors under the extended validated contract. Expansion can be much larger than input, so cap output before allocation and use checked count arithmetic. Empty groups should not trigger billions of empty appends.

Trace/tests: `2[a3[b]]z` yields `abbbabbbz`. Test nested groups, adjacent groups, malformed input, count overflow and output limits. Let M be decoded length and d nesting depth: this builder implementation can copy intermediate expansions at multiple levels, giving O(n + dM) time in a conservative bound and O(n + M) space. Do not claim O(input length).

Strong answer signals: distinguish parsing cost from expansion cost and bound amplification.

```
// Q140. Grammar: lowercase literals and positive-int repetitions. Cap expanded output.
private record DecodeFrame(StringBuilder prefix, int repetitions) {}
public static String decode(String encoded, int maxOutputLength) {
    Objects.requireNonNull(encoded);
    if (maxOutputLength < 0) throw new IllegalArgumentException("Negative output limit");
    Deque<DecodeFrame> frames = new ArrayDeque<>();
    StringBuilder current = new StringBuilder();
    int count = 0;
    boolean digits = false;
    for (int i = 0; i < encoded.length(); i++) {
        char c = encoded.charAt(i);
        if (c >= '0' && c <= '9') {
            count = Math.addExact(Math.multiplyExact(count, 10), c - '0');
            digits = true;
        } else if (c == '[') {
            if (!digits || count == 0) throw new IllegalArgumentException("Missing positive
count");
            frames.push(new DecodeFrame(current, count));
            current = new StringBuilder(); count = 0; digits = false;
        } else if (c == ']') {
            if (digits || frames.isEmpty()) throw new IllegalArgumentException("Malformed close");
            DecodeFrame frame = frames.pop();
            long size = (long) current.length() * frame.repetitions() + frame.prefix().length();
            if (size > maxOutputLength) throw new IllegalArgumentException("Output limit
exceeded");
            StringBuilder expanded = frame.prefix();
            if (!current.isEmpty())
                for (int repeat = 0; repeat < frame.repetitions(); repeat++)
expanded.append(current);
            current = expanded;
        } else {
            if (digits || c < 'a' || c > 'z') throw new IllegalArgumentException("Invalid
literal");
            if (current.length() >= maxOutputLength) throw new IllegalArgumentException("Output
limit exceeded");
            current.append(c);
        }
    }
    if (digits || !frames.isEmpty()) throw new IllegalArgumentException("Incomplete encoding");
    return current.toString();
}
```

Q141 Find the longest valid parentheses substring

Public prompt: find the longest contiguous balanced region, not merely whether the whole string is balanced. Longest Valid Parentheses.

Derive it: store unmatched opening indices plus a boundary before the current candidate region. A matched close exposes the index immediately before the valid suffix, giving its length by subtraction.

Choose versus alternatives: testing all substrings repeats work. DP ending at each index is also O(n), but its transition is less direct. A two-direction counter method uses O(1) extra space for this one-bracket-type problem; the stack makes positional boundaries easy to explain.

What if: using only total counts misses invalid ordering. Without the initial −1 boundary, a valid substring starting at zero is awkward to measure. When a close has no matching open, reset the boundary to that close.

Trace/tests: `)()())` has maximum length four. Test empty, all openings, nested pairs and multiple separated regions. O(n) time and O(n) stack; preserving the winning end index also returns the substring.

Strong answer signals: explain what each stored index means and why a bad close resets the boundary.

```
// Q141. Parentheses only; returns length, not the substring.
public static int longestValidParentheses(String text) {
    Objects.requireNonNull(text);
    Deque<Integer> positions = new ArrayDeque<>();
    positions.push(-1); // Boundary immediately before the current candidate region.
    int best = 0;
    for (int i = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        if (c == '(') positions.push(i);
        else if (c == ')') {
            positions.pop();
            if (positions.isEmpty()) positions.push(i);
            else best = Math.max(best, i - positions.peek());
        } else throw new IllegalArgumentException("Expected parentheses only");
    }
    return best;
}
```

Java scenarios with strong spoken answers

Q142 Can a method swap the caller's two object references

Source basis: Java method arguments.

Strong answer: Java passes values. For an object argument, that value is a reference. Reassigning two local parameters changes only those local copies; mutating a referenced object's state can be visible to the caller. Return the desired pair or mutate an explicitly owned container when that is the contract.

Reasoning and alternatives: distinguish the variable holding a reference from the object it identifies. A wrapper does not turn Java into pass-by-reference; it introduces shared mutable state.

What if/test: `a.add(x)` can affect the caller's list, whereas `a = new ArrayList<>()` cannot replace the caller's variable. Test aliasing where both arguments refer to the same object. **Signal:** give a concrete mutation/reassignment distinction, not just a slogan.

Q143 Is a record containing a list immutable and safe as a map key

Source basis: Record API, List copy contracts.

Strong answer: record components have final references; referenced objects can still be mutable. Copy the list on construction when the type owns it, and ensure its elements are also suitably immutable. Changing equality-relevant contents after insertion can invalidate hash-collection behavior.

Reasoning and alternatives: an unmodifiable view still reflects changes to its backing collection. A shallow snapshot isolates list structure but not mutable elements. Deep copying has costs and requires a domain contract.

What if/test: mutate the original list, then an element, and check which changes affect the record. Test null policy before choosing `List.copyOf`. **Signal:** distinguish final reference, unmodifiable collection, snapshot and deep immutability.

Q144 Why did a TreeSet keep one of two different objects

Source basis: TreeSet ordering contract.

Strong answer: a sorted set uses comparison result zero to identify equivalent entries. Comparing employees only by salary can collapse distinct employees with equal salaries. If identity matters, add a stable distinguishing key or choose a collection that permits duplicates.

Reasoning and alternatives: a HashSet uses equals/hashCode, while a List preserves duplicates. Pick the structure after deciding the intended uniqueness rule; do not patch the comparator merely to force every comparison nonzero.

What if/test: compare two equal-salary employees with different IDs, then test comparator transitivity and extreme numbers. Subtraction can overflow; use comparator factories. **Signal:** connect ordering equality to collection identity and preserve comparator laws.

Q145 Does ConcurrentHashMap make a contained ArrayList thread-safe

Prerequisite for Q145–Q148: multithreading foundations → concurrent collections → executors and completion stages.

Source basis: ConcurrentHashMap API.

Strong answer: it protects map operations, not arbitrary later mutations of its values. Multiple callers modifying the same ArrayList still race. Choose confinement, a suitable concurrent value, immutable replacement via an atomic map operation, or one clear lock around the compound invariant.

Why not just synchronize put: the mutation may happen after lookup, outside that lock. **What if/test:** two writers append to one value; also test removing/replacing that value while another caller retains its reference. **Signal:** identify the actual shared state and required atomic operation.

Q146 Should computeIfAbsent perform a slow remote call

Source basis: ConcurrentHashMap computeIfAbsent.

Strong answer: keep its mapping function short and simple; a long computation can block competing updates. For expensive loading, consider a bounded cache with explicit load coalescing, timeout and failure policy.

Trade-off: loading outside the map may duplicate work. Caching a future can coordinate callers but must define failed-future eviction and cancellation ownership. **What if/test:** the loader throws, returns null, or recursively updates the map. **Signal:** explain both contention and failure lifecycle instead of assuming “atomic” means cheap.

Q147 When should thenCompose replace thenApply

Source basis: CompletionStage API.

Strong answer: use `thenApply` when the function returns an ordinary value; use `thenCompose` when it returns another stage that should become part of the dependency chain. Otherwise the result is a nested future. Use combination when two independent stages are both required.

Reasoning and alternatives: blocking with `join` inside the mapping callback can occupy a limited executor and hide asynchronous dependencies. Async method names do not by themselves define a sufficient resource policy.

What if/test: let the second stage fail or finish later; verify the composed result follows it. Check both failure propagation and execution context. **Signal:** draw the dependency relationship and distinguish sequencing from parallel fan-out.

Q148 Why is a thread pool not growing to its configured maximum

Source basis: ThreadPoolExecutor sizing and queues.

Strong answer: after reaching core size, the executor normally tries to queue work. An unbounded queue accepts it, so the maximum size does not create more workers in the usual saturation path. Inspect queue depth, task wait time and downstream capacity before changing sizes.

Reasoning and alternatives: a bounded queue makes overload explicit; rejection, caller execution and shedding have different contracts. CallerRuns can slow producers but can also block an event loop and discards rejected work after shutdown.

What if/test: run tasks that wait for subtasks submitted to the same full pool; spare thread counts alone may not establish progress. **Signal:** explain the submission decision order and a deliberate rejection/deadline policy.

Q149 Which failure survives when both work and resource closing fail

Source basis: Try-with-resources tutorial.

Strong answer: if the body throws, that failure remains primary and close failures are suppressed; inspect `getSuppressed()` when diagnosing cleanup. Resources close in reverse declaration order. If the body succeeds, a close failure can be the thrown exception.

Reasoning and alternatives: a manual finally block can accidentally replace the original error or return over it. Use try-with-resources for resources whose ownership belongs to the method; do not close a borrowed resource without an ownership contract.

What if/test: make both body and close throw distinct messages and assert primary plus suppressed failures. **Signal:** discuss error preservation and ownership, not only “it closes automatically.”

Q150 Why did adding to Stream.toList fail

Source basis: Stream.toList API.

Strong answer: the result is unmodifiable. If callers need a mutable collection, collect into an explicit `ArrayList` or copy the result. `Collectors.toList()` does not promise a particular mutable implementation either.

Reasoning and alternatives: decide whether ownership permits mutation; avoid relying on a currently observed concrete class. Unmodifiable structure does not make its elements immutable.

What if/test: adding/removing must fail under the unmodifiable contract, while mutating a mutable element may still succeed. **Signal:** separate API guarantees from implementation accidents and document the return contract.

Q151 Why can decimal equality disagree with numeric ordering

Source basis: BigDecimal API.

Strong answer: `BigDecimal.equals` considers scale, while `compareTo` compares numeric value. Consequently 1.0 and 1.00 can compare as zero without being equal. Pick a domain policy for scale, currency and rounding before normalizing keys or values.

Reasoning and alternatives: binary floating point is inappropriate when exact decimal semantics are required; integer minor units work only with an agreed unit and range. Do not apply arbitrary rounding merely to make a test pass.

What if/test: compare HashSet and TreeSet behavior, division with a nonterminating decimal result, and rounding boundaries. **Signal:** relate representation and equality to the business contract.

Q152 Does loading a class always execute its static initializer

Source basis: Java initialization rules.

Strong answer: loading, linking and initialization are different stages. Specified active uses trigger initialization; reading a compile-time constant can be inlined without initializing the declaring class. Class-loading APIs also expose initialization choices.

Reasoning and alternatives: avoid guessing order solely from source appearance. Determine the exact operation and whether it requires initialization, then apply superclass rules. Static initialization is not a suitable place for arbitrary fragile remote work.

What if/test: contrast a compile-time constant with a nonconstant static field and explicit reflective loading with initialization disabled. **Signal:** reason from the trigger rather than asserting “at class load.”

Q153 Is volatile enough for publishing an object and incrementing a counter

Source basis: Java happens-before rules.

Strong answer: a volatile publication can establish visibility for preceding initialization writes to a reader observing that publication. It does not make subsequent mutable operations atomic. `count++` still combines a read, arithmetic and write, so competing increments can be lost.

Reasoning and alternatives: use an atomic primitive for a single atomic update, or locking when multiple fields must change together. Immutability or confinement may remove the need for shared mutation.

What if/test: publishing before initialization completes defeats the intended ordering; changing fields later needs its own synchronization. Use coordinated concurrency tests to probe invariants, while explaining that a passing stress run is not a proof of memory-model correctness. **Signal:** identify the specific happens-before edge and the invariant it does—and does not—protect.

Part 11 - Decision walkthroughs for the full practice roadmap

Every roadmap item below has either a worked implementation elsewhere in the guide or a decision walkthrough here. Walkthrough-only entries intentionally focus on deriving a solution; they are not additional tested Java programs. Write the implementation yourself, then use the stated trace and boundary conditions to review it.

Roadmap coverage index

Problem	Solution and reasoning
1. Two Sum	Decision walkthrough
2. Subarray Sum Equals K	Decision walkthrough
3. Longest Consecutive Sequence	Decision walkthrough
4. Three Sum	Decision walkthrough
5. Container With Most Water	Decision walkthrough
6. Trapping Rain Water	Decision walkthrough
7. Longest Substring Without Repeats	Decision walkthrough
8. Minimum Window Substring	Decision walkthrough
9. Permutation in String	Decision walkthrough
10. Lower Bound	Decision walkthrough
11. Search Rotated Array	Decision walkthrough
12. Minimum Feasible Processing Speed	Decision walkthrough
13. Merge Intervals	Decision walkthrough
14. Meeting Rooms II	Decision walkthrough
15. Insert Interval	Decision walkthrough
16. Reverse List	Decision walkthrough
17. Detect Cycle	Decision walkthrough
18. Merge K Sorted Lists	Decision walkthrough
19. Valid Parentheses	Decision walkthrough
20. Daily Temperatures	Decision walkthrough
21. Largest Rectangle in Histogram	Decision walkthrough
22. Level Order	Decision walkthrough
23. Lowest Common Ancestor	Decision walkthrough

Problem	Solution and reasoning
24. Serialize and Deserialize a Tree	Decision walkthrough
25. Top K Frequent	Decision walkthrough
26. Median of a Stream	Decision walkthrough
27. Kth Largest	Decision walkthrough
28. Number of Islands	Decision walkthrough
29. Clone Graph	Decision walkthrough
30. Course Schedule	Decision walkthrough
31. Network Delay	Decision walkthrough
32. Minimum-Cost Grid Path	Decision walkthrough
33. Redundant Connection	Decision walkthrough
34. Accounts Merge	Decision walkthrough
35. Combination Sum	Decision walkthrough
36. Subsets With Duplicates	Decision walkthrough
37. Word Search	Decision walkthrough
38. Coin Change	Decision walkthrough
39. Edit Distance	Decision walkthrough
40. Longest Increasing Subsequence	Decision walkthrough

Additional algorithm walkthroughs

R1 Two Sum

Derive and choose: enumerate pairs as an O(n²) baseline. For each current value x, the needed partner is target−x; a map of earlier values answers that lookup in expected O(1). Store indices because the result requests positions.

Why not others: sorting plus two pointers uses O(n log n) time and needs original indices; it is attractive if sorted input is already available or memory constraints favor that representation.

What if: insert x before lookup and a value can match itself. Duplicate values can still be a valid pair at different indices. Widen arithmetic if target−x can overflow. All-pairs output requires a different duplicate/output policy.

Trace/test and signal: `[3,3]`, target 6: first 3 is stored; second finds index zero. Test absent pair and one-element input. Expected O(n) time, O(n) space. Explain the “earlier indices only” invariant.

R3 Longest Consecutive Sequence

Derive and choose: use a set for membership, but grow a run only from values whose predecessor is absent. Otherwise the same long run is traversed repeatedly.

Why not others: sorting and scanning is a correct O(n log n) alternative with straightforward duplicate handling. A set gives expected O(n) work because each distinct element belongs to one expanded run.

What if: iterating duplicate input values as starts can repeat whole runs; iterate the set. Guard integer minimum/maximum when computing predecessor/successor, or store widened values. Sorting in place changes the input.

Trace/test and signal: `[100,4,200,1,3,2,2]` has run 1–4. Test empty, all duplicates and integer extremes. O(distinct values) space. Prove each distinct value is expanded at most once.

R4 Three Sum

Derive and choose: sort, fix one value, and use two pointers for the remaining pair. Sorted order tells which pointer to move when the sum is too low or too high.

Why not others: three nested loops cost O(n³). Repeated hash-based pair search gives expected O(n²) too, but deduplicating value triplets is often less direct. Sorting plus pointers gives O(n²) scanning after sorting.

What if: skipping all duplicates before finding a pair can lose valid repeated-value triplets; skip repeated anchors and skip repeated pair values after processing a match. Use long for the sum. Clarify unique value triplets versus all index triples.

Trace/test and signal: `[−1,0,1,2,−1,−4]` yields two unique zero-sum triplets. Test `[0,0,0,0]`, fewer than three elements and extremes. Count output and sorting workspace. Explain both pointer elimination and duplicate policy.

R5 Container With Most Water

Derive and choose: area is width times the shorter boundary. Start at both ends; moving the taller boundary inward cannot improve area while the shorter boundary stays fixed, so discard the shorter one.

Why not others: testing every pair costs O(n²). A stack can represent some boundary problems but is unnecessary because the shorter-boundary argument directly eliminates candidates.

What if: moving the taller pointer lacks that proof and can miss the optimum. Negative heights violate the model. Widen width×height before multiplication; define fewer than two bars as zero area.

Trace/test and signal: heights `[1,8,6,2,5,4,8,3,7]` yield area 49. Test equal ends, all zeros and two bars. O(n) time, O(1) auxiliary space. State the elimination argument rather than simply naming two pointers.

R6 Trapping Rain Water

Derive and choose: water above a bar is bounded by the smaller of the greatest heights to its left and right. Prefix/suffix maxima give an easy O(n)-space solution; two running maxima can reduce space to O(1).

Why not others: rescanning both sides for every bar costs O(n²). A monotonic stack is also O(n) and useful for explaining basin boundaries, but needs width calculations.

What if: in the two-pointer variant, process the side with the smaller known maximum because the other side already supplies a sufficient boundary. Confusing this with the container objective produces a wrong recurrence. Widths other than one require weighting accumulated water.

Trace/test and signal: `[3,0,2,0,4]` holds seven units. Test monotone input, a plateau and all zeros. O(n) time; use long for large totals. Explain why the chosen side's water is final before advancing.

R8 Minimum Window Substring

Public practice reference: Minimum Window Substring.

Derive and choose: maintain required multiplicities in a moving window. Expand until all required counts are satisfied; then shrink while still valid, recording the best range.

Why not others: examining every substring repeats character counts. Sorting destroys contiguity. A set loses repeated requirements, such as needing two `a` characters.

What if: updating the number of satisfied requirements on every occurrence instead of threshold crossings breaks duplicates. Define an empty-target policy; distinguish ASCII arrays from Unicode maps. Return a range first and create the result substring once.

Trace/test and signal: `ABAAC` with required `AAC` yields `AAC`. Test no match and repeated targets. Expected O(source length + target length) time and O(alphabet) counts. Explain why each pointer moves only forward and how validity changes at a boundary.

R9 Permutation in String

Derive and choose: a matching permutation has the same length and character counts as the pattern. Slide a fixed-width window, removing the outgoing character and adding the incoming one.

Why not others: generating all permutations is factorial work. Sorting every window repeats O(k log k) work; fixed-alphabet counts allow linear traversal.

What if: duplicates require counts, not membership. A mismatch counter must change only when a character's equality with the target count changes. If pattern length exceeds input length, fail immediately. Declare empty-pattern behavior.

Trace/test and signal: pattern `ab` in `eidbaooo` matches at `ba`; `aa` does not match a window with one a. O(n + k) time for fixed alphabet and O(alphabet) state. Explain the fixed-window invariant and test entry/exit of the same character.

R12 Minimum Feasible Processing Speed

Derive and choose: if speed v meets the deadline, every larger speed also does. Binary search this monotone feasibility predicate; calculate work time with ceiling division and stop early if the budget is exceeded.

Why not others: searching every speed is proportional to the numeric range. A greedy average ignores indivisible work units and per-item rounding.

What if: no speed is feasible when each nonempty work item needs at least one unit of time and the deadline is too short. `(pile + speed − 1)` can overflow; use long or division/remainder. An empty workload needs its own minimum-speed convention.

Trace/test and signal: piles `[3,6,7,11]` in eight hours need speed four. Test deadline equal to item count and one huge pile. O(n log M) time, O(1) space for maximum pile M. Prove monotonicity and state the feasible-bound invariant.

R14 Meeting Rooms II

Derive and choose: when a meeting begins, reuse the room whose current meeting ends first if it is available; otherwise allocate another. A min-heap stores active room end times.

Why not others: comparing all pairs does not directly produce peak overlap efficiently. Sorting start/end event arrays and sweeping is equally valid, often simpler when only the count is needed; a heap can also retain room assignments.

What if: for half-open meetings, end time equal to a new start frees a room. Closed intervals need different tie handling. Zero-length events need an explicit policy. Reject reversed endpoints.

Trace/test and signal: `[0,30), [5,10), [15,20)` needs two rooms. Test simultaneous starts and touching meetings. O(n log n) time and O(n) worst-case state. Explain why the peak active count is a lower bound and is achievable.

R15 Insert Interval

Derive and choose: with sorted, nonoverlapping existing intervals, copy those entirely before the new one, merge the contiguous overlap block, then copy the rest.

Why not others: appending and sorting is correct but spends O(n log n) on an ordering guarantee already supplied. An interval tree is useful for many dynamic updates, not necessary for one insertion.

What if: unsorted or overlapping input invalidates the three-phase assumption; normalize first or use general merge. Decide touching-boundary semantics and whether returned intervals may alias input arrays.

Trace/test and signal: insert `[2,5]` into `[1,3], [6,9]` to get `[1,5], [6,9]` under closed semantics. Test before all, after all, containing all and empty input. O(n) time plus O(n) output. State why overlaps form one contiguous block.

R16 Reverse Linked List

Derive and choose: maintain a reversed prefix and an untouched suffix. Save the next node before redirecting the current node's next pointer; then advance both boundaries.

Why not others: recursion is concise but consumes O(n) call-stack space. An explicit stack also uses O(n) space. In-place iteration uses O(1) auxiliary state when mutation is permitted.

What if: changing next before saving it loses the suffix. Shared external references observe the reversed links; copy nodes if preservation is required. A cyclic input violates the ordinary terminating traversal contract.

Trace/test and signal: `1→2→3` becomes `3→2→1`; the former head must end at null. Test empty, singleton and two nodes. O(n) time. Explain the prefix/suffix invariant after each pointer update.

R18 Merge K Sorted Lists

Derive and choose: the next globally smallest value must be among the current heads of nonempty lists. Keep those heads in a min-heap; after removal, add that node's successor.

Why not others: repeatedly scanning k heads costs O(Nk). Balanced pairwise merging also gives O(N log k) time and avoids heap bookkeeping; concatenating and sorting ignores existing order.

What if: disjoint acyclic input lists are assumed when reusing nodes. Shared tails can cause duplicated output or cycles under careless rewiring. Copy nodes when ownership or aliasing requires it. Equal values need only stable tie-breaking if the contract requests stability.

Trace/test and signal: merge `[1,4]`, `[1,3]`, and empty into `[1,1,3,4]`. O(N log(k+1)) time, O(k) frontier, plus output if copied. Test many empty lists and one very long list. Prove why no hidden non-head can be smaller.

R21 Largest Rectangle in Histogram

Derive and choose: each bar can define a rectangle extending until a smaller bar blocks it on either side. An increasing-height stack discovers the right boundary when a smaller incoming height arrives.

Why not others: expanding from every bar costs O(n²). Precomputing previous/next smaller boundaries is another O(n) stack formulation. A single-pass stack computes areas as boundaries become known.

What if: after popping index p, width is currentIndex minus the new stack top minus one; if empty, it extends to zero. Equal-height handling must be consistent. Add a final zero sentinel or drain the stack. Use long for area.

Trace/test and signal: `[2,1,5,6,2,3]` has area ten from heights 5/6. Test increasing, decreasing, all equal and zero bars. O(n) time and stack space. Explain both boundaries and why the popped bar cannot extend farther.

R23 Lowest Common Ancestor

Derive and choose: in a general binary tree, ask each subtree whether it contains a target. If targets are found on different sides, the current node is their lowest meeting point; otherwise propagate the found node upward.

Why not others: ancestor sets or parent-pointer paths are simple when parent links exist. A BST can use ordering to choose one branch, but that proof is invalid for an arbitrary binary tree.

What if: the usual recursion assumes both targets exist; otherwise it may return the one found as if it were an LCA. Track presence or validate membership when absence is allowed. Compare node identity, not only duplicated values.

Trace/test and signal: targets in different root branches yield root; a target that is ancestor of the other yields itself. Test same node, missing node and skewed tree. O(n) time and O(height) stack. State the meaning of the recursive return value and membership assumptions.

R24 Serialize and Deserialize a Tree

Derive and choose: values alone do not describe shape. Preorder traversal with explicit null markers provides a parseable recursive description: node, left subtree, right subtree.

Why not others: inorder values alone are ambiguous; level order plus null markers is also valid and avoids recursive traversal. JSON objects may be simpler for an application, with a defined schema and payload budget.

What if: omitting null markers makes a left-only child and right-only child indistinguishable. Values containing delimiters need escaping or a length-prefixed encoding. Deep trees, huge payloads and malformed input need depth/size/token limits.

Trace/test and signal: tree 1 with left child 2 and no right child can encode as `1,2,#,#,#`. Test `decode(encode(tree))` for shape and values, null root and negative values. O(n) nodes plus encoded-character processing; O(height) recursive stack. Explain framing and validate complete token consumption.

R29 Clone Graph

Derive and choose: preserve graph topology while creating new identities. On first seeing a node, allocate and register its clone before exploring neighbors; then reuse the mapping for every edge.

Why not others: recursively copying without a visited map loops on cycles and duplicates shared neighbors. BFS and DFS are both O(V + E); iterative BFS avoids deep call-stack limits.

What if: duplicate node labels rule out a value-keyed map. Self-loops must point to the clone itself. A graph reachable from one root excludes disconnected components unless they are supplied as additional roots.

Trace/test and signal: A↔B clones into new A′↔B′ with no edges to originals. Test self-loop, repeated edges, shared neighbors and null root. O(V + E) output and O(V) traversal/mapping state. State the one-original-to-one-clone invariant.

R32 Minimum-Cost Grid Path

Derive and choose: define vertices as cells and edges as allowed moves. If arbitrary moves have nonnegative costs, use Dijkstra from the start; define whether cost belongs to entering a cell or traversing an edge.

Why not others: BFS only minimizes the number of equal-cost moves. If movement is restricted to right/down, the graph is acyclic and a simpler DP gives O(rows × columns) time. For zero/one edge costs, 0–1 BFS is another specialized choice.

What if: negative moves break Dijkstra's proof; diagonal moves change connectivity. A minimum-total-cost path and a path minimizing its largest edge are different objectives and require different relaxation functions.

Trace/test and signal: in a 2×2 grid with entry costs `[[0,100],[1,1]]`, right/down total cost is minimized by going down then right. Test unreachable cells and one-cell grids. For ordinary four-neighbor nonnegative costs, heap-based work is O(RC log(RC)) with O(RC) state. Derive the graph and objective before naming an algorithm.

R33 Redundant Connection

Derive and choose: process undirected edges incrementally; if both endpoints already have the same representative, adding the edge creates a cycle. Use the disjoint-set implementation in Part 9.

Why not others: DFS reachability before each edge works but repeats traversal. One DFS is appropriate for finding cycles in a static graph, whereas union-find matches incremental connectivity.

What if: a directed graph requires a different cycle criterion. Parallel edges and self-edges may themselves be cycles under the chosen graph definition. “First redundant edge” and “last removable edge” can require different output handling.

Trace/test and signal: edges 1–2, 2–3, 1–3 flag the third edge. Initialization is O(V); processing is O(E α(V)) amortized and O(V) space. Explain what union returning false means, and match it to the exact requested edge.

R34 Accounts Merge

Derive and choose: shared email identity connects account records transitively. Map each email to a representative account and union accounts sharing an email; then group and sort emails by final root.

Why not others: a graph of email/account relationships plus DFS works too. Comparing every pair of accounts repeats expensive intersections. Equal display names alone are not evidence of shared identity.

What if: duplicate emails in one account must not duplicate output. Conflicting names within one connected group need a policy rather than arbitrary selection. Accounts without emails need explicit treatment. Never perform real identity merging from this toy rule without a domain contract.

Trace/test and signal: one account shares x with a second, which shares y with a third; all three merge even without direct overlap between first and third. Expected union work is near-linear in email occurrences plus output sorting; space is proportional to accounts and distinct emails. Explain the equivalence relation and transitivity.

R36 Subsets With Duplicates

Derive and choose: enumerate include/skip choices, but ensure equal values do not start the same branch twice. Sort and skip an equal candidate only when it is a duplicate sibling at the current recursion depth.

Why not others: generating every index subset then deduplicating outputs wastes work and requires canonical keys. Counting multiplicities and choosing 0..count copies of each value is an equally sound alternative.

What if: skipping every repeated value globally loses valid subsets containing two copies. Store a copy of the current path, not the mutable path itself. Sorting mutates input unless copied first.

Trace/test and signal: `[1,2,2]` has six unique subsets including `[2,2]`. Test empty and all-equal input. Worst-case O(n 2^n) output work and O(n) auxiliary path/recursion space. Explain the difference between duplicate sibling choices and legitimate repeated selections.

R37 Word Search

Derive and choose: try each matching start cell; recursively match the next character through permitted neighbors while marking cells used on the current path. Undo the mark when that branch returns.

Why not others: a global visited set rejects valid later paths; this constraint is path-specific. BFS that tracks entire used-cell sets can use large memory. A trie is useful when searching for many words together, not necessary for one word.

What if: reusing a cell is forbidden in the usual contract. If diagonal moves are allowed, branching and answers change. An in-place sentinel must not collide with valid input; restore it on every exit path.

Trace/test and signal: board `AB` cannot form `ABA` without reuse. Test repeated letters, word longer than cell count and failed branches followed by success. A simple upper bound is $O(RC \times 4^L)$, auxiliary $O(L)$ path depth, excluding an optional visited grid. Explain choose/explore/unchoose and preservation of input.

R40 Longest Increasing Subsequence

Derive and choose: an $O(n^2)$ DP computes the best subsequence ending at each index. For length only, maintain the smallest possible tail of each length; binary-search the first tail at least as large as the current value and replace it.

Why not others: sorting the input loses original order. Enumerating subsequences is exponential. The tails representation gives O(n log n) processing and O(n) state but does not by itself store an actual valid final subsequence.

What if: strictly increasing requires lower bound; nondecreasing uses upper bound. Reconstructing indices needs predecessor/position tracking. Replacing a tail improves future extension options without claiming the tails array is the selected subsequence.

Trace/test and signal: `[3,1,2]` changes tails `[3]`→`[1]`→`[1,2]`, length two. Test duplicates, decreasing input and empty input. Explain why a smaller tail of equal length dominates a larger one.

Machine coding decision walkthroughs

MC1 Bounded LRU cache

Use the LRU implementation and decision walkthrough in Module 8. **Derive:** combine direct lookup and recency order. **Alternative:** approximate eviction may improve concurrency when exact global recency is unnecessary. **What if:** TTL introduces a second expiry rule; synchronization must cover accesses that move entries. **Test:** access changes eviction, duplicate put replaces, and simultaneous operations preserve size/structure. A strong answer states whether nulls, TTL and loading are in scope before designing the API.

MC2 Rate limiter with an injected ticker

Use the token-bucket and sliding-window walkthroughs to select semantics first. **Derive:** burst credit favors a bucket; exact recent-count limits favor timestamp state. **Alternative:** a fixed window is cheaper but has a boundary-burst trade-off. **What if:** replacing real time with a ticker makes exact expiry tests deterministic; separate idle-key eviction from active-counter synchronization. **Test:** limits, exact boundaries, rollover, concurrent admission and key-cap rejection. A strong answer defines whether the method rejects, waits or reserves future capacity.

MC3 In-memory task scheduler

Derive and choose: place due times in a priority queue, wait until the earliest task is due and dispatch through a bounded executor. For ordinary production use, first consider `ScheduledExecutorService`; implementing the mechanism is the interview exercise.

Why not others: periodically scanning all tasks wastes O(number of tasks) work per tick and adds scheduling jitter. A timing wheel may suit huge numbers of coarse timers but adds complexity.

What if: cancellation races with dispatch, so define a state transition such as scheduled→running or canceled under one synchronization policy. One coordinator's exception must not stop future jobs. Wall-clock schedules and elapsed delays need different clock rules; in-memory work is lost on restart.

Trace/test and signal: enqueue a late job, then an earlier job; the waiter must wake and revise its deadline. Test cancel-before-dispatch, shutdown and executor rejection. Heap updates are O(log n); waiting tasks use O(n) state. Explain task ownership and precisely when cancellation stops being guaranteed.

MC4 Inventory reservation service

Derive and choose: available stock must never become negative, so check-and-reserve is one atomic business operation. Model reservation IDs and states so repeated reserve/release calls have deliberate semantics.

Why not others: a thread-safe map alone does not make a read-check-write sequence atomic. A global lock is simple; per-product locks improve unrelated-product concurrency but make multi-product acquisition ordering necessary. A database conditional update can enforce an invariant across service instances.

What if: expiry and explicit release can race; only one successful state transition returns stock. A retried reservation with changed quantity should not silently reuse an old result. Multi-product requests need all-or-nothing or clearly documented partial behavior.

Trace/test and signal: one unit and two simultaneous buyers permits at most one reservation. Test duplicate release, expired reservation, clock boundary and rollback. Explain the state machine and atomic boundary before introducing design patterns.

Explain the eight output puzzles

Puzzle	How to derive the answer	What changes and how to test it
Boxed integer identity	== compares references; small constant boxing has specified identity guarantees	Values outside the guaranteed range may be cached too; do not infer numeric equality from reference equality. Test `equals` and primitive unboxing separately
Literal concatenation	Constant expressions can be folded and interned; `new String` creates a distinct object	A runtime expression with nonconstant input does not have the same identity guarantee. Compare values with `equals`
Return in finally	The abrupt completion of finally replaces an earlier return/throw	Remove that return to preserve the original outcome. Test a body exception as well as a return
List modification	`Arrays.asList` is fixed-size; `List.of` is unmodifiable	`set` works for the former and changes its backing array, but not for the latter. A new ArrayList gives a mutable copy

Puzzle	How to derive the answer	What changes and how to test it
Mutable hash key	Lookup uses equality/hash-relevant state that should remain stable while stored	A mutation may break lookup; a collision can make a particular experiment still appear to work. Use immutable keys rather than relying on observed output
Removing during iteration	The iterator and collection can detect inconsistent structural modification	Fail-fast detection is not guaranteed on every path. Use iterator removal or a supported bulk operation and test skipped elements, not only exceptions
Floating-point decimal arithmetic	Binary representation approximates many decimal fractions	`BigDecimal` from decimal text gives decimal semantics; specify scale and rounding instead of comparing with an arbitrary epsilon for money
Initialization order	Determine whether an operation triggers initialization, then apply superclass/subclass rules	Compile-time constants and reflective loading options change the trigger. Compare loading with initialization explicitly

For these puzzles, the strong signal is the rule and a counterexample to an overgeneralization. Memorizing one observed output is weaker than explaining why another valid program or runtime setting changes it.

SDE-3 Terminology and Follow-up Audit

The guide should never require you to memorize a term you cannot explain. Use the following as a **depth map**: if one of these terms appears in an answer, you should be able to give the one-sentence explanation first, then go deeper only if the interviewer asks.

Module 1 — Core Java, Concurrency and JVM

`CAS` — Compare-And-Set

Simple meaning: an atomic operation that says, “change this value from A to B only if it is still A.”

Conceptually:

```
read current value
      ↓
is it still expected?
   yes ↓       no → retry/fail
update value
```

Atomics such as `AtomicInteger` use CAS-based operations. CAS avoids a traditional mutual-exclusion lock for the individual update, but it does **not** make an arbitrary multi-variable invariant automatically safe.

Follow-up: CAS can suffer from retries under contention. That is one reason `LongAdder` can outperform a single atomic counter for highly contended updates when an exact instantaneous value is not required.

`happens-before`

Simple meaning: a Java Memory Model ordering/visibility relationship. If action A happens-before action B, the effects that are required to be visible through that relationship are visible to B.

Do not describe it as “A literally executes earlier in wall-clock time.” It is a **memory-model guarantee**, not simply a timestamp relationship.

Important examples:

```
unlock(monitor) → later lock(same monitor)
volatile write → later volatile read of same variable
Thread.start() → actions in started thread
actions in thread → successful join() return
```

Follow-up: happens-before solves visibility/order guarantees; it does not make a compound operation such as `count++` atomic.

`JIT`

Simple meaning: the Just-In-Time compiler turns frequently executed bytecode into optimized native machine code while the application is running.

A useful mental model:

```
Java source
   ↓ javac
bytecode
   ↓ JVM
interpret/profile
   ↓
JIT optimization
   ↓
native machine code
```

The JVM can optimize based on runtime observations. Therefore a microbenchmark that ignores warm-up can be misleading.

Escape analysis

Simple meaning: the JVM analyzes whether an object or value can escape a method/thread. If it proves useful non-escaping properties, the JIT may optimize allocation and synchronization.

Do not promise:

> “Escape analysis always puts objects on the stack.”

That is an implementation optimization, not a Java language guarantee.

Metaspace

Simple meaning: JVM native memory used for class metadata. It is outside the Java heap.

Therefore:

```
container memory
 ≠
–Xmx
```

A Java process also uses memory for thread stacks, metaspace/class metadata, direct buffers, code cache, GC structures and native libraries.

Production follow-up: container OOM can happen even when heap usage appears below `–Xmx`.

Class loader

Simple meaning: a class loader is responsible for locating/loading class definitions into the JVM.

A useful senior-level point is that class identity is associated with both the **class name and the defining class loader**. Two classes with the same fully qualified name loaded by different class loaders can be different runtime types.

This is why class-loader leaks can matter in application servers, plugin systems and hot-reload environments.

Thread pool, queue and back-pressure

A thread pool is not just “N threads.”

The important execution sequence is:

```
task
 ↓
core threads available?
 ↓ no
queue
 ↓ queue full?
 ↓ yes
create thread up to max
 ↓ max reached?
 ↓ yes
rejection policy
```

Back-pressure means slowing/rejecting producers when downstream capacity is exhausted instead of allowing unlimited work to accumulate.

A bounded queue plus a deliberate rejection policy is therefore a capacity-protection mechanism.

Bulkhead

A **bulkhead** isolates resources so one overloaded dependency or workload cannot consume all capacity.

Example:

```
Payment calls → pool A
Search calls  → pool B
Email calls   → pool C
```

If email becomes slow, it should not consume every thread needed by payments.

This is the same principle as watertight compartments in a ship.

Virtual threads and pinning

A **virtual thread** is a lightweight JVM-managed thread designed to make high-concurrency blocking-style code practical.

It does **not**:

- make CPU work faster;
- increase database connection capacity;
- remove the need to bound scarce resources.

Pinning means a virtual thread can be prevented from unmounting from its carrier while performing certain operations, historically including some monitor-based blocking scenarios on older JDKs. Always answer this with the target JDK in mind; do not blindly apply old JDK 21 guidance to newer JDKs.

Module 3 — JPA / Hibernate

Persistence context

Think of the JPA persistence context as a managed set of entity instances associated with a unit of work.

Conceptually:

```
database row
    ↕
managed entity
    ↕
persistence context
```

Within the same persistence context, Hibernate can track entity identity and changes.

Dirty checking

Dirty checking means Hibernate detects changes made to managed entities and can generate SQL updates when the persistence context is flushed.

Example:

```
order.setStatus(PAID);
```

You do not necessarily call an explicit `UPDATE`. Hibernate tracks the managed entity and can issue the update during flush/commit.

Follow-up: dirty checking applies to managed entities; a detached/transient object is not automatically tracked in the same way.

N+1

N+1 means one query loads N parent records and then additional queries load related data individually:

```
1 query for parents
+
N queries for children
=
N+1 database round-trips
```

The fix depends on the access pattern: fetch join, entity graph, batch fetching, projection, or an explicit query can all be appropriate. “Always use eager loading” is not a good solution.

Optimistic locking

Optimistic locking assumes conflicts are relatively uncommon and detects them when updating.

Typical JPA form:

```
@Version
private long version;
```

Two transactions read version 5. One commits and changes it to 6. The other later tries to update version 5 and can fail rather than silently overwrite the first change.

Pessimistic locking

Pessimistic locking asks the database to lock the relevant rows while the transaction is working with them.

Use it when the concurrency contract genuinely requires database-level locking and the expected contention justifies the cost.

Follow-up: locks can increase blocking and deadlock risk. Do not present pessimistic locking as automatically “safer.”

Isolation level

An isolation level defines which concurrent transaction effects a transaction is allowed to observe.

When answering, separate:

```
database isolation guarantees
vs
application-level locking
vs
optimistic version checks
```

Do not claim that `@Transactional` by itself gives serializable behavior.

Module 4 — Microservices and distributed systems

Circuit breaker

A **circuit breaker** prevents repeated calls to a failing dependency from consuming local resources indefinitely.

Typical states:

```
CLOSED
  ↓ failures
OPEN
  ↓ after recovery delay
HALF_OPEN
  ↓ probe succeeds
CLOSED
```

A circuit breaker is not a substitute for timeouts. A call should have a bounded deadline before the circuit breaker even has a chance to help.

Saga

A **saga** coordinates a distributed business transaction as a sequence of local transactions.

Example:

```
Create order
   ↓
Reserve inventory
   ↓
Charge payment
```

If payment fails, the system may execute a compensating action such as releasing inventory.

A saga is not the same thing as a database transaction. It trades atomicity for a workflow with explicit compensation and consistency semantics.

Outbox pattern

The **outbox pattern** solves the problem of atomically updating a database and publishing an event.

Instead of:

```
DB commit
   ↓
publish Kafka event
```

where the two operations can disagree, write the business change and an outbox record in the **same database transaction**:

```
transaction:
  update business data
  insert outbox event
commit
```

A separate publisher then sends the outbox event.

Follow-up: this usually gives at-least-once delivery, so consumers still need idempotency.

Idempotency

An operation is **idempotent** when repeating the same logical request produces the same intended final effect.

For payments, for example:

```
POST /payments
Idempotency-Key: abc123
```

A retry with the same key should not charge the customer twice.

Idempotency is especially important when timeouts make it unclear whether the server completed the original request.

At-least-once vs exactly-once

At-least-once: a message should not be lost, but duplicates can occur.

Exactly-once: the system attempts to make the processing effect occur exactly once under a defined scope and set of guarantees. Do not casually equate Kafka's exactly-once processing semantics with “the external database/business side effect can never happen twice.”

For interviews, state the boundary of the guarantee.

Eventual consistency

Eventual consistency means replicas or services may temporarily disagree, but under the model's assumptions they converge if updates stop.

Do not equate eventual consistency with “random stale data.” The system still needs a defined convergence and conflict-resolution model.

Kafka — terminology you should be able to explain

Topic, partition and consumer group

A **topic** is a logical stream of records.

A **partition** is an ordered append-only sequence within that topic.

A **consumer group** is a set of consumers cooperating to process partitions. Within one consumer group, a partition is normally assigned to one consumer at a time.

Therefore:

```
topic
 ├── partition 0 → consumer A
 ├── partition 1 → consumer B
 └── partition 2 → consumer C
```

Partition count is therefore a major scalability constraint for parallel consumption.

Rebalance

A **rebalance** redistributes partition ownership among consumers in a consumer group.

It can happen when consumers join/leave or when group membership changes.

A rebalance is not free: it can temporarily affect processing and should be considered in latency and throughput design.

ISR

In Kafka, **ISR** means *in-sync replicas*. These are replicas considered sufficiently caught up to the leader under Kafka's replication rules.

When discussing durability, distinguish:

```
replication factor
vs
ISR
vs
acks
vs
min.insync.replicas
```

Do not collapse all four into “Kafka replication.”

Poison message

A **poison message** is a message that repeatedly fails processing.

If the consumer continually retries it without progress, it can block useful processing.

Typical strategies include:

```
bounded retries
dead-letter topic/queue
failure classification
alerting
manual replay
```

Security — terminology you should be able to explain

JWT

A JWT is a signed token containing claims.

Important distinction:

> A JWT is normally **encoded and signed**, not encrypted merely because it is a JWT.

Therefore sensitive secrets should not be placed in ordinary JWT claims.

JWKS

JWKS is a JSON Web Key Set: a published set of public keys that a resource server can use to validate signed tokens.

Mental model:

```
Issuer
  ↓ publishes
JWKS public keys
  ↓
Resource server validates JWT signature
```

OAuth2 vs OIDC vs PKCE

- **OAuth 2.0** — authorization/delegation framework.
- **OIDC** — authentication layer built on OAuth 2.0.
- **PKCE** — protects authorization-code flows against code interception by binding the authorization request to a verifier held by the client.

A common interview mistake is saying:

> "OAuth2 is authentication."

It is primarily an authorization framework; OIDC adds standardized authentication semantics.

CORS vs CSRF

CORS controls whether browser JavaScript from one origin may access resources from another origin.

CSRF tricks a browser into making an authenticated request using ambient credentials such as cookies.

They solve different problems.

mTLS

Mutual TLS means both sides authenticate during the TLS handshake using certificates.

Normal TLS commonly authenticates the server to the client; mTLS adds client authentication.

RBAC

Role-Based Access Control assigns permissions through roles:

```
user → role → permissions
```

It is different from fine-grained attribute/policy-based authorization.

SSRF

Server-Side Request Forgery occurs when an attacker can influence a server into making requests to destinations the attacker should not be able to reach directly.

Mitigations include strict destination allowlists, URL/IP validation, network egress controls and careful redirect handling.

System design terminology

CAP

CAP concerns distributed systems under a network partition:

```
Consistency
Availability
Partition tolerance
```

When a partition occurs, a distributed system cannot simultaneously guarantee both the strongest form of consistency and availability for all operations.

Do not use CAP as “you can only ever choose two.” The interesting question is what guarantee the system provides **during a partition**.

PACELC

PACELC extends the discussion:

```
If Partition:
    choose Availability or Consistency

Else:
    choose Latency or Consistency
```

It is useful when discussing real-world distributed database trade-offs beyond CAP.

Quorum

A quorum is a sufficient number of replicas participating in an operation to satisfy a consistency/durability rule.

Do not assume every database uses the same quorum formula or semantics. Always name the system and its configured behavior.

Consistent hashing

Consistent hashing maps both nodes and keys onto a logical hash space.

When nodes change, ideally only a relatively small portion of keys moves, unlike naive modulo hashing where changing the node count can remap a large fraction of keys.

Rate limiter

A rate limiter controls how much work a caller may perform over time.

Know at least:

```
Fixed window
Sliding window
Token bucket
Leaky bucket
```

For token bucket:

```
tokens accumulate at a configured rate
       ↓
request consumes tokens
       ↓
no token → reject/delay
```

It can allow controlled bursts while enforcing an average rate.

Keyset pagination

Instead of:

```
OFFSET 100000 LIMIT 20
```

keyset pagination uses the last seen ordered key:

```
WHERE (created_at, id) < (?, ?)
ORDER BY created_at DESC, id DESC
LIMIT 20
```

It can avoid scanning/skipping large numbers of earlier rows and gives stable continuation when the ordering is properly indexed.

Kubernetes deployment terminology

Rolling, blue-green and canary

Rolling deployment: gradually replaces old instances with new ones.

Blue-green: maintains two environments and switches traffic from the old environment to the new one.

Canary: sends a controlled fraction of traffic to the new version first, observes behavior, then expands traffic.

The important SDE-3 question is not “define them,” but:

> Which one gives me the safest rollback and smallest blast radius for this workload?

Readiness vs liveness vs startup

- **Startup probe:** “Has the application finished starting?”
- **Readiness probe:** “Should this instance receive traffic?”
- **Liveness probe:** “Is this process sufficiently unhealthy that it should be restarted?”

Do not put downstream dependency health blindly into liveness; a database outage should not necessarily cause every application pod to restart.

Live-coding terminology

Invariant

An **invariant** is a condition that remains true at important points during the algorithm.

Example for binary search:

```
target, if it exists, is still inside [lo, hi)
```

Example for a sliding window:

```
the current window satisfies the required constraint
```

Stating the invariant makes the solution easier to prove and debug.

Amortized complexity

Amortized complexity averages expensive operations over a sequence.

For a dynamic array:

```
most append operations → O(1)
occasional resize      → O(n)
overall                 → O(1) amortized append
```

This is different from claiming that every append is worst-case O(1).

Identity vs equality

Java's == on object references asks whether two references point to the same object.

`equals()` asks whether the objects are logically equal according to the class contract.

This distinction matters for:

```
HashMap
IdentityHashMap
graph cloning
entity comparison
caching
```

Interview rule for all remaining sections

Whenever an answer contains a specialized term, use this progression:

```
1. What is it?
2. Why does it exist?
3. How does it work?
4. Small example
5. Complexity / guarantee
6. Failure mode
7. Trade-off
8. SDE-3 follow-up
```

You do **not** need to say all eight points every time. The point is to make sure you can go one level deeper whenever the interviewer asks.

A strong senior answer is not the longest answer. It is an answer where every term you introduce can survive the next “why?”.

Part 12 - More Senior-Level Questions (Q154-Q173)

Twenty additional questions on topics the guide did not yet cover: consensus and data distribution, database internals, low-level concurrency, JVM performance, Spring operations, and engineering practice. Each follows the same layout as Part 10: **Source basis**, **Strong answer**, **Reasoning and alternatives**, **What if/test**, and **Signal**. Where no specific source is cited, the basis is general engineering practice, so check details against the documentation for your version before quoting numbers.

Distributed systems and data

Before consensus and sharding, review CAP, quorums and consistency terminology, plus SQL transactions. Then use Q154–Q160 for the deeper mechanisms.

Q154 How does Raft elect a leader and why do systems use consensus

Source basis: Raft paper.

Strong answer: Raft keeps a replicated log consistent across nodes by electing at most one leader per *term*. A follower that hears no heartbeat before a randomized election timeout becomes a candidate, increments the term and requests votes. A node grants at most one vote per term, and only to a candidate whose log is at least as up to date as its own. A candidate with a majority becomes leader; at most one leader is elected in a term, and some terms have no leader. The leader replicates entries through `AppendEntries`. To advance `commitIndex` by replica count, it must count an entry from its **current term**. Committing that entry also commits its preceding log prefix; a prior-term entry replicated to a majority alone is not sufficient.

Reasoning and alternatives: a majority quorum means 3 nodes tolerate 1 failure and 5 tolerate 2, and any two majorities overlap. Raft's election and log-safety rules preserve committed entries across leader changes. Systems such as etcd (Kubernetes' store) and Consul use Raft; others use Paxos variants. Consensus has coordination costs. It can govern metadata or durable application data, depending on the system; replicated databases also use consensus in their data path.

What if/test: a minority partition cannot elect a leader or commit, trading availability for consistency. Randomized timeouts reduce split votes. Test with fault injection: kill the leader, partition the network, and verify no committed write is lost. **Signal:** explain terms, majority overlap and the up-to-date-log rule, not just "a leader is elected".

Q155 What is consistent hashing and why use virtual nodes

Strong answer: nodes and keys are hashed onto a ring; each key belongs to the first node clockwise. Adding or removing a node moves only the keys in the adjacent range, roughly `K/N` of them, whereas `hash(key) % N` remaps almost every key when `N` changes. **Virtual nodes** give each physical node many ring positions, which evens out load and lets stronger machines take more positions.

Reasoning and alternatives: Dynamo-style stores, Cassandra and cache client libraries use it. For replication, copy each key to the next *R* distinct physical nodes. Alternatives: fixed hash slots (Redis Cluster uses 16,384) that are moved between nodes, or a lookup directory.

What if/test: hot keys stay hot regardless of the ring; they need caching, splitting or key salting. Test balance by measuring keys per node with realistic keys, and measure data moved when adding a node. **Signal:** quantify the remapping difference and name the virtual-node benefit.

Q156 What is MVCC and why do readers not block writers

Source basis: PostgreSQL MVCC introduction and explicit locking.

Strong answer: multi-version concurrency control keeps several versions of a row and gives each transaction a snapshot, so a reader sees the data as of its snapshot while writers create new versions. Ordinary snapshot reads generally avoid waiting for row updates by reading an eligible older version. Explicit locking reads and conflicting table/DDL locks can still block; write-write conflicts can wait or fail. MVCC does not mean the database uses no locks.

Reasoning and alternatives: PostgreSQL stores old row versions in the table and relies on `VACUUM` to reclaim them; InnoDB keeps old versions in undo logs. The visible-version rules depend on the isolation level (READ COMMITTED takes a fresh snapshot per statement, REPEATABLE READ one per transaction).

What if/test: long-running transactions prevent cleanup of old versions, causing table bloat and slower queries. Monitor oldest transaction age and dead tuples, and set statement and idle-in-transaction timeouts. **Signal:** connect MVCC to isolation levels and to the operational bloat risk.

Q157 When do you use SQL window functions

Source basis: PostgreSQL window function tutorial.

Strong answer: a window function computes a value over a set of related rows *without collapsing them*, unlike `GROUP BY`. Typical uses: top-N per group, running totals, ranking, and comparing a row with the previous one (`LAG`).

```
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC, id) AS rn
  FROM employee e
) t WHERE rn <= 3;            -- top 3 earners per department
```

Reasoning and alternatives: use `RANK`/`DENSE_RANK` instead of `ROW_NUMBER` when ties must share a position. A correlated subquery can do the same but is usually slower and harder to read. Portable JPQL has no window-function syntax. Use native SQL mapped to a projection, or a provider extension such as Hibernate HQL window functions; a projection alone does not add the missing syntax.

What if/test: an `ORDER BY` inside `OVER` without a frame uses a default frame that can surprise running totals. Check the plan with `EXPLAIN` and add an index matching the partition and order columns. **Signal:** write the top-N-per-group query from memory and explain tie handling.

Q158 What is a Bloom filter and where would you use one

Source basis: Guava BloomFilter contract.

Strong answer: a Bloom filter is a bit array plus *k* hash functions that answers "possibly present" or "definitely absent". For entries successfully inserted into the filter under its supported update contract, it has **no false negatives**, but can produce false positives. A filter that has not yet received a database update can still reject a newly created database key. Standard Bloom filters cannot delete entries; counting variants can.

Reasoning and alternatives: use it to skip expensive lookups for keys that do not exist, for example blocking cache-penetration queries before they hit the database, or checking SSTables in LSM-tree stores. The false-positive rate depends on bit-array size and the number of items; size it from the expected count and target rate. Guava provides `BloomFilter`. Alternatives: a cache of negative results, or a cuckoo filter if deletion matters.

What if/test: if the filter fills beyond its design capacity, false positives climb; rebuild it periodically. Test by inserting the expected number of items and measuring the observed false-positive rate. **Signal:** state the one-sided error guarantee and a concrete use.

Q159 How do systems resolve concurrent writes to the same data

Strong answer: options, from simplest to most capable: avoid conflicts with a single writer per key; **last-write-wins** using timestamps (simple but silently discards writes and is vulnerable to clock skew); **vector clocks** that detect concurrent updates and keep siblings for the application to merge; **CRDTs** whose state-based merge is associative, commutative and idempotent, or whose operation-based design meets its delivery and commutativity requirements. They can provide convergence without coordinating each update, subject to their model assumptions; or application-specific merge rules.

Reasoning and alternatives: pick based on the business meaning: an add-only set can merge by union, but a real shopping cart must also define removals and quantities; a bank balance cannot safely be last-write-wins. A linearizable store orders individual operations but does not make a multi-step read-modify-write atomic. Use an atomic update, compare-and-swap/version check or transaction for business invariants; coordination can add latency.

What if/test: clock skew makes last-write-wins lose newer data. Test by writing concurrently from two replicas during a partition and checking the converged result against the business rule. **Signal:** tie the resolution strategy to what the data means.

Q160 How do you choose a sharding strategy

Strong answer: common strategies are **range** (supports range scans but suffers hotspots with monotonic keys such as timestamps), **hash** (balances load but loses range queries), **directory/lookup** (flexible but the directory is a dependency) and **geo** (data residency and latency). Pick the shard key from the dominant access pattern, with high cardinality and even distribution.

Reasoning and alternatives: sharding adds cross-shard joins, transactions and resharding work, so exhaust simpler steps first: indexing, caching, read replicas and table partitioning. Use consistent hashing or many logical shards mapped to fewer physical nodes so resharding moves whole shards.

What if/test: a celebrity tenant or hot key overloads one shard; handle it with key salting, dedicated shards or caching. Test with production-like skewed traffic, not uniform random keys. **Signal:** justify the shard key and name the cost of resharding.

Low-level concurrency and JVM performance

Prerequisite: first work through the multithreading foundations, particularly shared-state invariants and atomic operations. The questions below assume those basics.

Q161 What is the ABA problem and how do you avoid it

Source basis: AtomicStampedReference.

Strong answer: a compare-and-set succeeds when the current value equals the expected value, but the value may have changed from A to B and back to A in between, so the CAS succeeds although the state is different. It matters in lock-free structures that reuse nodes or recycle identifiers.

Reasoning and alternatives: attach a version or stamp so equality also requires the same version: `AtomicStampedReference` (reference plus changing int stamp). `AtomicMarkableReference` has only one boolean bit: it suits protocols such as irreversible logical deletion but does not detect arbitrary A→B→A reuse if the mark

returns to its old value. On the JVM, garbage collection prevents many reference-reuse cases because a node cannot be reclaimed while referenced, but pooled objects and numeric values remain exposed. Often the better answer is to avoid hand-written lock-free code and use `ConcurrentLinkedQueue` or a lock.

What if/test: State a stamp wraparound assumption. Test A→B→A while retaining the old stamp and assert the stamped CAS fails; restoring both a reference and mark bit can let a marked CAS succeed. **Signal:** explain why CAS equality is not state equality.

Q162 StampedLock optimistic reads versus ReentrantReadWriteLock

Source basis: StampedLock.

Strong answer: `StampedLock.tryOptimisticRead()` returns a stamp without locking; the reader copies the fields it needs and calls `validate(stamp)`. If a writer intervened, the read is retried under a real read lock. For read-heavy data with short critical sections this avoids lock traffic.

Reasoning and alternatives: `StampedLock` is not reentrant, has no `Condition` support, and must be used carefully: never act on optimistic data before validating. `ReentrantReadWriteLock` is simpler and reentrant but reads still contend on the lock state. Often a `volatile` immutable snapshot or `ConcurrentHashMap` is simpler than either.

What if/test: using unvalidated data or acting on it before validation can expose an inconsistent snapshot. A delay before validation does not bypass the validation check, but it increases retry likelihood and can waste work. Verify under a mixed reader/writer stress test and compare performance with JMH before choosing the complexity. **Signal:** describe the read-copy-validate pattern and its restrictions.

Q163 What is false sharing

Strong answer: CPU caches move data in cache lines (commonly 64 bytes). If two threads write different variables that sit on the same line, each write invalidates the other core's copy, so performance collapses even though the threads never share data logically.

Reasoning and alternatives: mitigate by separating hot, independently written fields onto different lines through padding, or the JDK-internal `@Contended` annotation. Application use also needs module access (for example `--add-exports java.base/jdk.internal.vm.annotation=ALL-UNNAMED` for an unnamed module) and the VM option `-XX:-RestrictContended`; it is nonportable, not an ordinary application API. `LongAdder` spreads updates across padded cells for this reason. Another approach is per-thread accumulation merged later.

What if/test: the effect is hardware and layout dependent, so prove it with a JMH benchmark that pads versus does not pad, rather than assuming. **Signal:** know the cache-line mechanism and insist on measurement.

Q164 Why is object allocation cheap on the JVM, and what is escape analysis

Strong answer: allocation in the young generation is typically a pointer bump inside a thread-local allocation buffer (TLAB), so it needs no lock. Short-lived objects die young and are cheap to collect. The JIT's **escape analysis** can find objects that never leave a method and may eliminate the allocation (scalar replacement), keeping fields in registers.

Reasoning and alternatives: consequences: do not pool ordinary objects out of habit, because pooling adds synchronization and keeps garbage alive; optimize allocation only when profiling (JFR allocation profiling) shows it matters. Large allocation rates still increase GC frequency, and optimizations are not guaranteed.

What if/test: microbenchmarks can mislead because the JIT removes allocations that real code retains; use JMH with a blackhole and check allocation rate in a realistic load test. **Signal:** avoid cargo-cult object pooling and ground claims in profiling.

Spring operations and engineering practice

Q165 Spring Retry or Resilience4j, and what is the ordering trap with transactions

Strong answer: both provide retry with backoff. Spring Retry offers `@Retryable`/`RetryTemplate` through proxies; Resilience4j offers composable decorators (retry, circuit breaker, bulkhead, rate limiter) with metrics. Newer Spring Framework releases also bring retry support into the core framework, so check what your version provides.

Reasoning and alternatives: the retry boundary must enclose a **new physical transaction for each attempt**. An outer retry proxy alone does not guarantee that: if the caller already has a transaction and each attempt uses default `REQUIRED`, all attempts join the same transaction. A failure can leave it rollback-only, so a later retry cannot commit. Establish the transaction boundary deliberately, such as by invoking a separate proxied `REQUIRES_NEW` unit per attempt or using `TransactionTemplate` per attempt when independent commits are intended. Account for connection-pool use and the business consequences of separate commits. Retry only transient failures on idempotent operations, with exponential backoff and jitter, and combine with a circuit breaker so retries do not amplify an outage.

What if/test: retries around a call that is not idempotent create duplicates (charges, emails). Test failure injection: first two attempts fail and the third succeeds; verify transaction identities, rollback-only behavior and the actual side-effect/idempotency contract. **Signal:** state the retry-outside-transaction rule and the idempotency requirement.

Q166 How do you run scheduled jobs safely across many instances

Strong answer: `@Scheduled` runs on every instance, so a job executes N times with N replicas. Options: a distributed lock (ShedLock with a database or Redis lock and an expiry), Quartz with a clustered JDBC job store, or moving scheduling out of the app to a Kubernetes `CronJob` or Cloud Scheduler that calls one endpoint.

Reasoning and alternatives: make jobs **idempotent** regardless, because lock expiry, retries and failover can still cause a duplicate run. Set the lock duration longer than the typical run but short enough to recover, and decide what to do about missed runs (misfire policy).

What if/test: if a job outlives its lock, a second instance starts concurrently. Test by running two instances and killing one mid-job. **Signal:** combine a locking choice with idempotent job design.

Q167 How do you reduce Spring Boot startup time and memory

Strong answer: measure first: use the startup actuator endpoint with `BufferingApplicationStartup` to see which beans and phases are slow. Then trim: remove unused starters and auto-configurations, narrow component scanning, defer expensive initialization, and consider `spring.main.lazy-initialization=true`.

Reasoning and alternatives: lazy initialization moves cost to the first request and delays discovery of wiring errors, so use it knowingly. Heavier options: Class Data Sharing archives (AppCDS), Spring AOT with GraalVM native image (fast start and low memory, with build-time and reflection constraints), or checkpoint/restore approaches where supported. On serverless platforms combine with minimum instances and startup CPU boost.

What if/test: a faster start can hide slower peak throughput (native image) or first-request latency spikes (lazy beans). Compare cold start, memory and steady-state latency before and after. **Signal:** diagnose before optimizing and list the trade-off of each lever.

Q168 What are the risks of Lombok, and why use MapStruct

Strong answer: Lombok removes boilerplate, but `@Data` on JPA entities generates `equals`, `hashCode` and `toString` that can touch lazy collections, trigger extra queries or recurse through bidirectional associations. Builders and annotation-processor ordering can also confuse debugging and tooling. MapStruct generates mapping code at compile time (no reflection) between entities and DTOs.

Reasoning and alternatives: use records for immutable DTOs, which reduces the need for Lombok there; for entities write `equals`/`hashCode` deliberately. Configure MapStruct with `unmappedTargetPolicy = ERROR` so a new field cannot be silently unmapped.

What if/test: test the mapper with a contract test, and verify `toString` never loads lazy associations. **Signal:** name the entity-specific Lombok hazards and the compile-time-mapping benefit.

Q169 How do you prevent SQL injection beyond "use prepared statements"

Source basis: OWASP SQL Injection Prevention Cheat Sheet.

Strong answer: parameterize all *values*. The gaps are things that cannot be parameters: table or column names and sort direction in dynamic `ORDER BY`. Map user input to an allow-list of known identifiers instead of concatenating it. Also escape wildcard characters in `LIKE` when the input must be literal, avoid string-built JPQL/native queries, and review dynamic SQL inside stored procedures.

Reasoning and alternatives: an ORM does not make you immune if you concatenate into queries. Apply least privilege to the database account, validate input shape, and monitor for anomalous queries as defense in depth.

What if/test: include injection payloads and unusual sort parameters in automated tests, and run static analysis plus dynamic scanning in CI. **Signal:** handle the dynamic `ORDER BY` case with an allow-list.

Q170 What makes a test suite good beyond coverage, and what are mutation and property-based testing

Strong answer: coverage shows which lines ran, not whether tests would detect a bug. **Mutation testing** (for example PIT) changes the code in small ways and checks whether tests fail; surviving mutants reveal weak assertions. **Property-based testing** (for example jqwik) states a rule that must hold for all inputs, generates many cases and shrinks a failing case to a minimal example.

Reasoning and alternatives: use them selectively on core logic such as pricing, parsing and state machines, because mutation runs are slow. A healthy suite is fast, deterministic, behavior-focused, and has few mocks of types you own.

What if/test: quarantine flaky tests with an owner and deadline. Track defect escape rate and test duration, not just percentage coverage. **Signal:** argue that assertion quality matters more than line coverage.

Q171 Trunk-based development versus GitFlow

Strong answer: trunk-based development uses short-lived branches merged to the main branch at least daily, with **feature flags** hiding incomplete work and CI keeping main releasable. GitFlow uses long-lived `develop` and release branches, which suits scheduled, versioned releases but increases merge pain and delays integration.

Reasoning and alternatives: for services deployed continuously, trunk-based development shortens lead time and reduces risk because changes are small. Pair it with expand-and-contract database migrations and automated rollback. Flags need ownership and cleanup, or they become debt.

What if/test: a flag combination matrix can explode; remove flags after rollout and test both states of any flag that guards risky logic. **Signal:** connect branching strategy to delivery metrics (lead time, change failure rate).

Q172 What goes wrong with multi-region active-active

Strong answer: you must handle write conflicts across regions, cross-region replication lag, globally unique IDs, split-brain during partitions, and the latency of any synchronous cross-region coordination. A common design assigns each user or tenant a **home region** for writes, with other regions serving reads or standby, and uses a global load balancer for failover.

Reasoning and alternatives: active-passive is simpler and often meets the recovery objective. Truly multi-writer designs need a conflict strategy (see Q159) or a globally consistent database such as Spanner, accepting its cost and latency. Define RTO and RPO first.

What if/test: failover drills reveal hidden dependencies (a single-region cache, DNS TTLs, shared secrets). Run regular game days that fail a whole region. **Signal:** start from RTO/RPO and prefer the simplest topology that meets them.

Q173 When do you add a search engine such as Elasticsearch, and how do you keep it in sync

Strong answer: a search engine builds an **inverted index** (term to documents), applies analyzers (tokenizing, stemming) and ranks results (BM25 by default in Elasticsearch). It is built for full-text search, faceting and relevance, where an unindexed leading-wildcard `LIKE '%term%'` may require a scan. For simple needs, database full-text search or a PostgreSQL `pg_trgm` GIN/GiST index may be enough. A normal B-tree generally cannot accelerate a leading-wildcard `LIKE`, but trigram indexes support patterns such as `LIKE '%term%'` when useful trigrams can be extracted. Check the plan; patterns without extractable trigrams can still scan.

Reasoning and alternatives: treat the search index as a **derived store**, not the system of record. Populate it from change events (outbox or CDC) so it can be rebuilt; reindex into a new index and switch an alias for zero-downtime mapping changes. Search is near-real-time (a refresh interval), so a write is not instantly searchable.

What if/test: the index drifts from the database if events are lost. Add a reconciliation job comparing counts and checksums, and test a full rebuild from source. **Signal:** call the index derived data and explain how you rebuild it.

Part 13 - Spring Boot Internals, Coding-Style and Leadership Questions (Q174-Q193)

Twenty more questions in the same layout as Parts 10 and 12. Q174–Q182 cover Spring Boot internals, Q183–Q187 are coding-style questions with solutions, and Q188–Q193 cover leadership. Version-specific points (Boot 3.x, Spring Framework 6.2 and later) should be checked against the versions your target company runs.

Spring Boot annotations: usage to internals to senior diagnosis

Read this ladder before Q174–Q182. **Foundation:** explain the annotation's purpose and scope. **Mid-level:** name the processor and lifecycle phase. **Senior/SDE-3:** predict failures, demonstrate the boundary with a test and explain the operational trade-off. Seniority is depth of reasoning, not the number of annotation names memorized. This section targets Boot 3.5 / Framework 6.2; annotation packages, conditions and defaults must be checked when migrating versions.

S1 How does an annotation actually make something happen

Foundation answer: an annotation is metadata. Spring code must discover and interpret it; Java does not automatically implement dependency injection, transactions or HTTP routing because a class carries an annotation. A manually constructed object can carry `@Autowired` or `@Transactional` and still receive neither behavior.

Internal answer: configuration parsers and import selectors register bean definitions; bean-factory post-processors can change definitions before instantiation; bean post-processors handle injection/lifecycle and may wrap beans in proxies. MVC handler mappings register endpoints. Hibernate separately reads JPA mappings. Ask “which subsystem reads this annotation, when, and what object or metadata does it create?”

Senior follow-up: distinguish startup metadata processing from per-call interception. If a bean exists but advice never runs, check the actual object, proxy path and advisor rather than adding another component scan. Q176 compares the post-processor phases; Q179 explains repository proxies.

S2 What is inside @SpringBootApplication

Foundation answer: it combines `@SpringBootConfiguration` (a specialized `@Configuration`), `@EnableAutoConfiguration` and `@ComponentScan`. Place the application class in a deliberate root package so default discovery covers intended application components.

Internal answer: component scanning discovers application bean candidates; auto-configuration imports supply conditional infrastructure definitions. Boot 3.x discovers auto-configuration candidates through `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, then evaluates conditions and ordering. This is distinct from scanning every dependency package. Auto-configuration ordering influences definition processing, not a blanket bean instantiation order.

Senior follow-up: `scanBasePackages` customizes component scanning; do not assume it also relocates entity and Spring Data repository scanning. Use `@EntityScan` and `@EnableJpaRepositories` when needed, and wire persistence units explicitly for multiple databases. Demonstrate the distinction with a context containing a service, an entity and a repository in different packages. Continue with Q174 for the complete startup lifecycle.

S3 How do @Component, @Service, @Repository and injection work

Foundation answer: stereotypes mark component candidates; `@Service` communicates the service role. `@Repository` also marks persistence components eligible for configured exception translation. They do not automatically make every method transactional. Constructor injection expresses required dependencies; a single constructor normally needs no `@Autowired`.

Internal answer: scanning registers definitions; the container resolves constructor dependencies when creating beans. `AutowiredAnnotationBeanPostProcessor` processes annotated fields/methods. `@Qualifier` narrows candidates and `@Primary` favors a candidate for single-valued injection; it does not remove other beans from a collection. `@Resource` has different name/type resolution semantics from `@Autowired`.

Senior follow-up: resolve multiple candidates explicitly, prefer constructor injection, and treat a constructor dependency cycle as a design signal. A singleton is shared within a container, not automatically thread-safe. `@Scope("prototype")` does not yield a new object on every call when one prototype is injected once into a singleton; use an appropriate provider or scoped proxy if that lifecycle is intended.

S4 How do @Configuration, @Bean and @Import build the graph

Foundation answer: `@Bean` describes a factory method whose product is managed by the container; `@Import` brings in configuration classes, selectors or registrars. `@Configuration` identifies a configuration source.

Internal answer: `ConfigurationClassPostProcessor` parses configuration, imports and bean methods. Full configuration can intercept inter-bean method calls to preserve container semantics; `proxyBeanMethods=false` skips that enhancement. In lite mode a direct Java call to another bean method follows ordinary Java behavior, which can create an extra object when the method constructs one. Inject dependencies as bean-method parameters. See Q177.

Senior follow-up: explain which bean owns lifecycle/close operations and how a test proves identity. Do not interpret `@Order` as a universal dependency or initialization guarantee; express real construction dependencies through injection, and use `@DependsOn` only for an actual initialization dependency not otherwise represented.

S5 How do conditional and configuration annotations work internally

Foundation answer: `@Profile` selects definitions by active profiles. `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnBean`, `@ConditionalOnProperty` and `@ConditionalOnWebApplication` constrain auto-configuration using available types, beans, properties or application kind. `@ConfigurationProperties` binds grouped external properties; `@Value` injects an individual value/expression.

Internal answer: conditions are evaluated during configuration processing, some using the definitions available so far. Missing-bean conditions are intended for carefully ordered auto-configuration; arbitrary user configuration ordering can produce surprises. Properties are bound through Boot's binder, with relaxed naming and conversion. Register a configuration-properties type through scanning or an enable annotation; putting the annotation on an arbitrary unregistered class is insufficient. `@Validated` enables configured validation of bound properties with a validation implementation present.

Senior follow-up: use `ApplicationContextRunner` to test property absent/enabled/disabled, class absent/present and custom-bean override cases. A user bean should trigger the documented back-off behavior. Inspect the condition report before excluding infrastructure. Decide property defaults and fail-fast validation explicitly; avoid exposing secrets in diagnostics.

S6 How do @Transactional, @Async, @Cacheable and @Scheduled differ

Foundation answer: these need their respective infrastructure, such as transaction management, async enablement, caching enablement or scheduling enablement. They do not all have the same execution model.

Annotation family	Internal mechanism	Senior follow-up and failure case
`@Transactional`	Transaction advisor/interceptor and a selected transaction manager, normally through a proxy.	Self-invocation bypasses default proxy advice; checked exceptions do not roll back by default. Trace REQUIRED versus REQUIRES_NEW, rollback-only state and pool demand (H3/H5 in Part 16).
`@Async`	Async advisor dispatches an intercepted call to an executor.	A transaction/thread-local does not automatically cross threads. Bound queues, define rejection handling and observe failures, including void-returning methods.
`@Cacheable`, `@CachePut`, `@CacheEvict`	Cache advice computes a key and interacts with the configured cache manager.	Define tenant-aware keys and invalidation. Cache and transaction advisor order matters; an ordinary cache mutation is not automatically atomic with database commit.
`@Scheduled`	`ScheduledAnnotationBeanPostProcessor` discovers methods and registers tasks with scheduling infrastructure.	Each application instance can register the task. A scheduled method may still use other advice, but scheduling alone supplies no cluster-wide ownership or exactly-once guarantee. Test overlap, shutdown and retry behavior.

Senior answer: proxy-based AOP uses JDK interface proxies or subclass proxies depending on configuration. Subclass proxies cannot advise final methods; private methods cannot be advised through an external proxy call. Self-invocation calls the target directly. Moving an operation to a separate injected bean or using a suitable programmatic API makes the boundary explicit. Do not assume advisor ordering gives a particular transaction/cache/async composition; inspect and test the actual invocation path.

S7 How do web annotations become request handling

Foundation answer: `@RestController` combines controller discovery with response-body semantics. `@RequestMapping` and HTTP-specific mappings define routes and request conditions. `@PathVariable`/`@RequestParam` resolve inputs; `@RequestBody` uses message conversion. `@Valid` requests configured validation; `@ExceptionHandler` and `@RestControllerAdvice` provide exception-to-response handling.

Internal answer: `RequestMappingHandlerMapping` registers mappings; DispatcherServlet selects a handler; handler adapters resolve arguments, invoke the method and process the result through converters. These annotations are not all implemented by AOP proxies. `@Valid` is not an authorization rule, and MVC argument validation is distinct from service method validation through `@Validated` and its configured interceptor.

Senior follow-up: test missing/invalid/null input, unexpected fields, 400/406/415 outcomes and safe exception responses. Avoid returning persistence entities that trigger lazy loads during serialization; use deliberate DTOs. Q178 covers negotiation and the managed Jackson configuration.

S8 Which persistence and test annotations are commonly confused

Foundation answer: `@Entity` and relationship annotations belong to JPA/Hibernate; `@PersistenceContext` requests an EntityManager through persistence integration. Spring Data `@Query` supplies a query and `@Modifying` marks modifying query execution; neither alone guarantees a complete service transaction. See Part 16 for mapping annotations and lifecycle follow-ups.

Internal answer: Spring Data creates repository proxies rather than implementation classes from component scanning alone. Entity mapping, repository registration and transaction advice are separate stages. Inherited CRUD methods have defaults different from user-declared query methods (Q179).

Senior follow-up: `@SpringBootTest` loads broad application configuration; `@WebMvcTest` and `@DataJpaTest` constrain the test slice. A rollback-based persistence test can miss flush-time constraints or commit-only behavior. Flush where the assertion requires SQL, and use separate committed transactions where the contract concerns visibility or post-commit effects. Mocked repositories do not prove database behavior. Q182 explains context-cache cost.

S9 How should a senior candidate demonstrate the internals

Walk one request through controller mapping → injected service → transaction proxy → transaction manager → repository proxy → EntityManager/Session → JDBC/pool → database → flush/commit → DTO serialization. State which steps are startup registration and which run per request; where a connection is borrowed; and who closes owned resources.

Then challenge the path: a self-call, two databases, pool exhaustion, detached lazy state, duplicate concurrent writers, an async hop, and a failure near commit. For each, identify the boundary, predict the observable result and propose a targeted integration test. Naming internals without explaining their consequences is incomplete.

Primary references: Boot 3.5 SQL configuration, Boot 3.5 auto-configuration development, Spring annotation processing, Spring proxy mechanics, and declarative transactions. These are explanations and proposed integration checks; no live Spring/database verification is claimed.

Spring Boot internals

Q174 What happens inside SpringApplication.run

Strong answer: Boot creates `SpringApplicationRunListeners`, prepares the `Environment` (property sources, profiles), creates the `ApplicationContext` type chosen from the deduced web application type, applies initializers, loads bean definitions, then calls `refresh()`. During refresh, configuration classes are processed (component scanning, auto-configuration imports), singleton beans are created, and for web applications the embedded server is created and started. After refresh, `ApplicationRunner` and `CommandLineRunner` beans execute, and `ApplicationReadyEvent` is published, at which point the readiness state becomes accepting traffic.

Reasoning and alternatives: knowing the order tells you where to hook in: `EnvironmentPostProcessor` for property changes before the context exists, `ApplicationContextInitializer` for early context customization, `@PostConstruct` for per-bean setup, runners for post-start work, and `ApplicationReadyEvent` for "now safe to serve".

What if/test: a runner blocks readiness until it returns. Move optional work to a bounded background executor, but keep traffic refused until mandatory initialization is complete. Use a full SpringApplication startup test for runner/event/readiness order; a context runner or slice alone does not exercise that whole lifecycle. **Signal:** place each extension point correctly in the sequence.

Q175 How does Boot decide it is a web application and which server to start

Strong answer: `WebApplicationType` is deduced from the classpath: if Spring MVC's `DispatcherServlet` and servlet classes are present it is a servlet application; if only WebFlux's `DispatcherHandler` is present it is reactive; otherwise it is a plain application. When both MVC and WebFlux are present, MVC wins. The embedded server comes from the starter on the classpath: `spring-boot-starter-web` brings Tomcat, and you swap it by excluding Tomcat and adding the Jetty or Undertow starter.

Reasoning and alternatives: override the deduction with `SpringApplication.setWebApplicationType` or `spring.main.web-application-type`. Auto-configuration then creates a `ServletWebServerFactory` bean and applies `server.*` properties through customizers.

What if/test: adding a reactive library to an MVC project can change behavior unexpectedly, so inspect the conditions report (`--debug`) when startup differs from expectation. **Signal:** explain that classpath contents drive both the app type and the server.

Q176 BeanFactoryPostProcessor versus BeanPostProcessor

Strong answer: a `BeanFactoryPostProcessor` runs after bean *definitions* are loaded and before any ordinary bean is instantiated, so it can modify definitions (`ConfigurationClassPostProcessor` handles `@Configuration`, `@Bean` and `@ComponentScan` this way; `PropertySourcesPlaceholderConfigurer` resolves `${...}`). A `BeanPostProcessor` runs around the initialization of each bean *instance*, so it can wrap or replace it (autowiring annotations are processed this way, and AOP auto-proxy creation wraps beans in proxies).

Reasoning and alternatives: use the first to change configuration metadata and the second to decorate objects. Declare a `BeanFactoryPostProcessor` `@Bean` method as `static` so the configuration class is not instantiated too early.

What if/test: beans created before all post-processors are registered are not processed by them; Spring logs an "not eligible for getting processed by all BeanPostProcessors" message. Search the startup log for it when a proxy or annotation seems ignored. **Signal:** distinguish definition-time from instance-time hooks and cite one real example of each.

Q177 What does @Configuration(proxyBeanMethods = false) change

Strong answer: by default (`true`, "full" mode) Spring subclasses the configuration class with CGLIB so that calling one `@Bean` method from another returns the container's singleton instead of creating a new object. With `proxyBeanMethods = false` ("lite" mode) there is no subclass, startup is cheaper, and a direct call to another `@Bean` method is an ordinary Java call; it creates a fresh instance if that method constructs one, without container interception of the call.

Reasoning and alternatives: Boot's own auto-configurations use lite mode for speed and native-image friendliness. In lite mode, inject dependencies as method parameters instead of calling other `@Bean` methods. Keep full mode only if you rely on inter-bean method calls.

What if/test: switching a legacy configuration to lite mode can silently duplicate singletons. Add a test asserting the same instance is injected in both places. **Signal:** state the trade-off between convenience and startup cost.

Q178 How do content negotiation and Jackson configuration work in Spring MVC

Strong answer: the `Accept` header, the controller's `produces` and `consumes` conditions, and the registered `HttpMessageConverter`s together decide the representation. An unsupported `Accept` gives `406 Not Acceptable`; an unsupported request `Content-Type` gives `415 Unsupported Media Type`. Boot auto-configures one shared `ObjectMapper` via `spring.jackson.*` properties and registers modules found on the classpath (for example Java time); by default Boot disables failing on unknown properties and writing dates as timestamps.

Reasoning and alternatives: customize through properties or a `Jackson2ObjectMapperBuilderCustomizer` bean, not by constructing a separate `new ObjectMapper()` that ignores the shared configuration. For API evolution, being tolerant of unknown fields on input helps backward compatibility, while strict validation protects security-sensitive endpoints.

What if/test: a hand-built `ObjectMapper` used in tests may differ from production behavior; inject the bean or use `@JsonTest`. Test one request each for 406 and 415. **Signal:** know the two status codes and why you inject the managed mapper.

Q179 How does Spring Data JPA implement a repository interface you only declared

Source basis: Spring Data repository initialization.

Strong answer: at startup, repository scanning registers a factory bean for each interface. The factory creates a **JDK dynamic proxy** that delegates CRUD methods to `SimpleJpaRepository` and routes query methods to query lookups: an `@Query`, a named query, or a query **derived from the method name** by parsing it into parts. Query derivation is validated when the repository initializes: normally during startup, but lazy/deferred bootstrap modes change the timing. A misspelled property can therefore fail startup or later initialization.

Reasoning and alternatives: Inherited CRUD methods implemented by `SimpleJpaRepository` have transaction defaults (read-only at class level with write methods transactional). User-declared query methods, including default methods, have no transaction configuration by default. Without a service transaction, separate inherited CRUD calls normally run in separate transactions; do not assume a custom query participates or that a multi-call unit is atomic. Annotate declared query methods or, commonly, the service operation. Custom behavior can go in a repository fragment interface plus implementation class.

What if/test: assuming two repository calls share a transaction without a service-level `@Transactional` is a common bug. Write a test that fails the second call and asserts whether the first persisted. **Signal:** explain proxy creation, startup validation and default transaction behavior.

Q180 How does the embedded Tomcat threading model affect tuning

Strong answer: the NIO connector accepts connections and hands request processing to a worker pool. Boot's defaults are roughly 200 maximum worker threads, 100 for the accept queue (`accept-count`) and 8192 maximum connections. With supported Boot 3.x and Java 21+, `spring.threads.virtual.enabled=true` can switch request execution to virtual threads. This removes the usual fixed platform-worker pool limit for that path, not the need for admission limits, connection limits or downstream capacity controls. Verify exact defaults against the chosen Boot/Tomcat version.

Reasoning and alternatives: more threads do not fix slow downstream calls; they move the bottleneck to the database pool or the dependency. Size worker threads and connection pools together, set timeouts, and use bulkheads or a `Semaphore` to bound calls to scarce resources, especially with virtual threads.

What if/test: raising `threads.max` without raising the connection pool just queues requests inside the app. Load test and watch pool wait time, active threads and p99 latency together. **Signal:** tune the whole path (threads, pool, downstream), not one number.

Q181 How do Actuator health groups, liveness and readiness work

Strong answer: the health endpoint aggregates `HealthIndicator` beans. **Liveness** and **readiness** are availability states exposed as health groups and driven by `ApplicationAvailability` and `AvailabilityChangeEvent`; on Kubernetes the probe groups are enabled automatically. Readiness flips to refusing traffic during shutdown, which lets the platform drain the instance.

Reasoning and alternatives: keep liveness limited to "the process is irrecoverably broken", because including a database or downstream check there can cause restart storms when the dependency fails. Readiness may include critical dependencies cautiously. Custom indicators must be fast and time-bounded.

What if/test: a health check that calls a slow dependency can itself overload it under frequent probing; cache results or use a short timeout. Test by stopping the database and observing the probe states. **Signal:** explain why liveness must not depend on downstream services.

Q182 Why are Spring test suites slow, and what is context caching

Strong answer: the test framework caches an application context and reuses it for tests with the **same configuration key** (configuration classes, properties, active profiles, mock-bean definitions and so on). Every distinct combination starts a new context, so scattering different `@MockBean`/`@MockitoBean` sets or `@DirtiesContext` across classes multiplies startup cost.

Reasoning and alternatives: prefer plain unit tests for logic, slice tests (`@WebMvcTest`, `@DataJpaTest`) for one layer, and few full `@SpringBootTest` classes sharing a common base configuration. Recent Spring versions replace `@MockBean` with `@MockitoBean`; check the deprecation status for your version.

What if/test: measure by counting context startups in the test log. Reduce unique configurations, avoid `@DirtiesContext`, and reuse Testcontainers through a shared static container. **Signal:** connect suite speed to the number of distinct context configurations.

Coding-style questions

Q183 Implement a thread-safe TTL cache with a loader

Strong answer: use a `ConcurrentHashMap` and `compute` to serialize lookup/expiry/load for a key. Waiting callers reuse a successfully loaded value while it remains unexpired; load failures or a very short TTL can cause another load. This is a limited teaching sketch, not a complete production cache.

Explain before coding: Trace a miss: hold the per-key compute boundary, load the value, then start its TTL after loading. A later lookup checks expiry before reusing it. Explain the cost: a slow loader holds map synchronization, and untouched expired entries remain retained.

```
class TtlCache<K, V> {
    private record Entry<V>(V value, long expiresAt) {}
    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
    private final long ttlNanos;
    TtlCache(Duration ttl) {
        Objects.requireNonNull(ttl, "ttl");
        if (ttl.isZero() || ttl.isNegative()) throw new IllegalArgumentException("ttl must be
positive");
        this.ttlNanos = ttl.toNanos();
        if (ttlNanos <= 0) throw new IllegalArgumentException("ttl outside supported range");
    }

    V get(K key, Function<K, V> loader) {
        Objects.requireNonNull(key, "key");
        Objects.requireNonNull(loader, "loader");
        return map.compute(key, (k, old) -> {
            long now = System.nanoTime(); // sampled after acquiring this key's compute lock
            if (old != null && now - old.expiresAt() < 0) return old;
            V value = Objects.requireNonNull(loader.apply(k), "loader returned null");
            long expiresAt = System.nanoTime() + ttlNanos; // expiry starts after loading
            return new Entry<>(value, expiresAt);
        }).value();
    }
}
```

Reasoning and alternatives: Use `System.nanoTime()` for elapsed time. Subtraction tolerates wraparound only within the supported signed-difference interval (less than 2^63 nanoseconds); expiry comparisons require that bound for the interval being compared. See the Java 21 elapsed-time contract. This version evicts lazily and never removes entries that are not read again, so for production use Caffeine (size and time bounds, async loading, statistics).

What if/test: the loader runs while the map holds a bin lock, so it must be short and must not recursively update the same key. Test a slow loader, contention across an expiry boundary and concurrent callers. This teaching cache still evicts lazily, has no size bound, and may block other keys sharing a bin; use a mature cache for production. **Signal:** mention single-flight loading, monotonic time and the missing eviction.

Q184 Implement a minimal fixed thread pool

Explain before coding: A fixed pool separates submission from execution: producers hand tasks to a bounded queue and workers repeatedly take them. The sketch below illustrates that mechanism but has an acknowledged submit-versus-close race and is not a fully correct shutdown implementation. In an interview, identify that missing atomic admission/shutdown boundary before claiming accepted work will finish.

```
class MiniPool implements AutoCloseable {
    private final BlockingQueue<Runnable> queue = new LinkedBlockingQueue<>(100);
    private final List<Thread> workers = new ArrayList<>();
    private volatile boolean stopped;
```

```
    MiniPool(int n) {
        for (int i = 0; i < n; i++) {
            Thread t = new Thread(this::loop, "mini-" + i);
            workers.add(t); t.start();
        }
    }
    void submit(Runnable task) throws InterruptedException {
        if (stopped) throw new RejectedExecutionException("pool closed");
        queue.put(task);                       // bounded queue gives back-pressure
    }
    private void loop() {
        try {
            while (!stopped || !queue.isEmpty()) {
                Runnable r = queue.poll(100, TimeUnit.MILLISECONDS);
                if (r != null) { try { r.run(); } catch (Throwable ignored) { /* log in real code
*/ } }
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
    @Override public void close() throws InterruptedException {
        stopped = true;
        for (Thread t : workers) t.join();
    }
}
```

Reasoning and alternatives: it shows the core of `ThreadPoolExecutor`: worker threads draining a queue, a bounded queue, and graceful shutdown. Missing features: dynamic sizing, a rejection policy, `Future` results and thread factories.

What if/test: a task submitted while `close()` begins can slip through, and concurrent close/submit can leave work queued after workers exit. A production pool must serialize submission with shutdown, define rejection and interruption behavior, and ensure every accepted task is completed or explicitly cancelled. Test the race with latches; ordinary draining tests do not prove it safe. **Signal:** explain the worker loop and why the queue is bounded.

Q185 Write a Spring filter that adds a correlation ID to the logging context

Explain before coding: The request thread first validates or generates an ID, stores it in the logging context, runs the downstream chain and removes it in finally. The cleanup must also run if downstream code throws. This sketch assumes it owns that MDC entry; nested context ownership may require restoring a previous value instead.

```
@Component
class CorrelationIdFilter extends OncePerRequestFilter {
    private static final Pattern SAFE = Pattern.compile("[A-Za-z0-9._-]{1,64}");

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain)
            throws ServletException, IOException {
        String incoming = req.getHeader("X-Correlation-Id");
        String id = (incoming != null && SAFE.matcher(incoming).matches()) ? incoming :
UUID.randomUUID().toString();
        MDC.put("correlationId", id);
        res.setHeader("X-Correlation-Id", id);
        try { chain.doFilter(req, res); }
        finally { MDC.remove("correlationId"); }   // pooled threads must not leak context
    }
}
```

Reasoning and alternatives: validating the inbound value prevents log injection. Include `%X{correlationId}` in the log pattern or use JSON logging. With Micrometer Tracing, trace and span IDs may already be placed in the MDC, so reuse them rather than inventing a second ID.

What if/test: MDC does not propagate to new threads automatically; wrap executors with a `TaskDecorator` that copies it. Test with `MockMvc` asserting the response header and that the MDC is empty afterwards. **Signal:** clear the context in `finally` and sanitize the header.

Q186 Merge K sorted linked lists

Explain before coding: For lists [1,4] and [2,3], put 1 and 2 in the heap, emit 1 and offer its successor 4, then emit 2 and offer 3. Only the smallest remaining head can be next. This version relinks existing nodes and assumes a non-null array of sorted, acyclic, disjoint lists; shared nodes require a different contract.

```
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.val));
    for (ListNode head : lists) if (head != null) pq.add(head);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll();
        tail.next = node; tail = node;
        if (node.next != null) pq.add(node.next);
    }
    return dummy.next;
}
```

Reasoning and alternatives: the heap holds at most one node per list, giving O(N log k) time and O(k) extra space for N total nodes. Alternative: merge pairwise in a divide-and-conquer fashion, also O(N log k) with O(1) extra space for linked lists (excluding recursion).

What if/test: test empty input, all-empty lists, one list, and duplicate values across lists. **Signal:** give both complexities and explain why the heap stays size k.

Q187 Build a mini dependency-injection container

Explain before coding: Resolve A by resolving its constructor parameters first. The creating set represents the active dependency path: seeing A again before it is complete detects a cycle. Cache only successful instances. The sketch assumes one constructor per concrete class and single-threaded access.

```
class MiniContainer {
    private final Map<Class<?>, Object> singletons = new HashMap<>();
    private final Set<Class<?>> creating = new HashSet<>();

    @SuppressWarnings("unchecked")
    <T> T get(Class<T> type) {
        Object existing = singletons.get(type);
        if (existing != null) return (T) existing;
        if (!creating.add(type)) throw new IllegalStateException("Circular dependency: " +
type.getName());
        try {
            Constructor<?> ctor = type.getDeclaredConstructors()[0];      // assume one constructor
            Object[] args = new Object[ctor.getParameterCount()];
            Class<?>[] params = ctor.getParameterTypes();
            for (int i = 0; i < args.length; i++) args[i] = get(params[i]);
            ctor.setAccessible(true);
            Object instance = ctor.newInstance(args);
            singletons.put(type, instance);
            return (T) instance;
        } catch (ReflectiveOperationException e) {
            throw new IllegalStateException(e);
        } finally {
            creating.remove(type);
        }
    }
}
```

Reasoning and alternatives: this demonstrates constructor injection, singleton scope, and cycle detection, which is exactly why constructor cycles fail in Spring. Real containers add interface-to-implementation binding, scanning, scopes, lifecycle callbacks and thread safety.

What if/test: the sketch is not thread-safe and handles only concrete classes. Test a chain A→B→C and a cycle A→B→A. **Signal:** relate each simplification to the matching Spring feature.

Leadership

Q188 How do you run a design review and write an RFC or ADR

Strong answer: write a short document stating the problem, goals and non-goals, constraints, options considered with trade-offs, the recommendation, risks, a rollout and rollback plan, and open questions. Share it early with the people affected, set a review deadline, and record the decision with the reasoning (an Architecture Decision Record) and a date to revisit.

Reasoning and alternatives: the aim is a better decision, not approval of your preferred one: invite objections, use data or a small prototype to settle disputes, and separate reversible decisions (decide fast) from one-way doors (review thoroughly).

What if: the review becomes opinion battles; fix this by agreeing criteria first (latency, cost, delivery time, operability) and scoring options against them. **Signal:** show a written process, decision criteria and a record of the outcome.

Q189 How do you handle a team member who is underperforming

Strong answer: start with a private, specific conversation about observed outcomes, not personality. Clarify expectations, ask about blockers (skills, context, workload, personal issues), agree on measurable goals and a short timeline, and check in frequently with support such as pairing, smaller tasks or a mentor. Involve your manager or HR early if the pattern continues.

Reasoning and alternatives: many cases are unclear expectations or poor fit with the task rather than lack of ability. Document the discussions factually and keep the rest of the team's workload fair in the meantime.

What if: performance does not improve despite support; then follow the formal process promptly and respectfully, because delaying harms both the person and the team. **Signal:** be specific, empathetic and accountable, with a concrete example and outcome.

Q190 How do you prioritize technical debt against product features

Strong answer: quantify debt in business terms: incident count, time lost per release, onboarding time, lead time, or risk exposure (a vulnerable library). Rank by impact and cost to fix, tie work to roadmap goals ("this refactor removes the migration blocker for feature X"), and agree on a standing allocation (for example 15–20% of capacity) plus targeted projects for large items.

Reasoning and alternatives: the strongest framing is a risk and cost-of-delay conversation with product, not "the code is ugly". Pay debt incrementally alongside feature work where possible.

What if: product refuses any time for debt; then propose the smallest high-impact fix, measure the improvement and use the result to justify more. **Signal:** use metrics and negotiate trade-offs instead of demanding rewrites.

Q191 How do you drive a change across teams when you have no authority

Strong answer: build a case with evidence, find allies and early adopters, make adoption easy (templates, libraries, migration guides, automation), start with a pilot, publish the results, and ask leaders to remove blockers rather than mandate. Keep feedback loops open so the solution adapts to real needs.

Reasoning and alternatives: people adopt what saves them effort and what they helped shape. Mandates without support create resentment and shallow compliance.

What if: teams still resist; find out their constraints, adjust the approach, and escalate only for genuine risk (security, compliance) with clear data. **Signal:** describe influence through evidence, enablement and empathy, with a real adoption result.

Q192 How do you estimate work and communicate delays

Strong answer: estimate in ranges with stated assumptions, break work into small deliverables, separate known from unknown (add spikes for unknowns), and add buffer for integration and testing. Track progress against milestones. When a delay appears, tell stakeholders **early**, explain the cause and impact, and present options: reduce scope, move the date, or add resources, with your recommendation.

Reasoning and alternatives: a single-point estimate hides uncertainty; ranges and confidence levels make planning honest. Reduce risk by delivering the riskiest part first.

What if: pressure to commit to an unrealistic date; present the trade-off explicitly (scope, time, quality) rather than silently absorbing risk. **Signal:** early, transparent communication with options, not excuses.

Q193 How do you decide between building and buying a solution

Strong answer: compare **total cost of ownership** (licenses or usage fees versus engineering, operations and maintenance), time to value, fit to requirements, integration effort, security and compliance, vendor risk and lock-in, and whether the capability is a **differentiator** for the business. Build what differentiates the product; buy or adopt open source for commodity capabilities.

Reasoning and alternatives: run a time-boxed proof of concept against the hardest requirements, check exit strategy and data portability, and review support and roadmap. A hybrid (buy the platform, build thin extensions) is common.

What if: a vendor's pricing or roadmap changes later; keep an abstraction at the integration boundary and revisit the decision periodically. **Signal:** structure the decision with criteria and name the differentiator test.

Part 14 - System Design, Data Internals, JVM and Coding Questions (Q194-Q213)

Twenty more questions in the same layout as Parts 10, 12 and 13. Q194–Q199 are condensed system designs, Q200–Q202 cover Kafka and database internals, Q203–Q206 cover Java I/O and JVM topics, and Q207–Q213 are coding problems with solutions. Where no source is linked, the basis is general engineering practice, so confirm specifics against the documentation for your version.

Condensed system designs

Use the requirements-first design sequence before the examples: requirements → estimates → API/data → architecture → failure/scaling analysis → trade-offs. These prompts assume that interview process.

Q194 Design a distributed cache

Strong answer: partition keys across nodes with consistent hashing or fixed hash slots, replicate each partition to a replica for failover, and bound memory with an eviction policy (LRU, LFU or TTL). Clients route requests directly or through a proxy. Use cache-aside with a TTL so the database stays the source of truth.

Reasoning and alternatives: asynchronous replication means a failover can lose the most recent writes or serve stale reads; synchronous replication costs latency. Handle **hot keys** with a small local near-cache or by replicating the key, and **stampedes** with single-flight loading, jittered TTLs or early refresh.

What if/test: a node failure triggers a cold-cache load spike on the database, so rate-limit refills and test by killing a node under load. Track hit rate, evictions and p99 latency. **Signal:** cover partitioning, replication trade-offs, hot keys and stampedes, not just "use Redis".

Q195 Design a news feed or timeline

Strong answer: there are two models. **Fan-out on write** pushes each new post ID into every follower's precomputed timeline (fast reads, expensive writes, bad for accounts with millions of followers). **Fan-out on read** merges followed accounts' posts at request time (cheap writes, slower reads). A hybrid pushes for ordinary accounts and pulls for high-follower accounts, merging at read time.

Reasoning and alternatives: store posts in a durable database, and timelines as capped lists of post IDs in a cache, hydrating post content on read. Use cursor-based pagination, and apply ranking as a separate step if the feed is not purely chronological.

What if/test: deleting or editing a post must propagate to timelines (lazy filtering on read is simplest). Test a celebrity posting during peak and measure write amplification. **Signal:** name the celebrity problem and the hybrid solution.

Q196 Design a webhook delivery system

Strong answer: persist each event, then deliver it with worker processes that POST to the subscriber's URL. Sign the payload (HMAC with a per-endpoint secret plus a timestamp to prevent replay), include a unique event ID so receivers can deduplicate, use short timeouts, and retry with exponential backoff for a bounded period. Deliveries are at-least-once and unordered unless you build per-endpoint ordering.

Reasoning and alternatives: isolate endpoints (a queue or rate limit per endpoint) so one slow subscriber cannot starve others. Disable an endpoint after sustained failure, send failed events to a dead-letter store, and provide a replay tool. Protect against SSRF with an outbound egress policy and destination validation: allow only

approved schemes/hosts where possible; resolve and reject private, loopback, link-local and metadata addresses; revalidate every redirect and DNS resolution; and account for IPv4/IPv6 and DNS rebinding. A one-time hostname check or redirect following can bypass a denylist.

What if/test: a receiver that returns 200 before processing may lose events, so document that they should acknowledge only after durable receipt. Test with a flaky receiver and verify retry timing and deduplication. **Signal:** mention signatures, idempotency, isolation and SSRF.

Q197 Design a file upload and storage service

Strong answer: prefer direct-to-object-storage uploads when the security and processing requirements allow them; controlled application streaming is an alternative when inline inspection or another constraint requires it. The client requests a signed upload URL, then uploads directly to object storage, using resumable or multipart upload for large files. Store metadata (owner, size, content hash, status) in a database with states such as PENDING → READY. An object-finalized event triggers asynchronous processing (virus scan, thumbnails, indexing). Keep the object quarantined and unavailable for download until required validation and scanning finish; only then transition metadata to READY. Serve READY downloads through signed URLs or a CDN.

Reasoning and alternatives: deduplicate by content hash if storage cost matters, and apply lifecycle policies for expiry and tiering. Keep the object store and the metadata consistent with a reaper job that cleans abandoned uploads.

What if/test: a client may upload but never confirm, or confirm without uploading; validate against the object store before marking READY. Test interrupted and resumed uploads. **Signal:** keep large data off the app tier and handle the two-system consistency gap.

Q198 Design search autocomplete

Strong answer: serve suggestions from a precomputed structure: a trie or prefix index where each prefix maps to its top-k completions, built offline or by streaming aggregation from query logs and refreshed periodically. Cache results in memory and at the edge, since latency must be very low.

Reasoning and alternatives: store the top-k at each node (more memory, faster lookup) instead of traversing the subtree at query time. Debounce on the client, shard by prefix for scale, add typo tolerance and personalization separately, and filter abusive or sensitive terms.

What if/test: trending queries need a fast update path beside the batch rebuild. Test p99 latency with realistic prefix distributions. **Signal:** precompute top-k per prefix and justify the memory trade-off.

Q199 Design a leaderboard

Strong answer: use a Redis sorted set: `ZADD`/`ZINCRBY` update a member in O(log n); rank lookup is O(log n), while returning N members is O(log n + N). Modern Redis can use `ZRANGE ... REV`. Equal scores already tie-break lexicographically by member bytes. If packing a tie-breaker into scores, bound and prove the encoding preserves order and exactness: Redis scores are doubles, with exact integer representation only through ±2^53. Keep separate keys with TTLs for daily and weekly windows.

Reasoning and alternatives: a single sorted set is bounded by one node's memory and throughput. To scale, shard users and merge each shard's top N (exact for top N), or accept approximate ranks for far-down positions. Treat the sorted set as rebuildable from the event stream.

What if/test: concurrent updates are atomic per command, but rank queries during bursts can reflect in-flight changes. Test with simulated score bursts and verify rebuild from events. **Signal:** name the data structure and the scale-out path.

Kafka and database internals

Q200 Why is Kafka fast, and how do log segments, retention and compaction work

Source basis: Kafka design documentation.

Strong answer: each partition is an append-only log stored as segment files. Writes are sequential, which disks handle efficiently, and Kafka relies on the operating system page cache, batching, compression and zero-copy transfer (`sendfile`) to serve consumers. Retention deletes whole old segments by time or size. **Log compaction** instead keeps the latest record per key, and a record with a null value (tombstone) deletes a key after a delay.

Reasoning and alternatives: consumers pull by offset, so replay is just resetting the offset. Compaction suits changelogs and current-state topics; time retention suits event streams.

What if/test: compaction does not guarantee immediate removal of older values, and consumers can still see superseded records until cleaning runs. Test tombstone behavior and retention settings explicitly. **Signal:** explain performance through sequential IO and the page cache, not just "it's distributed".

Q201 What are ISR, acks and min.insync.replicas, and how can data be lost

Strong answer: the in-sync replica set (ISR) contains replicas caught up with the leader. With `acks=all` the leader waits for all current ISR members; `min.insync.replicas` makes the broker reject writes when the ISR is smaller than the threshold. A common setting is replication factor 3 with `min.insync.replicas=2`, tolerating one broker failure without losing acknowledged writes.

Reasoning and alternatives: with `acks=1` a leader crash after acknowledging but before replication loses the write. Allowing **unclean leader election** (a non-ISR replica becoming leader) trades data loss for availability, so it is normally disabled. Enable producer idempotence and retries to avoid duplicates and transient loss.

What if/test: `acks=all` with `min.insync.replicas=1` gives weaker protection than people assume. Test by stopping brokers and checking producer errors and acknowledged-write survival. **Signal:** show how the three settings combine, not each in isolation.

Q202 Which PostgreSQL index types do you know and when do you use each

Source basis: PostgreSQL index types.

Strong answer: B-tree (default) handles equality, ranges and ordering. A **partial index** (`WHERE status = 'ACTIVE'`) indexes only the rows queries need. **GIN** suits `jsonb`, arrays and full-text search. A **covering index** (`INCLUDE`) allows index-only scans. **BRIN** is a tiny index for very large tables whose values correlate with physical order (for example append-only timestamps). Expression indexes support queries on computed values such as `lower(email)`.

Reasoning and alternatives: every index slows writes and uses space, so base them on real queries and drop unused ones. Build on live tables with `CREATE INDEX CONCURRENTLY` to avoid blocking writes.

What if/test: index-only scans depend on the visibility map, so a table that is not vacuumed enough falls back to heap reads. Confirm with `EXPLAIN (ANALYZE, BUFFERS)`. **Signal:** match the index type to the access pattern and mention the write cost.

Java I/O and JVM

Q203 How do you process a very large CSV file in Java

Strong answer: stream it instead of loading it. Use a streaming CSV parser over a `Reader`; physical lines are not necessarily records because quoted fields may contain line breaks. Parse logical records, validate and process in **batches** (for example 1,000 rows per JDBC batch or a database bulk-load path), keeping memory bounded. Record progress so a failed run can resume.

Reasoning and alternatives: `Files.readAllLines` loads everything and fails on large files. For restartable, chunked processing with skip and retry policies use Spring Batch. Use a bounded queue between a reader thread and worker threads for parallelism with back-pressure.

What if/test: a single bad row should not abort a run silently; route rejects to an error file. Test with a file larger than the heap, quoted delimiters and multiline fields, malformed rows, and restart points at logical-record boundaries. **Signal:** bounded memory, batching and restartability.

Q204 How do Java NIO and selectors differ from classic blocking I/O

Strong answer: classic I/O uses one blocked thread per connection. NIO uses **channels**, **buffers** and a **selector** so a small number of threads can monitor many connections and handle only those that are ready. Netty and Tomcat's NIO connector build event loops on this model. `FileChannel.transferTo` can move file data to a socket without copying through user space (zero-copy).

Reasoning and alternatives: NIO scales connection count but code is more complex. On Java 21, virtual threads let you write simple blocking code that scales well for I/O-bound services. Direct `ByteBuffer`s live off-heap, reduce copying, but are costly to allocate and need monitoring.

What if/test: blocking work inside an event-loop thread stalls many connections. Test by adding a slow call and observing latency across unrelated requests. **Signal:** explain readiness multiplexing and the event-loop pitfall.

Q205 What are G1 humongous allocations and how do you spot them

Strong answer: in G1, an object at least half the size of a heap region is **humongous** and is allocated directly in contiguous old-generation regions, bypassing the young generation. Frequent humongous allocations can cause fragmentation, extra GC work and surprising full collections.

Reasoning and alternatives: spot them in GC logs (`-Xlog:gc*`) where humongous allocation is reported. Fixes: avoid very large arrays and buffers (process in chunks or stream), or increase the region size with `-XX:G1HeapRegionSize`. Fixing the allocation pattern is better than only tuning flags.

What if/test: a service that reads large payloads fully into a `byte[]` is a typical cause. Reproduce under load, compare GC logs before and after chunking. **Signal:** connect a symptom in GC logs to a code-level cause.

Q206 List.of versus unmodifiableList versus List.copyOf versus Arrays.asList

Strong answer: `List.of` creates a structurally unmodifiable list and rejects nulls. `Collections.unmodifiableList` is a read-only **view**: changes to the underlying list remain visible through it. `List.copyOf` makes a shallow, structurally unmodifiable copy; callers must not rely on object identity. `Arrays.asList` is fixed-size and writes through to the backing array. `Stream.toList()` returns an unmodifiable list that allows nulls.

Reasoning and alternatives: for safe publication of collection structure, return an unmodifiable snapshot or defensive copy. If the object graph must also be immutable, its elements must be immutable or deep-copied. The unmodifiable variants reject structural/content replacement. Arrays.asList permits set and replacement/reordering

operations that fit its fixed size, but cannot grow or shrink. Do not equate fixed-size with unmodifiable.

What if/test: returning `unmodifiableList(internalList)` and later mutating `internalList` surprises callers who iterate concurrently. Test by mutating the source after creating each variant. **Signal:** distinguish a view from a copy.

Coding problems

Q207 Longest consecutive sequence in O(n)

Explain before coding: Build a set, then start counting only at values whose predecessor is absent. For [100,4,200,1,3,2], only 1 starts the run 1–4; starting again at 2, 3 or 4 would repeat work. Handle integer endpoints so predecessor/successor arithmetic cannot wrap into a false neighbor.

```
int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);
    int best = 0;
    for (int n : set) {
        if (n == Integer.MIN_VALUE || !set.contains(n - 1)) { // avoid predecessor wraparound
            int len = 1;
            while ((long) n + len <= Integer.MAX_VALUE
                    && set.contains((int) ((long) n + len))) len++;
            best = Math.max(best, len);
        }
    }
    return best;
}
```

Reasoning and alternatives: each element is visited at most twice, so the time is O(n) with O(n) space. Sorting is simpler but O(n log n).

What if/test: test empty input, duplicates, negatives, one long run and both integer endpoints together. **Signal:** explain why starting only at sequence starts keeps it linear.

Q208 Product of array except self (no division)

Explain before coding: At each index, multiply the product strictly to its left by the product strictly to its right. For [1,2,3,4], the result is [24,12,8,6]. Prefix work fills the output; a backward suffix pass completes it without division, so zeros need no special division case. State the numeric range assumption.

```
int[] productExceptSelf(int[] a) {
    int n = a.length;
    int[] out = new int[n];
    if (n == 0) return out;
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];   // product of everything to the
left
    int right = 1;
    for (int i = n - 1; i >= 0; i--) { out[i] *= right; right *= a[i]; }
    return out;
}
```

Reasoning and alternatives: O(n) time, O(1) extra space beyond the output. Division fails with zeros and is often disallowed.

What if/test: test zeros, negatives and overflow (ask whether to use `long`). **Signal:** prefix and suffix products, and the zero case.

Q209 Rotting oranges (multi-source BFS)

Explain before coding: Enqueue every initially rotten orange at distance zero. All infections produced by one queue layer occur during the same minute; starting separate searches would repeat work and complicate the earliest-arrival calculation. Track remaining fresh cells to distinguish completion from unreachable cells.

```
int orangesRotting(int[][] g) {
    Objects.requireNonNull(g, "grid");
    if (g.length == 0) return 0;
    Objects.requireNonNull(g[0], "grid row");
    int R = g.length, C = g[0].length;
    for (int[] row : g) {
        Objects.requireNonNull(row, "grid row");
        if (row.length != C) throw new IllegalArgumentException("grid must be rectangular");
        for (int cell : row) {
            if (cell < 0 || cell > 2) throw new IllegalArgumentException("grid cells must be 0, 1
or 2");
        }
    }
    int fresh = 0;
    Deque<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++) {
            if (g[r][c] == 2) q.add(new int[]{r, c});
            else if (g[r][c] == 1) fresh++;
        }
    int minutes = 0;
```

```
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!q.isEmpty() && fresh > 0) {
        for (int s = q.size(); s > 0; s--) {
            int[] p = q.poll();
            for (int[] d : dirs) {
                int nr = p[0] + d[0], nc = p[1] + d[1];
                if (nr >= 0 && nc >= 0 && nr < R && nc < C && g[nr][nc] == 1) {
                    g[nr][nc] = 2; fresh--; q.add(new int[]{nr, nc});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

Reasoning and alternatives: start BFS from all rotten cells together so each level equals one minute. O(R × C) time and space.

What if/test: test empty and zero-column grids, no fresh oranges (0), unreachable fresh orange (-1), one cell and malformed nonrectangular input. **Signal:** know why multi-source BFS gives the minimum time.

Q210 Trapping rain water (two pointers)

Explain before coding: Water above a bar needs boundaries on both sides and is limited by the smaller available boundary. Move the side whose boundary is currently limiting, accumulating its deficit and updating its maximum. For [2,0,2], the middle holds two units; monotonic heights hold none.

```
long trap(int[] h) {
    Objects.requireNonNull(h, "heights");
    for (int height : h) {
        if (height < 0) throw new IllegalArgumentException("heights must be nonnegative");
    }
    int l = 0, r = h.length - 1;
    long leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (h[l] < h[r]) { leftMax = Math.max(leftMax, (long) h[l]); water += leftMax - h[l]; l++;
}
        else             { rightMax = Math.max(rightMax, (long) h[r]); water += rightMax - h[r]; r-
-; }
    }
    return water;
}
```

Reasoning and alternatives: water at a position is bounded by the lower of the tallest bars on each side; the pointer with the smaller height is already limited by its own side's maximum. O(n) time, O(1) space. The total is accumulated as `long`; the alternative uses two precomputed max arrays.

What if/test: test empty, monotonic and valley shapes, including a valley bounded by `Integer.MAX_VALUE` heights. **Signal:** justify why the smaller side can be processed safely.

Q211 Minimum window substring (sliding window)

Explain before coding: Track the missing character multiplicities. Expanding the right boundary eventually makes a window valid; then move the left boundary until removing a needed occurrence would invalidate it. For s=ADOBECODEBANC and t=ABC, the minimum is BANC. Repeated target characters need counts, not a set. This array-based sketch assumes ASCII input because its count array has 128 entries.

```
String minWindow(String s, String t) {
    if (t.isEmpty() || s.length() < t.length()) return "";
    int[] need = new int[128];                    // assumes ASCII; use a map for full Unicode
    for (char c : t.toCharArray()) need[c]++;
    int missing = t.length(), left = 0, bestStart = 0, bestLen = Integer.MAX_VALUE;
    for (int r = 0; r < s.length(); r++) {
        if (need[s.charAt(r)]-- > 0) missing--;
        while (missing == 0) {
            if (r - left + 1 < bestLen) { bestLen = r - left + 1; bestStart = left; }
            if (++need[s.charAt(left++)] > 0) missing++;
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
}
```

Reasoning and alternatives: expand the right edge until all characters are covered, then shrink the left edge while still valid. Each index moves at most once, so O(n).

What if/test: test repeated characters in `t`, no valid window, and `t` longer than `s`. **Signal:** explain the `missing` counter and the shrink condition.

Q212 Lowest common ancestor of a binary tree

Explain before coding: Each recursive call reports whether its subtree contains a target or an already found ancestor. If the left reports one target and the right the other, the current node is their split point. If one target is itself an ancestor, return it. State the assumption that both target objects occur in the tree.

```
TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lca(root.left, p, q), right = lca(root.right, p, q);
    return (left != null && right != null) ? root : (left != null ? left : right);
}
```

Reasoning and alternatives: if both subtrees return a node, the current node is the split point. O(n) time, O(h) stack. For a binary search tree you can walk down using the ordering in O(h).

What if/test: the solution assumes both nodes exist; confirm that or add a verification pass. Test when one node is an ancestor of the other. **Signal:** state the assumption and give the BST shortcut.

Q213 Time-based key-value store

Explain before coding: For one key with values at times 3 and 8, a read at 6 returns the value at 3: the greatest stored timestamp no later than the request. A sorted map directly supports that floor operation. Per-map concurrent replacements alone do not define a cross-key snapshot contract.

```
class TimeMap {
    private final Map<String, TreeMap<Integer, String>> store = new HashMap<>();

    void set(String key, String value, int timestamp) {
        store.computeIfAbsent(key, k -> new TreeMap<>()).put(timestamp, value);
    }
    String get(String key, int timestamp) {
        TreeMap<Integer, String> versions = store.get(key);
        if (versions == null) return "";
        Map.Entry<Integer, String> e = versions.floorEntry(timestamp);   // latest version at or
before timestamp
        return e == null ? "" : e.getValue();
    }
}
```

Reasoning and alternatives: `floorEntry` is O(log n). If timestamps always increase, an `ArrayList` with binary search uses less memory.

What if/test: this class is not thread-safe; for concurrency use `ConcurrentHashMap` with `ConcurrentSkipListMap`. Test a get before the first set, equal timestamps and overwriting. **Signal:** pick a floor lookup structure and discuss thread safety.

Part 15 - Full System Design Walkthroughs and DP/Graph Problems (Q214-Q222)

Q214–Q216 are full walkthroughs you can follow in a 45-minute design round. Each uses the same flow: requirements, estimates, API, data model, architecture, deep dives, failure handling, trade-offs and a timing plan. The numbers are illustrative assumptions to show the method; in an interview, state your own assumptions and adjust. Q217–Q222 are dynamic programming and graph problems with solutions.

Full system design walkthroughs

Use the requirements-first design sequence before the examples: requirements → estimates → API/data → architecture → failure/scaling analysis → trade-offs. These prompts assume that interview process.

Q214 Design a flash-sale checkout that never oversells inventory

Requirements. Functional: browse a product, add to cart, reserve stock, pay, confirm the order. Non-functional: never sell more units than exist, tolerate a sudden traffic spike (assume 1M users, 100K requests/second on the product page, 10K order attempts/second, 5,000 units), stay available if payment is slow, and stay idempotent on retries.

API. `POST /reservations {sku, qty}` returns a reservation with an expiry, `POST /orders {reservationId, paymentToken}` with an `Idempotency-Key` header returns the order, and `GET /orders/{id}` returns its status.

Data model. `stock(sku, available)`, `reservation(id, sku, qty, user, expires_at, status)`, `order(id, reservation_id, status, amount)`, and an idempotency table with a unique key scoped to the caller and operation plus a request fingerprint.

Architecture. A CDN serves the static product page. A **waiting room** (a queue that admits users at a controlled rate) protects the backend. The reservation service decrements stock atomically, the order service runs a saga (reserve → charge → confirm), and events flow through a message queue with an outbox.

Deep dive: preventing oversell. In one database transaction, validate positive quantity, perform `UPDATE stock SET available = available - :qty WHERE sku = :sku AND available >= :qty`, and create exactly one reservation if one row was updated. Zero updated rows means insufficient stock or a missing SKU; distinguish them if the API needs to. The database remains the inventory authority. A hot row can limit throughput: preallocate the 5,000 units into disjoint stock buckets and atomically decrement only a bucket with enough stock, or use a carefully reconciled reservation service. A Redis counter alone cannot prove the no-oversell guarantee across failover and database commits. Expiry releases a hold only after an atomic `ACTIVE → EXPIRED` transition that loses to a concurrent confirmation; credit stock once in the same transaction.

Failure handling. Move a reservation to `PAYMENT_PENDING` before charging so ordinary expiry does not release it mid-payment. A confirmed payment failure may release the reservation. A timeout is ambiguous: first query or reconcile the provider's payment state, using the same provider idempotency key, before refunding or releasing a hold. Bound how long pending holds can remain unresolved and escalate for reconciliation. Make order creation and compensation idempotent, and let a recovery worker finish interrupted state transitions. Reject traffic early with rate limiting and bot protection.

Trade-offs. A cache can serve availability hints but must not independently authorize a sale. A single database stock row is simple but limited by contention; disjoint buckets need careful allocation and reconciliation to avoid stranded units. Reserving before payment temporarily holds inventory. A waiting room protects capacity but needs an explicit fairness policy.

Timing plan. 5 min requirements and estimates, 5 min API and data, 10 min architecture, 15 min the oversell deep dive, 10 min failures and trade-offs. **Signal:** a precise oversell mechanism (conditional update, sub-counters, expiring reservations), a saga with compensation and a plan for the traffic spike.

Q215 Design a ride-hailing dispatch system

Requirements. Functional: drivers send locations, riders request a ride, the system finds and offers the trip to a nearby driver, tracks the trip and shows ETA. Non-functional: matching in a few seconds, high-volume location updates (assume 1M online drivers sending a location every 4 seconds, about 250K updates/second), no driver offered two trips at once.

API. Drivers stream locations over a persistent connection (WebSocket or gRPC) or `POST /drivers/{id}/location`. Riders call `POST /rides {pickup, dropoff}` and receive updates through push or streaming. Drivers respond with `POST /offers/{id}/accept` or `decline`.

Data model. The **latest** driver location lives in an in-memory geospatial store (for example Redis geospatial commands or an index keyed by geohash/S2/H3 cell), with driver state `AVAILABLE | OFFERED | ON_TRIP`. Trips, riders and drivers live in a durable database. Location history is written asynchronously to a stream and analytics storage.

Architecture. Location gateways (stateless, many instances) update the geo index. A matching service queries nearby available drivers in the pickup cell and neighboring cells, ranks them by ETA (from a routing service) and offers the trip to the best one with a timeout. A trip service owns the trip state machine, and a pricing service computes fares and surge per cell from supply and demand.

Deep dive: matching without double-booking. Choose one authoritative store for driver offer state. Atomically change `AVAILABLE → OFFERED` with a unique offer ID, expiry and version/fencing token, and persist the matching trip transition before notifying the driver. Acceptance must atomically verify that the offer is still current before moving to `ON_TRIP`. Decline and timeout may restore `AVAILABLE` only if they still own that offer; a late timeout must not clear a newer offer. Expanding search rings (nearby cells first, then wider) bounds candidate work per request.

Failure handling. Stale locations expire by a TTL; do not offer drivers whose location or availability is stale. Recover unexpired offers from the authoritative trip/offer store after a restart, and reconcile the geo index as a derived view. Partitioning by city or region limits the failure radius. Retry notifications without repeating a trip transition.

Trade-offs. Location updates need not be durable (the next update replaces them), which allows in-memory storage; trip state must be durable. Larger cells mean fewer lookups but more candidates to rank. Frequent updates improve accuracy but cost bandwidth and battery.

Timing plan. 5 min requirements and scale, 10 min geo index and updates, 15 min matching and state machine, 10 min failures and scaling, 5 min trade-offs. **Signal:** separating ephemeral location data from durable trip data, and the atomic driver-state transition.

Q216 Design a real-time clickstream analytics pipeline on GCP

Requirements. Functional: collect events from web and mobile clients, make near-real-time aggregates available (page views per minute, funnel counts) and keep raw events for later analysis. Non-functional: handle bursts (assume 200K events/second peak), tolerate duplicates and late events, keep results correct within

minutes, control cost.

API. `POST /events` accepts a batch of events, each with a client-generated `eventId`, timestamp, user or session ID and payload, validated against a schema.

Data model. Raw events in a date-partitioned, clustered BigQuery table (and an archive in Cloud Storage), aggregates in a separate table keyed by window and dimensions, and a dead-letter topic for invalid events.

Architecture. Clients send to a stateless collector on Cloud Run, which validates and publishes to **Pub/Sub**. A **Dataflow** (Apache Beam) streaming job reads the subscription, validates, deduplicates stable client `eventId` values within a defined retention period, enriches, computes windowed aggregates and writes raw and aggregated results to **BigQuery**. Route invalid records to a dead-letter destination explicitly in the pipeline. Dashboards query the aggregate tables.

Deep dive: time, duplicates and late data. Aggregate by **event time**, not arrival time, using windows with a watermark and allowed lateness. A late event updates an earlier window only while that window remains open under the chosen trigger and accumulation policy. Dataflow's default exactly-once mode covers committed pipeline results, not arbitrary external side effects. Deduplicate client retries by stable `eventId`; Pub/Sub message deduplication alone does not identify separately published copies of one client event. Choose a BigQuery sink mode with the required write guarantee, or reconcile duplicate rows downstream. State the end-to-end guarantee only after checking the source, sink and deduplication window. Partition BigQuery tables by event date and cluster by common filters to cut query cost.

Failure handling. The collector acknowledges only a confirmed publish; an ambiguous publish failure may still produce a duplicate when the client retries with the same `eventId`. Route poison records explicitly instead of relying solely on Pub/Sub subscription dead lettering after pipeline acknowledgement. If the pipeline is down, Pub/Sub retains messages within the configured retention period; recovery depends on backlog, quota and capacity. Monitor backlog age, watermark lag and error rates.

Trade-offs. A longer allowed lateness improves accuracy but delays final results and increases state. Streaming aggregation gives low latency at higher cost than batch recomputation; a common design keeps both, with batch as the correction layer. Schema evolution needs versioned, backward-compatible event formats.

Timing plan. 5 min requirements and scale, 10 min ingestion path, 15 min windows/dedupe/late data, 10 min failures and cost, 5 min trade-offs. **Signal:** event time versus processing time, idempotent sinks and honest delivery guarantees.

Dynamic programming and graph problems

Q217 Longest increasing subsequence in O(n log n)

Explain before coding: tails[i] is the smallest known ending value for an increasing subsequence of length i+1. For [3,1,2], replace tail 3 with 1, then extend with 2. A smaller tail leaves more future extension options; the tails array is a summary of possibilities, not necessarily one reconstructed subsequence.

```
int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];          // tails[i] = smallest tail of an increasing
subsequence of length i+1
    int size = 0;
    for (int x : nums) {
        int i = Arrays.binarySearch(tails, 0, size, x);
        if (i < 0) i = -(i + 1);                 // insertion point
        tails[i] = x;
        if (i == size) size++;
    }
    return size;
}
```

Reasoning and alternatives: each number replaces the first tail that is greater or equal, keeping tails as small as possible, which leaves the most room to extend. O(n log n) time. The simpler O(n²) DP (`dp[i] = 1 + max(dp[j])` for smaller earlier values) is acceptable first, then optimize.

What if/test: the array `tails` is not itself a valid subsequence, only its length is meaningful. Test empty input, all equal values (answer 1) and strictly decreasing input. **Signal:** explain what `tails` means and why equal values replace instead of extend.

Q218 Partition equal subset sum (0/1 knapsack)

Explain before coding: An equal partition exists exactly when a subset reaches half the total. Start with sum zero reachable and add each non-negative number once. Update sums downward so the current number cannot contribute repeatedly during one iteration; upward updates would solve a different problem.

```
boolean canPartition(int[] nums) {
    long sum = 0;
    for (int n : nums) {
        if (n < 0) throw new IllegalArgumentException("non-negative values required");
        sum += n;
    }
    if (sum % 2 != 0) return false;
    int target = Math.toIntExact(sum / 2); // array-backed DP requires a feasible target
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int n : nums)
        for (int s = target; s >= n; s--)        // go downward so each number is used at most once
            dp[s] |= dp[s - n];
    return dp[target];
}
```

Reasoning and alternatives: the question reduces to "can some subset sum to half the total". O(n × target) time, O(target) space. Iterating upward would let a number be reused, which solves the unbounded variant instead.

What if/test: this array-backed method assumes non-negative integers and a feasible target size; reject negative values and use another representation if the target is too large for memory. Test an odd total, an empty input (two empty subsets are allowed), a single element, and a case that needs several elements. **Signal:** explain the reverse loop and the reduction.

Q219 Longest common subsequence

Explain before coding: For two prefixes, matching last characters extend the best result for the two shorter prefixes. If they differ, discard the last character from either prefix and take the better result. For abc and ac, keeping a then c gives length two. Subsequence permits gaps; substring does not.

```
int longestCommonSubsequence(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                    ? dp[i - 1][j - 1] + 1
                    : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}
```

Reasoning and alternatives: `dp[i][j]` is the answer for the first `i` characters of `a` and the first `j` of `b`. O(m × n) time and space; keep only two rows to reduce space to O(n). Walk the table backwards to reconstruct the sequence itself.

What if/test: test empty strings, identical strings and no common characters. **Signal:** define the state precisely and mention the space optimization.

Q220 Word ladder (shortest transformation, BFS)

Explain before coding: Treat each valid word as a node and each one-letter transformation as an equal-cost edge. BFS explores all one-change candidates before two-change candidates. Mark a word visited on enqueue so repeated paths do not expand it again; the first target layer gives the shortest ladder.

```
int ladderLength(String begin, String end, List<String> words) {
    if (begin.equals(end)) return 1; // if a zero-transformation ladder is allowed
    Set<String> dict = new HashSet<>(words);
    if (!dict.contains(end)) return 0;
    Deque<String> q = new ArrayDeque<>();
    Set<String> seen = new HashSet<>();
    q.add(begin); seen.add(begin);
    int steps = 1;                                  // counts words in the sequence
    while (!q.isEmpty()) {
        for (int s = q.size(); s > 0; s--) {
            String w = q.poll();
            if (w.equals(end)) return steps;
            char[] c = w.toCharArray();
            for (int i = 0; i < c.length; i++) {
```

```
                char old = c[i];
                for (char x = 'a'; x <= 'z'; x++) {
                    if (x == old) continue;
                    c[i] = x;
                    String next = new String(c);
                    if (dict.contains(next) && seen.add(next)) q.add(next);
                }
                c[i] = old;
            }
        }
        steps++;
    }
    return 0;
}
```

Reasoning and alternatives: words are nodes and a single-letter change is an edge; BFS finds the shortest path in an unweighted graph. For each visited word of length L, generating 26L candidates costs O(26L²) in Java because constructing and hashing each candidate string costs O(L); memory is O(NL) for N dictionary words. Bidirectional BFS can reduce explored nodes on large dictionaries but must coordinate the two visited frontiers correctly.

What if/test: assume non-null, equal-length lowercase words; clarify whether a zero-transformation ladder counts as one word, and whether the end must appear in the dictionary. This implementation returns 1 when begin equals end. Test an unreachable target and a target missing from the dictionary. **Signal:** model it as a graph, mark visited when enqueuing, and offer bidirectional BFS.

Q221 Cheapest flights within K stops (Bellman-Ford style)

Explain before coding: Keep the cheapest cost using at most the previous round's number of edges. Read only that old array while writing the next one, so a single round adds at most one flight. With zero intermediate stops, at most one flight is allowed; reusing updated costs in the same round could accept a forbidden connection.

```
int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
    long INF = Long.MAX_VALUE / 4;
    long[] dist = new long[n];
    Arrays.fill(dist, INF);
    dist[src] = 0;
    for (int i = 0; i <= k; i++) {                  // at most k+1 edges
        long[] next = dist.clone();                 // use only last round's values
        for (int[] f : flights)
            if (dist[f[0]] < INF)
                next[f[1]] = Math.min(next[f[1]], dist[f[0]] + f[2]);
        dist = next;
    }
    return dist[dst] >= INF ? -1 : Math.toIntExact(dist[dst]);
}
```

Reasoning and alternatives: each round allows one more edge, so after `k + 1` rounds the array holds the cheapest cost using at most `k` intermediate stops. O((k + 1) × (E + V)) time and O(V) space, including each clone. Assume non-negative fares and a final answer in the `int` range; `long` distances avoid ordinary

intermediate overflow. Plain Dijkstra using only one best cost per city can discard a path with fewer edges that is needed under the stop limit; a correct priority-queue variant tracks stops in its state.

What if/test: cloning the array each round prevents chaining multiple newly relaxed edges in that same round, preserving the edge-count bound. Test unreachable destinations, `k = 0` and a case where a cheaper path needs too many stops. **Signal:** explain why the stop limit rules out standard Dijkstra and why the clone matters.

Q222 Redundant connection (Union-Find)

Explain before coding: Before adding an edge, ask whether both endpoints already share a representative. In the triangle (1,2), (2,3), (1,3), the third edge closes the existing path into a cycle. Otherwise union the two components; path compression and rank keep representative searches cheap.

```
int[] findRedundantConnection(int[][] edges) {
    int[] parent = new int[edges.length + 1];
    int[] rank = new int[parent.length];
    for (int i = 0; i < parent.length; i++) parent[i] = i;
    for (int[] e : edges) {
        int a = find(parent, e[0]), b = find(parent, e[1]);
        if (a == b) return e;                       // this edge closes a cycle
        if (rank[a] < rank[b]) parent[a] = b;
        else if (rank[a] > rank[b]) parent[b] = a;
        else { parent[b] = a; rank[a]++; }
    }
    return new int[0];
}

int find(int[] p, int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } // path halving
    return x;
}
```

Reasoning and alternatives: assume n edges over node labels 1 through n and an original tree plus one extra edge. The edge joining two already connected nodes closes the unique cycle. Path compression plus union by rank gives O(E α(n)) total union-find time after initialization; α is the inverse Ackermann function. A fresh DFS reachability check before every edge can take O(n²) overall.

What if/test: under the tree-plus-one-edge premise, the first edge that closes a cycle is the last edge of that cycle in input order, matching the usual tie rule. This implementation is not a general solver for arbitrary graphs or sparse labels. Test a simple triangle and a longer cycle. **Signal:** know when union-find beats DFS and name path compression and union by rank.

Version and source notes

The system-design figures are illustrative estimates. Delivery guarantees and sink behavior are based on Google Cloud Dataflow's exactly-once explanation, Pub/Sub to Dataflow behavior, and Dataflow's BigQuery write modes. These sources do not prove an arbitrary application's end-to-end exactly-once behavior.

Part 16 - Hibernate and SQL Questions (Q223-Q244)

Hibernate/JPA and SQL practice. SQL examples use `employee(id, name, dept, salary, manager_id)` unless stated. The SQL examples below were checked against SQLite for the portable cases. JPA snippets are illustrative; check provider behavior and SQL with the Hibernate 6.6 and Spring Data JPA 3.5 references at the end. PostgreSQL-specific notes are identified explicitly.

Start here: database connections and session management

Study this chapter in order: connections → pools → sessions and factories → transaction boundaries → entity lifecycle → mapping annotations → advanced Hibernate → SQL. The existing Q223–Q244 identifiers stay stable for bookmarks; their reading order now follows the topic rather than the number. These explanations target Java 17/21, Spring Boot 3.5, Spring Framework 6.2 and Hibernate 6.6 with `jakarta.persistence` imports. Framework flows are conceptual walkthroughs, not runnable or integration-tested applications.

H1 How do we make a connection with a database in plain Java

Strong answer: add the database's JDBC driver, supply its JDBC URL and credentials, then obtain a `java.sql.Connection` using `DriverManager.getConnection` or a configured `DataSource`. A JDBC 4 driver normally registers through service discovery; explicitly loading its class is usually unnecessary. Use a `PreparedStatement` with bound values, execute it, read the `ResultSet`, and close result sets, statements and owned connections with try-with-resources. The driver translates JDBC operations into the database protocol; Hibernate is not required.

Follow-up — what makes two writes atomic? JDBC connections default to auto-commit. For an owned connection, disable auto-commit, execute both writes on that connection, commit on success and roll back on failure. Closing a resource is not a substitute for an explicit transaction outcome. Do not manually commit or close a Spring-managed transactional connection.

Senior follow-up — what fails in production? Separate driver/URL mistakes, DNS or TLS failures, rejected credentials, database limits, and pool acquisition timeouts. Bound connection acquisition and query execution separately, keep secrets outside source control, and never concatenate user input into SQL. A bound parameter represents a value, not a table name or sort direction; allowlist those structural choices.

What if/test: make the second write violate a constraint and check that the first write rolls back. Simulate an unavailable database and distinguish a connection failure from a slow query.

H2 How does Spring Boot establish a database connection

Strong answer: with `spring-boot-starter-jdbc` or `spring-boot-starter-data-jpa`, a compatible driver and external `spring.datasource.url`, `username` and `password`, Boot can configure a `DataSource`; the normal starter pool is HikariCP. Boot binds configuration and evaluates auto-configuration conditions. JDBC code can use `JdbcTemplate`/`JdbcClient`; JPA adds an `EntityManagerFactory`, a provider such as Hibernate, and transaction integration. A repository ultimately executes SQL through JDBC connections obtained from the DataSource.

Internal flow: configuration → DataSource/pool → EntityManagerFactory/SessionFactory → transaction-bound EntityManager/Session → JDBC connection → database. This is a responsibility diagram: a Session need not hold a physical connection for its whole lifetime; acquisition/release depend on transaction and provider settings.

Follow-up — can Boot connect without a URL? It can select a supported embedded database on the classpath. A production database needs deliberate configuration. A custom DataSource can make the default configuration back off; multiple DataSources require explicit factory, repository and transaction-manager wiring.

`@Primary` alone does not define every repository's database.

Senior follow-up — how do you diagnose it? Inspect the condition evaluation report, effective configuration without revealing secrets, pool metrics and root driver exception. Keep schema migration ownership explicit. `ddl-auto=update` is not a production migration strategy.

H3 What is a connection pool, and is a Session a connection

Strong answer: a pool keeps reusable physical connections; borrowing typically returns a logical handle whose close returns it to the pool. A Hibernate Session instead tracks managed entities, identity and pending work. Increasing pool size cannot exceed the database's useful capacity and can worsen contention. Size against all application replicas and other database clients, then measure acquisition wait, transaction duration and database load.

Follow-up — what causes exhaustion? Leaked handles, long transactions, lock waits or slow queries can keep connections occupied. A remote HTTP call inside a database transaction can retain scarce resources while doing no SQL. Virtual threads do not create database capacity.

Senior follow-up — REQUIRES_NEW? An inner transaction can need a second connection while the outer transaction retains its resources. Many callers doing this simultaneously can exhaust the pool. Review boundaries and worst-case nested demand rather than blindly doubling capacity.

H4 What are Session, SessionFactory, EntityManager and EntityManagerFactory

Strong answer: `Session` is Hibernate's unit-of-work API; JPA's standard API is `EntityManager`. Both expose a persistence context that tracks managed entity identity and changes. `SessionFactory`/`EntityManagerFactory` hold expensive shared configuration and metadata and are normally created once per persistence unit. Factories are thread-safe; individual Sessions/EntityManagers are not. A persistence context is not a global cache or a database transaction.

Follow-up — why can an EntityManager be injected into a singleton service? Spring usually injects a shared proxy that resolves the appropriate transaction-bound EntityManager. The underlying mutable context is not made thread-safe. Do not pass it or managed entities to another thread and expect the transaction to follow.

Follow-up — openSession versus getCurrentSession? `openSession` creates a Session whose lifecycle you own. `getCurrentSession` resolves one through the configured current-session context; it is not automatically available in every environment. With Spring Data JPA, normally let Spring own the EntityManager and transaction lifecycle.

Senior follow-up — first-level versus second-level cache? First-level identity belongs to one persistence context and disappears when it is cleared/closed. Optional second-level cache spans contexts and needs provider/invalidation decisions; continue with Q231 after the mapping section.

H5 Is sessionManager the same as EntityManager or a transaction manager

Strong answer: there is no standard Hibernate/JPA API named `SessionManager`. An application may define a helper with that name, but ask which type the interviewer means. `EntityManager` manages entity operations and context; `SessionFactory` creates Sessions; Spring's `PlatformTransactionManager` coordinates begin, commit, rollback and resource binding. `JpaTransactionManager` integrates a JPA EntityManagerFactory; older native Hibernate integrations may use `HibernateTransactionManager`. Spring 6.2's `hibernate5` integration targets Hibernate 5.5/5.6; for this chapter's Hibernate 6.6 stack, use JPA integration with `JpaTransactionManager` (or appropriate JTA coordination). Servlet `HttpSession` is a different concept.

Follow-up — what happens on a service call? With proxy-based transaction management enabled, an external call through the Spring proxy invokes transaction advice. The manager starts or joins a transaction, associates persistence resources with the current thread, calls the method, then commits or rolls back and cleans up resources it owns. Joining an outer transaction does not commit that transaction when the inner method returns.

Senior follow-up — why did @Transactional do nothing? Check whether the instance is Spring-managed, whether the call crossed the proxy, method visibility/proxy limitations, the selected manager and propagation. Self-invocation bypasses default proxy advice. By default unchecked exceptions and Errors trigger rollback; checked exceptions require an appropriate rollback rule. Catching an exception can hide it from the advice, while an already rollback-only transaction can still fail on outer completion.

H6 What happens to an entity from load to commit

Strong answer: an entity starts transient, becomes managed through persist/load, can become detached after clear/close, and can be marked removed. Within a context, repeated entity lookup by identity normally yields the same managed instance. Hibernate dirty checking detects changes and flush synchronizes pending work with SQL; commit finalizes the database transaction. Flush can happen before commit, including before relevant queries, and does not guarantee durability.

Follow-up — persist versus merge? `persist` makes a new instance managed. `merge` copies state into a managed instance and returns that instance; the supplied detached object does not itself become managed. Work with the return value. Detached changes are not automatically tracked.

Senior follow-up — why does lazy loading fail? Accessing an uninitialized association after its Session is closed can produce `LazyInitializationException`. Fetch the needed data or project a DTO inside a bounded service transaction. Open Session in View can defer work into rendering, but can conceal extra SQL and reads outside the service transaction. Bulk JPQL/native writes can leave already managed objects stale; plan flush/clear/refresh boundaries. See Q232 for concurrent writers.

What if/test: compare instance identity inside and across contexts, update a detached object without merging, then flush and roll back: SQL execution must not be mistaken for committed data.

JPA and Hibernate annotations, then their follow-ups

H7 Which annotations define the entity and its columns

Strong answer: annotations are mapping metadata read by the persistence provider while building its model; they do not execute SQL on their own. An `@Entity` is not automatically a Spring component. Start with the purpose, explain the provider's interpretation, then describe the generated SQL and database invariant.

Annotation family	What the provider does	Follow-up and senior boundary
`@Entity`, `@Table`	Registers an entity and its table mapping; entity name and table name serve different query layers.	Does JPQL name the Java entity or the physical table? How does the naming strategy affect existing schemas?
`@Id`, `@GeneratedValue`, `@SequenceGenerator`	Defines identity and identifier generation.	When is an ID available, and how does it affect batching? Continue with Q226.
`@Column`, `@Basic`, `@Transient`, `@Access`	Maps basic state, excludes fields, or controls field/property access.	`nullable=false` is mapping/schema metadata, not request validation. A migrated database must actually enforce NOT NULL. Avoid accidentally mixing field and property access.

Annotation family	What the provider does	Follow-up and senior boundary
`@Enumerated`, `@Convert`, `@Converter`, `@Lob`	Chooses enum, custom-value or large-object mapping.	Ordinals break if constants reorder; strings also need a plan when names change. A LOB or lazy basic field is not automatically cheap.
`@Embeddable`, `@Embedded`, `@AttributeOverride`	Maps a value object into the owning entity's columns.	Does the value need independent identity/lifecycle? If yes, an entity may fit better.
`@EmbeddedId`, `@IdClass`, `@MapsId`	Represents composite or derived identity.	Define stable key equality and check foreign-key ownership; continue with Q224.
`@Version`	Adds an optimistic concurrency predicate and checks affected rows.	Two concurrent writers must not both silently succeed; carry the client's version through the API (Q232).
`@MappedSuperclass`, `@Inheritance`, `@DiscriminatorColumn`	Shares mapped attributes or selects entity inheritance strategy.	Compare joins, nullable columns and polymorphic query cost; a mapped superclass is not independently queryable as an entity.
`@PrePersist`, `@PreUpdate`, `@EntityListeners`	Invokes lifecycle callbacks during managed operations.	Bulk/native SQL and external writers do not necessarily invoke these callbacks. Continue with auditing in Q233.

H8 How do relationship annotations change SQL and lifecycle

Strong answer: `@ManyToOne`, `@OneToMany`, `@OneToOne` and `@ManyToMany` describe cardinality. `@JoinColumn` or `@JoinTable` describes foreign-key/join-table storage; `mappedBy` identifies the inverse side using the owning Java property, not a database column. Change the owning side and keep both sides of a bidirectional relationship consistent.

Follow-up — fetch, cascade and orphan removal? Fetch determines loading; cascade propagates selected entity operations; orphan removal deletes a dependent child removed from an owning relationship. They are independent. JPA defaults to EAGER for to-one relationships and LAZY for to-many; LAZY is a hint, not a guarantee of one specific SQL plan. Cascade REMOVE on shared entities is dangerous. Database ON DELETE CASCADE is a separate mechanism.

Senior follow-up — which annotation fixes N+1? None universally. `@EntityGraph` selects an endpoint-specific fetch plan; Hibernate `@BatchSize` and `@Fetch` change loading strategies; query fetch joins are another option. Measure statement count, rows and memory, especially for collections with pagination. `@OrderBy` sorts by mapped attributes; `@OrderColumn` persists list positions and can cause index-shift updates. `@ElementCollection` stores values without independent entity identity. Continue with Q224, Q225, Q223 and Q229 below.

Follow-up — provider-specific annotations? `@SoftDelete`, `@SQLRestriction`, `@Immutable` and cache annotations have Hibernate-specific behavior. Pin the provider version, inspect actual SQL and test native/bulk access and external writers. They do not replace constraints, authorization or a migration plan.

Advanced Hibernate: annotations, queries and operational follow-ups

Q226 IDENTITY, SEQUENCE or UUID for primary keys

Strong answer: with Hibernate 6.6, `IDENTITY` prevents JDBC insert batching for those entities because the database must generate each identifier during insert. A database `SEQUENCE` can support batching; when using a pooled optimizer, align its allocation/increment configuration with the actual database sequence and verify schema validation. Random UUIDs can reduce B-tree locality and enlarge keys; time-ordered identifiers may improve locality, subject to storage byte order and database index behavior.

Reasoning and alternatives: use UUIDs when ids must be generated client-side or across systems, accepting larger keys and the index cost.

What if/test: insert 10,000 rows with each strategy and compare the statement count and time. **Signal:** connect id strategy to batching and index locality.

Q224 Why is a lazy @OneToOne hard, and what does @MapsId do

Strong answer: on the inverse `mappedBy` parent side, Hibernate may need a secondary query to learn whether a child exists, so `LAZY` alone may not defer loading. Lazy-state bytecode enhancement can help when a bidirectional association is required. On the owning side the foreign key is available to form a proxy. With `@MapsId`, the child's primary key also serves as its foreign key to the parent; this avoids a separate child identifier column, but it does not eliminate the need for a foreign-key constraint or guarantee no index.

Reasoning and alternatives: put the foreign key on the side you load more often, or model the child as a separate lookup by the parent's id.

What if/test: log SQL when loading the parent and count extra selects. **Signal:** explain why null-ability forces the extra query.

Q225 How should you map @ManyToMany

Strong answer: when links are unique and order is unimportant, use a `Set` with stable entity equality and helper methods that keep both sides of a bidirectional mapping in sync. Avoid cascading `REMOVE` to the other entity: it can be shared by other owners. An unordered bag-style `List` can trigger delete-and-reinsert work on the join table; verify the mapping's SQL instead of assuming all lists behave alike.

Reasoning and alternatives: when the relationship has its own data (a role granted on a date) or you need to query it, use an explicit join entity with two `@ManyToOne` associations.

What if/test: remove one link and inspect the SQL for bulk deletes. **Signal:** prefer an explicit join entity once the relationship has attributes.

Q223 What goes wrong with @ElementCollection on a List

Strong answer: in Hibernate, an unordered `List` element collection can be mapped as a bag, whose duplicate values give rows no stable position or identity. Removing one value may delete all rows for the owner and re-insert the survivors. A `Set` or indexed `List` with `@OrderColumn` can permit more targeted SQL, but list index shifts may still update multiple rows. The exact plan depends on mapping and version; inspect generated SQL.

Reasoning and alternatives: for anything that grows, has its own lifecycle or needs queries, model a real entity with `@OneToMany` instead.

What if/test: turn on SQL logging, remove one element and count the statements. **Signal:** you know the delete-all-and-reinsert behavior and when to promote the collection to an entity.

Q229 Batch fetching versus subselect fetching

Strong answer: batch fetching (`@BatchSize`, `default_batch_fetch_size`) groups lazy loads for several parents into bounded `IN` queries. `@Fetch(FetchMode.SUBSELECT)` uses a subselect tied to the parent load to initialize the matching collection roles for parents held in the persistence context; it can fetch far more children than the one collection you touched.

Reasoning and alternatives: batching bounds each query's parent IDs and works with paging. Subselect fetching can reduce query count for a known, modest parent result, but inspect generated SQL, pagination behavior and total rows fetched before choosing it.

What if/test: load 100 parents and count statements under each setting. **Signal:** choose by result size and paging needs, not by habit.

Q228 What does @Transactional(readOnly = true) really do

Strong answer: Spring's `readOnly = true` is a transaction hint, not a write prohibition. With Hibernate integration, Spring Data JPA documents a manual flush mode, which avoids normal dirty checking/flush work. A JDBC read-only hint may also be propagated depending on transaction manager and driver. Replica routing requires an explicitly configured router; the annotation does not move reads to a replica by itself.

Reasoning and alternatives: it is a hint, not a security boundary. Keep the same transaction rules and do not rely on it to block writes.

What if/test: modify an entity inside a read-only transaction and check whether anything persists on your stack. **Signal:** call it an optimization hint and verify behavior for your versions.

Q230 Which projection type should you use for read endpoints

Strong answer: project to the fields the API returns. In Spring Data JPA, a closed interface projection exposes known top-level properties and can let the query select fewer columns. Open projections with expression-based accessors limit this optimization; nested properties can require joins and materialize more data. Explicit DTO constructor expressions and tuples can also select narrow columns when the query names them.

Reasoning and alternatives: returning entities for list screens loads unused columns and risks lazy-loading surprises. Keep entities for writes.

What if/test: compare the generated SQL and memory for an entity query versus a projection. **Signal:** separate read models from write models.

Q231 How do second-level cache concurrency strategies differ

Strong answer: `READ_ONLY` suits immutable data. `NONSTRICT_READ_WRITE` can allow stale reads around updates. `READ_WRITE` coordinates updates to a single cached entity more strictly, but it does not provide serializable transaction isolation. `TRANSACTIONAL` requires a provider that supports its stronger transactional semantics. Verify cache-provider support and cross-node behavior.

Reasoning and alternatives: cache only read-mostly entities with clear invalidation, and never cache what must be strongly consistent across instances unless the provider guarantees it.

What if/test: run two application instances, update through one, and read through the other. **Signal:** name the strategy and the staleness it allows.

Q232 How do you stop lost updates across HTTP requests

Strong answer: return the entity version in a strong ETag, require `If-Match` on update, and enforce the precondition atomically with the write (for example using an `@Version` update or conditional SQL). For an HTTP `If-Match` failure, return `412 Precondition Failed`; an application-level version field may use `409 Conflict` by contract.

Reasoning and alternatives: `@Version` detects concurrent writers to the entity loaded in the update transaction, but it does not know the version the client saw unless that value travels through the API. A read-only comparison followed by an unguarded update is racy; the final write must check the version.

What if/test: two clients fetch the same resource, both update, and the second must fail. **Signal:** know that the version must round-trip through the API.

Q227 How do you implement soft delete safely

Strong answer: Hibernate 6.4+ provides `@SoftDelete` for entities and supported collection tables; prefer it when its semantics fit. On older versions, custom delete SQL plus a restriction/filter can implement soft delete, but verify entity loads, associations, bulk operations and caches. `@Where` was deprecated in favor of `@SQLRestriction` in newer Hibernate 6 releases; neither turns arbitrary native SQL into a filtered query.

Reasoning and alternatives: unique constraints still see deleted rows, so use a partial unique index covering only live rows. Native queries and bulk updates bypass the filter, and cascades need thought. If you need history rather than hiding, use an audit table.

What if/test: delete a row, then insert the same unique value again and query natively. **Signal:** mention the unique-constraint trap and the native-query gap.

Q233 How do you add auditing

Strong answer: Spring Data auditing (`@CreatedDate`, `@LastModifiedDate`, `@CreatedBy` with an `AuditorAware` bean) records who and when on the current row. Hibernate Envers stores revisions for audited entities when changes pass through it.

Reasoning and alternatives: Envers costs storage and write time and does not automatically cover native SQL or every external writer. Compliance-grade trails need a threat model, restricted mutation rights, retention policy and tamper-evident storage; triggers or change-data-capture can be part of that design.

What if/test: update an entity twice and confirm both who and when, and the history rows. **Signal:** choose between column-level auditing and full history by need.

Q234 How do you manage schema migrations with Flyway or Liquibase

Strong answer: Flyway versioned migrations and Liquibase changesets record what ran and their checksums. Treat an applied migration as immutable; fix forward with a new migration rather than silently changing history. Flyway repeatable migrations suit replaceable views or procedures. Baseline an existing database deliberately, and keep Hibernate schema generation at `validate` or `none` in production when migrations own the schema.

Reasoning and alternatives: make migrations backward compatible (expand, migrate, contract) so rolling deploys work. Use the migration tool's locking/coordination or a single deployment step; do not assume every application replica can safely run arbitrary DDL concurrently.

What if/test: run the migration against a copy of production-sized data and time lock duration. **Signal:** immutable history plus backward-compatible changes.

SQL foundations before advanced scenarios

D1 How does SQL from a Java application reach the database

Strong answer: JDBC sends a prepared statement and bound values through the driver. The database parses/plans and executes it against the transaction's visible data; JDBC exposes rows or an update count. JPA/HQL translates entity-oriented queries into SQL first; native SQL already names the database's tables/columns. A method named `save` does not promise one immediate INSERT: entity state, identifiers and flush timing matter.

Follow-up — which SQL concepts come first? Explain primary/foreign/unique keys, NOT NULL and CHECK constraints, SELECT/INSERT/UPDATE/DELETE, then INNER/LEFT joins, WHERE, GROUP BY, HAVING, ORDER BY and pagination. A join can multiply rows; `COUNT(*)` counts rows while `COUNT(column)` excludes nulls. SQL has no guaranteed result order without ORDER BY; use a unique tie-breaker for stable paging.

Senior follow-up — how do you investigate a slow query? Capture the actual SQL and safe parameter shapes, inspect the execution plan on representative data, compare estimated/actual rows, and check indexes, statistics, lock waits and transaction length. `EXPLAIN ANALYZE` executes the statement on databases such as PostgreSQL; use caution with writes. An index trades read speed for write/storage cost and is not guaranteed to be chosen.

D2 What does a transaction guarantee, and which guarantees must you name

Strong answer: atomicity gives all-or-nothing commit, constraints protect declared invariants, isolation controls concurrent visibility, and durability describes committed persistence under the database's configured guarantees. Auto-commit makes individual statements transactions, not a multi-call service operation. Name the database and isolation level before claiming a specific anomaly is prevented.

Follow-up — two writers? Use atomic conditional SQL, constraints or an appropriate lock/version strategy; an application read followed by an unconditional write races. Roll back a failed transaction and retry the whole safe operation with fresh state when appropriate. A connection-loss timeout near commit can leave the outcome unknown; use a durable idempotency key or reconcile rather than assuming rollback.

Progression: first grouping (Q238), null-aware anti-joins (Q235), ranking (Q236), hierarchy traversal (Q237), then deadlocks and write skew (Q239–Q240), upserts (Q241), data distribution (Q242), paging (Q243) and duplicate cleanup (Q244).

Q238 WHERE versus HAVING, and conditional aggregation

```
SELECT dept,
       COUNT(*) AS headcount,
       SUM(CASE WHEN salary > 100 THEN 1 ELSE 0 END) AS high_earners
FROM employee
GROUP BY dept
HAVING COUNT(*) >= 2;
```

Strong answer: `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation. `CASE` inside an aggregate counts or sums by condition portably (PostgreSQL also has `FILTER`).

Reasoning and alternatives: push filters into `WHERE` when possible so fewer rows are grouped.

What if/test: compare plans when a filter sits in `WHERE` versus `HAVING`. **Signal:** know the order of evaluation.

Q235 NOT IN versus NOT EXISTS

```
-- customers with no orders (orders.customer_id may contain NULL)
SELECT c.id FROM customer c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

Strong answer: if the subquery contains a `NULL`, `NOT IN` cannot return a row whose value matches none of the non-null entries: the remaining comparison is unknown, not true. `NOT EXISTS` avoids that specific null trap. A `LEFT JOIN ... IS NULL` anti-join also works when the tested right-hand column is non-nullable for matched rows.

Reasoning and alternatives: `NOT EXISTS` also expresses intent clearly and usually plans as an anti-join.

What if/test: insert an order with a null customer and compare both forms. **Signal:** explain three-valued logic.

Q236 Find the N-th highest salary

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r
  FROM employee WHERE salary IS NOT NULL
) t WHERE r = :n;
```

Strong answer: bind a positive `:n`. `DENSE_RANK` treats equal non-null salaries as one rank, so ties do not skip a position. The query returns one row for the N-th distinct salary, or no row when it does not exist.

Reasoning and alternatives: `ORDER BY salary DESC LIMIT 1 OFFSET n-1` over distinct salaries also works, but returns nothing, not null, when n is too large, depending on how it is written.

What if/test: include tied top salaries and a table with fewer than N distinct values. **Signal:** choose `DENSE_RANK` over `RANK` or `ROW_NUMBER` for this.

Q237 Walk an org hierarchy with a recursive CTE

```
WITH RECURSIVE chain(id, name, depth, path) AS (
  SELECT id, name, 0, ',' || id || ',' FROM employee WHERE id = 1
  UNION ALL
  SELECT e.id, e.name, c.depth + 1, c.path || e.id || ','
  FROM employee e JOIN chain c ON e.manager_id = c.id
  WHERE instr(c.path, ',' || e.id || ',') = 0
)
SELECT id, name, depth FROM chain ORDER BY depth, id;
```

Strong answer: an anchor query seeds the result and the recursive part joins to the previous iteration until no rows are added. This SQLite example records visited IDs in a delimiter-guarded path, so a cycle on one branch is not expanded forever. Use a database-native cycle clause or array/path expression where available.

Reasoning and alternatives: a depth limit is a resource bound, not a proof that a hierarchy is acyclic; it can truncate legitimate data. Add an index on `manager_id` and validate ownership or tenant scope before traversing a production hierarchy.

What if/test: add a cycle in the data and confirm your guard stops it. **Signal:** mention termination and cycle protection.

Q239 What causes database deadlocks and how do you handle them

Strong answer: two transactions each hold a lock the other needs, typically by touching the same rows in opposite order. The database detects the cycle and aborts one transaction. Retry the whole transaction from a fresh read when the operation is safe to repeat; never retry only the failed SQL statement against stale assumptions.

Reasoning and alternatives: lock rows in a consistent order, keep transactions short, add indexes so fewer rows are locked, and retry on the deadlock error code (for example `40P01` in PostgreSQL).

What if/test: reproduce with two sessions updating two rows in opposite order. **Signal:** prevention plus an idempotent retry path.

Q240 What is write skew and how do you prevent it

Strong answer: two transactions read overlapping data, each updates a different row, and together they break an invariant (two doctors both go off call). PostgreSQL's `REPEATABLE READ` uses snapshot isolation and can allow this write skew because the writes do not conflict on one row.

Reasoning and alternatives: use `SERIALIZABLE` and retry aborted transactions, enforce the invariant with a database constraint where possible, or coordinate through a shared guard row/lock in every writer. Locking only the rows currently returned by a predicate may miss future rows or other access paths.

What if/test: run both transactions concurrently under the default isolation level. **Signal:** explain the guarantee for the named database and isolation level rather than treating every vendor's `REPEATABLE READ` as identical.

Q241 How do you write an idempotent upsert

```
INSERT INTO account(id, balance) VALUES (1, 100)
ON CONFLICT(id) DO UPDATE SET balance = excluded.balance;
```

Strong answer: an upsert relies on a unique key and performs insert-or-update as one database statement (`ON CONFLICT` in PostgreSQL and SQLite; vendor alternatives differ). This example sets the same balance on repeat, so the stored balance is unchanged on an identical retry. It does not make audit triggers or other side effects automatically idempotent.

Reasoning and alternatives: select-then-insert races under concurrency. With JPA, use a database-specific native upsert or recover from a duplicate-key conflict in a fresh transaction after the failed transaction has rolled back; do not continue blindly in a transaction marked failed.

What if/test: run the statement twice and from two connections at once. **Signal:** rely on a unique constraint, not application checks.

Q242 Partitioning versus sharding

Strong answer: partitioning splits a logical table into partitions in one database (range, list or hash). When the planner can use the partition key, partition pruning skips irrelevant partitions; time-range retention can become dropping a partition. Sharding distributes data across separate database instances or servers.

Reasoning and alternatives: choose partitioning for table management or query pruning when one database can still meet capacity needs; sharding is a separate operational step when data or write load must be distributed. PostgreSQL requires a unique or primary-key constraint declared on a partitioned table to include all partition-key columns, with additional restrictions on expressions.

What if/test: check `EXPLAIN` shows partition pruning for your common query. **Signal:** keep the two terms distinct and start with partitioning.

Q243 Why can COUNT(*) and offset pagination be slow, and what is the alternative

Strong answer: an exact `COUNT(*)` over a filtered MVCC snapshot must establish which rows are visible; it is not generally a constant-time metadata lookup, though an index-only scan or selective plan can reduce cost. Deep offset pagination still processes or skips many earlier rows, so latency often grows with the offset.

Reasoning and alternatives: with a stable total ordering and matching index, use keyset pagination and fetch `limit + 1` rows to know whether another page exists. Use approximate counts or a maintained counter only when the UI accepts their freshness and precision limits.

What if/test: time page 1 versus page 10,000 under each approach. **Signal:** question whether the UI really needs a total.

Q244 Remove duplicate rows but keep one

```
DELETE FROM person WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM person WHERE email IS NOT NULL
  ) t WHERE rn > 1
);
```

Strong answer: number rows within each non-null email group and delete everything after the first, then add a matching unique constraint so duplicates cannot return. Define normalization (for example case folding) and the survivor rule before running it. The query intentionally leaves null emails untouched; SQL unique constraints often permit multiple nulls.

Reasoning and alternatives: preview affected rows, back up the data, run the delete in a transaction, and coordinate concurrent writers while establishing the unique constraint. If the survivor needs related rows or richer fields, migrate their references and data before deleting.

What if/test: verify counts before and after, and that the constraint now rejects a duplicate. **Signal:** fix the data and add the constraint that prevents recurrence.

Version and source notes

Connection setup follows the JDBC connection tutorial and Boot 3.5 SQL configuration. JDBC tutorial examples predate Java 17/21, but the connection APIs described here remain applicable. Hibernate mapping, identifier, cache, association and soft-delete behavior above is calibrated to the Hibernate ORM 6.6 user guide. The

transaction and projection points use the Spring Data JPA transaction reference and Spring Data JPA 3.5 projection reference. PostgreSQL-specific isolation and partitioning details should be checked against the deployed version's transaction-isolation and partitioning documentation. SQL query syntax and plans remain database-specific.

Part 17 - More Hibernate and SQL Questions (Q245-Q272)

Baseline: Hibernate 6.x / Jakarta Persistence 3.1 and PostgreSQL where named. Framework and PostgreSQL examples require integration testing on the target stack.

Q245 When does Hibernate flush, and what surprises people

Strong answer: with the default auto mode it flushes at commit and before queries that could be affected by pending changes, so a query can see unsaved changes made in the same transaction.

Reasoning and alternatives: native SQL behaves differently between the native Session API and the JPA EntityManager, so do not assume.

What if/test: persist an entity, run a native query that should see it, and check the result on your stack.

Signal: you can explain why a flush can fire mid-method and trigger constraint errors early.

Q246 Enums and attribute converters

Strong answer: ORDINAL breaks when someone reorders the enum; STRING is safer and readable. An AttributeConverter maps a stable code, such as a one-letter status, to the enum.

Reasoning and alternatives: back it with a database check constraint so bad values cannot be stored by other writers.

What if/test: reorder the enum constants and confirm stored data still reads correctly.

Signal: persistence format is a contract, independent of code order.

Q247 Large objects and BLOB columns

Strong answer: large binary fields load with the entity unless lazily fetched, which usually needs bytecode enhancement. Prefer storing files in object storage and keeping a reference and metadata in the row.

Reasoning and alternatives: if you must keep them in the database, put them in a separate table or entity so list queries never touch them.

What if/test: log SQL for a list query and check whether the blob column is selected.

Signal: keep wide, rarely used columns out of hot queries.

Q248 What is @NaturalId for

Strong answer: Hibernate @NaturalId identifies a business key and enables natural-ID lookup and optional caching in addition to a surrogate primary key. It is immutable by default, but `@NaturalId(mutable = true)` explicitly supports changes and requires synchronization of lookup state. ISBN may be a stable key; email often changes and should not be called inherently immutable.

Enforce uniqueness in the database. Test lookup before and after an allowed change, and understand session/cache effects. The annotation does not replace the entity's primary-key identity or a database constraint.

Hibernate 6.6 natural IDs

Q249 Why not return entities from REST controllers

Strong answer: bidirectional associations cause infinite recursion in JSON, lazy proxies break serialization, and exposing entities leaks fields and invites mass assignment on input.

Reasoning and alternatives: map to DTOs or records for each use case; this also stabilizes your API against schema changes.

What if/test: add a new column and verify the API response does not change.

Signal: separate the persistence model from the API contract.

Q250 Composite keys: @EmbeddedId or @IdClass

Strong answer: both model multi-column keys. @EmbeddedId uses an embeddable key class, which needs equals, hashCode and serializability. @IdClass repeats the fields on the entity.

Reasoning and alternatives: composite keys complicate repositories, joins and foreign keys, so a surrogate key plus a unique constraint is often simpler.

What if/test: load the same row twice in one session and confirm identity behaves.

Signal: prefer surrogate keys unless the natural composite is truly the identity.

Q251 Native queries with paging and sorting in Spring Data

Strong answer: Spring cannot always derive the count query for a native query, so supply countQuery when using Pageable. Sort fields from user input must be validated against an allow-list.

Reasoning and alternatives: map public sort names to known column names instead of passing them straight to the query.

What if/test: send a malicious sort parameter and confirm it is rejected.

Signal: treat sorting as an injection surface.

Q252 equals and hashCode when proxies are involved

Strong answer: a lazy proxy is a subclass, so getClass() == other.getClass() can be false for the same entity. Compare by business key or id, use instanceof carefully, and access state through getters, not fields.

Reasoning and alternatives: if you must reach the real class, unproxy it explicitly.

What if/test: compare a loaded entity with a reference obtained by getReference for the same id.

Signal: you know proxies break naive class checks.

Q253 Configuring multiple data sources with JPA

Strong answer: for separate local JPA transactions, configure each DataSource, entity manager factory, entity package scan and transaction manager, and bind each repository group to the correct factory/manager. Select the manager explicitly where ambiguous. Auto-configuration back-off is conditional on the beans and release, so inspect the effective configuration.

A local transaction manager does not make changes in two databases atomic. JTA/XA can coordinate supported resources when configured; alternatively use explicit distributed-workflow semantics such as an outbox/saga. Test failure after the first database write and state the actual guarantee.

Q254 Routing reads to replicas

Strong answer: a routing data source can pick the primary or a replica by checking whether the current transaction is read-only. Wrap it in a lazy connection proxy so the choice happens after the transaction attributes are known.

Reasoning and alternatives: replication lag breaks read-your-writes, so send reads that must see recent writes to the primary.

What if/test: write then immediately read through a replica under artificial lag.

Signal: mention lag and the lazy proxy.

Q255 What bites teams migrating to Hibernate 6

Strong answer: the move to the Jakarta namespace, stricter query parsing, changes to default sequence naming and allocation, replaced annotations (for example @Where and @Type), and different JSON and type mapping.

Reasoning and alternatives: upgrade stepwise, run the full test suite with SQL logging, and compare generated statements.

What if/test: diff the DDL and the SQL log before and after the upgrade.

Signal: a migration plan driven by evidence, not hope.

Q256 Batch insert pattern

Strong answer: enable JDBC batching, persist in a loop, and every batch size call flush() then clear() so the persistence context does not grow. Use sequence-based ids, because IDENTITY blocks batching.

Reasoning and alternatives: chunk the work into several transactions so one failure does not roll back millions of rows.

What if/test: insert 100,000 rows and compare statement count and heap use with and without clear().

Signal: you control memory, statements and transaction size together.

Q257 How do databases execute joins

Strong answer: nested loop (good when the outer side is small and the inner side is indexed), hash join (equi-joins on large unsorted inputs) and merge join (inputs already sorted on the join key). The optimizer chooses from statistics.

Reasoning and alternatives: a missing index or a bad estimate can flip the plan to a slow choice.

What if/test: read the join type in EXPLAIN and check estimated versus actual rows.

Signal: connect the join algorithm to the data shape.

Q258 Why do query plans change or regress

Strong answer: the planner relies on statistics. Stale or skewed statistics, data growth, or correlated columns make row estimates wrong and produce a worse plan.

Reasoning and alternatives: refresh statistics, add extended statistics for correlated columns, and watch for

parameter-dependent plans with prepared statements.

What if/test: compare estimated and actual rows after a large data load.

Signal: diagnose estimates before touching the query.

Q259 Why did the optimizer ignore my index

Strong answer: common causes are a function on the indexed column, an implicit type conversion (a text column compared with a numeric parameter), a leading wildcard, low selectivity, an OR across columns, stale statistics, or a table small enough that a scan is cheaper.

Reasoning and alternatives: fix the predicate (match types, use an expression index) before forcing hints.

What if/test: compare plans with the correctly typed parameter.

Signal: list several causes, not just "functions on columns".

Q260 Using the database as a work queue with SKIP LOCKED

Strong answer: PostgreSQL `SELECT ... FOR UPDATE SKIP LOCKED` lets workers skip rows locked by peers. Locks belong to the transaction and disappear on commit or rollback. Holding the transaction through short database work can serialize claims; if you commit a claim before a long external task, persist claim state with a lease and define expiry/recovery.

A worker crash or lost acknowledgement can repeat an external effect. Use idempotency and, where required, fencing; a database row lock alone does not protect work after its transaction ends. Test crash-before-commit, crash-after-effect and lease expiry. A broker may be preferable when queue volume or delivery features justify it.

Q261 What are advisory locks for

Strong answer: PostgreSQL advisory locks are application-defined locks on a numeric key, useful for singleton jobs or migrations. Prefer the transaction-scoped form, because session-level locks can leak through pooled connections.

Reasoning and alternatives: a leader-election library or a lock table also works if you need portability.

What if/test: start two instances and confirm only one runs the job.

Signal: mention the pooling pitfall.

Q262 Savepoints and aborted transactions

Strong answer: a PostgreSQL statement error leaves the transaction unable to execute ordinary statements until rollback. A savepoint created before the failure allows rollback to that savepoint. Catching a Java exception alone does not repair database or persistence-context state.

Spring NESTED propagation requires transaction-manager savepoint support. A JDBC manager can support it; JpaTransactionManager does not provide true nested JPA entity-state transactions, and its optional savepoints apply to JDBC work. JPA exceptions may mark the transaction rollback-only. Prefer a fresh appropriate transaction when retrying a failed JPA operation; test the actual manager and driver.

Spring JpaTransactionManager

Q263 Storing money correctly

Strong answer: use NUMERIC/DECIMAL or integer minor units, never floating point. In Java use BigDecimal built from strings, with an explicit scale and rounding mode.

Reasoning and alternatives: store the currency with the amount, and allocate rounding remainders deliberately when splitting.

What if/test: sum 0.1 ten times in double versus decimal.

Signal: explain the representation error, not just "use BigDecimal".

Q264 timestamp versus timestamptz and time zones

Strong answer: in PostgreSQL timestamptz represents an instant, normalized to UTC, while timestamp has no zone. Map to Instant or OffsetDateTime, store UTC, and keep the user's zone separately for scheduling.

Reasoning and alternatives: daylight saving makes recurring local times tricky, so store the zone ID and local time for those.

What if/test: run the app and database in different time zones and compare stored values.

Signal: separate instants from local calendar times.

Q265 JSONB column or relational columns

Strong answer: JSONB suits flexible, sparse or evolving attributes and can be indexed with GIN. Core, frequently filtered and joined fields belong in real columns with constraints and statistics.

Reasoning and alternatives: a hybrid model is common: stable columns plus a JSONB column for extras.

What if/test: query on a JSONB field with and without the index and compare plans.

Signal: justify the schema flexibility against lost constraints.

Q266 Views versus materialized views

Strong answer: a view stores a query; a materialized view stores query results that become stale between refreshes. In PostgreSQL, REFRESH MATERIALIZED VIEW CONCURRENTLY requires an already-populated view and a qualifying UNIQUE index using plain column names, covering all rows (no partial predicate or expression-only index). A generic unique index is not sufficient.

Measure refresh cost and the staleness budget. Summary tables maintained by jobs or events are alternatives with their own consistency and failure semantics.

PostgreSQL materialized-view refresh requirements

Q267 Gaps and islands: longest login streak

```
-- Integer day numbers; duplicate activity on the same day counts once.
WITH days AS (
    SELECT DISTINCT user_id, day_no FROM logins
    WHERE day_no IS NOT NULL
), numbered AS (
    SELECT user_id, day_no,
           day_no - ROW_NUMBER() OVER (
               PARTITION BY user_id ORDER BY day_no
           ) AS grp
    FROM days
)
SELECT user_id, MIN(day_no) AS start_day, MAX(day_no) AS end_day,
       COUNT(*) AS days
FROM numbered
GROUP BY user_id, grp
ORDER BY days DESC, user_id, start_day;
```

Strong answer: consecutive day numbers minus row numbers share a constant difference. Grouping that difference produces streaks. The query returns every streak longest first; use a per-user ranking if the requirement is exactly one longest streak per user. DISTINCT prevents duplicate days from splitting a streak. Test gaps, ties, duplicates, null dates and single-day runs.

Q268 Why constraints belong in the database

Strong answer: primary keys, foreign keys, unique, check and not-null constraints enforce integrity for every writer and under concurrency, where application checks race. Deferrable foreign keys help with circular inserts.

Reasoning and alternatives: index foreign key columns yourself in PostgreSQL, since it does not do so automatically, to speed joins and deletes.

What if/test: try to insert an orphan and a duplicate from two sessions.

Signal: the database is the last line of defense.

Q269 Row-level security for multi-tenancy in PostgreSQL

Strong answer: enable PostgreSQL row-level security and define both visibility and permitted-write policies. For a UUID tenant column, an example predicate is `tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::uuid`. Set a verified tenant with a transaction-local setting before any tenant data access. A missing value should deny access, and pooled connections must not retain the preceding request's tenant.

Use an application role that neither owns the table nor has superuser/BYPASSRLS privileges. FORCE ROW LEVEL SECURITY subjects an owner to policies, but does not constrain superusers or BYPASSRLS roles. An application-set tenant variable assumes trusted application code; it is not protection from a database client able to change that variable. Test SELECT and writes, missing tenant context, connection reuse and privileged paths.

PostgreSQL row security

Q270 How do you find what is blocking a query

Strong answer: in PostgreSQL inspect pg*stat*activity for state, wait events and transaction start time, use pg*blocking*pids, and look for sessions that are idle in transaction. Protect the system with lock*timeout, statement*timeout and an idle-in-transaction timeout.

Reasoning and alternatives: cancel the query first, and terminate the session only if needed.

What if/test: hold a lock in one session and watch the blocker in another.

Signal: diagnose, then add timeouts to prevent recurrence.

Q271 NULL traps in SQL

```
SELECT COUNT(*) AS all_rows, COUNT(score) AS non_null_scores,
       AVG(score) AS avg_score
FROM exam;
SELECT id FROM exam WHERE score = NULL; -- UNKNOWN, not TRUE: no matches
SELECT id FROM exam WHERE score IS NULL;
```

Strong answer: SQL uses three-valued logic. Ordinary comparison with NULL evaluates to UNKNOWN, so WHERE rejects it. COUNT(*) counts rows; COUNT(column), AVG and SUM ignore null inputs, while an all-null SUM/AVG is NULL. Aggregates have individual contracts, so do not generalize this to every aggregate.

Also test NOT IN with a null in its input; NOT EXISTS with the intended correlation is often the clearer anti-join. Unique-constraint treatment of nulls and default null ordering depend on the database and options.

Q272 Backup and restore strategy

Strong answer: combine periodic physical base backups with continuous log archiving for point-in-time recovery, plus logical dumps for portability. Define RPO and RTO first, encrypt backups and store a copy in another region.

Reasoning and alternatives: replication is not a backup, since it replicates mistakes such as accidental deletes.

What if/test: restore to a new instance on a schedule and time it against the RTO.

Signal: an untested backup is not a backup.

Daily SDE-3 Java interview practice sets

Five ElevatePrep sets, 12 scenarios each, dated 7-9 October 2026. These add 60 scenarios to the numbered question bank. The source baseline and fragment limitations are stated within each set.

Daily practice set 1

Day 1 · 7 October 2026 — Foundations under production pressure

Twelve new questions, using your Senior Java Interview Master Guide as the baseline. Assume **Java 21, without preview features**. Framework assumptions are stated where relevant. All scenarios are illustrative.

Practise each spoken answer in 45–90 seconds, then use the follow-ups to test your understanding.

---

1. A customer object is used as a `HashMap` key. After changing its region, lookup with that same object fails. Why?

Say aloud:
“A hash map uses a key’s hash code to locate a bucket, which is a group of candidate entries, and equality to identify the matching key. Equal keys must have equal hash codes. If region participates in those calculations, changing it after insertion breaks the lookup assumptions. The entry remains stored according to its original hash, while the next lookup uses the new hash.

I would define the key using stable identity, such as an immutable customer ID, and keep changing attributes in the value. Synchronizing the map would not solve this identity problem. If mutation is unavoidable, remove the entry before changing the key and then reinsert it. Once mutation has already happened, rebuilding the map from its entries may be needed.”

- **Example:** Insert a key representing `(customerId=42, region="east")`; change region to `"west"`; `get(key)` may return `null` even though iteration still shows the entry.
- **Cost and guarantee:** Expected O(1) lookup with well-distributed hashes and constant-cost equality; occasional resizing costs O(n). Stable keys preserve the lookup contract.
- **Trade-off and failure:** Immutable keys may require separate key objects. Accidentally including changing fields in generated equality recreates the bug.
- **Follow-ups:**
  “Does `ConcurrentHashMap` fix it?” → No; concurrency protection does not repair mutable identity.
  “Can unequal objects share a hash?” → Yes; equality resolves collisions.

---

2. Does `record DeliveryPlan(List<String> stops)` give you an immutable delivery plan?

Say aloud:
“A record is a concise Java data carrier with generated accessors and value-oriented methods. Its fields are final, meaning the references cannot be reassigned after construction. That does not freeze objects those references point to.

If I store the caller’s mutable list directly, the caller can change the plan later. I would make a defensive copy in the constructor: a copy that prevents the caller’s subsequent structural changes from affecting my object. `List.copyOf` provides an unmodifiable list and rejects null elements.

For strings, this is sufficient because strings themselves are immutable. For mutable stop objects, copying the list still shares those objects, so I would also need immutable stop values or appropriate element copies. Immutability has to cover the reachable state that matters to the contract.”

```
record DeliveryPlan(List<String> stops) {
    DeliveryPlan {
        stops = List.copyOf(stops);
    }
}
```

- **Cost and guarantee:** Copying n references is O(n) time and space when a copy is needed. Existing suitable immutable lists may be reused.
- **Trade-off and failure:** Copying costs memory; shallow copying fails when elements remain mutable.
- **Follow-ups:** “Copy on every accessor call?” → Unnecessary here. “Allow null stops?” → Define another explicit representation or validation policy.

Java's exact list guarantees: Java 21 `List` documentation.

---

3. A background thread replaces routing configuration while request threads read it. When is `volatile` enough?

Say aloud:
"`volatile` gives a field visibility and ordering guarantees across threads. It exists because an ordinary shared field does not reliably communicate updates without synchronization. A write to a volatile field happens-before subsequent reads of that field: earlier writes become visible through that publication.

For configuration, I would construct a complete immutable snapshot and assign it to a volatile reference. Each request would read that reference once into a local variable, then use that snapshot throughout its decision. This prevents mixing fields from different configuration versions.

Volatile does not make compound operations atomic. Atomic means an operation acts as one indivisible step relative to competing operations. Two writers that both read, modify, and replace configuration can still overwrite one another's changes. That requires additional coordination."

```
private volatile RoutingConfig current;

// Reader: one snapshot per decision
RoutingConfig snapshot = current;
return routeUsing(snapshot);

// Writer: build fully before publication
current = validatedReplacement;
```

- **Guarantee:** Readers see a safely published snapshot; this does not freeze mutable objects inside it.
- **Cost:** Reference access is constant work; constructing the snapshot depends on its size.
- **Trade-off and failure:** Immutable snapshots simplify reads but allocate on updates. Reading `current` repeatedly can mix versions.
- **Follow-ups:** "Two writers?" → Serialize updates with a lock or use an atomic conditional update. "Does volatile guarantee fairness?" → No.

---

4. An executor has eight core threads and a maximum of 64, yet its queue grows while only eight threads work. What happened?

Say aloud:
"An executor manages worker threads and pending tasks. In `ThreadPoolExecutor`, the queue policy determines when the pool grows. Once the core workers exist, it normally tries to queue new tasks before creating additional workers. With an unbounded queue, queuing keeps succeeding, so the configured maximum does not normally help.

I would first measure task duration and the downstream service's capacity. Then I would choose bounded workers and a bounded queue, plus an explicit rejection policy. Backpressure means slowing or refusing incoming work when capacity is exhausted; it exists to stop overload becoming unlimited memory growth.

A queue absorbs a short burst, but it cannot fix sustained excess demand. I would also enforce deadlines so requests do not wait until their results are already useless."

- **Example:** Eight workers, maximum 64, bounded queue of 100: growth beyond eight becomes possible when the queue fills.
- **Guarantee/cost:** Pending queue occupancy is bounded; completion time is not. More workers consume resources and may overload dependencies.
- **Failure/trade-off:** Running rejected work on the submitting thread slows submissions but can block a latency-sensitive caller.
- **Follow-ups:** “Larger queue?” → More burst absorption, more waiting. “More threads?” → Only if dependencies can sustain them.

Verified behavior: Java 21 `ThreadPoolExecutor`.

---

5. JVM heap usage keeps rising. How do you distinguish a memory leak from normal allocation?

Say aloud:
“The heap holds Java objects. Garbage collection reclaims objects that are no longer reachable from live application state. A memory leak in Java usually means objects remain reachable even though the application no longer needs them.

I would compare memory behavior under equivalent workloads and examine the amount still retained after relevant collection cycles. A sawtooth pattern can be normal: allocation raises usage, collection lowers it. A rising retained baseline is stronger evidence of retention, but it can also reflect a growing legitimate workload.

Next, I would identify which object types are accumulating and inspect their retaining paths—the references keeping them alive. For example, a static map storing every completed request is a concrete explanation. I would fix its ownership or retention policy before attempting garbage-collector tuning.”

- **Example:** A completed-request map grows by 10,000 entries per hour because nothing removes entries.
- **Cost/guarantee:** Retaining n request objects requires space proportional to their combined reachable size. Collection cannot reclaim strongly reachable entries.
- **Trade-off and failure:** Bounded retention reduces memory but may remove useful history. Heap inspection can be expensive; reproduce with representative load where possible.
- **Follow-ups:**
  “High allocation but stable retained heap?” → Investigate temporary-object creation and collection overhead.
  “Process memory grows while heap stays stable?” → Investigate non-heap memory and other process resources.

---

6. In a Spring Boot service, `importAll()` calls its own `@Transactional importOne()`. A checked exception leaves partial data. Explain both risks.

Say aloud:
“A transaction groups database changes so they commit or roll back together. In Spring’s default proxy mode, a proxy is a wrapper around a managed service that applies transaction behavior when calls pass through it.

A method calling another method on the same object bypasses that wrapper. Therefore, the inner annotation does not establish its requested transaction boundary. An existing outer transaction may still apply.

The second issue is rollback policy. Under Spring’s traditional defaults, unchecked exceptions and errors trigger rollback, while checked exceptions do not automatically do so. Configuration can change that.

I would put the intended database operation behind a managed service boundary, choose whether each item or the whole import is atomic, and configure rollback for the relevant checked exception explicitly."

- **Example:** Another service calls `importService.importOne(...)`, annotated with `@Transactional(rollbackFor = ImportException.class)`.
- **Guarantee:** Atomicity covers participating transactional database changes, not an already-sent email.
- **Trade-off/failure:** Whole-import transactions hold resources longer; per-item transactions permit partial completion.
- **Follow-ups:** "Exception swallowed?" → Normal return may allow commit. "Spring 6.2+?" → Global all-exception rollback configuration is available; inspect actual settings.

Source: Spring transaction annotations.

---

7. A page of 50 orders executes 51 database queries when order lines are accessed. How would you fix it without breaking pagination?

Say aloud:
"This is the N-plus-one query problem: one query loads the parent records, then one additional query per parent loads related data. Lazy loading means delaying related-data retrieval until access; it can avoid unnecessary work, but here it creates many database round trips.

I would verify the actual queries, then design retrieval around the response. One option is first paging ordered order IDs, then fetching those orders and their lines in a second query. I would restore the first query's ordering explicitly.

A fetch join retrieves an association alongside its parent. However, fetching a collection multiplies result rows, so combining that with pagination can produce costly in-memory limiting in Hibernate. I would avoid treating a collection fetch join as a universal one-query fix."

- **Example:** Fetch 50 IDs ordered by creation time and ID; fetch associated lines using those IDs.
- **Cost:** Usually two data queries, plus a count query if requested; materialization still scales with returned orders and lines.
- **Trade-off/failure:** Two reads can observe intervening changes; define required consistency.
- **Follow-ups:** "Set everything eager?" → May still cause extra queries. "Huge line collections?" → Bound or separately page them.

Reference: Hibernate 7.0 query guide.

---

8. A client times out while creating a booking and retries. How do you prevent a second booking?

Say aloud:
"A timeout means the client did not receive a result; it does not prove the server failed. Idempotency means repeating the same logical operation does not repeat its intended effect.

I would accept a client-generated operation key, scoped to the authenticated customer and operation type. In the same database transaction, I would record that key, create the booking, and save the outcome. A uniqueness constraint—a database rule rejecting duplicate key values—arbitrates concurrent requests.

I would also store a request fingerprint, a stable representation or digest of the meaningful request fields. Reusing a key with different input should fail rather than return an unrelated result. Once the original transaction commits, a retry can return the saved outcome. This design's guarantee applies to the local transactional work;

remote side effects need their own coordination.”

- **Example:** Customer 17 sends key `book-8f2` twice with identical room and dates; both receive booking 901.
- **Guarantee/cost:** One committed local effect per retained scoped key. Storage grows with retained operations; database lookup cost depends on the index.
- **Trade-off and failure:** Expiring keys saves space but limits the retry window. A separate “check then insert” without database uniqueness races.
- **Follow-ups:**
  “Two requests arrive together?” → Let the database arbitrate; return/retry according to the winning transaction’s outcome.
  “Response lost after commit?” → Return the stored result.

---

9. A Kafka consumer updates a database and crashes before committing its offset. What happens, and what would you design?

Say aloud:
“A Kafka partition is an ordered portion of a topic, and an offset identifies a record’s position within it. A committed consumer offset records where that consumer group should resume.

If the database commit succeeds but the offset commit does not, the record can be processed again. That is the duplicate-processing window associated with at-least-once handling.

For a local database update, I would insert a unique processed-event ID and apply the business change in the same database transaction. A repeated event then produces no second business change. Only after successful database processing would I advance the Kafka offset.

Kafka transactions can coordinate Kafka records and offsets, but do not automatically include an arbitrary external database. I would state the exact system boundary before claiming exactly-once behavior.”

- **Example:** Event `inventory-adjustment-72` is recorded with its stock update; replay encounters the existing event ID.
- **Guarantee:** Duplicate database effects are prevented within the retained-ID scope; handler execution may repeat.
- **Trade-off/failure:** Deduplication costs storage. Committing offsets first risks skipping unfinished work.
- **Follow-ups:** “Parallel processing?” → Advance only past completed earlier records. “Dedup insert fails?” → Roll back appropriately before handling the duplicate.

Reference: Kafka 4.0 delivery semantics.

---

10. A service completes 200 requests per second with average response time of 150 ms. How many requests are in progress, and what does that tell you?

Say aloud:
“Little’s Law relates average work in a system to throughput multiplied by average time in that system. It helps connect observed traffic to concurrency rather than guessing from thread counts.

Here, 200 requests per second multiplied by 0.15 seconds gives 30 requests in the system on average. The measurement boundary matters: if response time includes queue waiting, those 30 include queued requests.

That does not mean 30 worker threads or 30 database connections is the correct configuration. Requests may wait without occupying those particular resources, and averages hide bursts. I would measure how long each scarce resource is actually held, test under representative load, and leave measured headroom. The relationship assumes a stable flow; it is not a promise that an overloaded system's queue will stop growing."

- **Example:** If every request holds one database connection for 40 ms, average connection occupancy is approximately `200 × 0.04 = 8`.
- **Guarantee:** An average relationship under consistent boundaries and stable conditions; no worst-case latency guarantee.
- **Trade-off and failure:** Extra capacity costs money; excessive parallelism can slow the database. Underprovisioning causes queuing.
- **Follow-ups:**
  "Latency doubles at the same throughput?" → Average in-progress work doubles.
  "Can this size the pool alone?" → No; validate variability and saturation experimentally.

---

11. A user is logged in and has the `USER` role. Why is `GET /invoices/123` still potentially unsafe?

Say aloud:
"Authentication establishes who the caller is. Authorization decides what that caller may do. A role can grant access to an invoice endpoint without granting access to every invoice.

I would enforce object-level authorization, meaning a permission decision about the specific requested invoice. In a system serving multiple customer organizations, called tenants, the query should be scoped using the authenticated caller's permitted tenant and any required ownership rules.

The server must verify that scope; a tenant ID supplied by the client is not proof of membership. For an unauthorized invoice, I would return a policy-consistent response that does not unnecessarily reveal whether it exists. I would test access between users and tenants. Unpredictable invoice IDs can reduce guessing, but they do not replace permission checks."

- **Example:** Look up invoice 123 within the caller's verified tenant, then apply any finer permission rule.
- **Guarantee/cost:** Enforces the chosen access policy when applied on every relevant path; query cost depends on indexing.
- **Trade-off/failure:** Central rules improve consistency but must express domain exceptions. Unscoped caches can leak results despite correct queries.
- **Follow-ups:** "Administrator?" → Explicit broader permission. "Update operation?" → Enforce scope in the mutation too.

Reference: OWASP authorization guidance.

---

12. Count contiguous subarrays whose sum equals `k`. Values may be negative. Explain and implement the solution.

Say aloud:
"A subarray is a contiguous section of an array. I would use prefix sums: running totals from the beginning. They let me calculate a section's sum by subtracting the total before it from the total at its end.

If the current total is `s`, I need to know how many earlier totals equal `s minus k`. Each occurrence identifies a different valid starting position, so I store frequencies rather than just remembering whether a total occurred.

I seed total zero with frequency one to represent the position before the array starts. For each value, I update the total, count matching earlier totals, and only then record the current total. That ordering avoids counting an empty subarray when `k` is zero. Unlike a usual shrinking-window approach, this works with negative numbers."

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

static long countSubarrays(int[] values, int k) {
    Objects.requireNonNull(values, "values");

    Map<Long, Long> frequencies = new HashMap<>();
    frequencies.put(0L, 1L);

    long prefix = 0;
    long count = 0;

    for (int value : values) {
        prefix += value;
        count += frequencies.getOrDefault(prefix - k, 0L);
        frequencies.merge(prefix, 1L, Long::sum);
    }
    return count;
}
```

- **Example:** `[1, -1, 1]`, `k = 1` → **3**: the first element, the last element, and the entire array.
- **Complexity:** Expected O(n) time with well-distributed hashing; O(n) additional space.
- **Guarantee/failure:** Handles negatives, repeated totals, and an empty array. Using `int` for totals or the answer risks overflow; `long` handles the bounds of an `int[]`.
- **Trade-off:** Extra memory buys a single-pass solution; enumerating all start/end pairs costs O(n²) time.
- **Follow-ups:**
  "Why not a set?" → Repeated totals represent different valid starts.
  "All zeros, length n?" → `n × (n + 1) / 2` valid subarrays when `k = 0`.
  "Why not sliding window?" → Negative values destroy the monotonic relationship between expanding the window and increasing its sum.

The code is illustrative and has not been executed in this run.

Today's coverage: Core Java identity and immutability; concurrency visibility and executor overload; JVM retention; Spring transactions; JPA/Hibernate query design; microservice idempotency; Kafka replay; capacity estimation; authorization; prefix-sum coding.

Tomorrow's progression: Move from individual guarantees to interacting operations: multi-field concurrency, cancellation, concurrent database updates, pagination during writes, retry budgets, and event ordering, with more coding practice.

Daily practice set 2

Set 2 · 7 October 2026 — Coordinating concurrent work and failures

This is today's second run: **12 new questions**, checked against the earlier set and cumulative ledger. Baseline: your Senior Java Interview Master Guide, using **Java 21 without preview features**.

Practise the "Say aloud" paragraphs in 45–90 seconds, then tackle the follow-ups. All examples are illustrative.

---

1. Writing a report throws an exception, and closing its output stream also throws. Which exception should the caller receive?

Say aloud:
"I would use try-with-resources, Java's language construct for closing resources automatically. A resource is something such as a file or socket whose lifetime needs explicit management; garbage collection does not provide timely closure.

If the body throws and closing also throws, try-with-resources preserves the body's exception as the primary failure. The closing failure becomes a suppressed exception—additional diagnostic information attached to the primary exception. This prevents cleanup from hiding the original problem.

Resources close in reverse declaration order. If acquiring a later resource fails, already acquired resources still close. I would also establish ownership: a method should normally close resources it creates and owns, while a borrowed stream may belong to its caller. Automatic cleanup is useful only when the lifetime boundary is correct."

```
try (var out = Files.newOutputStream(path)) {
    writeReport(out); // Application-specific operation
}
```

- **Example:** Writing throws `IOException("disk full")`; closing also fails. The write exception escapes, with the close exception available through `getSuppressed()`.
- **Guarantee/cost:** Each successfully initialized, non-null resource gets a closing attempt during normal language-level exit. For r resources, there are r attempts; their duration depends on the resource.
- **Trade-off/failure:** A manual `finally` that throws can replace the original failure. Closing a caller-owned stream can break subsequent work. Process termination can prevent cleanup entirely.
- **Follow-ups:** "Only closing fails?" → That closing exception escapes. "Does closing guarantee durable storage?" → No; closure and durability are separate contracts.

Reference: Java 21 try-with-resources rules.

---

2. An in-memory seat allocator uses separate atomic counters for available and reserved seats. Can its totals still become inconsistent?

Say aloud:
"Yes. An atomic counter makes an individual operation indivisible, but it does not combine operations on different counters. Our invariant—the condition that must always remain true—is that available plus reserved equals capacity, with neither count negative.

A reservation contains several steps: check availability, decrease available, and increase reserved. Two callers can both pass the check, or a reader can observe the state between updates.

I would protect validation, both updates, and any combined read using the same lock. A lock allows one participating thread at a time into that protected operation. For a small in-memory allocator, this makes the business rule easy to verify. All access must follow that discipline. The guarantee stops at this Java process; multiple service instances need coordination around shared authoritative state.”

```
synchronized boolean reserve(int seats) {
    if (seats <= 0) throw new IllegalArgumentException("seats");
    if (seats > available) return false;

    available -= seats;
    reserved += seats;
    return true;
}
```

- **Example:** Available is 1; two callers each request 1. The lock permits one success and one rejection.
- **Guarantee/cost:** O(1) state work per request; lock waiting has no fixed upper bound. Combined snapshots must acquire the same lock.
- **Trade-off/failure:** Serialization limits concurrency. Exposing either field through an unsynchronized reader breaks the observation guarantee.
- **Follow-ups:** “Could one count be derived?” → Yes; storing capacity and reserved removes redundant state. “What about many independent shows?” → Separate locks by show can reduce contention, with careful lifecycle management.

---

3. A `CompletableFuture` times out after 200 ms, but the remote operation continues for five seconds. Is that a Java bug?

Say aloud:
“No. A future represents a computation’s eventual result; it is not necessarily the owner of the running computation. `orTimeout` completes the future exceptionally if it has not completed in time. It does not automatically stop the underlying network request.

I would distinguish the response deadline, which limits how long the caller waits, from cancellation, which asks the actual operation to stop. The network client needs appropriate time limits and, where supported, an operation-specific cancellation handle.

Even cancellation cannot prove that a remote side effect did not happen. The server may have completed it before receiving the cancellation. Also, `orTimeout` changes the future it is called on. If several callers share that future, I would apply an individual caller’s timeout to a copy rather than unexpectedly timing out everyone.”

```
CompletableFuture<Quote> callerView =
        sharedWork.copy().orTimeout(200, TimeUnit.MILLISECONDS);
```

- **Example:** The caller receives a timeout while the original quote request continues. Cancelling it requires coordination with the client performing that request.
- **Guarantee/cost:** Bounds the caller’s intended waiting policy, subject to scheduling delays; it does not guarantee termination or rollback. Algorithmic complexity is not the useful measure here.
- **Trade-off/failure:** Continued work wastes resources; cancelling shared work may harm other callers.

- **Follow-ups:** “Does `cancel(true)` interrupt `CompletableFuture` work?” → Its interrupt flag has no effect on processing. “Should capacity be released on timeout?” → Only when the underlying operation actually releases the resource.

Reference: Java 21 `CompletableFuture`.

---

4. A request occasionally runs under the previous customer’s tenant context. How could a thread pool cause this?

Say aloud:
“`ThreadLocal` stores a separate value for each thread. It can make request context available without passing it through every method, but its lifetime follows the thread, not the request.

A thread pool reuses worker threads. If request A stores tenant A and fails before removing it, request B may run on that worker and observe tenant A. This can become an authorization defect, not just incorrect logging. The stored value can also retain objects for longer than intended.

At the outer request boundary, I would set trusted context and remove it in a finally block. For nested temporary context, I would restore the previous value instead. Work submitted to another executor needs deliberate context transfer and cleanup; ordinary thread-local values do not automatically follow the task.”

```
tenantContext.set(verifiedTenant);
try {
    handleRequest();
} finally {
    tenantContext.remove();
}
```

- **Example:** A worker handles tenant A, then a request with no tenant context. Without cleanup, the second request may inherit A accidentally.
- **Guarantee/cost:** Cleanup removes this thread’s stored association. Retained memory otherwise scales with live worker associations and their reachable objects.
- **Trade-off/failure:** Thread-local access simplifies signatures but hides dependencies. Copying mutable context into asynchronous tasks creates additional sharing risks.
- **Follow-ups:** “Why not inheritance between threads?” → Pool workers may predate the request; thread creation is not task submission. “Simpler alternative?” → Pass an immutable context explicitly.

Reference: Java 21 `ThreadLocal`.

---

5. Two buyers read stock quantity 1 and both attempt a purchase. How does a JPA version field help, and where does it stop helping?

Say aloud:
“Optimistic locking detects conflicting updates instead of reserving exclusive access when data is read. It exists to prevent one writer silently overwriting another writer’s changes.

With a JPA version field, each buyer reads both quantity and version. A typical update succeeds only if the stored version still matches the version originally read, and then advances it. If both buyers read version seven, one update can win; the other encounters a conflict.

The losing operation must roll back. If retry is appropriate, it needs a fresh transaction, fresh data, and a new availability check. It must not blindly repeat the old decision.

This protects the versioned entity. It does not automatically enforce a business rule spread across independently updated rows, and direct bulk updates need explicit attention to version handling."

```
-- Illustrative version-check mechanism:
UPDATE stock
SET remaining = 0, version = 8
WHERE id = 42 AND version = 7;
```

- **Example:** Buyer A changes version 7 to 8. Buyer B's version-7 update affects no row; a fresh read reveals no stock.
- **Guarantee/cost:** Detects stale updates to participating versioned entities. Indexed row access is efficient, but contention may cause repeated failed attempts.
- **Trade-off/failure:** Good when conflicts are uncommon; frequent conflicts waste work. Detection may occur when changes are sent to the database or when the transaction commits.
- **Follow-ups:** "Can two different rows violate a shared limit?" → Yes; coordinate around the shared rule. "Does versioning replace quantity validation?" → No.

Reference: Jakarta Persistence 3.2 locking specification.

---

6. A Spring Boot service has 20 database connections. Twenty requests each start a transaction, then call an audit service using `REQUIRES_NEW`. Why might everything stall?

Say aloud:
"Transaction propagation defines how a called operation participates in an existing transaction. `REQUIRES_NEW` starts an independent transaction, suspending the outer one.

With a typical local database transaction manager, the outer transaction keeps its connection while the inner transaction needs another. If all twenty requests already hold the twenty connections, none can acquire a connection for auditing. They wait until a timeout or some other intervention releases resources.

I would first decide whether the audit must commit independently. If it should commit only with the business change, a shared transaction is a better fit. If independent persistence is required, I would bound concurrent outer work and provide capacity for the inner work. I would also inspect database lock dependencies: more connections cannot fix an inner operation waiting on a row locked by its suspended outer transaction."

- **Example:** Twenty outer connections remain occupied; twenty inner audit calls queue for a connection.
- **Guarantee/cost:** Independent transactions can commit separately. With one inner level, a request can simultaneously hold two connections.
- **Trade-off/failure:** An audit record may survive a business rollback. More pool capacity consumes database resources and does not remove lock conflicts.
- **Follow-ups:** "Would one extra connection help?" → It can permit progress under simple assumptions, but is not a universal sizing rule. "Does the outer rollback undo the audit?" → No.

Reference: Spring Framework transaction propagation.

---

7. A descending activity feed duplicates entries between pages when new rows arrive. How would you redesign pagination?

Say aloud:
“Offset pagination skips a number of rows before returning a page. Inserts near the beginning shift those positions, so the next offset can revisit rows already shown.

I would use keyset pagination: continue after the last ordering key seen, rather than after a row count. A cursor carries that boundary. For descending creation time, I would also include a unique ID so rows with identical timestamps have a deterministic order.

The next query requests rows older than the timestamp boundary, or rows with the same timestamp and a smaller ID. An index matching the filters and ordering helps it seek directly to that boundary.

This improves traversal through changing data, but it does not create a frozen dataset. Updated ordering fields, deletions, and changing filters still require an explicit consistency policy.”

```
WHERE tenant_id = :tenant
  AND (
       created_at < :lastTime
       OR (created_at = :lastTime AND id < :lastId)
  )
ORDER BY created_at DESC, id DESC
LIMIT :pageSize;
```

- **Example:** After `(10:00, 105)`, request keys below that pair. A new `(10:01, 120)` does not shift the continuation boundary.
- **Guarantee/cost:** With a suitable index and filters, approximately O(log N + page size) retrieval; verify the query plan. Stable ordering keys prevent position-shift duplicates.
- **Trade-off/failure:** Efficient sequential navigation, but arbitrary “jump to page 500” becomes harder. Mutable ordering keys can still cause skips or repeats.
- **Follow-ups:** “Need an exact export?” → Use a consistent database snapshot or materialized result. “Is an encoded cursor authorization?” → No; independently enforce tenant and filter scope.

---

8. Three layers each allow three total attempts. How can one request produce 27 calls to the deepest dependency?

Say aloud:
“Retries multiply when each layer repeats the work of the layers below it. If three layers each allow three total attempts, one original request can produce three cubed, or twenty-seven, attempts at the deepest dependency.

I would assign retry ownership to the layer that understands whether the operation can safely repeat. An end-to-end deadline limits the total time, including waiting and retries. A retry budget limits additional attempts across traffic, so a struggling dependency does not receive unlimited extra load.

Between eligible attempts, backoff increases the delay, while jitter randomizes it so many clients do not retry together. Neither creates capacity; both only shape demand. I would stop when the deadline or budget is exhausted and avoid retrying permanent validation failures or side effects whose outcome cannot safely be repeated.”

- **Example:** A 900 ms budget allows a 250 ms first attempt, 100 ms delay, and another attempt capped by the remaining time, with response overhead reserved.

- **Guarantee/cost:** With a attempts at d independently retrying layers, worst-case downstream attempts can reach a^d. One owner with limit a avoids that multiplication for a single call chain.
- **Trade-off/failure:** Retries can recover transient failures but increase load and latency. Random delay alone does not cap retries.
- **Follow-ups:** “What does a 10% budget mean?” → At most one extra attempt per ten original requests over a defined window. “Retry a timeout?” → Only when repeating the operation is safe and time remains.

---

9. Kafka events for an order use the same key. Event 12 goes to a retry topic, while event 13 succeeds. Why is the order still wrong?

Say aloud:
“A Kafka partition is an ordered log, but log order is not automatically the order in which business effects complete. Routing related events to one partition helps only if processing preserves that order.

A retry topic is a separate log used to delay failed processing. Moving event twelve there and continuing with thirteen allows thirteen’s effect to happen first. Kafka has not reordered the original partition; the application has changed the processing path.

For strict order, I would prevent later events for that order from taking effect until the failed event is resolved. I could also store the expected business sequence with the order and accept only the next sequence. A gap then triggers bounded waiting or recovery. That choice sacrifices availability for the affected order, so the business must decide whether strict sequencing is necessary.”

- **Example:** `12 = approve order`, `13 = dispatch order`. Applying dispatch first violates the workflow.
- **Guarantee/cost:** Sequential processing preserves effects only within its coordinated scope. Sequence validation can be constant application work plus a database operation; buffering gaps needs bounded storage.
- **Trade-off/failure:** Pausing a whole partition also delays unrelated keys. Per-key coordination permits more concurrency but requires more state and recovery logic.
- **Follow-ups:** “Skip old sequence numbers?” → Only when known to be already applied or safely obsolete. “Increase partition count?” → Key routing may change; plan migration if historical per-key order matters.

Kafka’s log guarantees: Kafka 4.0 design documentation. The retry scenario illustrates an application-level consequence.

---

10. A refresh token is stolen. Why does issuing a replacement token on every refresh help, and what race does it introduce?

Say aloud:
“An access token authorizes API calls. A refresh token obtains replacement access tokens, allowing access tokens to expire quickly without requiring frequent sign-in.

Refresh-token rotation replaces the refresh token after each successful use and invalidates its predecessor. The server retains the relationship between replacements, often called a token family. If an invalidated predecessor appears again, that reuse can reveal theft. The server can revoke the active replacement, forcing fresh authorization.

The transition must be atomic so two simultaneous uses cannot independently create valid successor branches. However, legitimate requests can also race—for example, two browser tabs refreshing together, or a retry after a lost response. I would coordinate refreshes in the client and define recovery explicitly. A broad grace period reduces accidental sign-outs but gives a stolen token more opportunity for reuse.”

- **Example:** `R1 → R2`; later reuse of R1 causes the active family to be revoked.
- **Guarantee/cost:** Rotation detects reuse of an invalidated token; it does not prevent the thief's first successful use. It requires server-side state and coordinated updates.
- **Trade-off/failure:** Strong replay detection can force legitimate users to sign in again after races.
- **Follow-ups:** "Are existing access tokens immediately invalid?" → Not necessarily; local validation may accept them until expiration. "Can rotation identify which caller is the attacker?" → No.

Reference: RFC 9700, refresh-token protection.

---

11. Merge overlapping maintenance intervals without changing the caller's input. What invariant makes the algorithm correct?

Say aloud:
"I would first clarify the endpoint convention. For this problem, intervals are closed: both endpoints belong to the interval, so intervals touching at an endpoint overlap.

I would copy the input and sort by start time. Then I maintain one current merged interval. Because starts are sorted, the next interval either overlaps the current interval or begins after it ends. On overlap, I extend the current end to the larger end. Otherwise, I emit the current interval and start another.

The invariant is that everything already emitted is complete and cannot overlap any future interval. Future starts cannot move backward. This explains why comparing with the current interval is sufficient. Sorting dominates the runtime. Copying the list and using immutable interval values avoids changing the caller's data."

```
// Uses java.util.ArrayList, Comparator, List, and Objects.
record Interval(int start, int end) {
    Interval {
        if (start > end) {
            throw new IllegalArgumentException("start > end");
        }
    }
}

static List<Interval> merge(List<Interval> input) {
    Objects.requireNonNull(input, "input");
    var sorted = new ArrayList<>(input);
    sorted.forEach(x -> Objects.requireNonNull(x, "interval"));
    sorted.sort(Comparator.comparingInt(Interval::start));

    var result = new ArrayList<Interval>();
    if (sorted.isEmpty()) return result;

    Interval current = sorted.get(0);
    for (int i = 1; i < sorted.size(); i++) {
        Interval next = sorted.get(i);
        if (next.start() <= current.end()) {
            current = new Interval(
                    current.start(),
                    Math.max(current.end(), next.end()));
        } else {
            result.add(current);
```

```
            current = next;
        }
    }
    result.add(current);
    return result;
}
```

- **Example:** `[1,4], [4,7], [2,3], [10,12]` → `[1,7], [10,12]`.
- **Complexity/guarantee:** O(n log n) time; O(n) additional space including copy and output. Handles empty input, nesting, duplicates, and negative endpoints.
- **Trade-off/failure:** Sorting a caller-owned list saves a copy but mutates input. Comparing integers by subtraction can overflow; `comparingInt` avoids that.
- **Follow-ups:** “Half-open intervals `[start,end)`?” → Touching endpoints do not overlap; use < if merging only overlaps, and define empty-interval handling. “Input already sorted?” → O(n) scan.

---

12. Given build tasks and prerequisites, return a valid execution order or report a dependency cycle. How would you implement it?

Say aloud:
“I would model this as a directed graph: each task is a vertex, and an edge from prerequisite to dependent means the prerequisite must come first. A topological order is an ordering that respects every such edge.

I would use Kahn’s algorithm. For each task, count its unmet prerequisites; that count is called its indegree. Initially, tasks with zero indegree can run. I put them in a queue, remove one, append it to the result, and reduce the counts of its dependents. A dependent enters the queue when its count reaches zero.

If I process every task, the result is valid. Otherwise, some tasks remain blocked by a cycle. I must initialize every task, including isolated tasks, and be explicit about edge direction. Multiple valid orders are possible.”

```
// Uses java.util.ArrayDeque, ArrayList, List, and Objects.
// Each edge is [prerequisite, dependent]; task IDs are 0..n-1.
static List<Integer> buildOrder(int n, int[][] edges) {
    if (n < 0) throw new IllegalArgumentException("negative n");
    Objects.requireNonNull(edges, "edges");

    List<List<Integer>> dependents = new ArrayList<>(n);
    for (int i = 0; i < n; i++) {
        dependents.add(new ArrayList<>());
    }
    int[] remaining = new int[n];

    for (int[] edge : edges) {
        if (edge == null || edge.length != 2
                || edge[0] < 0 || edge[0] >= n
                || edge[1] < 0 || edge[1] >= n) {
            throw new IllegalArgumentException("invalid edge");
        }
        dependents.get(edge[0]).add(edge[1]);
        remaining[edge[1]]++;
    }
```

```
    var ready = new ArrayDeque<Integer>();
    for (int task = 0; task < n; task++) {
        if (remaining[task] == 0) ready.addLast(task);
    }

    var order = new ArrayList<Integer>(n);
    while (!ready.isEmpty()) {
        int task = ready.removeFirst();
        order.add(task);
        for (int next : dependents.get(task)) {
            if (--remaining[next] == 0) ready.addLast(next);
        }
    }

    if (order.size() != n) {
        throw new IllegalArgumentException("dependency cycle");
    }
    return order;
}
```

- **Example:** Tasks 0–3, edges `0→2`, `1→2`, `2→3` → valid order `[0,1,2,3]`. Adding `3→1` creates a cycle.
- **Complexity/guarantee:** O(V + E) time and space, where V is tasks and E is dependency entries. Duplicate edges are counted and removed consistently.
- **Trade-off/failure:** Detects a cycle but does not identify its exact path. Reversing the edge direction produces the wrong build semantics.
- **Follow-ups:** “Always choose the smallest ready task?” → Use a minimum-priority queue, costing O(E + V log V). “Execute tasks concurrently?” → Release dependents only after prerequisites successfully finish, not when they are merely scheduled.

The code examples are illustrative and were not executed in this run.

Today’s coverage: Resource ownership and exception preservation; multi-field concurrency; cancellation; thread-local lifecycle; JPA optimistic locking; Spring transaction resource use; pagination under writes; retry amplification; Kafka processing order; refresh-token security; interval merging; dependency ordering.

Next day’s progression: Virtual-thread resource limits, JVM CPU diagnosis, cache invalidation races, reliable database-to-Kafka publication, schema evolution, Spring startup validation, and sliding-window/heap coding.

The cumulative ledger now tracks **24 distinct questions across two sets**.

Daily practice set 3

Set 3 · 7 October 2026 · 12 new questions

Checked against the previous **24 questions**. Baseline: your Senior Java Interview Master Guide, using **Java 21 without preview features**, with explicit version differences below.

Practise each **Say aloud** answer in 45–90 seconds, then attempt its follow-ups. All scenarios are illustrative.

---

1. You switch to virtual threads, but 10,000 requests still overwhelm a database with 40 connections. Why?

Say aloud:
“A virtual thread is a lightweight Java thread scheduled by the JVM onto an operating-system thread. It makes waiting for supported blocking operations cheaper, which helps applications handle many concurrent requests.

It does not create database capacity. With forty connections, thousands of requests can still wait while retaining request data and consuming memory.

I would limit database concurrency and bound how much work the service admits. A semaphore—a counter of available permits—can limit entry to a scarce operation, but unlimited callers waiting for permits still create a backlog.

I would also inspect the Java version. Pinning means a virtual thread cannot release its carrier, the operating-system thread executing it. Java 21 can pin during blocking work inside synchronized code. Java 25 has removed that particular limitation, although other pinning situations remain.”

- **Example:** Permit at most 30 concurrent reporting queries, preserving capacity for other work; reject or time-limit excess demand.
- **Guarantee/cost:** At most 30 participating operations hold permits. Waiting requests still consume memory; release permits in `finally`.
- **Trade-off/failure:** Lower concurrency protects the database but may reject traffic. Virtual threads do not accelerate CPU-intensive computation.
- **Follow-ups:** “Pool virtual threads?” → Generally create one per task; limit the scarce resource. “Replace every `synchronized`?” → No; inspect runtime version and measured blocking.

References: Java 21 virtual threads, Java 25 virtual threads.

---

2. Splitting 10.00 into three independently rounded shares produces 9.99. How should Java code preserve the total?

Say aloud:
“`BigDecimal` represents decimal values with configurable precision and rounding. I would construct amounts from decimal text, because constructing from a double can preserve an earlier binary approximation.

Rounding means choosing a representable amount when the exact result needs more digits. It needs a business rule: how many decimal places are allowed, when rounding occurs, and who receives any remainder.

For ten units divided three ways, independently rounding every share to 3.33 loses one cent. I would calculate the base shares, calculate the remainder against the original total, and distribute it deterministically.

For this two-decimal example, the result could be 3.34, 3.33, and 3.33. The invariant is that allocated amounts sum exactly to the original. Currency and allocation rules belong in the contract; two decimal places are not universal.”

```
BigDecimal total = new BigDecimal("10.00");
BigDecimal base = total.divide(
        BigDecimal.valueOf(3), 2, RoundingMode.DOWN);
BigDecimal remainder =
        total.subtract(base.multiply(BigDecimal.valueOf(3)));
// base = 3.33; remainder = 0.01
```

- **Guarantee/cost:** O(n) allocation steps for n recipients; arithmetic cost also depends on digit count.
- **Trade-off/failure:** Deterministic remainder allocation preserves totals but needs a fairness rule. Negative amounts need separately defined rounding behavior.
- **Follow-ups:** “Exact division by three?” → Can throw without rounding. “Round each intermediate result?” → Only if required; repeated rounding changes totals.

Reference: Java 21 `BigDecimal`.

---

3. After `entityManager.merge(detachedOrder)`, you modify `detachedOrder`. Why might the update disappear?

Say aloud:
“A persistence context is the set of entity objects a JPA entity manager currently tracks. A managed entity belongs to that context; a detached entity represents persisted data but is no longer tracked there.

Merge copies state into a managed instance and returns that instance. It does not generally turn the detached argument into the managed object.

If I modify the original afterward, those later changes are not automatically tracked. I should use the returned object within the transaction.

For API updates, I often prefer loading the managed entity and applying an explicit command containing only allowed changes. Merging a partially populated object is dangerous because ordinary null fields may represent actual replacement state, rather than ‘the caller did not provide this field.’ That distinction belongs in the update contract.”

```
Order managed = entityManager.merge(detachedOrder);
managed.setDeliveryNote("Leave at reception");
// Run within an appropriate transaction.
```

- **Guarantee/cost:** Dirty checking—detecting managed-object changes—can persist updates at flush or commit. Database work depends on mappings and loaded state.
- **Trade-off/failure:** Merge supports detached workflows but can copy unintended state across a large object graph.
- **Follow-ups:** “Does merge commit?” → No. “What about unfetched lazy fields?” → JPA requires ignoring those during merge; they differ from explicitly populated nulls.

Reference: Jakarta Persistence 3.2 merge semantics.

---

4. A Spring Boot service starts successfully with a negative timeout and an impossible retry configuration. Where should validation happen?

Say aloud:
“I would validate configuration during startup, before accepting requests. External configuration is deployment-supplied input, so parsing it successfully does not prove it is meaningful.

Configuration binding converts properties into a typed object. In Spring Boot, I would register a configuration-properties class and enable validation using `@Validated`, with a compatible validation implementation available.

Individual constraints handle required values and simple bounds. Cross-field validation checks relationships—for example, whether the intended attempt durations and waiting periods fit within the operation’s total budget.

I would distinguish missing values from deliberate defaults. A required timeout should not silently become zero. I would also report the invalid property clearly without exposing credentials. Failing startup makes a bad deployment visible instead of allowing intermittent request failures."

- **Example:** `attemptTimeout=800 ms`, `attempts=3`, `totalBudget=1 s` conflicts with a policy promising three full attempts.
- **Guarantee/cost:** Checks configuration at initialization; it does not prove a remote dependency is available. Cost scales with validated properties.
- **Trade-off/failure:** Strict validation prevents unsafe operation but can block rollout. Refreshed configuration needs validation again.
- **Follow-ups:** "Nested settings?" → Apply cascading validation with `@Valid`. "Validate a duration?" → Check required presence and positive/range semantics explicitly.

Reference: Spring Boot configuration validation.

---

5. You invalidate a cache after every database update. How can an old value still reappear?

Say aloud:
"In cache-aside, the application checks the cache first, reads the database on a miss, and stores the result. It reduces database reads, but reading and filling the cache are separate operations.

A reader can fetch the old database value, pause, and then resume after a writer commits a new value and invalidates the cache. That reader now installs the old value after invalidation.

Expiration limits how long that installed value remains, but does not remove the race. For stronger protection, I could maintain a generation number, meaning a version of the cache's validity. A reader records the generation before loading and installs its result only if that generation is still current.

The comparison and installation must be atomic. The writer must advance the generation reliably, and the generation must survive deletion of the cached value."

- **Example:** Reader loads price 80 → writer commits 90 and invalidates → reader caches 80.
- **Guarantee/cost:** Generation checks reject fills crossing a completed generation change. They do not make database commit and invalidation atomic.
- **Trade-off/failure:** Expiration is simpler but permits stale reads. Stronger coordination adds metadata, operations, and recovery requirements.
- **Follow-ups:** "Delete twice?" → Reduces some races but is not a proof. "Strict current price required?" → Validate against authoritative data at purchase time.

---

6. An order commits to the database, but publishing its Kafka event fails. How do you avoid losing the event?

Say aloud:
"I would use a transactional outbox: a database table containing events that must be published. It exists to close the failure gap between changing database state and notifying another system.

The order and its outbox event are inserted in the same database transaction. Either both commit or neither does. A separate relay reads committed events and publishes them to Kafka.

The relay must tolerate retries. If publication succeeds and the relay crashes before recording completion, it may publish the same event again. Each event therefore needs a stable identifier, and consumers must handle duplicates.

This guarantees a durable publication obligation alongside the order, not instantaneous delivery. I would monitor the oldest unpublished event and backlog size, and define recovery for events that repeatedly fail."

```
Database transaction: insert order + insert outbox event → commit
Relay: read committed event → publish → record completion
```

- **Guarantee/cost:** One additional durable record per event. Eventual publication requires a recovering relay, available infrastructure, and retained events.
- **Trade-off/failure:** Adds storage and operational work; duplicate delivery remains possible.
- **Follow-ups:** "Publish first?" → Could publish an order that later rolls back. "Relay options?" → Poll the table or use change-data capture, which reads committed database changes.

Reference: Debezium outbox documentation.

---

7. A Kafka event changes `amount_cents` into `amount_micros` while keeping the same Protobuf field number. Why is that dangerous?

Say aloud:
"A schema defines a message's fields and meaning. Protocol Buffers encode fields using numeric identifiers, so keeping the same field number and integer type can allow old software to parse new bytes successfully.

That is binary compatibility, but it is not semantic compatibility: the receiver may interpret the number using the wrong unit. A consumer expecting cents could calculate an amount ten thousand times too large.

I would introduce a new field number, retain the old meaning, and deploy consumers that understand both representations before changing producers. During transition, precedence and consistency rules must be explicit.

I would also test historical events, because Kafka consumers may replay data written before the deployment. Once a field is removed, its number should be reserved so it cannot later acquire a different meaning."

```
int64 amount_cents = 1;          // Existing meaning retained
optional int64 amount_micros = 2; // New representation
```

- **Guarantee/cost:** Additive binary evolution helps mixed versions parse messages; business compatibility still requires agreement.
- **Trade-off/failure:** Dual representations increase payload and validation complexity. Conflicting values need a defined response.
- **Follow-ups:** "Absent versus zero?" → Explicit presence distinguishes them. "Convert through JSON?" → Unknown-field preservation differs; test that path separately.

Reference: Official Protobuf evolution guidance.

---

8. How would you rotate an outbound API credential across 100 service instances without an outage?

Say aloud:
"Credential rotation replaces a secret used to authenticate a service. It limits the lifetime of exposed credentials, but distributed instances do not all change at exactly the same moment.

For routine rotation, I would create a second credential while the provider still accepts the first. I would distribute the new secret through controlled secret storage, update clients, and verify that every instance is using the new version.

Some clients capture credentials when constructed, so updating a file or environment source may not update existing clients. Those clients need safe replacement, with old work allowed to finish where appropriate.

After confirming adoption and accounting for in-flight requests, I would revoke the old credential. I would track credential version identifiers, never secret values. A confirmed compromise may require immediate revocation even if that causes temporary failures."

- **Example:** Provider accepts K1 and K2 → instances adopt K2 → old requests drain → revoke K1.
- **Guarantee/cost:** Overlap supports a gradual transition only if the provider permits concurrent credentials. Work scales with clients and instances.
- **Trade-off/failure:** Longer overlap improves availability but extends exposure. A forgotten instance fails after revocation.
- **Follow-ups:** "Provider allows one key?" → Coordinate cutover or use a supported intermediary. "Rollback after compromise?" → Do not restore the compromised secret.

Reference: OWASP secrets lifecycle guidance.

---

9. Transfers from account A to B and B to A occasionally freeze. CPU usage is low. How do you investigate and prevent this?

Say aloud:
"A deadlock occurs when operations form a waiting cycle with no path to progress. Here, one transfer may hold A's lock while waiting for B, while another holds B's lock while waiting for A.

I would inspect thread stacks and lock ownership to confirm the cycle. Low CPU supports a waiting hypothesis, but is not enough to establish deadlock.

To prevent this two-lock cycle, every transfer should acquire account locks in the same global order, such as ascending immutable account ID. The ordering must be total: distinct accounts cannot tie without another ordering rule. Lock objects must also be shared consistently.

For Java 21, traditional management APIs detect certain platform-thread deadlocks but do not cover virtual threads. A negative detector result therefore cannot rule out every application waiting cycle."

```
A → B: lock smaller ID → lock larger ID → transfer
B → A: lock smaller ID → lock larger ID → transfer
```

- **Guarantee/cost:** Consistent ordering prevents cycles among locks following that order. Waiting time remains unbounded.
- **Trade-off/failure:** Ordering simplifies reasoning but requires discipline across all paths. Other locks can reintroduce cycles.
- **Follow-ups:** "Same account?" → Treat as a defined no-op or validation error. "Does locking make transfer rollback automatic?" → No; validate before mutation and define failure handling.

Reference: Java 21 thread-management coverage.

---

10. An endpoint waits for 20 downstream calls. Each finishes within 200 ms 99% of the time. Will the endpoint meet that threshold 99% of the time?

Say aloud:
“No. Fan-out means one request launches several downstream operations. When every result is required, completion depends on the slowest operation, so slow-tail behavior becomes more visible.

Assuming the calls are independent and start together, the probability that all twenty finish within two hundred milliseconds is 0.99 raised to the twentieth power, about 81.8 percent. Roughly 18.2 percent of requests would wait longer, even before local overhead.

Independence is only a simplifying assumption. Shared infrastructure creates correlated delays, so I would measure the combined endpoint directly.

I would question whether all twenty results are necessary, combine calls where possible, and define which results may be omitted or served from older data. A deadline bounds waiting policy, but returning partial data requires an explicit product contract.”

- **Example:** A dashboard can omit optional recommendations; a financial total may require every component.
- **Guarantee/cost:** Twenty calls create twenty units of downstream work. Parallelism reduces elapsed time relative to serial execution, not total work.
- **Trade-off/failure:** Partial responses improve availability but may mislead users unless clearly represented.
- **Follow-ups:** “Perfectly correlated calls?” → The independence calculation no longer applies. “Duplicate slow calls?” → May reduce latency but increases load and requires safe repeatability.

---

11. Find the longest contiguous sequence containing at most `k` **distinct integers. Explain why your solution is linear.**

Say aloud:
“I would use a sliding window: a contiguous range whose left and right boundaries move through the array. A frequency map records how many times each value occurs inside that range.

For each new right-hand element, I increment its frequency. If the number of distinct values exceeds k, I advance the left boundary, decrementing frequencies and removing values whose frequency reaches zero.

Once the window is valid, I update the best length. Shrinking cannot introduce a new distinct value, so moving left eventually restores the constraint.

Although the code contains a loop inside a loop, each element enters once and leaves at most once. The total number of boundary movements is linear. With expected constant-time map operations, total runtime is expected O(n). I would define zero, negative k, and empty-input behavior explicitly.”

```
// Imports: java.util.HashMap, java.util.Objects
static int longestAtMostKDistinct(int[] values, int k) {
    Objects.requireNonNull(values, "values");
    if (k < 0) throw new IllegalArgumentException("negative k");
    if (k == 0) return 0;

    var counts = new HashMap<Integer, Integer>();
    int left = 0;
    int best = 0;

    for (int right = 0; right < values.length; right++) {
        counts.merge(values[right], 1, Integer::sum);

        while (counts.size() > k) {
            int removed = values[left++];
            int remaining = counts.get(removed) - 1;
            if (remaining == 0) counts.remove(removed);
            else counts.put(removed, remaining);
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

- **Example:** `[4, 4, 7, 9, 7, 7]`, k=2 → **4**, from `[7, 9, 7, 7]`.
- **Complexity/guarantee:** Expected O(n) time; O(min(n, k+1)) map entries, including temporary expansion.
- **Trade-off/failure:** Frequencies require extra memory. A set alone cannot tell whether removing one occurrence eliminates a value.
- **Follow-ups:** “Exactly k distinct?” → Update the answer only when the valid window contains k. “Return the range?” → Save its boundaries when improving the best length.

---

12. You receive integer readings continuously. How can you maintain the kth-largest reading without storing everything?

Say aloud:
“I would maintain a minimum heap containing at most k readings. A heap is a data structure that keeps its smallest element readily accessible while supporting efficient insertion and removal.

The invariant is that it contains the largest k readings seen so far, counting duplicates. Until it reaches size k, I insert everything. After that, a new reading replaces the smallest retained reading only if it is larger.

The heap’s smallest value is then the kth-largest overall: exactly k retained readings are at least that large, and discarded readings cannot improve the retained group.

This uses memory proportional to k rather than the stream length. Before k readings arrive, the answer is unavailable. I would also clarify whether duplicates count, because kth-largest distinct value is a different requirement.”

```
// Imports: java.util.Objects, java.util.PriorityQueue
// Batch wrapper around the same streaming update rule.
static int kthLargest(int[] values, int k) {
    Objects.requireNonNull(values, "values");
    if (k < 1 || k > values.length) {
        throw new IllegalArgumentException("invalid rank");
    }

    var largest = new PriorityQueue<Integer>();

    for (int value : values) {
        if (largest.size() < k) {
            largest.offer(value);
        } else if (value > largest.peek()) {
            largest.poll();
            largest.offer(value);
        }
    }
    return largest.peek();
}
```

- **Example:** `[8, 3, 8, 2, 10]`, `k=3` → **8**; duplicates occupy separate ranks.
- **Complexity/guarantee:** O(n log(k+1)) total time, O(k) space, O(1) answer lookup after enough readings.
- **Trade-off/failure:** Excellent for insertion-only streams. If old readings expire, discarded values may become relevant again.
- **Follow-ups:** “Need all readings sorted?” → This heap does not retain enough information. “Increase k later?” → Cannot recover discarded readings without another data source.

Java operation costs: Java 21 `PriorityQueue`.

Today’s coverage: Virtual-thread capacity; decimal allocation; JPA managed versus detached state; Spring configuration validation; stale cache fills; transactional outbox; Kafka message evolution; credential rotation; deadlock diagnosis; fan-out latency; sliding windows; heap-based selection.

Next day’s progression: Generics variance, class-loader retention, business constraints spanning database rows, JPA relationship ownership, circuit-breaker recovery, Kafka ownership during rebalancing, server-side request forgery, and binary-search/deque coding.

The ledger now records **36 distinct questions across three sets**. Code examples were reviewed but not executed.

Daily practice set 4

Set 4 · 8 October 2026 — Ownership, recovery, and invariants

Twelve new scenarios, following the 36 questions in Sets 1–3. Baseline: **Java 21, no preview features**. Framework references: Jakarta Persistence 3.2, Kafka Java client 4.0, Spring Boot 3.5, PostgreSQL 17, and Resilience4j 2.x circuit-breaker concepts. These are explicit assumptions, not latest-version claims. All scenarios are original illustrations, not personal experience.

Practise each **Say aloud** answer in 45–90 seconds. Java method examples below are extracted and exercised by the repository tests with imports and a class wrapper supplied. Configuration and operational sequences are contextual examples, not complete deployed systems. Stable question IDs are **S04-Q01–S04-Q12**, in the order below.

---

1. Your copy helper accepts `List<Number>`, but a caller has `List<Integer>`. How would you design its type signature safely?

Say aloud:
“Java generics are invariant: a list of integers is not a subtype of a list of numbers. Otherwise code receiving a number list could insert a decimal into an integer list. I would describe how the method uses each argument. The source produces values, so it can contain an unknown subtype of T. The destination consumes those values, so it can accept T or a superclass of T. That gives extends on the source and super on the destination.

The compiler then checks the transfer without casts. This does not make the source immutable or the destination writable at runtime; an unmodifiable destination still rejects additions. I would also specify that the lists must be distinct and not overlapping views, because modifying the destination while iterating an aliased source can fail. Type safety and mutation policy are separate contracts.”

```
static <T> void appendAll(List<? extends T> source, List<? super T> target) {
    Objects.requireNonNull(source, "source");
    Objects.requireNonNull(target, "target");
    if (source == target) throw new IllegalArgumentException("same list");
    // Contract: no overlapping backing storage; target supports add.
    for (T value : source) target.add(value);
}
```

- **Example:** Append integers `[2, 5]` to an empty `ArrayList<Number>`. Adding a `Double` through the source reference remains forbidden.
- **Guarantee/cost:** Compile-time transfer safety; O(n) with constant-cost iteration and amortized constant-cost target additions. No atomic rollback on a failed addition.
- **Trade-off/failure:** Flexible arguments, more complex signatures; raw types can defeat checks. Null elements follow the destination's policy.
- **Follow-ups:** “Read from `? super T`?” → Only `Object` is guaranteed. “Is extends read-only?” → No; operations such as clear may still work.

Reference: Oracle wildcard guidance, whose Java 8 variance rules also apply to Java 21.

---

2. A plugin host grows in metaspace after every plugin reload. Why might deleting the plugin reference not help?

Say aloud:
“A class loader is the object that defines classes, and metaspace is HotSpot's native-memory area for class metadata. Replacing a plugin field does not necessarily make its loader unreachable. A host-owned listener registry might still reference a plugin callback, which keeps plugin objects and their defining loader alive.

I would compare repeated reloads under a controlled workload and inspect reference paths from long-lived roots to old loaders. I would also inspect plugin-created threads, context class loaders, and scheduled callbacks. The fix is an explicit unload lifecycle: stop new plugin work, wait for active work to finish, unregister callbacks, close owned resources, then release references.

Increasing the metaspace limit only postpones failure if old generations remain reachable. Conversely, rising usage alone does not prove a leak; generated classes or legitimate new plugins can explain growth. I need evidence of obsolete generations retained.”

- **Example:** `hostRegistry → oldPluginListener → oldPluginClass → oldLoader` remains after the current plugin field is replaced.
- **Guarantee/cost:** Loader reclaimability is a prerequisite for class unloading; prompt unloading is not guaranteed. Investigative cost depends on dump size and retained graph, not a useful fixed Big-O for the incident.
- **Trade-off/failure:** Waiting for active callbacks makes unload slower but avoids executing closed plugin state. A weak registry alone does not stop threads or close resources.
- **Follow-ups:** “Call GC?” → It cannot collect reachable loaders and is no lifecycle contract. “Static fields always leak?” → An unreachable loader and its internal cycles can be collected.

Reference: Java 21 class unloading rules.

---

3. Two dispatchers each see two active couriers, then independently take one offline. Both rows have versions. Can the depot still end with no courier?

Say aloud:
“Yes. Write skew is when transactions read a shared condition but change different rows, allowing their combined result to break that condition. Each version check can succeed because neither transaction updates the other's courier row. Row-level optimistic locking does not automatically enforce a rule spanning several rows.

For PostgreSQL 17, repeatable read can still allow this anomaly. One option is serializable isolation, which makes committed outcomes equivalent to some serial execution and may abort a conflicting transaction. The application must retry the entire transaction with fresh reads and a bounded policy. Another option is a shared depot guard row: under read committed, every operation changing membership locks that row before reading the active count and updating couriers.

The guard works only if all writers participate. I would include administrative tools and bulk jobs in that contract, and avoid irreversible external work inside a transaction that may retry.”

- **Example:** A reads `{A,B}` and disables A; B reads `{A,B}` and disables B. Their distinct row versions do not conflict.
- **Guarantee/cost:** A serializable solution must include all relevant invariant-changing transactions. The guard serializes changes per depot, with contention proportional to that depot's traffic.
- **Trade-off/failure:** Serializable transactions can abort; guard locking reduces throughput. Locking only the row being disabled leaves the anomaly.

- **Follow-ups:** “Retry just UPDATE?” → No, repeat the decision and reads. “Empty depot?” → Define creation and deletion rules, and keep the guard stable.

Reference: PostgreSQL 17 transaction isolation.

---

4. A managed project's `tasks` list includes a new task, but the task's project foreign key is absent. What JPA relationship rule did the code miss?

Say aloud:
“In a bidirectional relationship, the owning side is the side whose reference controls the stored relationship. With a project collection mapped by the task's project field, the task's many-to-one field owns the foreign key. Adding only to the inverse project collection changes the Java view without establishing that owning reference.

I would use a domain operation that sets task.project and adds the task to project.tasks, maintaining both directions. I would separately ensure the new task is persisted, either explicitly or through an intentional persist cascade. Cascade means propagating a persistence operation; it does not choose relationship ownership.

I would flush to send pending changes to the database, clear the persistence context to discard the managed-object cache, and read again. The assertion then checks stored state rather than the in-memory list. Moving or removing tasks needs a deliberate contract, especially if orphan removal is configured to delete children.”

- **Example:** Mapping: `Project.tasks @OneToMany(mappedBy="project")`; `Task.project @ManyToOne`. Inside a transaction, set both references and persist the new task if cascade does not do so.
- **Guarantee/cost:** Correct owning references determine relationship persistence. SQL count depends on mapping, identifiers, batching, and flush behavior.
- **Trade-off/failure:** A helper centralizes consistency but must define duplicate additions and reparenting. Nulling a mandatory foreign key can fail at flush.
- **Follow-ups:** “Does `mappedBy` name a column?” → It names the owning Java attribute. “Cascade REMOVE everywhere?” → Dangerous for shared entities; model lifecycle ownership first.

Reference: Jakarta Persistence 3.2 relationship ownership and synchronization.

---

5. A failed dependency recovers, but every replica sends a burst when its circuit breaker starts admitting traffic again. How would you control recovery?

Say aloud:
“A circuit breaker is a state machine that stops admitting calls after observed failures or slow calls. Its open state rejects calls; half-open permits a limited sample to test recovery. I would distinguish those probes from normal traffic and bound both their count and duration.

A breaker usually belongs to one process, so three allowed probes across a hundred replicas can still mean three hundred simultaneous calls. I would combine staggered recovery with a bulkhead, meaning a concurrency limit protecting downstream capacity, and suitable transport deadlines. The breaker alone does not limit concurrency in its closed state.

I would classify failures carefully: an invalid customer request should not necessarily imply a broken dependency. Recovery metrics should show admitted probes, failures, rejection rates, and downstream saturation. A successful probe is evidence for recovery, not proof the dependency can immediately sustain the previous peak.”

- **Example:** 80 replicas × 2 simultaneous probes permits up to 160 probes; a per-process setting is not a fleet-wide quota.

- **Guarantee/cost:** Constant-size admission decisions can be cheap, but state/window maintenance is implementation-specific. A breaker does not cancel already-running calls or guarantee availability.
- **Trade-off/failure:** Slow reopening protects the dependency but delays service recovery. Fallback responses need an explicit freshness or incompleteness contract.
- **Follow-ups:** “Window size equals concurrency?” → No. “Probe hangs?” → Bound call duration and half-open waiting; check library configuration.

Reference: Resilience4j circuit-breaker states and limits. Check exact property support in the deployed 2.x version.

---

6. A Kafka partition is reassigned while an asynchronous worker still processes its old records. How do you prevent stale completion from advancing progress?

Say aloud:
“Rebalancing changes which consumer owns a partition. A worker dispatched earlier can finish after ownership changes, so I would attach an assignment generation to dispatched work. A generation is a local identifier for the current ownership period. The consumer thread accepts completion only if that identifier is still current.

For each partition I track the highest contiguous completed position, not just the largest completed offset. If offset forty-two finishes before forty-one, committing forty-three would skip unfinished work after a crash. On revocation I stop dispatch for the affected partitions, bound any drain, and commit only safe progress while ownership permits. On lost partitions I assume ownership is already gone.

The local generation prevents stale bookkeeping, not stale database writes. External effects still need deduplication or storage-enforced fencing, where the destination rejects writes from superseded owners. Consumer API calls stay on the consumer's owning thread.”

- **Example:** Offset 41 pending, 42 complete: next safe offset remains 41. Completion tagged generation 8 is ignored after reassignment to generation 9.
- **Guarantee/cost:** Tracking out-of-order completions takes O(w) space for w outstanding records. Bound w and paused-partition buffering; keep polling within client liveness requirements.
- **Trade-off/failure:** Draining can delay rebalancing. A local generation must not be mistaken for a globally authoritative storage fence.
- **Follow-ups:** “Lost versus revoked?” → Lost may mean another member already owns it; do not rely on a final commit. “Exactly once externally?” → Requires an external atomicity/idempotency design.

Reference: Kafka 4.0 rebalance callbacks.

---

7. A document-preview API downloads a user-provided URL. Is checking that the string starts with `https://` sufficient?

Say aloud:
“No. Server-side request forgery means an attacker makes the server access a destination the attacker should not reach directly. HTTPS protects a connection; it does not authorize its destination. An apparently valid URL could target an internal administration service or redirect there.

I would prefer selecting known document providers instead of accepting arbitrary URLs. Where arbitrary fetches are required, I would isolate the fetcher, limit outbound network access, parse URLs with one consistent parser, and restrict schemes, ports, and destinations. DNS translates names to addresses; DNS rebinding changes those answers between validation and connection. The actual connected address must satisfy policy, including IPv4 and IPv6 handling, while preserving proper TLS hostname checks.

I would disable redirects or revalidate every hop. I would also cap bytes and time, and avoid forwarding application credentials. Input checks and network restrictions should reinforce each other."

- **Example:** A public image endpoint returns a redirect to a link-local metadata address. The second destination must not inherit trust from the first.
- **Guarantee/cost:** Policy applies to each connection and redirect, not merely the original text. DNS/network latency dominates; bytes and hop limits bound work.
- **Trade-off/failure:** Strict allowlists reduce supported providers. A public address can still host hostile or oversized content; destination safety does not make a document safe to render.
- **Follow-ups:** "Hostname suffix match?" → Parse and compare canonical domain boundaries, not substrings. "Resolve once, connect by hostname?" → A second resolution can defeat validation.

Reference: OWASP SSRF prevention guidance.

---

8. `computeIfAbsent` performs a slow remote catalog lookup. Why can a thread-safe map still suffer stalls?

Say aloud:
"Thread safety protects the map's structural and operation contracts; it does not make a slow mapping function harmless. ConcurrentHashMap computes an absent mapping atomically, and some other updates can wait while that computation runs. A network lookup can therefore turn a small map operation into a long contention point.

I would keep the atomic step short. If duplicate remote reads are acceptable, load outside the map and publish with putIfAbsent. If callers must share one load, I would atomically install a lightweight future placeholder, and only the winner would schedule the lookup on a bounded executor outside the map operation. That is single-flight loading: concurrent callers share one in-progress operation.

I would complete the placeholder on every terminal path, including scheduling rejection, and remove failed entries conditionally using the same placeholder identity. Otherwise an old failure can remove a newer successful entry. Capacity, expiration, and load deadlines remain separate policies."

- **Example:** `putIfAbsent(key, mine)` chooses one owner; on failure `remove(key, mine)` avoids deleting a replacement. This is a design sequence, not a complete cache implementation.
- **Guarantee/cost:** Expected constant-time map lookup under suitable hashing; remote latency and contention dominate loading. Null results and exceptions do not establish a `computeIfAbsent` value.
- **Trade-off/failure:** Shared loads reduce duplicate work but couple callers. Do not let one caller's cancellation cancel everyone's load; bound retained placeholders.
- **Follow-ups:** "Exactly once forever?" → No; removal/failure permits another load. "Does the map protect mutable values?" → No, values need their own state contract.

Reference: Java 21 ConcurrentHashMap atomic computations.

---

9. During a rolling release, requests are cut off even though Spring Boot graceful shutdown is configured. What would you investigate?

Say aloud:
"Graceful shutdown means stopping admission and giving active work a bounded opportunity to finish. It depends on the entire termination sequence, not just one application property. I would confirm that the process receives a normal termination signal, that traffic routing stops selecting the instance, and that the platform waits long enough before force-killing it.

For Spring Boot 3.5, I would explicitly configure graceful server shutdown and inspect the lifecycle timeout per phase. That timeout is not necessarily the total process shutdown budget. I would measure routing propagation, request drain time, and other lifecycle phases when choosing the platform allowance.

I would also distinguish HTTP requests from detached background tasks: a request returning does not prove its asynchronous side effect finished. Those tasks need their own shutdown ownership and durable recovery. I would test termination under representative long requests and persistent connections, since embedded server behavior and upstream routing affect what new traffic observes."

```
# Spring Boot 3.5 contextual configuration; size budgets from measurements.
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=25s
```

- **Example:** A platform force-kills after 20 seconds while an admitted request needs 23 seconds. A 25-second application phase budget cannot override that kill.
- **Guarantee/cost:** Bounded draining is best effort, not guaranteed completion after crashes. Extra deployment time and temporarily overlapping replicas cost capacity.
- **Trade-off/failure:** Longer grace periods slow releases; unbounded jobs cannot be drained safely forever. Readiness routing and active connection handling are distinct.
- **Follow-ups:** "SIGKILL?" → No cleanup opportunity. "Client saw a timeout?" → The effect may still have committed; use the operation's recovery contract.

Reference: Spring Boot 3.5 graceful shutdown.

---

10. A worker's lease expires during a long pause; a replacement starts, then the old worker resumes. Why is a lease check before writing unsafe?

Say aloud:
"A lease is permission valid for a limited period. Checking it and then writing leaves a race: the worker can pause between those actions, and the lease can expire before its write arrives. A fencing token is an increasing ownership number that the protected destination uses to reject obsolete owners.

I would require lease acquisition to issue a higher token and the destination to atomically compare the token with its highest accepted ownership number while applying a write. Once token eleven has been accepted, a delayed write carrying ten must fail. Merely including the number in a log does nothing.

This guarantee has a boundary: a lower token can still be accepted before the destination has learned a higher one. If the requirement is immediate rejection at lease expiry, the destination must enforce authoritative lease validity too. Repeated requests from the current owner separately need operation IDs or sequencing; fencing is not deduplication."

- **Example:** Destination accepts owner 11 and records its token atomically; owner 10's late update is rejected. Equal-token retries need their own rules.
- **Guarantee/cost:** Protects a destination that enforces a monotonic fence; requires atomic comparison plus mutation. Cross-resource enforcement is a separate distributed consistency problem.
- **Trade-off/failure:** Requires storage cooperation and durable token state. Local clock checks or a random owner UUID alone do not provide ordering.
- **Follow-ups:** "Counter resets?" → Can resurrect stale ownership; preserve monotonicity across recovery. "Object store cannot compare tokens?" → Use a supported conditional-write/version mechanism or redesign ownership.

This is a conceptual protocol, not a claim about a particular lock provider. It extends Q6's external-effect boundary without treating Kafka assignment generations as global fencing tokens.

---

11. Given ordered nonnegative parcel weights and at most `days` shipments, find the minimum daily capacity without reordering or splitting parcels.

Say aloud:
“I would binary-search the answer rather than a position in the input. A capacity is feasible if a greedy scan can ship all parcels within the allowed number of days. The predicate is monotonic: if a capacity works, a larger capacity also works, because the same grouping remains valid.

The lower bound is the heaviest parcel, and the upper bound is the sum, which ships everything in one day. For each midpoint, I keep adding parcels to the current day until the next would exceed capacity, then start another day. Greedily taking the longest legal prefix cannot require more days than cutting that prefix earlier.

A feasible midpoint moves the upper bound down; an infeasible one raises the lower bound. I use long for sums and capacity, reject negative weights and nonpositive days, and define empty input as requiring zero capacity. The array remains unchanged.”

```
static long minimumCapacity(int[] weights, int days) {
    Objects.requireNonNull(weights, "weights");
    if (days < 1) throw new IllegalArgumentException("days must be positive");
    long low = 0, high = 0;
    for (int weight : weights) {
        if (weight < 0) throw new IllegalArgumentException("negative weight");
        low = Math.max(low, weight);
        high += weight;
    }
    while (low < high) {
        long mid = low + (high - low) / 2;
        int used = 1;
        long load = 0;
        for (int weight : weights) {
            if (load + weight > mid) {
                used++;
                load = 0;
            }
            load += weight;
        }
        if (used <= days) high = mid;
        else low = mid + 1;
    }
    return low;
}
```

- **Example:** `[4, 2, 5, 3]`, two days → **8**, using `[4,2]` and `[5,3]`. Capacity 7 needs three days.
- **Guarantee/cost:** O(n log(S−M+2)) time, O(1) auxiliary space, with sum S and maximum M (both zero for empty input). Nonnegative int-array sums fit long.
- **Trade-off/failure:** Multiple scans exchange runtime for constant space. Negative weights invalidate the greedy reasoning; arbitrary reordering changes the problem.

- **Follow-ups:** “Exactly days?” → Clarify empty shipments and parcel count. “Return grouping?” → Run a final greedy pass at the computed capacity.

---

12. Produce the maximum in every width-k window of an integer array in linear time, including duplicates and negative values.

Say aloud:
“I would use a monotonic deque, a queue accessible at both ends whose stored candidate values stay in decreasing order. It stores indices so I can expire entries that have left the window. Before adding a new index, I remove older candidates from the back whose values are no larger than the new value.

Those removed candidates can never win again: the new value is at least as large and expires later. The front is therefore the largest remaining value in the current window. Equal values can discard the older index safely because the newer one lasts longer.

Each index is appended once and removed at most once, either by expiration or domination, so all nested-loop work totals linear time. I would validate that k is between one and the input length, avoid mutating the array, and distinguish auxiliary deque space from the returned output.”

```
static int[] windowMaxima(int[] values, int k) {
    Objects.requireNonNull(values, "values");
    if (k < 1 || k > values.length) throw new IllegalArgumentException("invalid window");
    var candidates = new ArrayDeque<Integer>();
    int[] result = new int[values.length - k + 1];
    for (int right = 0; right < values.length; right++) {
        while (!candidates.isEmpty() && candidates.peekFirst() <= right - k) {
            candidates.removeFirst();
        }
        while (!candidates.isEmpty() && values[candidates.peekLast()] <= values[right]) {
            candidates.removeLast();
        }
        candidates.addLast(right);
        if (right >= k - 1) result[right - k + 1] = values[candidates.peekFirst()];
    }
    return result;
}
```

- **Example:** `[5,5,-2,4,1]`, k=3 → `[5,5,4]`. At the last window both fives have expired.
- **Guarantee/cost:** O(n) amortized total time; O(k) auxiliary storage and O(n−k+1) output. Individual insertions can remove several entries.
- **Trade-off/failure:** The deque deliberately discards values, so it cannot answer arbitrary later range queries. Storing only values makes expiry with duplicates harder to get right.
- **Follow-ups:** “k=1?” → Every element is its own maximum. “Window minimum?” → Reverse the domination comparison. “Empty input?” → No valid positive k under this contract.

Today's coverage: S04-Q01 generics variance; Q02 class-loader lifecycle; Q03 cross-row write skew; Q04 JPA relationship ownership; Q05 circuit-breaker recovery; Q06 Kafka assignment ownership; Q07 SSRF prevention; Q08 atomic map loading; Q09 graceful shutdown; Q10 destination-enforced fencing; Q11 binary search on capacity; Q12 monotonic deque maxima.

Next set will build on: safe schema migrations during mixed-version deployments, read replicas and read-your-writes behavior, Spring transaction event boundaries, concurrent queue publication, JVM native memory outside metaspace, Kafka tombstones and compaction, secure file processing, and graph shortest paths. Continue with Set 5 even if another run happens on 8 October; do not repeat the existing 48 scenarios.

Daily practice set 5

Set 5 · 9 October 2026 — Safe transitions and shortest paths

Twelve new scenarios following the 48 questions in Sets 1–4. Baseline: **Java 21 without preview features**, HotSpot 21, Spring Framework 6.2 (as used with Spring Boot 3.5), Jakarta Persistence 3.2, PostgreSQL 17, and Kafka 4.0. These are reference versions, not claims about the latest releases. Examples are original illustrations, not personal experience.

Practise each **Say aloud** answer in 45–90 seconds. Stable IDs: **S05-Q01–S05-Q12**, in order. The two Java coding examples are methods and nested records compiled with a supplied `java.util.*` import and class wrapper. Other snippets describe protocols or deliberately faulty examples; they are not complete services.

---

1. A producer puts a mutable job into a blocking queue, then changes its payload. What can the consumer safely assume?

Say aloud: “A happens-before relationship is a memory-ordering guarantee: earlier writes become visible to the receiving thread through a specified synchronization action. A BlockingQueue provides that guarantee for actions before the object is placed in the queue and actions after another thread accesses or removes that element. So building the payload before enqueueing safely publishes those earlier writes.

It does not freeze the object. If the producer changes the payload after enqueueing while the consumer reads it, the queue does not order that later mutation. I would transfer ownership and prohibit further producer mutation, or enqueue an immutable snapshot, including copies of mutable nested values. The choice depends on copying cost and whether ownership is enforceable.

I would also define bounded admission and interruption handling. Thread-safe transport solves publication; payload ownership and overload remain application responsibilities.”

- **Example:** `job.amount = 7; queue.put(job); job.amount = 9;` gives no queue-based guarantee that the consumer observes the later value. Prefer a completed immutable job before `put`.
- **Guarantee/cost:** Publication covers pre-enqueue actions. A snapshot of n payload elements costs O(n); queue operation cost depends on implementation and contention.
- **Trade-off/failure:** Ownership transfer avoids copies but requires every alias to cooperate. A shallow copy leaves nested mutable state shared.
- **Follow-ups:** “Does put close the queue?” → No built-in shutdown protocol. “What if interrupted?” → Propagate or restore interruption according to the caller contract; do not silently lose the job.

Reference: Java 21 BlockingQueue contract.

---

2. A container is killed for memory use while the Java heap stays below its limit. What would you investigate beyond heap leaks?

Say aloud: “The heap is only part of a Java process's memory. Native memory is allocated outside that heap, including thread stacks, class metadata, compiled code and native library allocations. Direct buffers also use memory outside the ordinary object heap. A container limit applies more broadly than the maximum heap setting, so a healthy heap does not rule out memory exhaustion.

I would first confirm the container's termination reason and compare its memory accounting with process resident memory, thread count and direct-buffer metrics. On HotSpot I would use Native Memory Tracking, enabled at startup, and compare a baseline with later summaries. It tracks JVM categories but does not explain every third-party native allocation.

The fix follows the growing category: bound threads, release owned native resources or reduce retained buffers. I would budget native headroom before changing the heap ceiling.”

- **Example:** A 2 GiB container with a 1.5 GiB heap budget can still exceed its limit through stacks, buffers and other accounted memory; the remaining 0.5 GiB is not automatically adequate.
- **Guarantee/cost:** NMT adds overhead and is diagnostic evidence, not a complete process accounting ledger. Reserved virtual address space is different from committed or resident memory.
- **Trade-off/failure:** A larger heap can reduce available native headroom. Stable NMT with rising resident memory calls for OS/native investigation, not an automatic heap-leak diagnosis.
- **Follow-ups:** “Enable NMT after the incident?” → Plan startup enablement; it cannot be started dynamically if disabled. “Heap dump sufficient?” → It may expose owners, but not every native allocation.

Reference: HotSpot 21 diagnostic tools and NMT.

---

3. You must replace `customer_name` with `display_name` while old and new service replicas overlap. Why is a direct column rename risky?

Say aloud: “A rolling deployment runs several application versions at once. Renaming a column immediately breaks old SQL, even if the new application works perfectly. I would use expand and contract: first add a compatible representation, migrate readers and writers, and remove the obsolete representation only after its users are gone.

During overlap I would choose one authoritative value and a synchronization rule covering every writer. For example, keep the old column authoritative while a database trigger copies its value to the new column. Backfill historical rows in bounded batches with concurrency-safe conditions, verify convergence, then switch readers. Only after old writers are retired can a later release change write authority.

Rollback compatibility, jobs and reporting queries belong in the plan. Backfill completion alone does not prove safety when old replicas can still change data.”

- **Example:** Add nullable `display_name`; synchronize old-column writes; backfill; compare values; roll out new readers; retire all old clients; later switch writes and remove the old column.
- **Guarantee/cost:** An application protocol, not an automatic ALTER TABLE guarantee. Backfill is O(rows) data work and generates WAL, PostgreSQL's write-ahead log; DDL can acquire disruptive locks.
- **Trade-off/failure:** Extra schema and synchronization cost buy rollback time. Naive dual writes in only new replicas miss updates from old replicas; a stale backfill can overwrite newer data.
- **Follow-ups:** “Add NOT NULL immediately?” → Only after existing rows and every active writer satisfy it; plan validation/locking. “When drop the column?” → After observed non-use and the agreed rollback window.

Reference: PostgreSQL 17 ALTER TABLE behavior; the rollout protocol above is a design proposal.

---

4. A successful profile update is followed by an old value on refresh. How would you provide read-your-writes with database replicas?

Say aloud: “Read-your-writes means a client sees its own completed changes on subsequent reads. An asynchronous replica may not have replayed a successful primary commit yet, so load-balancing the next read can violate that expectation without losing the write.

The simplest targeted solution is to route the dependent read to the current primary, using a fresh transaction snapshot. If we need replica reads, a more involved protocol returns a replication progress marker that is at least as new as the commit. The chosen replica must replay through that marker before a fresh read, with a bounded wait and primary fallback.

A fixed delay is a latency guess, not a guarantee. Failover, cached responses and old transaction snapshots also matter. I would scope the guarantee explicitly rather than promise that all readers instantly see all writes.”

- **Example:** Update returns version 42; the next profile read goes to the primary. A replica-only alternative waits for verified replay progress, not merely receipt of WAL.
- **Guarantee/cost:** Primary reads trade replica capacity for simpler session consistency. PostgreSQL synchronous replication with `remote_apply` waits for replay on the configured synchronous standby set, not every arbitrary replica.
- **Trade-off/failure:** Marker routing needs topology/failover handling and deadlines. Old snapshots or stale caches can defeat a correct routing decision.
- **Follow-ups:** “Stick to primary for five seconds?” → Useful heuristic unless lag is strictly bounded. “Primary fails?” → State whether acknowledged writes survive the configured replication policy.

Reference: PostgreSQL 17 standby and synchronous replication.

---

5. An AFTER_COMMIT Spring event listener changes an entity, but the change does not persist. Why, and is the event durable?

Say aloud: “An AFTER*COMMIT listener runs after the publishing transaction has committed. Spring may still expose the old transaction's resources, which makes data access appear possible, but there is no further commit of that transaction to save new changes. I would put required business updates inside the original transaction, or invoke a separate proxied service with REQUIRES*NEW when a genuinely independent post-commit database write is appropriate.

That new transaction can fail after the original work succeeds. Also, an in-process event is not a durable task: a process crash after commit can prevent the listener from completing. If eventual execution is required, persist the work in the original transaction and process it with recovery.

I would distinguish timing from reliability. AFTER_COMMIT describes when a callback runs; it does not promise delivery, retries or atomicity with later effects.”

- **Example:** An order commits; its listener assigns an audit entity field. Use an explicit independent transaction for optional audit persistence, or persist mandatory audit/work alongside the order.
- **Guarantee/cost:** Default phase is AFTER_COMMIT; without a transaction the listener is skipped unless fallback execution is enabled. Independent work adds another commit and connection usage.
- **Trade-off/failure:** Separate commits allow partial success. Retried callbacks need duplicate protection; enabling fallback changes the no-transaction case rather than creating a transaction.

- **Follow-ups:** “Listener throws?” → Original commit cannot be rolled back. “Async listener?” → Does not inherit a thread-bound transaction; define its own boundary and failure handling.

Reference: Spring Framework 6.2 TransactionalEventListener contract.

---

6. A JPQL bulk update changes thousands of rows, but managed objects still show old values. What contract did you bypass?

Say aloud: “The persistence context is JPA's collection of managed entity instances and their tracked state. JPQL is the Jakarta Persistence query language. A bulk JPQL update operates directly on database rows rather than changing each managed object. JPA does not automatically synchronize those existing instances with the bulk result, and bulk operations bypass ordinary optimistic version checks.

I would isolate the bulk operation in a deliberate transaction boundary. If managed changes must be retained, flush them—synchronize pending changes to the database—before the bulk statement, then clear the context and reload what is needed. Clearing detaches all managed entities, so doing it casually can discard unflushed work and invalidate caller assumptions.

If the operation needs per-entity lifecycle behavior or conflict detection, I would use entity updates or explicitly design predicates and version changes. Bulk throughput is valuable only if the domain semantics remain correct.”

- **Example:** Load account A as ACTIVE; bulk-set matching accounts to SUSPENDED; A remains stale in memory until refreshed or detached and reloaded.
- **Guarantee/cost:** One bulk statement avoids per-entity materialization, but database work still depends on matched rows and indexes. Persistence-context synchronization must be handled explicitly.
- **Trade-off/failure:** Flush/clear affects more than the target rows. A later stale write can undermine the bulk result if version handling is absent.
- **Follow-ups:** “Version increment alone enough?” → It can invalidate older versions, but the bulk decision still needs explicit conflict semantics. “Test approach?” → Load, bulk-update, inspect stale state, clear, then reload in a real provider/database test.

Reference: Jakarta Persistence 3.2 bulk update and delete semantics.

---

7. A Kafka consumer restores an old snapshot after a long outage and resurrects a deleted key. How can compaction cause this?

Say aloud: “Compaction keeps the latest known record for a key while eventually removing older records. A tombstone is a record with a key and a null value, used to mark deletion. Tombstones themselves can eventually be removed, so a compacted topic is not an unlimited history of every delete.

If a consumer restores an old local snapshot containing a deleted key and resumes after the tombstone has disappeared, it may never learn to remove that key. I would pair snapshots with offsets and a recovery policy that guarantees required deletes remain available, or discard the old state and rebuild from an authoritative source when that window is exceeded.

I would monitor consumer recovery time against deletion retention. Compaction is asynchronous and preserves ordering of retained records, but it does not turn an arbitrary stale snapshot into a safe recovery point.”

- **Example:** Snapshot contains `K=active`; log later receives `(K,null)`; an outage exceeds the available delete history. Reusing that snapshot without reconciliation can retain K forever.

- **Guarantee/cost:** Offsets do not get renumbered after compaction; gaps are normal. A complete rebuild costs the retained log scan plus state storage. Tombstone retention bounds safe scan/recovery assumptions.
- **Trade-off/failure:** Longer retention uses more storage; short retention narrows recovery time. An empty string value is not a null tombstone.
- **Follow-ups:** “Immediate erasure?” → No, cleaning is asynchronous; backups and downstream copies need separate policies. “Replay all events?” → A compacted topic is unsuitable for complete event-history auditing.

Reference: Kafka 4.0 log compaction.

---

8. A ZIP upload is small, but extraction exhausts disk and writes outside its target folder. What controls belong in the processing boundary?

Say aloud: “An archive can expand into much more data than its compressed size, and entry names can attempt path traversal, meaning escape from the intended directory. I would treat extraction as untrusted computation with both path and resource controls.

Use a private, newly created workspace and server-generated storage names. Resolve and normalize each entry under its extraction root, reject absolute or escaping paths, and disallow links and unsupported entry types. Normalization alone is insufficient if an attacker can alter filesystem links while extraction runs, so workspace ownership and safe file creation matter.

Enforce limits while streaming actual decompressed bytes: total bytes, per-entry bytes, file count, nesting, time and concurrent jobs. Do not trust archive metadata. Publish only fully validated output; on failure, clean up the owned workspace and keep partial files inaccessible.”

- **Example:** Reject `../../config.yml` and stop an archive whose actual expansion exceeds 100 MiB even if its claimed size is 1 MiB. The numbers are illustrative policy values.
- **Guarantee/cost:** Work is bounded by enforced output and processing budgets; decompression can consume CPU before reaching byte limits. Isolation and deadlines complement counters.
- **Trade-off/failure:** Strict policies reject some legitimate archives. Extension/MIME checks and malware scanning are layers, not substitutes for safe extraction and private storage.
- **Follow-ups:** “Nested archives?” → Reject or apply one shared recursive budget. “Duplicate names?” → Reject or define collision handling; do not silently overwrite trusted output.

Reference: OWASP file upload guidance; extraction policy above is an illustrative design.

---

9. A stream uses `peek` to write audit records and then calls `count`. Why can auditing disappear?

Say aloud: “A stream describes a computation rather than promising that every intermediate callback will run. Peek is an intermediate operation, and Java permits optimizations that skip traversal when the terminal result can be derived directly. For a sized source with no size-changing operations, count may already know the answer, so an audit side effect in peek can be skipped.

I would separate required side effects from incidental observation. An explicit loop or a suitable terminal operation makes intended traversal clear, but it still does not make several external writes atomic or retry-safe. If auditing is mandatory business state, it belongs in a transaction or another durable protocol.

I would keep mapping and filtering functions free of externally required side effects. That makes sequential and parallel execution easier to reason about and prevents correctness from depending on an optimizer's chosen traversal."

- **Example:** `List.of(3, 8).stream().peek(audit::write).count()` must not be relied on to produce two audit writes. It is a deliberately faulty contextual fragment.
- **Guarantee/cost:** Stream result semantics can hold without callback traversal. An explicit n-element loop does O(n) visits plus the cost of each effect.
- **Trade-off/failure:** `forEach` does not provide rollback; parallel `forEach` does not preserve encounter order. Failure may leave only part of the effects completed.
- **Follow-ups:** "Use forEachOrdered?" → Preserves encounter order where defined, not transactional atomicity. "No terminal operation?" → A lazy pipeline does not execute just because it was constructed.

Reference: Java 21 Stream, count and peek contracts.

---

10. A team replaces a request counter with LongAdder, then uses its sum to enforce a hard admission limit. Is that safe?

Say aloud: "LongAdder spreads contended updates across internal variables to improve statistical counting throughput. Its sum is not an atomic snapshot while updates are happening. More fundamentally, reading a count, checking a limit and incrementing are separate actions: several callers can all pass the same check.

For admission I would use a semaphore, a counter of available permits, with an atomic try-acquire operation, releasing exactly once when the admitted work finishes. For a custom state rule, a compare-and-set loop can atomically validate and replace one state value. Compare-and-set means update only if the value still equals the one observed.

I would keep LongAdder for telemetry, where concurrent observations need not define permission. A process-local limiter still does not enforce a fleet-wide quota; that needs a separate shared authority or explicitly allocated per-instance budgets."

- **Example:** With count 9 and limit 10, two callers can both read 9 and increment. A one-permit semaphore admits at most one of them before a release.
- **Guarantee/cost:** LongAdder favors throughput at a space cost. Atomic admission has contention; no universal constant latency is promised. A quiescent sum, with no concurrent updates, is accurate.
- **Trade-off/failure:** Incorrect release ownership can inflate permits. Replacing LongAdder with AtomicLong while keeping separate get/check/increment still leaves the race.
- **Follow-ups:** "sumThenReset for exact intervals?" → Concurrent updates do not yield an exact interval boundary. "Request times out?" → Release on actual owned-work completion, according to the resource being limited.

Reference: Java 21 LongAdder contract.

---

11. Several depots are starting points in an unweighted directed road graph. Find each vertex's fewest hops from any depot.

Say aloud: “I would use multi-source breadth-first search. Breadth-first search processes vertices in increasing hop distance with a first-in, first-out queue. Instead of running it separately from every depot, seed every distinct depot at distance zero, then expand all of them together.

When I first discover an unvisited neighbor, assign the current distance plus one and enqueue it immediately. Marking on enqueue ensures a vertex enters the queue once, including when several depots reach it. Because every edge costs one hop and the queue visits layers in order, first discovery gives the minimum over all sources.

I would return minus one for unreachable vertices, accept an empty source set, and validate every endpoint before traversal. This computes directed reachability; for undirected roads the caller must include both directions. It does not solve unequal travel-time weights.”

```
static int[] nearestDepot(int[][] edges, int[] depots) {
    Objects.requireNonNull(edges, "edges");
    Objects.requireNonNull(depots, "depots");
    int n = edges.length;
    for (int[] row : edges) {
        Objects.requireNonNull(row, "row");
        for (int v : row) if (v < 0 || v >= n)
            throw new IllegalArgumentException("endpoint");
    }
    for (int s : depots) if (s < 0 || s >= n)
        throw new IllegalArgumentException("depot");
    int[] distance = new int[n];
    Arrays.fill(distance, -1);
    ArrayDeque<Integer> queue = new ArrayDeque<>();
    for (int s : depots) if (distance[s] == -1) {
        distance[s] = 0;
        queue.addLast(s);
    }
    while (!queue.isEmpty()) {
        int u = queue.removeFirst();
        for (int v : edges[u]) if (distance[v] == -1) {
            distance[v] = distance[u] + 1;
            queue.addLast(v);
        }
    }
    return distance;
}
```

- **Example:** Edges `0→1→2`, `3→2`, isolated 4; depots `[0,3,3]` produce `[0,1,1,0,-1]`.
- **Guarantee/cost:** With V vertices, E edges and S source entries, O(V+E+S) time including validation, O(V) auxiliary/output space; inputs remain unchanged and must not be mutated concurrently.
- **Trade-off/failure:** Distances omit actual paths and depot identity. Weighted edges invalidate first-discovery optimality; duplicate edges and self-loops are harmless.
- **Follow-ups:** “Return paths?” → Store a predecessor when first discovered. “Tied depots?” → Define a tie rule; current queue/source order only determines implicit discovery preference.

12. Now roads have nonnegative travel costs. How do you find the cheapest route when a vertex first discovered may later become cheaper?

Say aloud: “I would use Dijkstra's algorithm: repeatedly process the currently smallest tentative distance. Relaxation means trying an edge to see whether it improves the destination's best known cost. With nonnegative weights, a current minimum entry cannot later be beaten through a longer unfinished route.

Java's priority queue does not provide an efficient decrease-key operation, so I insert a new entry for each strict improvement. When removing an entry, I skip it if its distance no longer matches the best known value. This handles stale entries without scanning the queue to delete them.

I would reject negative weights, use long for accumulated distance, and preserve an explicit unreachable sentinel. Zero-cost edges, cycles and parallel edges are valid. If negative costs are part of the domain, I would choose another algorithm and consider negative cycles.”

```
record Road(int to, int cost) {}
record Visit(int vertex, long distance) {}

static long[] cheapestRoutes(Road[][] roads, int source) {
    Objects.requireNonNull(roads, "roads");
    int n = roads.length;
    if (source < 0 || source >= n) throw new IllegalArgumentException("source");
    for (Road[] row : roads) {
        Objects.requireNonNull(row, "row");
        for (Road road : row) {
            Objects.requireNonNull(road, "road");
            if (road.to() < 0 || road.to() >= n || road.cost() < 0)
                throw new IllegalArgumentException("road");
        }
    }
    long[] best = new long[n];
    Arrays.fill(best, Long.MAX_VALUE);
    best[source] = 0;
    PriorityQueue<Visit> queue = new PriorityQueue<>(
        Comparator.comparingLong(Visit::distance));
    queue.add(new Visit(source, 0));
    while (!queue.isEmpty()) {
        Visit visit = queue.remove();
        int u = visit.vertex();
        if (visit.distance() != best[u]) continue;
        for (Road road : roads[u]) {
            long candidate = visit.distance() + road.cost();
            if (candidate < best[road.to()]) {
                best[road.to()] = candidate;
                queue.add(new Visit(road.to(), candidate));
            }
        }
    }
    return best;
}
```

- **Example:** `0→1:10`, `0→2:1`, `2→1:2`, isolated 3 gives `[0,3,1,MAX]`. The queued distance 10 becomes stale and is ignored.
- **Guarantee/cost:** With V vertices and E edges, O(V+E log(E+2)) time and O(V+E) auxiliary/output space for this duplicate-entry heap implementation, including full validation. E counts parallel edges. With int vertex counts and nonnegative int costs, shortest simple paths plus one examined edge fit in long; arbitrary long edge costs would require overflow handling.
- **Trade-off/failure:** Extra heap entries simplify code but cost memory. `Long.MAX_VALUE` means unreachable. No caller mutation; no concurrent graph mutation permitted.
- **Follow-ups:** “Early exit at discovery?” → Unsafe; exit when the target is removed with its current minimum. “Negative weights?” → Bellman–Ford is an option, O(VE), with reachable negative-cycle detection. “All weights one?” → Use breadth-first search.

Topic coverage: S05-Q01 queue publication and ownership; Q02 native memory; Q03 mixed-version database migration; Q04 replica consistency; Q05 transaction events; Q06 JPA bulk state; Q07 Kafka deletion recovery; Q08 secure archive processing; Q09 stream side effects; Q10 telemetry versus admission; Q11 multi-source BFS; Q12 weighted shortest paths.

Next set builds on: database index selectivity and query plans, HTTP conditional updates, Unicode string contracts, interruptible task shutdown, Kafka transaction visibility, certificate rotation, and union-find or dynamic-programming coding. Set 6 should introduce new decisions and failure cases rather than repeat these scenarios.

Part 18 - SDE-3 Interview Simulation Lab

This section converts the guide from a reference book into an interviewer-led drill. The simulations are derived from topics and question areas already covered in this guide. They are intentionally structured as interviewer escalation, not as memorization cards. How to run a simulation For each scenario: 1 Answer Stage 1 aloud in 60-90 seconds. 2 Continue only when the answer is internally consistent. 3 Treat each Stage 2-5 question as a real interviewer interruption. 4 Give a concrete decision, not a list of technologies. 5 If you do not have production evidence, say so and explain how you would verify the decision. 6 Score yourself after the simulation using the rubric below. Simulation scoring 1 - Mechanism Technical correctness Explain the mechanism clearly 2 - Why Design reasoning Explain why the design exists and alternatives 3 - Failure Failure-mode thinking Explain what breaks and how it appears 4 - Trade-off Senior judgment State what you gain, what you sacrifice, and when you would change the choice 5 - Production Operational maturity Name evidence, metrics/logs/traces/tests, rollback or recovery where relevant Target: 4/5 or better consistently. A 5/5 answer is not the longest answer; it is the answer that makes a defensible engineering decision and explains how you would prove it.

Simulation 1 - HashMap internals and the red-black tree follow-up

Source areas: HashMap internals, red-black tree deep dive, hashCode()/equals(), collision behavior, treeification. Interviewer: “Explain how HashMap works internally.”

Stage 1 - Mechanism: Explain hash spreading, bucket/bin selection, linked-list bins, tree bins, lookup, and resize at the level appropriate to the JDK version being discussed.

Stage 2 - Why: 1 Why is capacity kept as a power of two? 1 Why is the load factor around 0.75 in the common implementation? 1 Why does a collision-heavy bin eventually use a red-black tree rather than an AVL tree?

Stage 3 - Failure: 1 Two different keys have the same hash. What happens? 1 What if hashCode() is poor but equals() is correct? 1 Why can a treeified bin still have lookup behavior that is not simply “always O(log n)”?

Stage 4 - Trade-off: 1 What do you gain from treeification and what complexity does it add? 1 Why not treeify every bucket immediately?

Stage 5 - Production: 1 If a service suddenly shows CPU growth and slower map-heavy code, what evidence would distinguish ordinary load from pathological hashing or oversized maps? 1 What would you inspect before changing the code?

SDE-3 target: Do not stop at “HashMap is O(1).” Explain average behavior, collision handling, resize cost, equality semantics, and implementation/version boundaries. Red flag: Claiming that the whole HashMap becomes a tree, or that every lookup in a treeified bin is guaranteed O(log n) regardless of key comparison behavior.

Simulation 2 - Thread pools, queues and back-pressure

Source areas: thread pool, queue and back-pressure; concurrency/JVM sections. Interviewer: “A Spring service is receiving more work than downstream systems can process. How would you design the executor?”

Stage 1 - Mechanism: Explain worker threads, queueing, task submission, execution, and rejection.

Stage 2 - Why: 1 Why not simply create more threads? 1 Why does queue size matter? 1 How does the workload type change your sizing decision?

Stage 3 - Failure: 1 What happens when downstream latency increases while request rate stays constant? 1 How can an unbounded queue hide an overload problem? 1 What happens when the executor saturates?

Stage 4 - Trade-off: 1 Bounded queue vs unbounded queue? 1 Caller-runs/rejection vs dropping work? 1 More concurrency vs protecting the dependency?

Stage 5 - Production: 1 Which metrics would you watch: active threads, queue depth, task wait time, execution time, rejection count, dependency latency, request latency? 1 What evidence would tell you the bottleneck is downstream rather than the executor itself?

SDE-3 target: Treat the executor as part of the service's capacity-control system, not merely a place to configure poolSize.

Simulation 3 - ConcurrentHashMap and shared mutable state

Source areas: ConcurrentHashMap and concurrency questions. Interviewer: “Is putting an ArrayList inside a ConcurrentHashMap enough to make the data thread-safe?”

Stage 1 - Mechanism: Separate thread safety of the map from thread safety of the object stored as its value.

Stage 2 - Why: Explain why concurrent access to the container does not automatically serialize mutations inside a mutable value.

Stage 3 - Failure: Give a concrete race involving read-modify-write on the contained list.

Stage 4 - Trade-off: Compare synchronized access, concurrent collections, immutable replacement, and redesigning the state ownership.

Stage 5 - Production: Explain how you would reproduce or detect the race with concurrency tests, invariants, logging/metrics, or a focused stress test.

SDE-3 target: Identify the actual shared mutable state and synchronization boundary rather than naming a “thread-safe collection” and stopping.

Simulation 4 - Spring @Transactional and ordering traps

Source areas: Spring Retry/Resilience, transactions, Spring internals. Interviewer: “You have a transactional service method that calls a retryable operation. What can go wrong?”

Stage 1 - Mechanism: Explain that Spring behavior such as transactions/retry is commonly applied through proxies/interceptors and that ordering affects the effective behavior.

Stage 2 - Why: Why does the transaction boundary matter when retries occur?

Stage 3 - Failure: 1 What if the operation partially changes state before failing? 1 What if the retry happens inside one transaction versus across transaction boundaries? 1 What if the call bypasses the proxy?

Stage 4 - Trade-off: Compare retrying the whole transaction with isolating the retryable operation. Discuss idempotency and side effects.

Stage 5 - Production: What logs/traces/transaction metrics would prove how many attempts occurred and which transaction boundaries were actually used?

SDE-3 target: Talk about proxy boundaries, transaction scope, idempotency, and failure semantics - not just annotations.

Simulation 5 - Spring Boot startup and dependency wiring

Source areas: SpringApplication.run, dependency injection, repository implementation, startup performance. Interviewer: “The service startup time doubled after a release. How do you investigate?”

Stage 1 - Mechanism: Describe the broad startup phases: environment/configuration, application context creation, bean discovery/creation, auto-configuration and embedded server startup.

Stage 2 - Why: Why can adding a dependency or auto-configuration materially affect startup?

Stage 3 - Failure: Distinguish slow classpath scanning, bean initialization, external calls during startup, database initialization, and configuration problems.

Stage 4 - Trade-off: Lazy initialization can improve startup but move work to first request. Explain when that is useful and what it risks.

Stage 5 - Production: Explain how you would use startup timing, application metrics/logs, profiling or framework-supported diagnostics to identify the slow phase before changing configuration.

SDE-3 target: Diagnose the startup phase before applying blanket lazy initialization or removing auto-configuration.

Simulation 6 - Hibernate N+1 and fetch strategy

Source areas: Hibernate/JPA, N+1, fetching, SQL visibility. Interviewer: “An endpoint returns 100 orders and suddenly executes hundreds of SQL statements. Explain.”

Stage 1 - Mechanism: Explain how loading a parent collection and then lazily accessing associated entities can produce repeated SQL.

Stage 2 - Why: Why does lazy loading exist, and why can it be useful despite the N+1 risk?

Stage 3 - Failure: Explain how the problem appears in production: query count, DB latency, connection pressure, endpoint latency.

Stage 4 - Trade-off: Compare fetch join, entity graphs, batch fetching, DTO projections, and changing the default fetch strategy. Explain why “make everything eager” is not a universal fix.

Stage 5 - Production: How would you prove the fix? Query count, SQL logs/profiling, DB metrics, endpoint latency, and representative data volume.

SDE-3 target: Optimize the access pattern rather than changing every association to eager loading.

Simulation 7 - @Transactional(readOnly = true)

Source areas: Q228 and transaction-related Hibernate/JPA material. Interviewer: “What does @Transactional(readOnly = true) really do?”

Stage 1 - Mechanism: Explain that it communicates a read-only transaction intent to the transaction/data-access stack; exact behavior depends on the framework, transaction manager, database and ORM configuration.

Stage 2 - Why: Why can the hint help, and why is it not a magic “make this query faster” switch?

Stage 3 - Failure: What assumptions become dangerous if a developer believes it prevents all writes under every configuration?

Stage 4 - Trade-off: Discuss consistency, transaction scope, flush behavior, connection usage, and the difference between application intent and database-enforced read-only semantics.

Stage 5 - Production: How would you verify the actual behavior in your deployed stack rather than relying on a generic claim?

SDE-3 target: State the version/configuration boundary explicitly and verify actual SQL/transaction behavior.

Simulation 8 - Hibernate flush surprises

Source areas: Q245 and Hibernate flush behavior. Interviewer: “Why did an apparently read-only query cause an INSERT/UPDATE to be flushed first?”

Stage 1 - Mechanism: Explain the relationship between the persistence context, pending changes and flush timing.

Stage 2 - Why: Why does an ORM sometimes flush before query execution?

Stage 3 - Failure: How can unexpected flushes affect latency, constraint errors, locks, or transaction behavior?

Stage 4 - Trade-off: Discuss explicit flush boundaries versus allowing ORM-managed flush behavior.

Stage 5 - Production: Prove the sequence with SQL logging/profiling and transaction traces; do not infer it solely from Java source order.

SDE-3 target: Think in terms of persistence-context state and actual SQL order.

Simulation 9 - Kafka consumer lag and scaling

Source areas: Kafka terminology, Kafka internals, messaging and distributed systems. Interviewer: “Consumer lag keeps increasing. Add more consumers?”

Stage 1 - Mechanism: Explain partitions, consumer groups and the relationship between partition parallelism and consumers.

Stage 2 - Why: Why can adding consumers stop helping once partition parallelism is exhausted?

Stage 3 - Failure: Distinguish slow processing, downstream latency, insufficient partitions, rebalances, errors/retries and producer-side bursts.

Stage 4 - Trade-off: More partitions can increase parallelism but affect ordering/operational characteristics. Explain the decision rather than treating partition count as free.

Stage 5 - Production: Which evidence would you inspect: lag by partition, processing latency, consumer assignment/rebalance behavior, downstream latency, error rate and throughput?

SDE-3 target: Diagnose why lag exists before scaling consumers blindly.

Simulation 10 - Kafka delivery semantics and idempotency

Source areas: Kafka/messaging, idempotent consumers. Interviewer: “How do you make an event consumer safe when the same event can be delivered more than once?”

Stage 1 - Mechanism: Explain idempotent processing and the difference between delivery semantics and business-effect semantics.

Stage 2 - Why: Why is consumer-side idempotency often necessary even when the messaging system offers stronger delivery features?

Stage 3 - Failure: Describe a crash between applying the business change and recording the processing result.

Stage 4 - Trade-off: Compare idempotency keys, unique constraints, transactional state changes and deduplication stores.

Stage 5 - Production: Explain how you would verify duplicate delivery behavior and recovery after consumer crashes.

SDE-3 target: Design around the business side effect, not only the broker's delivery guarantee.

Simulation 11 - Circuit breaker and retry interaction

Source areas: circuit breaker, retry, resilience and microservices. Interviewer: “A downstream service is failing. We already have retries and a circuit breaker. Why is the system still melting down?”

Stage 1 - Mechanism: Explain retries, failure counting, open/closed/half-open behavior at a conceptual level.

Stage 2 - Why: Why can retries amplify load during a dependency failure?

Stage 3 - Failure: Explain retry storms, increased latency, thread/connection exhaustion and cascading failure.

Stage 4 - Trade-off: Discuss retry count, backoff, jitter, timeout, circuit-breaker threshold, fallback and which failures should not be retried.

Stage 5 - Production: Identify the evidence needed: dependency latency/error rate, retry count, circuit state, request concurrency, queue depth and saturation signals.

SDE-3 target: Treat timeout + retry + circuit breaker + concurrency as one control system.

Simulation 12 - API idempotency

Source areas: REST/API design, microservices and idempotent operations. Interviewer: “A client times out after creating an order. It retries. How do you prevent duplicate orders?”

Stage 1 - Mechanism: Explain an idempotency key and durable association between the key and the resulting operation.

Stage 2 - Why: Why does a client timeout create ambiguity even when the server may have completed the operation?

Stage 3 - Failure: Consider concurrent duplicate requests, partial failure and key reuse.

Stage 4 - Trade-off: Discuss key retention, storage, conflict handling, response replay and scope of uniqueness.

Stage 5 - Production: Explain how you would measure duplicate attempts, idempotency conflicts and recovery behavior.

SDE-3 target: Recognize the fundamental “unknown outcome after timeout” problem.

Simulation 13 - Database indexing and query diagnosis

Source areas: SQL, PostgreSQL indexes, query performance. Interviewer: “This query is slow. Add an index.”

Stage 1 - Mechanism: Explain what an index changes and why it can reduce data access for suitable predicates.

Stage 2 - Why: Why is an index not automatically beneficial for every column or query?

Stage 3 - Failure: Discuss write overhead, storage, poor selectivity and a query plan that still chooses a scan.

Stage 4 - Trade-off: Explain composite-index column order and matching the access pattern rather than creating indexes indiscriminately.

Stage 5 - Production: Use the execution plan plus representative production-like data and query latency to validate the change.

SDE-3 target: Start with the query and execution plan, not the index definition.

Simulation 14 - SQL NULL semantics

Source areas: Q271 and SQL material. Interviewer: “Why did WHERE column != 'X' fail to return rows where column is NULL?”

Stage 1 - Mechanism: Explain SQL three-valued logic and that NULL represents an unknown/missing value rather than an ordinary comparable value.

Stage 2 - Why: Why do ordinary equality/inequality predicates not behave like Java null comparisons?

Stage 3 - Failure: Identify filters, joins, aggregates and uniqueness assumptions that can be affected.

Stage 4 - Trade-off: Explain explicit IS NULL/IS NOT NULL logic and the business meaning of missing data.

Stage 5 - Production: Validate with targeted test cases containing NULL and non-NULL values before shipping the query change.

SDE-3 target: Connect SQL semantics to the business meaning of missing data.

Simulation 15 - JVM allocation and escape analysis

Source areas: JVM performance and allocation questions. Interviewer: “Object allocation is cheap on the JVM. Then why is this service generating GC pressure?”

Stage 1 - Mechanism: Explain allocation paths at a high level and the role of garbage collection.

Stage 2 - Why: Why can short-lived allocations still matter at very high allocation rates?

Stage 3 - Failure: Explain allocation rate, object lifetime, retained objects and GC overhead as different possible causes.

Stage 4 - Trade-off: Discuss whether reducing allocations is worth code complexity; avoid optimizing without evidence.

Stage 5 - Production: Use allocation profiling, GC logs/metrics, heap information and latency evidence to identify the dominant source.

SDE-3 target: Do not equate “allocation is cheap” with “allocation is free.”

Simulation 16 - Kubernetes probes and graceful shutdown

Source areas: Kubernetes deployment, probes and graceful shutdown. Interviewer: “Pods are healthy, but deployments occasionally drop requests. Where do you look?”

Stage 1 - Mechanism: Explain readiness, liveness and shutdown behavior separately.

Stage 2 - Why: Why should readiness and liveness represent different questions?

Stage 3 - Failure: Consider terminating pods still receiving traffic, probes that are too aggressive, slow startup, and application shutdown that does not drain work.

Stage 4 - Trade-off: Probe frequency/timeouts versus false positives and slow detection.

Stage 5 - Production: Correlate deployment events, pod lifecycle, readiness transitions, request errors, termination timing and application shutdown logs.

SDE-3 target: Think about the entire pod lifecycle, not only the probe endpoint.

Simulation 17 - Spring MVC content negotiation and Jackson

Source areas: Spring MVC, content negotiation and Jackson configuration. Interviewer: “The same controller returns different representations depending on the request. Explain.”

Stage 1 - Mechanism: Explain content negotiation and serialization/deserialization at a framework level.

Stage 2 - Why: Why is the Accept/content-type contract important?

Stage 3 - Failure: Discuss incompatible media types, serialization configuration, unexpected field exposure and client/server contract mismatch.

Stage 4 - Trade-off: Global Jackson configuration versus endpoint-specific behavior.

Stage 5 - Production: Verify request headers, negotiated media type, serialized payload, server configuration and contract tests.

SDE-3 target: Trace the request from HTTP headers through handler selection to serialization.

Simulation 18 - Spring Data JPA repository abstraction

Source areas: Spring Data repository implementation. Interviewer: “I only declared an interface. Where does the implementation come from?”

Stage 1 - Mechanism: Explain runtime proxy/repository infrastructure at a conceptual level.

Stage 2 - Why: What does the abstraction buy the team, and what does it hide?

Stage 3 - Failure: What happens when the generated query does not match the actual access pattern or causes excessive SQL?

Stage 4 - Trade-off: Repository abstraction versus explicit queries/DTOs/native SQL for complex workloads.

Stage 5 - Production: Validate generated SQL, query count, execution plans and transaction boundaries.

SDE-3 target: Know when abstraction improves productivity and when the hidden behavior needs to become explicit.

Simulation 19 - Designing a real-time clickstream pipeline

Source areas: full system-design walkthroughs, GCP clickstream analytics pipeline. Interviewer: “Design a real-time clickstream analytics pipeline.”

Stage 1 - Mechanism: Define producers, ingestion, durable messaging/storage, stream processing, serving/analytics and consumers.

Stage 2 - Why: Explain why each boundary exists and where buffering/decoupling is useful.

Stage 3 - Failure: Walk through duplicate events, consumer lag, malformed events, processor failure and downstream unavailability.

Stage 4 - Trade-off: Discuss latency versus cost, delivery semantics, retention, partitioning and replayability.

Stage 5 - Production: Define SLOs and evidence: ingestion rate, processing lag, end-to-end latency, error/dead-letter rate and data-quality signals.

SDE-3 target: Start with requirements and guarantees before drawing boxes.

Simulation 20 - Multi-tenant data isolation

Source areas: PostgreSQL row-level security and database/system-design material. Interviewer: “How do you guarantee tenant A cannot read tenant B data?”

Stage 1 - Mechanism: Define the tenant boundary and where enforcement occurs.

Stage 2 - Why: Why is application-only filtering potentially weaker than defense in depth?

Stage 3 - Failure: Consider missing predicates, incorrect tenant context, privileged paths, background jobs and SQL written outside the normal repository path.

Stage 4 - Trade-off: Compare application filtering, row-level security, separate schemas/databases and operational complexity.

Stage 5 - Production: Explain authorization tests, negative cross-tenant tests, audit evidence and monitoring for tenant-boundary violations.

SDE-3 target: Treat tenant isolation as a security invariant that should be enforced and tested at multiple layers where appropriate.

Simulation 21 - System-design interviewer changes the scale

Interviewer: “Your design works for 10,000 requests per second. Now make it work for 100,000.”

Stage 1 - Mechanism: Identify which components scale horizontally and which have hard limits.

Stage 2 - Why: Explain the bottleneck rather than multiplying every component by ten.

Stage 3 - Failure: Find the first saturation point: database, partition count, connection pool, queue, downstream dependency, hot key or single coordinator.

Stage 4 - Trade-off: Explain caching, partitioning, batching, asynchronous processing, replication or data-model changes and the consistency/cost consequences.

Stage 5 - Production: Define the capacity metric and load-test evidence needed to validate the new design.

SDE-3 target: Scaling is a bottleneck-identification exercise, not a shopping list of infrastructure.

Simulation 22 - Production incident: latency spike

Interviewer: “p99 latency went from 300 ms to 3 seconds after a release. What do you do?”

Stage 1 - Mechanism: Establish the request path and distinguish application, dependency and queueing latency.

Stage 2 - Why: Explain why p99 can move dramatically even when average latency looks acceptable.

Stage 3 - Failure: Consider downstream latency, thread/connection pool saturation, GC, lock contention, DB query regression and retries.

Stage 4 - Trade-off: Decide between rollback, mitigation, capacity increase or code/configuration change based on evidence.

Stage 5 - Production: Use traces, latency percentiles, saturation metrics, error/retry rates and deployment correlation. State what you would check first and why.

SDE-3 target: Show a hypothesis-driven incident loop: observe → narrow → mitigate → verify → prevent recurrence.

Simulation 23 - Coding escalation: from correct to production-quality

Interviewer: “Implement the coding problem.”

Stage 1 - Mechanism: Produce a correct solution and state time/space complexity.

Stage 2 - Why: Explain the data structure/algorithm choice and the invariant that makes it correct.

Stage 3 - Failure: Identify edge cases, invalid inputs if relevant, integer overflow risks, duplicate values, empty inputs and boundary conditions.

Stage 4 - Trade-off: Compare the chosen solution with a simpler/brute-force solution and explain when the simpler one could be acceptable.

Stage 5 - Production: Discuss readability, test strategy, observability if this were production code, and how you would benchmark or load-test it when performance matters.

SDE-3 target: The interviewer should hear the reasoning before the code becomes a typing exercise.

Simulation 24 - Final architecture challenge: defend your decision

Interviewer: “You proposed solution A. I think B is better. Convince me.”

Stage 1 - Mechanism: Restate the requirement and the mechanism of A.

Stage 2 - Why: State the assumptions under which A is preferable.

Stage 3 - Failure: Acknowledge where B is stronger and where A fails.

Stage 4 - Trade-off: Compare latency, consistency, availability, complexity, cost, operational burden and migration risk as relevant to the design.

Stage 5 - Production: Define the measurement that would make you reverse the decision.

SDE-3 target: Do not defend a technology emotionally. Defend a decision with requirements, assumptions, trade-offs and measurable evidence.

Part 19 - 30-Minute SDE-3 Mock Interview Protocol

Use this for a daily timed drill. Keep expected answers hidden until the candidate has reasoned through the problem.

Minutes	Exercise
0-3	Warm-up: answer one core Java question. Allow interruption after 30-60 seconds.
3-10	Technical escalation: choose Simulation 1-18 and work through all five stages.
10-20	System design: choose Simulation 19-21; clarify requirements before proposing an architecture.
20-26	Incident/debugging: use Simulation 22; prioritize hypotheses and the evidence needed to reject each.
26-30	Coding explanation: use Simulation 23; state the invariant, complexity and boundaries before coding.

Score technical correctness, reasoning, failure-mode thinking, trade-offs, production verification and communication from 1 to 5 each. A practice target is 24/30, with focused revision where a dimension repeatedly scores below 4. This is a study rubric, not a hiring guarantee.

The spoken pattern is: how it works, why it exists, what breaks, the trade-off and how to verify the behavior.

Part 20 - Interviewer Follow-Up Rules

1. Let the candidate reason before revealing the expected answer.
2. Challenge assumptions and ask why; do not only test vocabulary.
3. Change one constraint at a time: traffic, failure, ordering, latency or dependencies.
4. Ask for evidence: how would the candidate verify the claim?
5. Ask about a credible alternative and when it would be better.
6. Reward explicit uncertainty about version/configuration-dependent behavior.
7. Stop when the candidate reaches a defensible decision; judgment matters more than endless trivia.

What interview-ready should sound like

Start with a concise mechanism and design reason, then deepen the answer when challenged. A strong answer can explain failure modes, trade-offs and production evidence without inventing personal experience.

Part 21 - Production Resilience, Delivery and JVM Operations (Q273-Q292)

Twenty additional questions on topics the guide covers only briefly or not at all: cache stampedes, distributed transactions, change data capture, clocks, backpressure, Kafka failure handling, API evolution, safe deployments, connection pooling, garbage-collector choice, leak diagnosis, native images, class loading, alerting and secrets.

Every question uses the same four-tier ladder, so you can stop at the tier that matches the interview or your current level:

Tier	Purpose	Use it to
1. Foundation	Plain-language definition, key terms and a small example	Learn the topic, or check you can explain it to a non-specialist
2. Core answer	The 30-second version	Open your spoken answer
3. Follow-ups	Three escalating probes (warm-up, applied, senior), each with expected reasoning	Practise answering before reading the reasoning
4. Deep dive	Internals, a realistic failure, the trade-off and how to verify	Reach a defensible decision with evidence

Baseline: Java 21 without preview features, Spring Boot 3.x, PostgreSQL, Kafka and Kubernetes. Q289 is explicitly a Java 21 preview comparison. The two Java fragments are compiled and exercised with supplied application types in the repository's Part 21 fixture; they are sketches, not complete applications. Framework, database, cloud and Kubernetes scenarios require testing on your stack. Defaults and flags change between versions, so confirm them against the documentation for yours. Metrics, numbers and incidents are illustrative. Where no source is linked, the basis is general engineering practice.

Reviewed 9 October 2026: provider-specific cache synchronization, transaction ordering, ETag validity, retry counts, DNS caching, diagnostic impact, cooperative cancellation and scoped-value bindings are qualified below.

Group	Questions
Distributed systems and data	Q273-Q278
APIs and delivery	Q279-Q285
JVM and Java	Q286-Q290
Operations	Q291-Q292

Q283 deliberately avoids graceful shutdown, which Q19 already covers.

Distributed systems and data

Q273 What is a cache stampede and how do you handle hot keys

Tier 1 - Foundation

In plain language: a cache keeps a ready copy of something expensive to compute. A **stampede** happens when a very popular copy disappears and many requests all rebuild it at the same moment. A **hot key** is a different problem: one entry is requested so often that a single cache node cannot serve it.

Key terms:

- **Cache hit / miss:** the value was / was not found in the cache.
- **TTL (time to live):** how long an entry may be served before it expires.
- **Eviction:** the cache removing an entry to make room.
- **Request coalescing (single flight):** letting one caller rebuild a value while the others wait for its result.
- **Near-cache:** a small in-process copy in front of a shared cache.

Small example: a product page is cached for 60 seconds and receives 5,000 requests per second. When the entry expires, rebuilding takes 200 ms, so roughly 1,000 requests miss during that gap and all query the database for the same row.

Tier 2 - Core answer (30-second version)

> A stampede is many simultaneous misses on one expired key, which hit the database exactly when the cache was meant to protect it. Reduce it with request coalescing, TTL jitter, serving stale data while one caller refreshes, and pre-warming. A hot key is a separate issue, where one key exceeds one node's capacity, and it needs a near-cache, key replication or splitting the value. I would verify by expiring a hot key under load and counting database queries.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does adding random jitter to TTLs help?

Expected reasoning: keys written together, for example after a deploy or a bulk warm-up, expire together, which produces a synchronized wave of misses. Jitter spreads expiry over time. It does not help a single popular key expiring on its own, which needs coalescing or early refresh.

Probe 2 (applied): You use `@Cacheable(sync = true)` on 50 instances. Is the stampede solved?

Expected reasoning: Spring delegates synchronized loading to the cache provider; the annotation alone does not promise a cluster-wide lock. With an instance-local provider such as Caffeine, 50 instances can still perform up to 50 simultaneous loads. A distributed provider can have different locking guarantees, so check its documentation before adding another lock. If the actual load count is unacceptable, serve stale values while one instance refreshes or use a short-lease distributed lock. The loader must still be safe to run twice, because a lock can expire mid-load. See Spring synchronized caching.

Probe 3 (senior): Traffic to one key doubles during a sale and one cache node saturates. What do you do?

Expected reasoning: this is a hot key, not a stampede, and hashing does not spread one key. Options are a small near-cache with a short TTL in each application instance, replicating the value under several suffixed keys and reading a random one, or splitting the value. Accept the staleness window these introduce and set it deliberately.

Tier 4 - Deep dive

Internals: single flight in one JVM is a map of in-flight loads.

```
// Coalesces concurrent loads for one key inside ONE JVM only.
private final ConcurrentHashMap<String, CompletableFuture<Product>> inFlight =
        new ConcurrentHashMap<>();

CompletableFuture<Product> load(String id) {
    var mine = new CompletableFuture<Product>();
    var existing = inFlight.putIfAbsent(id, mine);
    if (existing != null) return existing.copy();     // isolate each caller's completion
    try {
        mine.complete(fetchAndPopulateCache(id));     // the winning caller does the work
    } catch (Throwable t) {
        mine.completeExceptionally(t);
        if (t instanceof InterruptedException) Thread.currentThread().interrupt();
        if (t instanceof Error error) throw error;
    } finally {
        inFlight.remove(id, mine);
    }
    return mine.copy();
}
```

The winning caller performs the fetch synchronously; returning a future does not move that work to an executor. Each caller receives a defensive copy of the future, so its cancellation, timeout or explicit completion cannot change the shared result. Cancelling a copy does not stop the fetch.

Probabilistic early refresh lets each request, with a probability that rises as expiry approaches, refresh the value before it expires, so expiry rarely arrives with a crowd waiting. Stale-while-revalidate keeps the old value for a short grace period while one caller fetches a new one.

Realistic failure: after a deploy every instance starts with a cold cache at once. Misses saturate the database connection pool, queries time out, and timeouts are not cached, so the cache never warms and the system stays down until traffic is shed. Fixes are staged traffic, pre-warming the hot keys, caching short-lived negative results and a limit on concurrent loads.

Trade-off: coalescing and stale serving trade freshness for load protection. Distributed locks add a component that can fail. Near-caches add an inconsistency window between instances.

How to verify: expire one hot key under load and count database queries for it. The target is about one per instance or fewer, not hundreds. Flush the cache in staging while load-testing and watch database connections and latency.

Q274 Two-phase commit, saga or outbox: how do you keep two systems consistent

Source basis: PostgreSQL PREPARE TRANSACTION.

Tier 1 - Foundation

In plain language: one business action sometimes has to change two separate systems, such as two services or a database and a message broker. There is no automatic guarantee that both changes happen or neither does. Three techniques address this, and they solve different problems.

Key terms:

- **Atomicity:** all of a change happens, or none of it does.
- **Coordinator and participants:** in two-phase commit (2PC), the coordinator asks each participant to prepare, then to commit or roll back.
- **Compensation:** an action that undoes the business effect of an earlier step, such as refunding a payment.
- **Saga:** a sequence of local transactions where failure triggers compensations for the steps already done.
- **Outbox:** an event stored in the same local transaction as the business data, then published separately.
- **Idempotent:** safe to run more than once with the same effect.

Small example: placing an order reserves stock in the inventory service and then charges payment in the payment service. If payment fails after stock is reserved, the reservation must be released.

Tier 2 - Core answer (30-second version)

> Two-phase commit gives atomic commit across resources, but participants hold locks until the coordinator decides, so a coordinator failure blocks them. Across independent services I prefer a saga, which runs local transactions and compensates on failure, so intermediate states are visible and compensation must be retried. The outbox pattern is narrower: it makes "update my database and publish an event" atomic by recording the event in the same transaction. Delivery is at least once, so consumers must be idempotent.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why can two-phase commit block?

Expected reasoning: after a participant votes yes in the prepare phase it must be able to commit, so it keeps its locks until it learns the decision. If the coordinator crashes after prepare and before announcing the decision, participants wait. Recovery needs the coordinator's log, or an operator.

Probe 2 (applied): A saga's payment step fails after stock was reserved. What happens, and who drives it?

Expected reasoning: the saga runs the compensation, which releases the reservation. An orchestrator can drive this centrally, while in choreography services react to each other's events. The compensation must be idempotent and retried until it succeeds, and a saga that cannot finish needs a timeout, an alert and a manual path.

Probe 3 (senior): Someone says "we use an outbox, so we have exactly-once across services." Respond.

Expected reasoning: the outbox makes recording intent atomic with the local write. Publishing is a separate step that can repeat after a crash, so delivery is at least once. Consumers need deduplication, for example by event id, to get effectively-once processing. It does not give atomicity across two services' databases.

Tier 4 - Deep dive

Internals: in 2PC the coordinator sends *prepare*, each participant durably records that it can commit and votes, and the coordinator logs the decision and sends *commit* or *rollback*. In Java this is usually JTA with XA resources. In PostgreSQL, `PREPARE TRANSACTION` persists a transaction for a later `COMMIT PREPARED` or `ROLLBACK PREPARED`, which needs `max_prepared_transactions` above its default of 0.

Realistic failure: a coordinator crashes after prepare and nobody resolves the prepared transactions. Their locks stay, other transactions queue behind them and an abandoned prepared transaction can hold back vacuum, so table bloat grows. Query `pg_prepared_xacts` and make resolving them an operational procedure. For sagas, the common failure is a compensation that itself fails or cannot undo the effect (an email already sent), so design a **pivot step** and treat the effects after it as retryable forward steps.

Trade-off: 2PC coordinates atomic commit at the cost of availability and coupling to XA-capable resources. Isolation is a separate property of the participants and their concurrency-control protocol; 2PC alone does not guarantee globally serializable execution. Sagas keep services autonomous but expose intermediate states and push consistency logic into the application.

How to verify: crash the coordinator, or a service, between every pair of steps. Confirm the system completes or compensates, that each step is safe to repeat, and that a stuck saga appears on a dashboard.

Q275 Change data capture or a polling outbox publisher

Source basis: Debezium documentation.

Tier 1 - Foundation

In plain language: with the outbox pattern, events wait in a database table. Something must move them to Kafka. A **polling publisher** is application code that repeatedly queries the table. **Change data capture (CDC)** instead reads the database's own change log and emits each committed change.

Key terms:

- **WAL (write-ahead log):** PostgreSQL's durable log of every change.
- **Logical decoding / replication slot:** PostgreSQL's mechanism to stream committed changes, with a slot remembering how far a consumer has read.
- **Connector:** a CDC component such as Debezium running in Kafka Connect.
- **At-least-once:** every event arrives, possibly more than once.
- **Event id:** a unique id that lets a consumer recognize a repeat.

Small example: after an order row and its outbox row commit, a job runs `SELECT ... FROM outbox WHERE published = false`, sends the rows and marks them sent. A CDC connector would instead see the outbox insert in the log and emit it.

Tier 2 - Core answer (30-second version)

> Both deliver outbox rows to Kafka. Polling is simple but adds query load and latency, and needs care for ordering and duplicate publishers. CDC reads the transaction log, so it has low latency and commit order, but you operate another component, and a stalled PostgreSQL replication slot makes the database retain WAL and can fill the disk. Both are at least once, so consumers deduplicate.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why must consumers deduplicate with either approach?

Expected reasoning: a publisher can send an event and crash before recording that it did, and a connector can restart from an earlier position. The event is then delivered again. A stable event id and a processed-ids check, or an idempotent operation, makes this harmless.

Probe 2 (applied): How do you keep events for one order in order?

Expected reasoning: use the aggregate id as the Kafka message key so the events share a partition. With CDC, changes are read in commit order. With polling, several publishers running in parallel can send events from the same aggregate out of order, so partition the work by aggregate or use one publisher.

Probe 3 (senior): The CDC connector has been down for a day. What is the risk?

Expected reasoning: the replication slot keeps the WAL the connector has not yet consumed, so retained WAL grows and can exhaust disk. Monitor slot lag, alert on it, drop slots nobody uses, and consider `max_slot_wal_keep_size` to cap retention, knowing an invalidated slot means the connector needs a fresh snapshot.

Tier 4 - Deep dive

Internals: Debezium uses PostgreSQL logical decoding through an output plugin (commonly `pgoutput`) and a publication. Its outbox event router transformation can turn outbox rows into Kafka messages keyed by aggregate id. A common design inserts the outbox row and lets CDC pick it up from the log, so the table need not be retained for long.

Realistic failure: a polling publisher that reads `WHERE id > :last` can skip events. Transaction A takes id 100 and commits late, transaction B takes id 101 and commits first, the poller advances past 101 and never sees 100. Use a published flag, or read with a visibility-safe approach, rather than trusting id order. For CDC, the failure is the slot growth described above.

Trade-off: polling is easy to understand and operate but costs queries and delay. CDC gives timeliness and ordering but adds infrastructure, connector upgrades, schema-change handling and snapshot procedures.

How to verify: kill the publisher or connector after sending and before acknowledging, and confirm the event is delivered again but processed once. Stop the connector for an hour in a test environment and watch slot lag and disk usage.

Q276 Why can't you order events by wall-clock time across machines

Source basis: How to do distributed locking (fencing tokens).

Tier 1 - Foundation

In plain language: every computer has its own clock, and they are never perfectly in step. If you rely on "which timestamp is later" to decide which event happened later on different machines, you can get the wrong answer.

Key terms:

- **Wall-clock time:** the calendar time of a machine, such as `System.currentTimeMillis()`.
- **Monotonic clock:** a clock that only moves forward, suited to measuring durations, such as `System.nanoTime()`.
- **NTP:** the protocol that adjusts machine clocks, sometimes by stepping them.
- **Clock skew:** the difference between two machines' clocks.
- **Logical clock:** a counter that orders events without using real time.

- **Fencing token:** an increasing number issued with each lock or lease, checked by the resource being protected.

Small example: two servers update a user profile. Server B's clock is two seconds behind. Its later update gets an earlier timestamp, so "last write wins" discards it.

Tier 2 - Core answer (30-second version)

> Machine clocks drift and get corrected, so timestamps from different machines cannot reliably order events, and last-write-wins by timestamp can silently lose a newer write. Use `nanoTime` for durations within a process and a logical mechanism for ordering, such as a serialized per-aggregate version, Lamport clocks, version vectors or hybrid logical clocks. A database sequence orders number allocation, not transaction commits; use a commit-log position or serialize the relevant writes when commit order matters. For leases and locks, add a fencing token so a stale holder cannot write.

Tier 3 - Follow-ups

Probe 1 (warm-up): When do you use `nanoTime` and when `currentTimeMillis`?

Expected reasoning: `nanoTime` for elapsed time, timeouts and measurements, because it is monotonic and unaffected by clock adjustments. `currentTimeMillis` when you need a calendar time to show or store, knowing it can jump.

Probe 2 (applied): What can a Lamport clock tell you, and what can it not?

Expected reasoning: it gives an order consistent with causality: if event A caused event B, A's number is smaller. It cannot tell whether two events with different numbers were causally related or concurrent. Version vectors can detect concurrent writes, at the cost of metadata that grows with the number of writers.

Probe 3 (senior): A client holds a lock lease, pauses for a long garbage collection, and the lease expires. What goes wrong and how do you prevent it?

Expected reasoning: after the pause the client still believes it holds the lock and writes while a new holder is active. A fencing token solves this. The lock service issues an increasing token, the client sends it with each write, and the resource rejects any token lower than one it has already seen.

Tier 4 - Deep dive

Internals: NTP can slew a clock gradually or step it, and a step can move wall time backwards. Hybrid logical clocks keep a physical component close to real time plus a counter, so timestamps stay ordered and readable. Systems such as ZooKeeper (transaction ids) and etcd (revisions) provide monotonically increasing numbers that can serve as fencing tokens. Google Spanner exposes bounded clock uncertainty with TrueTime and waits out the uncertainty when committing, which needs hardware most teams do not have.

Realistic failure: a Redis-based lock with an expiry protects a nightly job. A pause makes two workers run it at once, and both update the same rows. The lock was correct as a mutual-exclusion hint but not safe as the only protection, because nothing at the data layer checked who held it.

Trade-off: logical clocks, aggregate versions and commit-log positions provide different ordering guarantees and need coordination or extra metadata. A sequence alone is not commit order: transaction A can allocate 100 and commit after transaction B allocates 101. PostgreSQL sequence allocation is not rolled back with the transaction; see sequence semantics. Wall-clock timestamps are cheap and human-readable but unreliable for ordering. Consensus-based services give strong guarantees at the price of latency and availability constraints.

How to verify: inject a `Clock` and move it backwards in tests, and confirm ordering and expiry logic do not break. Pause a lock holder past its lease in an integration test and verify the resource rejects its late write.

Q277 What is backpressure and what happens without it

Tier 1 - Foundation

In plain language: backpressure is a way for a slow part of a system to tell a fast part to slow down, so the amount of work waiting stays bounded. Without it, work piles up until the system becomes slow or runs out of memory.

Key terms:

- **Producer / consumer:** the side that creates work and the side that processes it.
- **Bounded / unbounded queue:** a queue with a maximum size, or none.
- **Load shedding:** rejecting some work on purpose to protect the rest.
- **Rejection policy:** what an executor does when it cannot accept more work.
- **Little's law:** items in a system = arrival rate x average time each spends in it.

Small example: an API accepts uploads and hands each to an executor backed by an unbounded queue. The downstream storage service slows down, the queue grows, memory rises, and every request waits longer until the service fails.

Tier 2 - Core answer (30-second version)

> Backpressure keeps work in flight bounded by letting the consumer slow the producer. Use bounded queues and decide what happens when they fill: block, reject, run on the caller, or drop low-value work. Apply it at each boundary: reactive streams use `request(n)`, HTTP services return 429 or 503 with Retry-After, and Kafka consumers limit and pause polling. I would test it by driving the service at twice its capacity.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why is `Executors.newFixedThreadPool` risky?

Expected reasoning: it uses an unbounded `LinkedBlockingQueue`. When producers outpace the threads, the queue grows without limit, so memory and waiting time grow and nothing pushes back on the caller. Use a `ThreadPoolExecutor` with a bounded queue and an explicit rejection policy.

Probe 2 (applied): How does `CallerRunsPolicy` create backpressure, and what is its risk?

Expected reasoning: when the queue is full, the submitting thread runs the task itself, so it cannot submit more until it finishes, which naturally slows the producer. The risk is that the submitting thread may be a request thread or an event-loop thread, and running slow work there can block unrelated requests.

Probe 3 (senior): How would a Kafka consumer apply backpressure?

Expected reasoning: bound the records per poll with `max.poll.records`, and when downstream is saturated, pause assigned partitions and keep calling `poll()` so the consumer stays in the group. Stopping polling for longer than `max.poll.interval.ms` makes the broker consider the consumer failed and triggers a rebalance.

Tier 4 - Deep dive

Internals: a `ThreadPoolExecutor` adds threads beyond the core size only when its queue is full, so with an unbounded queue the maximum pool size is never reached (see Q148). Little's law makes queues measurable: at 100 requests per second and 0.5 seconds of processing, about 50 requests are in flight, and a queue holding 10,000 requests means a new arrival waits roughly 100 seconds. Reactive Streams makes demand explicit: a subscriber calls `request(n)` and the publisher must not send more than requested.

Realistic failure: a service sheds load with 503s and clients retry immediately, so the retry traffic keeps the service overloaded. Return `Retry-After`, use exponential backoff with jitter on clients, and cap retries (see Q281).

Trade-off: blocking the producer preserves work but can stall upstream threads, rejecting is fast but visible to clients, dropping loses data, and large buffers hide the problem until they fail. Choose per flow: payments should not be dropped, telemetry may be.

How to verify: drive the service at twice capacity. A healthy design shows stable latency for accepted requests and quick rejections for the rest. An unhealthy one shows growing latency and memory, then failure.

Q278 Design retries and a dead-letter topic for a Kafka consumer

Source basis: Spring for Apache Kafka: non-blocking retries.

Tier 1 - Foundation

In plain language: a Kafka consumer reads messages from a partition in order. Sometimes processing a message fails. You need a plan that retries when it might help, stops when it will not, and keeps failed messages somewhere you can inspect them.

Key terms:

- **Transient failure:** one that may succeed later, such as a dependency being briefly down.
- **Permanent failure:** one that will never succeed, such as a malformed message.
- **Poison pill:** a message that fails every time and blocks progress if handled badly.
- **Offset commit:** recording how far the consumer has read.
- **Backoff:** a growing delay between attempts.
- **Dead-letter topic (DLT):** a topic holding messages that exhausted their retries.
- **Idempotent consumer:** processing the same message twice has the same effect as once.

Small example: an order event arrives with invalid JSON. Retrying a thousand times will not fix it, and if the consumer keeps retrying it, every later message on that partition waits.

Tier 2 - Core answer (30-second version)

> Separate transient from permanent failures. Retry transient ones with backoff, and send permanent ones straight to a dead-letter topic with the error and original coordinates. Blocking retry keeps order while retrying but stalls the partition; moving a failed record to a DLT and replaying it later can still reorder effects. Non-blocking retry topics keep the partition moving but give up ordering for a key. Wrap deserializers so bad bytes do not become a poison pill, alert on dead-letter depth, and make consumers idempotent because retries and replays redeliver.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why is "retry forever" a bad policy?

Expected reasoning: a permanent failure never succeeds, and with blocking retry it holds up the whole partition, so lag grows for healthy messages. Bound the attempts, then divert the message to a DLT.

Probe 2 (applied): What do you give up with non-blocking retry topics?

Expected reasoning: a failed message is republished to a retry topic and the main partition continues, so a later message for the same key can be processed before the earlier one finishes retrying. If strict order per key matters, block or quarantine that key until the failure is resolved; bounded blocking retries followed by DLT diversion do not preserve order on later replay.

Probe 3 (senior): The message cannot even be deserialized. Why does the normal error handler not help, and what do you do?

Expected reasoning: deserialization fails inside the consumer before your listener or its error handler runs, so the same record is fetched again each time. Wrap the deserializer with `ErrorHandlingDeserializer`, so the failure becomes a recoverable error that the handler can route to the DLT.

Tier 4 - Deep dive

Internals: Spring Kafka's `DefaultErrorHandler` with a back-off retries on the consumer thread, and a `DeadLetterPublishingRecoverer` publishes the exhausted record to a DLT with headers describing the exception and the original topic, partition and offset. `@RetryableTopic` creates retry topics (for example `orders-retry-0`) with delays, plus a DLT. Offset advancement depends on the container acknowledgement mode and recovery settings. Configure the recoverer to propagate DLT send failures (for example, `setFailIfSendResultIsError(true)`) and verify that unsuccessful recovery does not advance past the record. A successful asynchronous send submission alone is not proof of durable publication. See Spring Kafka recovery and offset handling.

Realistic failure: the DLT fills for weeks because nobody owns it. A replay after the fix then pushes thousands of old messages through at once and overloads a dependency. Give the DLT an owner, an alert on depth and age, a documented replay tool with rate limiting, and a rule for discarding records.

Trade-off: blocking retry protects order until a record is skipped or diverted, and delays everything behind it. Non-blocking retry maximizes throughput and loses per-key order. Sending to a DLT early protects the stream but moves the problem into an operational queue.

How to verify: publish a malformed record, a record that fails twice and then succeeds, and a record that always fails. Confirm each ends up where intended, nothing is lost, and DLT depth triggers an alert. Fail the DLT send and check that the source record remains recoverable; replay an earlier record after a later one and verify the ordering policy.

APIs and delivery

Q279 How do you version and evolve an API without breaking clients

Source basis: RFC 8594: The Sunset HTTP header.

Tier 1 - Foundation

In plain language: an API is a promise to its callers about requests, responses and behavior. Changing it can break callers you do not control. Evolving an API well means making most changes in a way that existing clients keep working, and having a plan for the rare change that cannot.

Key terms:

- **Breaking change:** a change that makes a correctly written existing client fail.
- **Tolerant reader:** a client that ignores fields and values it does not recognize.
- **Deprecation:** announcing that something will be removed.
- **Sunset:** the date after which it stops working, which RFC 8594 lets a server signal in a header.
- **Contract test:** a test that checks the API still satisfies what its consumers rely on.

Small example: renaming the response field `phone` to `phoneNumber` breaks every client that reads `phone`. Adding a new optional field `phoneVerified` does not.

Tier 2 - Core answer (30-second version)

> Prefer compatible evolution: adding optional fields and endpoints is safe for tolerant readers, while removing, renaming or changing meaning is breaking. When a break is unavoidable, version explicitly in the URI, a header or the media type, run both versions for a published period, and signal retirement with Deprecation and Sunset headers. Measure who still calls the old version, and use consumer-driven contract tests to catch breaks before release.

Tier 3 - Follow-ups

Probe 1 (warm-up): Is adding a value to a response enum a non-breaking change?

Expected reasoning: not automatically. A client with a strict switch may fail on an unknown value. It is only safe if the contract says clients must tolerate unknown values, so state that rule from the first version.

Probe 2 (applied): Compare URI versioning with header versioning.

Expected reasoning: URI versions are visible, easy to route, cache and test from a browser. Header or media-type versions keep resource URIs stable and match the idea that the URI identifies the resource, but are harder to see, test and cache correctly. Choose one convention for the whole organization.

Probe 3 (senior): How do you retire `v1` safely?

Expected reasoning: announce a date, send deprecation and sunset signals, identify remaining callers from request metadata such as a client id header, contact them, reduce traffic gradually and then remove it. Do not remove on schedule if important consumers remain, unless the policy says so and that was communicated.

Tier 4 - Deep dive

Internals: a common structure keeps one domain model and maps it to separate request and response models per API version, so version differences stay at the edge. Versioning the whole API gives clients one number to track, while versioning single resources limits churn but is harder to reason about. Recent Spring versions add built-in API versioning support, so check whether yours does rather than assuming. Event schemas need the same discipline, and a schema registry can enforce backward, forward or full compatibility modes between producer and consumer versions.

Realistic failure: a team changes `amount` from cents to dollars without changing the name. No schema checks fail, no client crashes, and totals are silently wrong. Changes in meaning are the most dangerous breaks because tooling does not see them. Add a new field with a new name, deprecate the old one, and document units in the contract.

Trade-off: every supported version is more code, tests and operational burden, but removing versions too quickly damages trust and forces clients to upgrade in lockstep with you.

How to verify: run the previous release's client test suite or contract tests against the new server in CI. Add a field and confirm old clients keep working. Track per-version call counts before retirement.

Q280 What are ETags and conditional requests used for

Source basis: RFC 9110: Conditional requests.

Tier 1 - Foundation

In plain language: an ETag is a label the server attaches to a specific version of a resource, such as a fingerprint. The client can send it back to ask, "has it changed?" or to say, "only update it if it has not changed." That makes caching cheaper and prevents one user from silently overwriting another's edit.

Key terms:

- **ETag:** an opaque validator for a representation of a resource.
- `If-None-Match`**:** the client's "send it only if it differs from this ETag."
- `If-Match`**:** the client's "apply this change only if the resource still has this ETag."
- `304 Not Modified`**:** the response when the cached copy is still current.
- `412 Precondition Failed`**:** the response when an `If-Match` condition is false.
- `428 Precondition Required`**:** the response when a conditional header is required and missing.
- **Lost update:** two writers read the same version and the later write erases the earlier one.

Small example: two admins open the same customer record. The first saves a new address. The second then saves a change to the phone number, based on the old copy. With `If-Match`, the second save fails with `412` instead of overwriting the address.

Tier 2 - Core answer (30-second version)

> An ETag identifies a specific version of a resource. For reads, `If-None-Match` returns 304 with no body when nothing changed. For writes, `If-Match` makes the update conditional, and the server returns 412 if the resource changed since the client read it, which prevents lost updates. I would derive a strong ETag from JPA `@Version` only if every change to the selected representation advances that version, including any related data it contains, and distinguish representation variants. Otherwise I need a representation-wide revision or hash as well as the database concurrency check.

Tier 3 - Follow-ups

Probe 1 (warm-up): What is the difference between how `304` and `412` are used?

Expected reasoning: `304` answers a conditional read: the cached copy is still valid, so no body is sent. `412` answers a conditional write: the precondition failed, so the change was not applied.

Probe 2 (applied): How do you map this to JPA?

Expected reasoning: expose `@Version` as a strong ETag only when it changes for every observable change to the representation. If the response includes related entities, an independent child update may leave the root version unchanged; use an aggregate revision or representation hash and ensure the write transaction validates all relevant revisions atomically. Include a variant identifier when negotiated representations differ. For a single versioned entity, compare the client's `If-Match` with the current version and let the persistence layer enforce the version in `UPDATE ... WHERE version = ?`. An application-only comparison followed by an unguarded save leaves a race. Map a stale conditional update to `412`; `409` can describe a conflict outside an HTTP precondition. See RFC 9110 strong validators.

Probe 3 (senior): What does Spring's `ShallowEtagHeaderFilter` do, and what does it not do?

Expected reasoning: it computes an ETag from the response body and answers `304` when it matches `If-None-Match`, which saves bandwidth. The server still builds the full response, so it saves no processing, and it is not a concurrency control for writes.

Tier 4 - Deep dive

Internals: ETag values are quoted strings, and a weak validator is prefixed `W/`. RFC 9110 says `If-Match` uses strong comparison, while `If-None-Match` uses weak comparison. `If-None-Match: *` on a `PUT` can mean "create only if it does not exist." A server can require conditional headers on updates and answer `428` when they are missing.

Realistic failure: an ETag computed from a content hash that leaves out one field means changes to that field never alter the ETag, so clients are told their copy is current when it is not. Another failure is differing ETags across instances behind a load balancer because of per-node compression or ordering, which defeats caching and breaks `If-Match`.

Trade-off: conditional writes force clients to read before writing and handle conflicts. That is the right cost when overwrites matter, and unnecessary friction for data where last-write-wins is acceptable.

How to verify: read a resource in two clients, update it from the first, then update from the second with the old ETag. Expect `412` and an intact first update. Change a related field included in the representation without updating the root entity, and confirm the ETag still changes. Check distinct negotiated representations use valid validators. Request with a current `If-None-Match` and confirm `304` and no body.

Q281 How do you set timeouts and retries so one slow dependency does not take everything down

Tier 1 - Foundation

In plain language: a call to another service can be slow or never return. If callers wait forever, threads and connections fill up and the failure spreads. Timeouts limit how long you wait, and retries try again when a failure may be temporary, but careless retries make an outage worse.

Key terms:

- **Connect timeout / read timeout:** limits for establishing a connection and for waiting for data.
- **Deadline:** the total time allowed for the whole user request.
- **Backoff and jitter:** growing, randomized delays between retries.
- **Idempotent:** repeating the operation has the same effect as doing it once.
- **Retry budget:** a cap on how much traffic may be retried.
- **Circuit breaker:** a switch that stops calls to a failing dependency for a while.

Small example: service A calls B, which calls C, and both call layers allow three retries after the initial attempt. That is four attempts per layer, so one failing request can become 4 x 4 = 16 attempts against C, which is already struggling. With three call layers allowing three total attempts each, the multiplier would be 3 x 3 x 3 = 27. Always distinguish retries from total attempts.

Tier 2 - Core answer (30-second version)

> Every remote call needs a timeout, and timeouts should shrink down the call chain inside one overall deadline. Retry only safe, idempotent operations on retryable errors, with exponential backoff and jitter, at one layer, and cap retries with a budget, because retries multiply across layers. Add a circuit breaker so a clearly failing dependency fails fast, and a bulkhead so it cannot use every thread.

Tier 3 - Follow-ups

Probe 1 (warm-up): Which operations are safe to retry?

Expected reasoning: reads and operations that are idempotent by design, or made idempotent with an idempotency key. A non-idempotent `POST` such as "charge the card" can duplicate its effect if the first attempt actually succeeded but the response was lost.

Probe 2 (applied): Why add jitter to backoff?

Expected reasoning: without randomization, many clients that failed together retry together, producing synchronized load spikes. Jitter spreads retries over time.

Probe 3 (senior): Where do you put retries, and how do deadlines help?

Expected reasoning: retry at one layer, usually the closest to the caller that can judge whether retrying is appropriate, and avoid retries in every layer. Pass the remaining deadline downstream, as gRPC does, so a downstream call is never given more time than the caller has left and abandoned work stops.

Tier 4 - Deep dive

Internals: timeouts bound thread occupancy. With 200 worker threads and a dependency that hangs for 30 seconds, even a modest request rate exhausts the pool within seconds, and unrelated endpoints stop responding. A circuit breaker moves between closed, open and half-open states using failure rate or slow-call thresholds. A retry budget limits retries to a small fraction of normal traffic, so a widespread failure cannot be multiplied. Resilience4j's aspect order matters, as discussed in Q165.

Realistic failure: a dependency slows from 50 ms to 5 s. Clients retry, the dependency receives three times the load, and recovery is impossible until traffic is shed. The fix is shorter timeouts, one retry layer, a budget, a breaker, and a fallback that serves cached or reduced data.

Trade-off: a short timeout fails requests that would have succeeded slowly, and a long one ties up resources. Retries improve success under transient failure and worsen overload. Set timeouts from the dependency's measured latency, for example a high percentile with headroom, then revisit them.

How to verify: inject latency and errors into a dependency in staging. Confirm callers time out within budget, total attempts stay bounded, and unrelated endpoints remain healthy.

Q282 Blue-green, canary and rolling deployments: how do you choose, and what about the database

Tier 1 - Foundation

In plain language: releasing a new version without downtime means old and new code run side by side for a while. Different strategies control how traffic moves, and the database and message formats must work with both versions at once.

Key terms:

- **Rolling update:** replace instances a few at a time.
- **Blue-green:** run a full second environment and switch traffic to it.
- **Canary:** send a small share of real traffic to the new version first.
- `maxSurge` / `maxUnavailable`**:** Kubernetes settings for how many extra or missing pods are allowed during a rolling update.
- **Expand/contract:** change the schema in compatible steps, adding first and removing later.

Small example: you add a column. During the rollout, old pods that do not know the column and new pods that need it both talk to the same database.

Tier 2 - Core answer (30-second version)

> Rolling updates replace instances gradually and are the Kubernetes default. Blue-green switches between two full environments and makes rollback a switch back, at the cost of double capacity. A canary exposes a small share of real traffic first and widens it only if error rate and latency stay healthy, and it needs traffic splitting. In every case old and new versions overlap, so database changes and message formats must be backward compatible, using expand/contract.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why must a database change be backward compatible during a rolling update?

Expected reasoning: for part of the rollout, old pods still run against the new schema. A change that removes or renames something the old version uses makes it fail mid-deployment.

Probe 2 (applied): What do you watch during a canary, and how do you decide?

Expected reasoning: error rate, latency percentiles, saturation and a key business metric, compared against the baseline version over enough traffic to be meaningful. Define automatic abort thresholds beforehand. One percent of a low-traffic endpoint may be too few requests to show a problem.

Probe 3 (senior): The new version is bad and the migration already ran. How do you roll back?

Expected reasoning: roll back the code, not the schema. Because the migration was an expansion that the old code tolerates, the old version works against it. Destructive "contract" steps happen only in a later release, after the new version is stable.

Tier 4 - Deep dive

Internals: a Kubernetes Deployment creates a new ReplicaSet and shifts replicas while readiness probes gate traffic, constrained by `maxSurge` and `maxUnavailable`, both of which default to 25%. Canary and blue-green traffic shifting needs an ingress controller, service mesh or a rollout controller such as Argo Rollouts or Flagger. Expand/contract in practice:

1. Expand: add the new column or table, nullable or with a default, without breaking the old version.
2. Deploy code that writes both forms atomically while continuing to read the old form. Wait until every old-only writer, including jobs, has stopped.
3. Backfill in bounded batches using row locking or version checks so a stale backfill cannot overwrite a concurrent update. Validate completeness and agreement.
4. Switch reads to the new form while maintaining both writes for the rollback window. A non-null new value is not proof of freshness while old-only writers remain.
5. After the rollback window, stop using the old form and remove it in a later contract release. See GitLab's migration guidance for staged changes and synchronization approaches.

The same applies to message formats, since old and new consumers or producers may overlap.

Realistic failure: a migration renames a column in the same release that uses the new name. Requests handled by old pods fail as soon as they access the renamed column, even if new pods become ready. Readiness alone cannot make an incompatible schema safe.

Trade-off: blue-green gives fast switching and simple rollback but needs double capacity and complicates stateful data. Canary gives the safest real-traffic validation and needs routing and observability. Rolling is simplest but exposes users to the new version gradually, with slower rollback.

How to verify: deploy a deliberately bad version to a small slice and confirm the metric gate stops it. Run old and new versions against the migrated schema in CI before releasing. Interleave an old-only write, a dual write and a backfill; verify that reads stay correct and rollback works throughout its supported window.

Q283 Why do pooled HTTP connections fail intermittently after idle periods

Tier 1 - Foundation

In plain language: opening a network connection is expensive, so HTTP clients keep connections open and reuse them. Something in the path, such as the server, a load balancer or a firewall, may close a connection that has been idle too long. The client does not notice until it tries to use it, and that request fails.

Key terms:

- **Keep-alive:** keeping a connection open for further requests.
- **Connection pool:** a set of reusable open connections.
- **Idle timeout:** how long an unused connection may stay open before something closes it.
- **Stale connection:** one the other side has already closed.
- **Connection reset:** an error shown when you use a connection the peer has dropped.
- **Max connections per route:** the pool limit for one destination.

Small example: a service calls a partner API every few minutes. The first call after a quiet period often fails with "connection reset," and the next call works.

Tier 2 - Core answer (30-second version)

> Pooled connections that sit idle can be closed by the server, load balancer or NAT, and the client discovers it only when it reuses the connection, so the first request after a quiet period fails. Make the client's idle timeout shorter than the shortest idle timeout in the path, evict or validate idle connections before reuse, and retry idempotent requests once. Size request capacity from request rate times latency, accounting for HTTP/2 multiplexing, and coordinate connection lifetime with DNS cache expiry so reconnects can notice address changes.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does only the first request after idle fail?

Expected reasoning: the idle connection was closed by the other side. The failure on reuse removes it from the pool, and the next request opens a fresh connection, which works.

Probe 2 (applied): How do you fix it?

Expected reasoning: set the client's maximum idle time below the lowest idle timeout in the path (server, load balancer, firewall), evict idle connections in the background or validate a connection that has been idle before reuse, and retry once for idempotent requests. The settings and names differ between clients such as Apache HttpClient, Reactor Netty and the JDK client, so check the documentation.

Probe 3 (senior): How do you size the pool, and what do exhaustion symptoms look like?

Expected reasoning: by Little's law, in-flight requests are roughly request rate times average latency, per destination, plus headroom. With HTTP/1.1 without pipelining, that approximates occupied connections. HTTP/2 multiplexes streams on a connection, so size stream capacity and connections using client/server limits and measurements; see RFC 9113 streams. Too small a pool shows as requests waiting to lease a connection and "connection request timeout" errors, even though the dependency is healthy. Too large a pool can overload the dependency.

Tier 4 - Deep dive

Internals: when a peer closes a TCP connection the client may not learn of it until it writes, and then receives a reset. A race remains even with validation, because the peer can close the connection between the client's check and its write; a bounded retry can recover when the operation is safe to repeat. An existing pooled connection keeps its original peer address. A maximum connection lifetime forces eventual replacement, but a new connection can still use a cached DNS address. For clients using the JVM resolver, `networkaddress.cache.ttl` controls positive caching and `-1` means cache forever; some clients have their own resolvers and cache settings. Coordinate connection replacement with the actual resolver's finite TTL and failover behavior. See Java 21 DNS caching.

Realistic failure: a load balancer closes idle connections after 60 seconds, while the client keeps them for five minutes. Roughly one request per quiet period fails, in a pattern that looks random and appears only in production. The check is to compare the idle timeouts of every hop with the client's setting.

Trade-off: aggressive eviction and short lifetimes cost more handshakes, including TLS, and add latency and CPU. Long-lived idle connections save that cost and risk stale reuse. Retries cover the gap only for idempotent calls.

How to verify: reproduce by waiting past the idle timeout and sending one request, then confirm the fix removes the failure. In a separate failover test, change the resolved address and verify new connections use it within the combined DNS-cache and connection-lifetime policy. Expose pool metrics such as leased, available and pending connections, through Micrometer where the client supports it, and alert on pending requests.

Q284 What does a PodDisruptionBudget protect against, and what can it not do

Source basis: Kubernetes: Specifying a Disruption Budget.

Tier 1 - Foundation

In plain language: Kubernetes sometimes removes pods on purpose, for example when a node is drained for an upgrade. A PodDisruptionBudget (PDB) says how many pods of an application may be unavailable at once during such planned removals, so maintenance does not take the whole service down.

Key terms:

- **Voluntary disruption:** a planned removal such as a node drain or an eviction.
- **Involuntary disruption:** an unplanned loss such as a node crash or an out-of-memory kill.
- **Eviction:** the API request to remove a pod gracefully.
- `minAvailable` / `maxUnavailable`**:** the budget expressed as pods that must stay, or may go.
- **Topology spread:** a scheduling rule that places replicas across nodes or zones.

Small example: an app has three replicas and a PDB with `minAvailable: 2`. Draining a node can evict one pod at a time, and the drain waits if another is already unavailable.

Tier 2 - Core answer (30-second version)

> A PodDisruptionBudget limits how many pods can be down during voluntary disruptions like node drains, by making the eviction API refuse an eviction that would break the budget. It does not protect against involuntary failures such as a node crash or an out-of-memory kill, and it does not add capacity. Use it with enough replicas spread across nodes and zones, and avoid budgets that cannot be satisfied, because they block maintenance.

Tier 3 - Follow-ups

Probe 1 (warm-up): Does a PDB stop an out-of-memory kill?

Expected reasoning: no. A PDB governs voluntary evictions. A crash, an OOM kill or a node failure is involuntary, so it still happens regardless of the budget.

Probe 2 (applied): One replica and `minAvailable: 1`. What happens during a node drain?

Expected reasoning: the eviction would take availability below the budget, so it is refused and the drain stalls. The same occurs when `minAvailable` equals the replica count. Either run more replicas, or use `maxUnavailable` and accept a short outage deliberately.

Probe 3 (senior): How do you make sure replicas survive a single node or zone loss?

Expected reasoning: a PDB is not enough, because it does not decide placement. Use topology spread constraints or pod anti-affinity to place replicas on different nodes and zones, run enough replicas, and combine with readiness probes and graceful shutdown.

Tier 4 - Deep dive

Internals: when an eviction would violate a budget the API server rejects it with a `429` response, and tools such as `kubectl drain` retry. A direct `kubectl delete pod` is not an eviction and bypasses the budget. Cluster autoscaler scale-down and managed-platform node upgrades generally respect PDBs, though managed upgrades may stop honoring a budget after a time limit, so check your provider's documentation.

Realistic failure: a team sets `minAvailable: 100%` on every service. A cluster upgrade then stalls for hours because no pod can ever be evicted, and an engineer force-deletes pods to finish, losing the protection the budget was meant to give.

Trade-off: a strict budget protects availability and slows or blocks maintenance. A loose one speeds upgrades and risks more simultaneous loss. Pick the budget from the service's real tolerance and replica count.

How to verify: run `kubectl get pdb` and read the allowed-disruptions value. Drain a node in a test cluster and confirm evictions wait rather than dropping below the budget. Create an unsatisfiable PDB and observe the stuck drain, so the symptom is recognizable.

Q285 Service mesh or library-based resilience

Tier 1 - Foundation

In plain language: a service mesh puts a small proxy next to each service so the platform handles network concerns such as encryption, retries and traffic routing. A resilience library does similar work inside the application code. The choice is about where the logic lives and who controls it.

Key terms:

- **Sidecar:** a proxy container deployed alongside each application container.
- **Data plane / control plane:** the proxies that carry traffic, and the component that configures them.
- **mTLS:** mutual TLS, where both sides authenticate with certificates.

- **Traffic splitting:** sending a percentage of requests to different versions.
- **Fallback:** an alternative response when a call fails.

Small example: a mesh retries a failed call to a backend automatically, with no code change. A library lets the code, on failure, return the last cached price instead.

Tier 2 - Core answer (30-second version)

> A mesh moves cross-cutting network behavior, such as mTLS, retries, timeouts, traffic splitting and telemetry, into the platform and works across languages. A library runs in the process, so it can use business context, such as returning cached data or failing one operation without affecting another. A mesh adds latency, resources and operational complexity, and the biggest trap is stacked retries from the mesh, the client and the application. Choose one owner for each behavior.

Tier 3 - Follow-ups

Probe 1 (warm-up): What can a mesh give you that a library cannot?

Expected reasoning: uniform behavior across languages without code changes, automatic mTLS between services with certificate rotation, consistent telemetry, and platform-controlled traffic management such as canary routing.

Probe 2 (applied): What can a library do that a mesh cannot?

Expected reasoning: decisions that depend on application meaning. A mesh sees requests and responses, not business context, so it cannot choose a sensible fallback value or treat two operations in the same service differently based on domain rules.

Probe 3 (senior): How do you prevent retry amplification?

Expected reasoning: list every layer that can retry, including mesh, client library and application, and compute the worst case by multiplying. Assign retries to one layer, restrict retry conditions to idempotent requests and specific failures, and add a retry budget.

Tier 4 - Deep dive

Internals: proxies, commonly based on Envoy, intercept traffic and are configured by the control plane. Certificates are issued and rotated automatically for mTLS. Each hop adds some latency and resource use, which you should measure for your workload. Some meshes offer modes without a per-pod sidecar, and their maturity and limits vary by version, so check yours.

Realistic failure: a mesh policy retries on any 5xx, including for a non-idempotent `POST`. A timeout on a payment endpoint then triggers a retry, and the customer is charged twice. Restrict retry rules to idempotent methods or require idempotency keys. Debugging also gets harder: a `503` may come from the proxy, not the application, so learn the proxy's response flags and read its logs.

Trade-off: a mesh gives consistent policy and security at scale in return for platform complexity and a team to run it. Library resilience is simple for a few services in one language and drifts across teams in a larger fleet.

How to verify: inject failures and watch whether traffic to the failing service multiplies. Check which layer made each retry. Test that mTLS is enforced by attempting a plaintext call.

JVM and Java

Q286 How do you choose a garbage collector

Source basis: JEP 439: Generational ZGC.

Tier 1 - Foundation

In plain language: the JVM automatically frees objects that nothing can reach any more. Different collectors make different trade-offs between how long the application pauses, how much work it completes, and how much memory the collector itself needs.

Key terms:

- **Heap:** the memory where Java objects live.
- **Young / old generation:** new objects versus long-lived ones, collected at different frequencies.
- **Stop-the-world pause:** a moment when application threads are halted for collection work.
- **Throughput:** the share of time spent running the application rather than collecting.
- **Allocation rate:** how fast the application creates objects.

Small example: a payment API has an acceptable average latency but its 99th-percentile latency spikes every few minutes, and the spikes line up with garbage collection pauses.

Tier 2 - Core answer (30-second version)

> Start from the goal. G1 is the default on modern JDKs and a good general choice, with a pause-time target it tries to meet. ZGC targets very low pauses that do not grow with heap size, at some cost in throughput and memory. Parallel GC maximizes throughput for batch work, and Serial suits tiny heaps. Confirm which collector is actually running, tune from GC logs and load tests, and often fix allocation behavior before changing collectors.

Tier 3 - Follow-ups

Probe 1 (warm-up): What does `-XX:MaxGCPauseMillis` do?

Expected reasoning: it sets a pause-time goal for G1, 200 ms by default. G1 adjusts its work to try to meet it, but it is a target, not a guarantee, and a very low target can reduce throughput.

Probe 2 (applied): Your container runs a different collector than you expected. Why?

Expected reasoning: the JVM selects a collector from the resources it detects. In a container with very few CPUs or little memory it can choose Serial instead of G1. Check with `-Xlog:gc` or `-XX:+PrintFlagsFinal`, and set the collector explicitly if it matters.

Probe 3 (senior): When would you move from G1 to ZGC?

Expected reasoning: when measured pauses violate a latency objective, especially with large heaps, and the throughput and memory cost is acceptable. In JDK 21 generational mode is enabled with `-XX:+UseZGC -XX:+ZGenerational`, and later releases changed the defaults, so check your JDK. Compare both under realistic load before committing.

Tier 4 - Deep dive

Internals: G1 splits the heap into regions and collects the ones with most garbage first, using young and mixed collections. Very large objects (humongous allocations, see Q205) are handled specially and can cause trouble. If evacuation cannot find space it falls back to a long full collection. ZGC does most work concurrently using load barriers and concurrent relocation, so pause times stay short regardless of heap size, with extra memory and CPU overhead.

Realistic failure: a team raises the heap to stop long pauses, and the container is killed by the kubelet for exceeding its memory limit, because the heap is only part of the JVM's total memory (it also uses metaspace, thread stacks and native memory). The real cause was a high allocation rate from a hot loop.

Trade-off: lower pauses generally cost throughput and memory, higher throughput generally costs longer pauses. Tuning flags rarely beats reducing allocation or sizing the heap and container correctly.

How to verify: enable `-Xlog:gc*` and examine pause distribution, allocation rate and old-generation behavior. Run the same load with G1 and ZGC and compare p99 latency, throughput and memory.

Q287 How do you find a memory leak in a production JVM

Source basis: JDK 21 jcmd diagnostic impact.

Tier 1 - Foundation

In plain language: in Java a leak is not lost memory. It is objects that are still reachable, so the collector cannot free them, but the program no longer needs them. Memory use keeps growing until the process fails.

Key terms:

- **GC root:** a starting reference the collector trusts, such as a static field or a thread stack.
- **Reachable:** an object connected to a GC root through references.
- **Heap dump:** a snapshot of all objects in the heap.
- **Retained size:** the memory that would be freed if an object were removed.
- **Dominator tree:** a view showing which objects keep the most memory alive.
- **Native Memory Tracking (NMT):** JVM tooling to see non-heap memory use.

Small example: code stores each session in a static `Map` keyed by session id and never removes finished sessions, so the map grows with every user.

Tier 2 - Core answer (30-second version)

> First establish which memory is growing: Java heap, metaspace, native memory, or the container limit being hit. A heap leak shows old-generation usage that keeps rising after full collections. Start with existing metrics and JFR evidence; take a histogram or heap dump only after assessing its impact, then use a tool like Eclipse MAT to find the dominator and follow the path to the GC root. Typical causes are static collections, unbounded caches, forgotten listeners, ThreadLocals in pooled threads and class-loader leaks.

Tier 3 - Follow-ups

Probe 1 (warm-up): How do you tell a leak from a heap that is simply too small?

Expected reasoning: look at memory after full collections over time. A leak shows a rising floor that never comes back down. An undersized heap shows a stable floor that is close to the limit, with frequent collections.

Probe 2 (applied): The pod restarts with no Java exception. Is it a heap leak?

Expected reasoning: not necessarily. If Kubernetes reports `OOMKilled`, the container exceeded its memory limit, and the cause may be native memory, direct buffers, thread stacks or metaspace, not the Java heap. Check NMT and the container's memory metrics, not only heap usage.

Probe 3 (senior): What are the risks of taking a heap dump from production?

Expected reasoning: it can pause the application, needs disk space comparable to the heap, and contains data such as personal information and secrets. Take it from one instance that has been removed from rotation if possible, store and delete it under the data policy, and start with metrics or JFR sampling where they can answer the question. A histogram produces less output than a dump but is also documented as high impact; do not treat it as a cheap, pause-free first step.

Tier 4 - Deep dive

Internals: useful commands:

```
jcmd <pid> GC.class_histogram | head -30      # HIGH impact; head only trims output
jcmd <pid> GC.heap_dump /tmp/heap.hprof       # pauses the app; check disk space
jcmd <pid> JFR.start settings=profile duration=5m filename=/tmp/rec.jfr
# Native memory needs the JVM started with -XX:NativeMemoryTracking=summary:
jcmd <pid> VM.native_memory summary
```

Taking two histograms some time apart shows which classes grow, but each scan can impose a substantial pause or trigger collection depending on options and collector. The JDK classifies both `GC.class_histogram` and `GC.heap_dump` as high impact. Limit production use to an assessed diagnostic window; truncating output with `head` does not reduce heap-inspection work. In MAT, the dominator tree and retained size point at the structure holding the memory, and the path to GC roots shows who holds the reference. JFR's old-object sampling can show where leaking objects were allocated with less overhead than a dump. Starting the JVM with `-XX:+HeapDumpOnOutOfMemoryError` captures evidence at failure.

Realistic failure: a pooled thread stores a request context in a `ThreadLocal` and never clears it. Because the pool reuses threads, each holds its last context forever, and a large object graph stays reachable. The fix is to clear the value in a `finally` block or avoid the ThreadLocal.

Trade-off: diagnostic data costs pause time, disk and privacy risk, while waiting for a crash costs availability. Gather the cheapest evidence that can answer the question first.

How to verify: reproduce growth in a soak test, apply the fix and confirm that the post-collection floor levels off.

Q288 What are the trade-offs of GraalVM native images for Spring Boot

Source basis: Spring Boot native images.

Tier 1 - Foundation

In plain language: a normal Java application starts a virtual machine, loads classes and compiles hot code while running. A native image is compiled ahead of time into a standalone executable, so it starts faster and needs less memory, in exchange for restrictions on dynamic features.

Key terms:

- **AOT (ahead-of-time) compilation:** compiling before the program runs.
- **JIT (just-in-time) compilation:** compiling hot code while the program runs.

- **Closed-world assumption:** the build must know every class that can be reached.
- **Reflection hints:** build-time metadata telling the compiler which classes are used reflectively.
- **Warm-up:** the time a JVM needs to reach full speed.

Small example: a function that must scale from zero starts in milliseconds as a native image, instead of seconds as a JVM application.

Tier 2 - Core answer (30-second version)

> A native image gives much faster startup and lower idle memory, which suits serverless, scale-to-zero and dense deployments. The cost is the closed-world assumption: reflection, proxies and resources must be known at build time, and Spring Boot's AOT processing generates many of the hints but libraries can need more. Builds are slower and heavier, tooling differs, and peak throughput after warm-up can be lower than a tuned JVM. I would choose it only when startup time or footprint is a real constraint, and test the native build itself.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why does reflection cause problems?

Expected reasoning: the build analyzes reachable code and removes the rest, and reflection finds classes by name at run time, which the analysis cannot see unless told. Missing hints produce runtime failures that never occurred on the JVM.

Probe 2 (applied): It works on the JVM and fails in the native image. How do you find the cause?

Expected reasoning: look for missing reflection, proxy, resource or serialization hints. Run the tests against the native build, use the GraalVM tracing agent to find what the application uses dynamically, and add hints through Spring's runtime hints API or registration annotations.

Probe 3 (senior): When is the JVM the better choice?

Expected reasoning: for long-running services with stable load, where warm-up is amortized and peak throughput, mature tooling and simple builds matter more than startup. Compare measured startup, memory and steady-state latency, and consider alternatives that keep the JVM, such as lazy initialization, trimmed auto-configuration and class-data sharing.

Tier 4 - Deep dive

Internals: the native-image builder performs a static analysis from the entry points, includes only reachable code, initializes some classes at build time and produces an executable with its own runtime and garbage collector. Spring Boot's AOT step generates bean definitions and hints ahead of time, so the application does

less discovery at startup. Features that rely on runtime class generation or loading behave differently, and some JVM tooling such as standard JFR and heap dump workflows differs or is unavailable, so check what your GraalVM distribution supports.

Realistic failure: a DTO serialized by Jackson works in all JVM tests. In the native build the JSON comes back empty or fails because no hint registered its members for reflection. It reaches production because only JVM tests ran. Run the integration tests in native mode in CI.

Trade-off: startup and memory gains come with longer builds, a build that can need several gigabytes of memory, less flexibility with dynamic libraries and possibly lower peak throughput. The operational question is whether the gain changes your cost or scaling behavior.

How to verify: build both forms, run the same test suite against each, and measure startup time, resident memory and steady-state latency under realistic load.

Q289 What are structured concurrency and scoped values (a later-version comparison)

Source basis: Java 21 StructuredTaskScope API and Java 21 ScopedValue API.

Label: in Java 21 both are **preview** APIs requiring `--enable-preview`, and their status and API shape have changed in later releases. This question is a comparison with the guide's Java 21 baseline. Check the final status in your JDK before relying on either.

Tier 1 - Foundation

In plain language: structured concurrency treats several related tasks as one unit of work that starts together and finishes together, like a block of code. Scoped values let code share a read-only binding with everything called inside a bounded region, as a safer alternative to thread-local variables. The bound object itself can still be mutable.

Key terms:

- **Structured concurrency:** subtasks cannot outlive the scope that started them.
- `StructuredTaskScope`**:** the Java 21 preview class that groups subtasks.
- **Fork / join:** start a subtask, then wait for the group.
- **Scoped value:** a read-only binding for the duration of a scope; it does not freeze the bound object.
- `ThreadLocal`**:** a mutable per-thread variable, with a lifetime tied to the thread.

Small example: loading a page needs a user and their orders. Both are fetched concurrently, and if one fails the other should be cancelled and the failure reported.

Tier 2 - Core answer (30-second version)

> Structured concurrency groups related tasks so they cannot outlive the scope, errors propagate and a failure can cancel the siblings, which avoids leaked threads and orphaned futures. Scoped values share read-only bindings, preferably to immutable data such as a request id, with code called within a bounded scope, and are meant as a safer alternative to ThreadLocal, especially with many virtual threads. In Java 21 both are preview, so I would not ship them to production without a deliberate decision.

Tier 3 - Follow-ups

Probe 1 (warm-up): What problem does structured concurrency solve compared with hand-built futures?

Expected reasoning: with ad hoc futures, a failed task may leave others running, an exception can be lost, and no code block owns the lifetime. A scope gives the tasks a clear owner and a defined end.

Probe 2 (applied): How does `ShutdownOnFailure` behave when one subtask throws?

Expected reasoning: the first failing subtask shuts down the scope and interrupts unfinished siblings. `join()` waits for all subtasks to finish or the scope to shut down, so it can return before an interrupted sibling stops. `throwIfFailed()` reports the first failure. The implicit `close()` at the end of the try-with-resources block waits for remaining threads to terminate; a sibling that ignores interruption can block closure indefinitely. Cancellation is cooperative: use interruptible work and bounded dependency calls, and do not treat `joinUntil()` as a hard bound on scope closure.

Probe 3 (senior): Why prefer a scoped value to a `ThreadLocal` here?

Expected reasoning: a `ThreadLocal` is mutable, lives as long as its thread, and with pooled threads can leak data between requests. Copying many thread-locals into many virtual threads also costs memory. A scoped-value binding is read-only within its scope, can be rebound in a nested scope, and is inherited by subtasks forked in a structured scope. The referred-to object is not automatically immutable; use immutable objects or synchronize shared mutation.

Tier 4 - Deep dive

Internals:

```
// Java 21 only with --enable-preview; the API may differ in later releases.
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var user   = scope.fork(() -> loadUser(id));
    var orders = scope.fork(() -> loadOrders(id));
    scope.join().throwIfFailed();              // failure triggers sibling interruption
    return new Page(user.get(), orders.get());
} // close waits for unfinished subtasks to terminate
```

By default each fork runs in a new virtual thread. `ShutdownOnSuccess` is the counterpart for "first result wins." A scoped value is bound with `ScopedValue.where(...)` and read within that call tree. Neither mechanism replaces a transaction or a dependency timeout. `joinUntil()` bounds that join, but `close()` still waits for unfinished subtasks; their work must respond to interruption or have its own timeout.

Realistic failure: a team uses a preview API in a library, upgrades the JDK, and the API changes shape or the preview flag is no longer accepted, so the build breaks. Preview features are for evaluation, and production use needs an owner who tracks the JEPs.

Trade-off: the structured form is easier to reason about and clean up. In return it requires newer APIs and, in Java 21, preview flags. `CompletableFuture` works everywhere and is flexible, and cancellation and lifetime handling are manual.

How to verify: make one subtask throw and confirm the sibling receives interruption. Hold that sibling behind a test latch and verify `join()` can return while it is unfinished; release it and verify `close()` waits for its termination. Confirm the scoped binding disappears outside its scope, a nested rebind restores the outer binding, and a bound mutable object remains mutable. Use immutable objects or synchronization before sharing it with subtasks.

Q290 ClassNotFoundException versus NoClassDefFoundError, and how class loading causes dependency bugs

Tier 1 - Foundation

In plain language: Java loads classes on demand from the classpath. If a class is missing, or two versions of a library are present, the program fails at run time rather than at compile time. The two error names look similar but describe different situations.

Key terms:

- **Classpath:** where the JVM looks for classes.
- **Class loader:** the component that finds and loads a class.
- **Parent delegation:** a loader asks its parent before looking itself.
- **Transitive dependency:** a library your dependency depends on.
- `ServiceLoader`**:** the Java mechanism for discovering implementations of an interface.

Small example: your code compiles against library version 2, but another dependency pulls in version 1, so at run time a method that exists in 2 is missing.

Tier 2 - Core answer (30-second version)

> `ClassNotFoundException` happens when code asks to load a class by name and it is not found. `NoClassDefFoundError` means a class that was there at compile time is missing at run time, or failed to initialize. Most real incidents in a Maven project come from version conflicts between transitive dependencies, which show up as `NoSuchMethodError` or `LinkageError`. Diagnose with `mvn dependency:tree`, then pin or exclude versions, and use the Enforcer plugin to catch conflicts at build time.

Tier 3 - Follow-ups

Probe 1 (warm-up): What is the difference between the two errors?

Expected reasoning: `ClassNotFoundException` is a checked exception from an explicit load such as `Class.forName` or reflection. `NoClassDefFoundError` is an error raised when the JVM, while linking or running code that was compiled against a class, cannot find or initialize it.

Probe 2 (applied): A `NoClassDefFoundError` **appears for a class that is on the classpath. Why?**

Expected reasoning: its static initializer probably failed earlier. The first failure throws `ExceptionInInitializerError`, and every later use of the class throws `NoClassDefFoundError`. Find the first error in the logs, not the repeated one.

Probe 3 (senior): You get `NoSuchMethodError` **after upgrading one dependency. How do you diagnose and prevent it?**

Expected reasoning: the compiled code called a method that the version loaded at run time does not have. Run `mvn dependency:tree` to find the competing versions, pin one in `dependencyManagement` or exclude the other, and add the Enforcer plugin's dependency convergence rule so it fails the build next time.

Tier 4 - Deep dive

Internals: loaders delegate to their parent first, so the first copy found along the delegation path wins. Servlet containers and plugin systems often change that order, which can load a different version than you expect. `ServiceLoader` finds providers through `META-INF/services` files, or `provides` in `module-info.java`. Building a

shaded jar can silently drop providers, because several jars contribute a file with the same path, so merge them, for example with Maven Shade's `ServicesResourceTransformer`.

Realistic failure: two JSON libraries each register a provider, and the shaded jar keeps only one service file. The application starts, but the missing provider never loads, and behavior differs from the unshaded build. It appears only in the packaged artifact.

Trade-off: pinning versions gives reproducibility and takes ongoing maintenance, and excluding a transitive dependency risks removing something a library needs. Shading avoids conflicts, at the cost of larger artifacts and merge details.

How to verify: print the actual source of a class with `SomeClass.class.getProtectionDomain().getCodeSource()` to see which jar was loaded. Run the Enforcer convergence rule in CI. Test the packaged artifact, not only the build directory.

Operations

Q291 How do you design alerts that wake people for the right reasons

Source basis: Google SRE Workbook: Alerting on SLOs.

Tier 1 - Foundation

In plain language: an alert should tell a person that users are being hurt and that action is needed. Alerts that fire for harmless internal fluctuations teach people to ignore them, so the real ones get missed.

Key terms:

- **SLI (service level indicator):** a measurement of user experience, such as the fraction of successful requests.
- **SLO (service level objective):** the target for an SLI, such as 99.9% over 30 days.
- **Error budget:** the allowed failure, which is 100% minus the SLO.
- **Burn rate:** how fast the error budget is being consumed relative to plan.
- **Symptom / cause:** what users feel versus the internal reason.
- **Runbook:** written steps for responding to an alert.

Small example: an alert that fires whenever CPU exceeds 80% wakes someone every night and rarely indicates a user problem. An alert that fires when the success rate drops below the SLO at a damaging pace does.

Tier 2 - Core answer (30-second version)

> Alert on symptoms users feel, such as errors and latency, not on every internal cause. Define an SLI and SLO and treat the remainder as an error budget, then page on burn rate, using a long and a short window together to cut noise and reset quickly. Every page should be actionable, link a runbook and have an owner. Keep cause-level metrics on dashboards for diagnosis, and remove alerts that fire without needing action.

Tier 3 - Follow-ups

Probe 1 (warm-up): Why alert on symptoms rather than causes?

Expected reasoning: many causes never reach users, and some user-facing problems have causes you did not anticipate. A symptom alert catches both. Cause metrics help the responder after the page.

Probe 2 (applied): What does a burn rate of 14.4 mean for a 30-day SLO?

Expected reasoning: the error budget is being consumed 14.4 times faster than the rate that would use it all in exactly 30 days. Sustained for one hour, that consumes about 2% of the budget (14.4 / 720 hours), which justifies a page.

Probe 3 (senior): How do you alert on a low-traffic service?

Expected reasoning: with few requests, one failure can swing the ratio and cause noise. Use longer windows, synthetic probes that generate steady traffic, or alert on absolute error counts alongside ratios.

Tier 4 - Deep dive

Internals: for a 99.9% SLO the budget is 0.1%. A burn rate of 1 means errors at 0.1%, using the budget exactly over the window. The workbook's illustrative thresholds for a 30-day window are a burn rate of about 14.4 over one hour (about 2% of the budget) as a page, about 6 over six hours (about 5%) as a page, and about 1 over three days (about 10%) as a ticket. Each pairs a long window with a short one, commonly one-twelfth the length, so the alert stops soon after the problem does.

Realistic failure: a team pages on pod restarts and CPU. Most pages are self-healing, engineers start muting the channel, and a real regression in checkout success goes unnoticed for an hour because nobody trusts the pager.

Trade-off: fast detection means more false alarms, and conservative thresholds miss short incidents. Burn-rate alerts balance detection speed against noise but depend on a good SLI. A poor SLI, such as one that excludes failing paths, makes the alert useless.

How to verify: replay a past incident against the alert rules and check that it would have paged at the right time. Inject errors in staging and confirm the burn-rate alert fires and clears. Review each page in a regular session and fix or remove noisy ones.

Q292 How do you manage secrets for services on GKE

Tier 1 - Foundation

In plain language: secrets such as database passwords and API keys must reach the application without being stored in code, images or plain config files. A service should prove who it is to a secrets store, receive only what it needs, and have credentials that can be replaced without a rebuild.

Key terms:

- **Secret store:** a managed service such as Google Secret Manager.
- **Workload identity:** a platform identity for a pod, so it needs no stored key file.
- **Least privilege:** granting only the access that is needed.
- **Rotation:** replacing a credential with a new one on a schedule or after exposure.
- **Kubernetes** `Secret`**:** an object holding sensitive data, base64-encoded.

Small example: a service needs a database password. Instead of a key file in the image, its pod authenticates through workload identity and reads the password from the secret store.

Tier 2 - Core answer (30-second version)

> Keep secrets out of Git, images and plain config. Store them in a managed secret store, give each workload its own identity with Workload Identity so it needs no long-lived key file, and grant only the secrets it needs. Kubernetes Secrets are base64, not encryption, so rely on access control and

> encryption at rest. Plan rotation from the start with secret versions and clients that can pick up new values, and have a tested process for a leaked credential.

Tier 3 - Follow-ups

Probe 1 (warm-up): Is a Kubernetes `Secret` encrypted?

Expected reasoning: base64 is an encoding, not protection. Secrets are protected by who can read them (RBAC) and by encryption at rest on the cluster, so restrict access and confirm how your cluster stores them.

Probe 2 (applied): Why avoid service-account key files?

Expected reasoning: a key file is a long-lived credential that can be copied, committed or leaked, and must be rotated manually. Workload Identity exchanges the pod's identity for short-lived tokens, so there is nothing static to steal.

Probe 3 (senior): A credential leaks. What do you do?

Expected reasoning: revoke or disable it first, issue a new one and roll it out, then investigate where it was used with audit logs. Rotation that was never exercised before is a common reason this takes hours.

Tier 4 - Deep dive

Internals: Workload Identity Federation for GKE maps a Kubernetes service account to an IAM identity, and the workload obtains short-lived credentials through the platform. Secrets can be mounted as files with the Secret Manager CSI driver, or read through a client library or framework integration such as Spring Cloud GCP, whose current options you should check in its documentation. Secret versions allow staged rotation: create a new version, update consumers, then disable the old one. For databases, a smooth rotation often means two valid credentials for a window.

Realistic failure: a key file is committed to a repository and found by a scanner. Deleting the file does not help, because history keeps it, so the key must be revoked and replaced. If rotation has never been rehearsed, the service goes down when the credential is revoked.

Trade-off: environment variables are easy but can leak through crash dumps, debug output and child processes, while mounted files with tight permissions reduce that exposure. Spring Boot 3 masks values in the `env` and `configprops` actuator endpoints by default, but verify your configuration and keep those endpoints restricted.

How to verify: rotate a database password in a non-production environment and confirm the service recovers without manual steps. Try to read a secret from a workload that should not have access and confirm it is denied. Review audit logs to confirm access is recorded.

Appendix A Complete producer-consumer implementation

One producer sends a separate end marker after all data. The owner observes task completion and cancels both tasks when either fails or the owner is interrupted. The data value 20 is no longer overloaded as a termination signal. Executor shutdown waits for cooperative tasks; a callback that ignores interruption can still prevent termination. This in-memory example provides no durability.

```
import java.util.Objects;
import java.util.concurrent.*;
import java.util.function.IntConsumer;
```

```
/** One producer and one consumer; callbacks must cooperate with interruption. */
public final class ProducerConsumerDemo {
    private record Item(int value, boolean end) {}
    private static final Item END = new Item(0, true);

    public static void run(int count, IntConsumer consume)
            throws InterruptedException, ExecutionException {
        if (count < 0) throw new IllegalArgumentException("negative count");
        Objects.requireNonNull(consume, "consume");
        BlockingQueue<Item> queue = new ArrayBlockingQueue<>(10);
        try (ExecutorService executor = Executors.newFixedThreadPool(2)) {
            CompletionService<Void> completion = new ExecutorCompletionService<>(executor);
            Future<Void> producer = completion.submit(() -> {
                for (int i = 0; i < count; i++) queue.put(new Item(i + 1, false));
                queue.put(END);
                return null;
            });
            Future<Void> consumer = completion.submit(() -> {
                while (true) {
                    Item item = queue.take();
                    if (item.end()) return null;
                    consume.accept(item.value());
                }
            });
            try {
                // Observe whichever task completes first, including early failure.
                completion.take().get();
                completion.take().get();
            } finally {
                // A failed or interrupted side must not strand its peer.
                producer.cancel(true);
                consumer.cancel(true);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        run(20, value -> System.out.println("Consumed: " + value));
    }
}
```

Java 21 BlockingQueue and ExecutorService lifecycle.

Appendix B Alignment and validation notes

Edition v25, 9 October 2026. The Markdown is the editable source for the PDF.

Alignment: Parts 01-16, Part 21 and five daily sets are copied from the current local ElevatePrep authored sources. Heading levels and navigation are adapted for one document; the answers and code retain their website wording. The two algorithm companions remain available through the website's guide. Q245-Q272, the

simulation lab, mock protocol and interviewer follow-up rules are retained from the supplied v23 document as supplemental material. They are not represented as newly reviewed website content.

Preserved repair: ElevatePrep's Q184 deliberately presents a shutdown race as an exercise. Appendix C retains v23's corrected bounded pool as a worked alternative. Appendix A retains the complete producer-consumer implementation. Neither example is a production executor implementation.

Validation for this edition: See the adjacent validation report for actual structural, SQL, Java, PDF and repository checks. Earlier edition check totals are historical and have not been carried forward as v25 results. Part 21 was independently reviewed against primary documentation; its fixture checks coalescing, successful and failed cleanup, cooperative cancellation, join versus close, nested scoped bindings and mutable bound objects. Cache-provider, database, DNS and HTTP guidance is source-backed rather than live infrastructure evidence. Framework mappings, PostgreSQL-specific execution, cloud deployments, Kubernetes and external services were not exercised end to end. Published reference versions are baselines rather than claims about the newest release. Compilation of wrapped fragments does not make them complete applications. No zero-defect guarantee is made.

Appendix C Repaired bounded thread pool

Worked alternative to the intentionally incomplete Q184 exercise in the ElevatePrep-aligned question bank. This preserves the supplied reviewed edition's atomic submission/shutdown boundary, Future-visible failures and explicit rejection.

Worked solution: minimal fixed thread pool

```
class MiniPool implements AutoCloseable {
    private final Object lock = new Object();
    private final java.util.ArrayDeque<java.util.concurrent.FutureTask<?>> queue =
            new java.util.ArrayDeque<>();
    private final java.util.List<Thread> workers = new java.util.ArrayList<>();
    private final int capacity;
    private boolean stopped; // guarded by lock

    MiniPool(int threads) { this(threads, 100); }
    MiniPool(int threads, int capacity) {
        if (threads < 1 || capacity < 1)
            throw new IllegalArgumentException("positive threads and capacity required");
        this.capacity = capacity;
        for (int i = 0; i < threads; i++) {
            Thread worker = new Thread(this::loop, "mini-" + i);
            workers.add(worker);
            worker.start();
        }
    }

    java.util.concurrent.Future<?> submit(Runnable task) {
        var future = new java.util.concurrent.FutureTask<Void>(
                java.util.Objects.requireNonNull(task), null);
        synchronized (lock) {
            if (stopped || queue.size() == capacity)
```

```
                throw new java.util.concurrent.RejectedExecutionException("closed or full");
            queue.addLast(future);
            lock.notifyAll();
        }
        return future;
    }

    private void loop() {
        while (true) {
            java.util.concurrent.FutureTask<?> task;
            synchronized (lock) {
                while (queue.isEmpty() && !stopped) {
                    try { lock.wait(); }
                    catch (InterruptedException ignored) {
                        // Workers are private; graceful shutdown is signalled by stopped.
                    }
                }
                if (queue.isEmpty()) return;
                task = queue.removeFirst();
            }
            task.run(); // FutureTask captures task failure for Future.get().
            Thread.interrupted(); // do not leak a task's interrupt status to the next task
        }
    }

    @Override public void close() throws InterruptedException {
        if (workers.contains(Thread.currentThread()))
            throw new IllegalStateException("close must be called by an external owner");
        synchronized (lock) {
            stopped = true; // atomic with the submission decision
            lock.notifyAll();
        }
        // Accepted tasks drain. If this wait is interrupted, shutdown remains in effect.
        for (Thread worker : workers) worker.join();
    }
}
```

Reasoning and alternatives: it shows the core of `ThreadPoolExecutor`: worker threads draining a bounded queue, explicit rejection, Future results and graceful shutdown. The exercise omits dynamic sizing and configurable thread factories.

What if/test: submission and shutdown share a lock, so an accepted task is queued before shutdown can finish. A full queue rejects immediately; this is an explicit overload policy, not blocking backpressure. Task failures are visible through the returned Future. Call close from an external owner; it drains accepted tasks and can wait indefinitely for a task that never finishes. Interruption of the owner stops its wait but leaves graceful shutdown in progress. For production use ThreadPoolExecutor; the exercise omits forced cancellation, worker replacement and robust thread-start failure handling. Test rejection, task failure and concurrent submit/close with latches.
Signal: explain the worker loop and why the queue is bounded.